Bank Marketing Profiles

Maksym Khavil

2025-07-25

Contents

1 Explanatory Data Analysis **2**
- 1.1 Loading data . . . 2
- 1.2 Binary Data Analysis . . . 2
- 1.3 Multiclass Data analysis . . . 4
- 1.4 Numerical Data . . . 16
- 1.5 Feature interactions . . . 30
- 1.6 Treatment of 'unknown' values . . . 81
- 1.7 Conclusion . . . 82

2 Conducting Statistical Tests **82**
- 2.1 Setup . . . 82
- 2.2 Distributions . . . 83
- 2.3 Catgorical vs Target . . . 89
- 2.4 Numerical vs Target . . . 90
- 2.5 Numerical vs Numerical . . . 92
- 2.6 Categorical vs Categorical . . . 93
- 2.7 Numerical vs Categorical . . . 94
- 2.8 Conclusion . . . 95

3 Modelling **95**
- 3.1 Setup . . . 95
- 3.2 Baseline Models . . . 98
- 3.3 Feature Selection and Engineering . . . 108
- 3.4 Final model . . . 123
- 3.5 Assumption Verification . . . 124
- 3.6 Final Decision Tree . . . 131
- 3.7 Wrap up . . . 132

4 Profiling **132**
- 4.1 Setup . . . 132
- 4.2 Data analysis . . . 137
- 4.3 Creating Profiles . . . 153
- 4.4 Profiles . . . 165
- 4.5 Conclusion . . . 168

Here is a complete research conduct on Bank Marketing dataset, where we derive a profile for future campaign based on data. In this research we do explanatory data analysis, modeling customer behaviour using statistical logistic regression and profile the population based on obtained knowledge.

1 Explanatory Data Analysis

1.1 Loading data

The dataset is loaded using a function from `R/load_data.R` script, we will also load required libraries.

```
library(corrplot)

source(here::here("R", "constants.R"))
source(here::here("R", "load_data.R"))

data = load_bank_data()
head(data)
```

```
##   age         job marital education default balance housing loan  contact day
## 1  30  unemployed married   primary      no    1787      no   no cellular  19
## 2  33    services married secondary      no    4789     yes  yes cellular  11
## 3  35  management  single  tertiary      no    1350     yes   no cellular  16
## 4  30  management married  tertiary      no    1476     yes  yes  unknown   3
## 5  59 blue-collar married secondary      no       0     yes   no  unknown   5
## 6  35  management  single  tertiary      no     747      no   no cellular  23
##   month duration campaign pdays previous poutcome  y
## 1   oct       79        1    -1        0  unknown no
## 2   may      220        1   339        4  failure no
## 3   apr      185        1   330        1  failure no
## 4   jun      199        4    -1        0  unknown no
## 5   may      226        1    -1        0  unknown no
## 6   feb      141        2   176        3  failure no
```

Let us separate the data into positive and negative samples for the ease of further analysis.

```
pos_labeled = data[data$y == "yes",]
neg_labeled = data[data$y == "no",]
print(sprintf("Number of positive samples: %d", nrow(pos_labeled)))
```

```
## [1] "Number of positive samples: 521"
```

```
print(sprintf("Number of negative samples: %d", nrow(neg_labeled)))
```

```
## [1] "Number of negative samples: 4000"
```

```
print(sprintf("Positive rate:              %.3f", nrow(pos_labeled) / nrow(data)))
```

```
## [1] "Positive rate:              0.115"
```

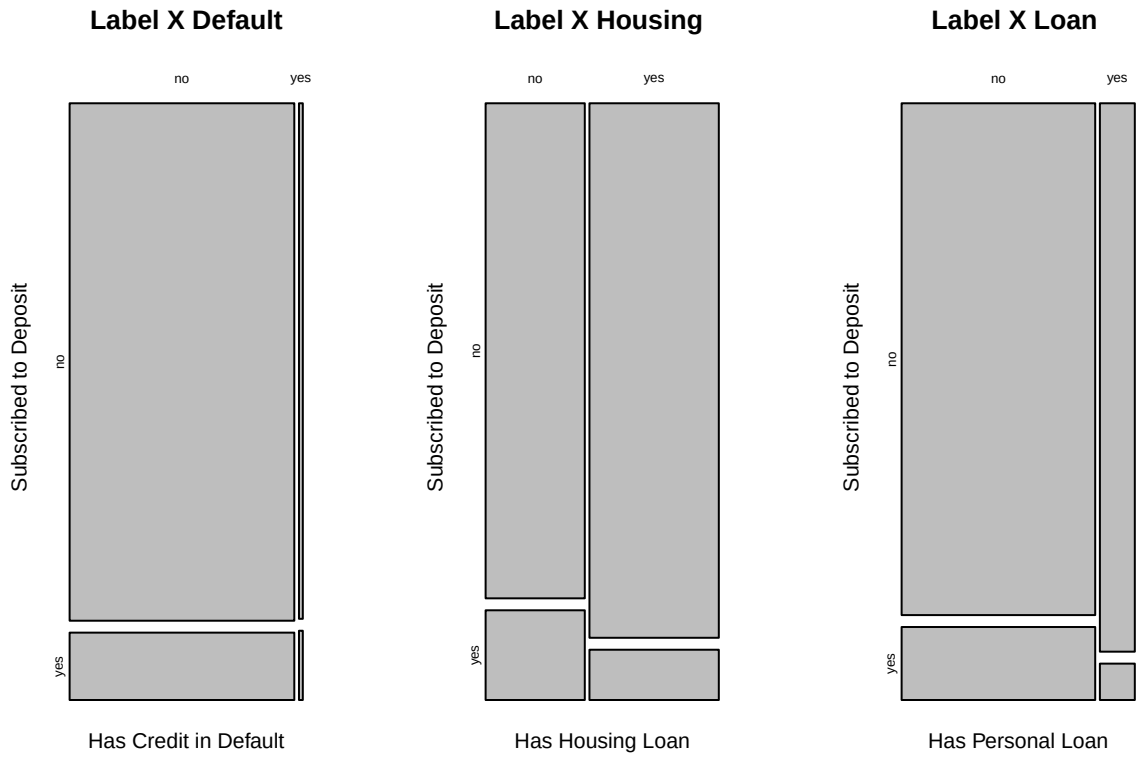
It is clear that data is strongly skewed towards negative outcome (the contacted person has not subscribed to the deposit) with only 11.5% of samples being positive.

1.2 Binary Data Analysis

We will plot contingency tables between binary features and target variable to investigate their relations. We want to check if those variables interact with target variable in any way.

```
par(mfrow=c(1, 3))

ct = table(data$default, data$y)
mosaicplot(
    ct, ylab="Subscribed to Deposit", xlab="Has Credit in Default",
    main="Label X Default"
```

```
)

ct = table(data$housing, data$y)
mosaicplot(
    ct, ylab="Subscribed to Deposit", xlab="Has Housing Loan",
    main="Label X Housing "
)

ct = table(data$loan, data$y)
mosaicplot(
    ct, ylab="Subscribed to Deposit", xlab="Has Personal Loan",
    main="Label X Loan "
)
```



Mosaic plots show that standalone **default** feature does not differentiate between target classes, due to its values being extrmely skewed towards negative answer and both groups have similar proportions of people subscribed to deposit.

Feature **loan**, representing whether the client has a personal loan, is also dominated by negative answers, though not as severe as **default**, and a population without the personal loan is more likely to subscribe to deposit.

The biggest distinction between target classes is given by **housing**, a feature indicating if client has housing loan. It is balanced between two classes and a person that had not borrowed for a house is more likely to subscribe to term deposit.

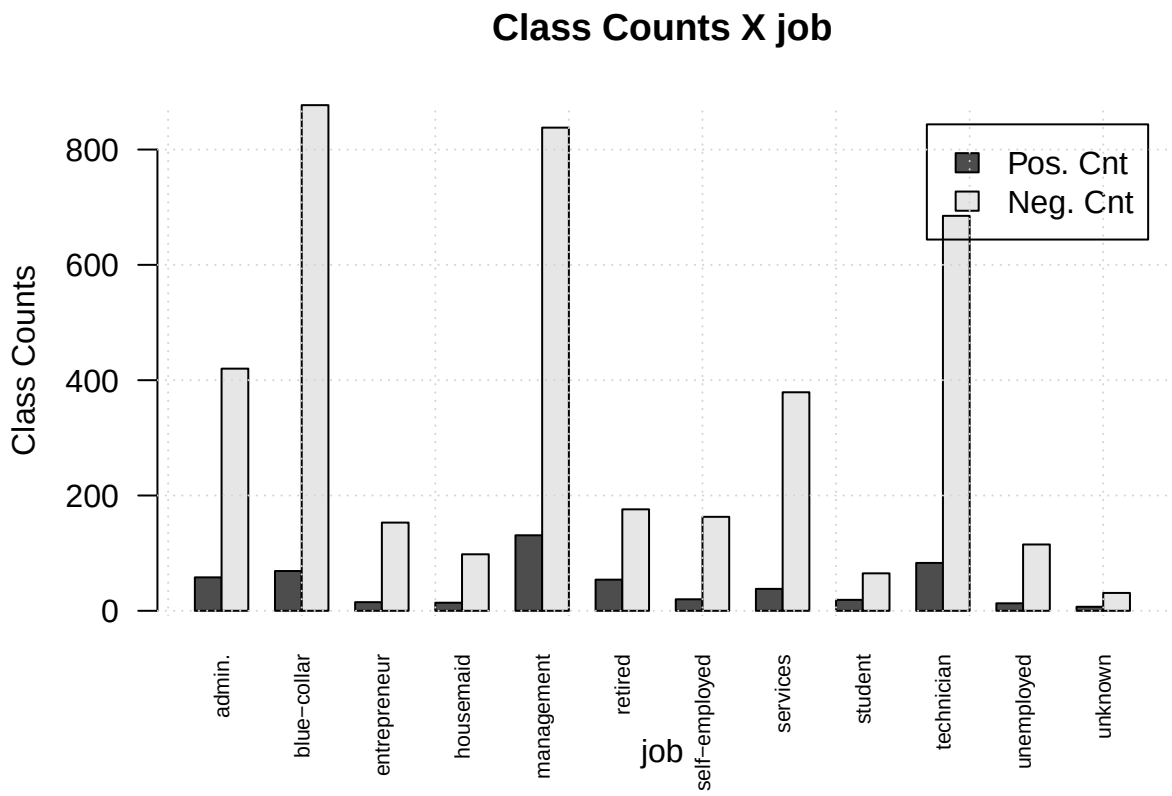
1.3 Multiclass Data analysis

For each multiclass feature we will plot absolute counts and rates of both classes to examine feature structure and interactions with target variable. By those means we want to understand whether different classes are more favorable to subscribe to the deposit, as well as study class distributions.

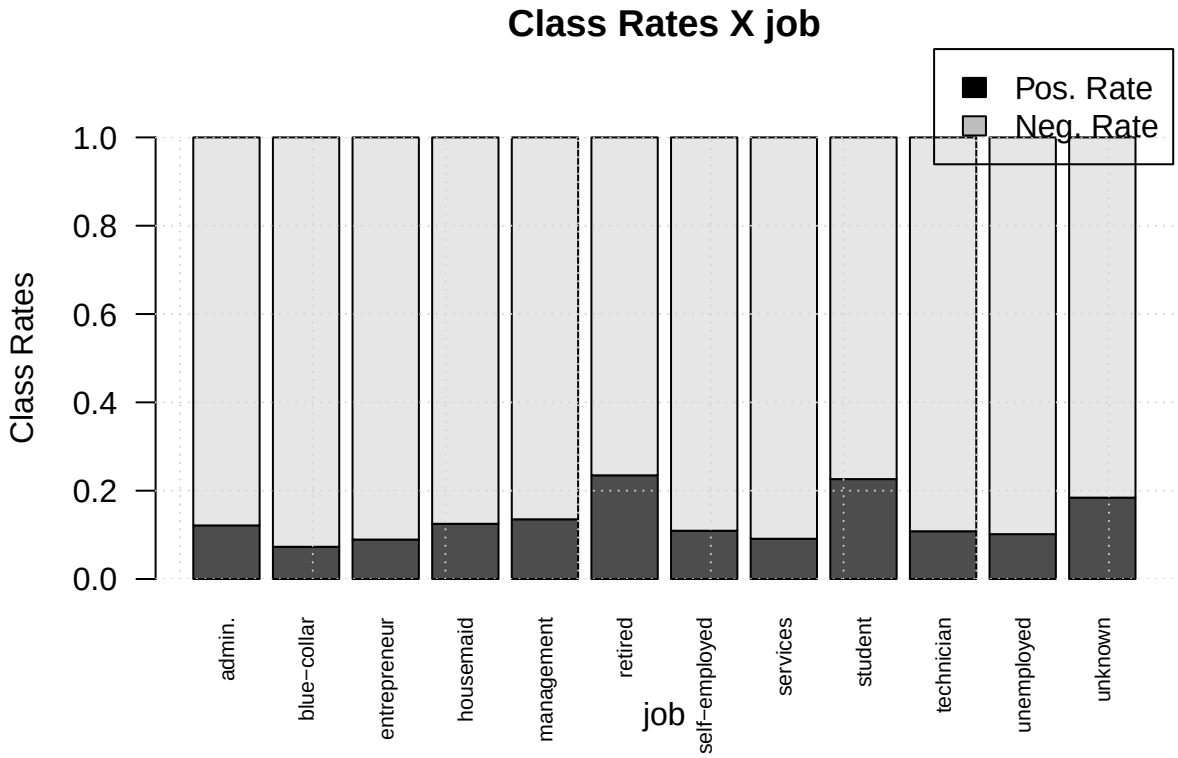
```
plot_count_bars <- function(col, ...) {
    pos <- aggregate(x = list(count = pos_labeled$y), by = list(column = pos_labeled[,col]), FUN = leng
    neg <- aggregate(x = list(count = neg_labeled$y), by = list(column = neg_labeled[,col]), FUN = leng
    data_matrix <- matrix(
        c( pos$count,  neg$count), nrow = 2, byrow = TRUE
    )
    colnames(data_matrix) <- pos$column
    rownames(data_matrix) <- c("Pos. Cnt", "Neg. Cnt")
    barplot(
        data_matrix,
        beside = TRUE,
        legend = rownames(data_matrix),
        main = sprintf("Class Counts X %s", col),
        xlab = col, ylab = "Class Counts",
        ...
    )
    grid()
}

plot_rate_bars <- function(col, ...) {
    par(mar=c(5, 4, 6, 2))
    total_cnt <- aggregate(x = list(count = data$y), by = list(column = data[,col]), FUN = length)$coun
    pos <- aggregate(x = list(count = pos_labeled$y), by = list(column = pos_labeled[,col]), FUN = leng
    neg <- aggregate(x = list(count = neg_labeled$y), by = list(column = neg_labeled[,col]), FUN = leng
    data_matrix <- matrix(
        c( pos$count / total_cnt,  neg$count / total_cnt), nrow = 2, byrow = TRUE
    )
    colnames(data_matrix) <- pos$column
    rownames(data_matrix) <- c("Pos. Rate", "Neg. Rate")
    barplot(
        data_matrix,
        main = sprintf("Class Rates X %s", col),
        ylim=c(0,1),
        xlab = col, ylab = "Class Rates",
        ...
    )
    grid()
    legend(
        "topright",
        legend = rownames(data_matrix),
        xpd = T,
        fill=c("black", "grey"),
        inset=c(0, -0.2)
    )
}
```

1.3.1 Job

```
plot_count_bars("job", cex.names = 0.7, las=2)
```



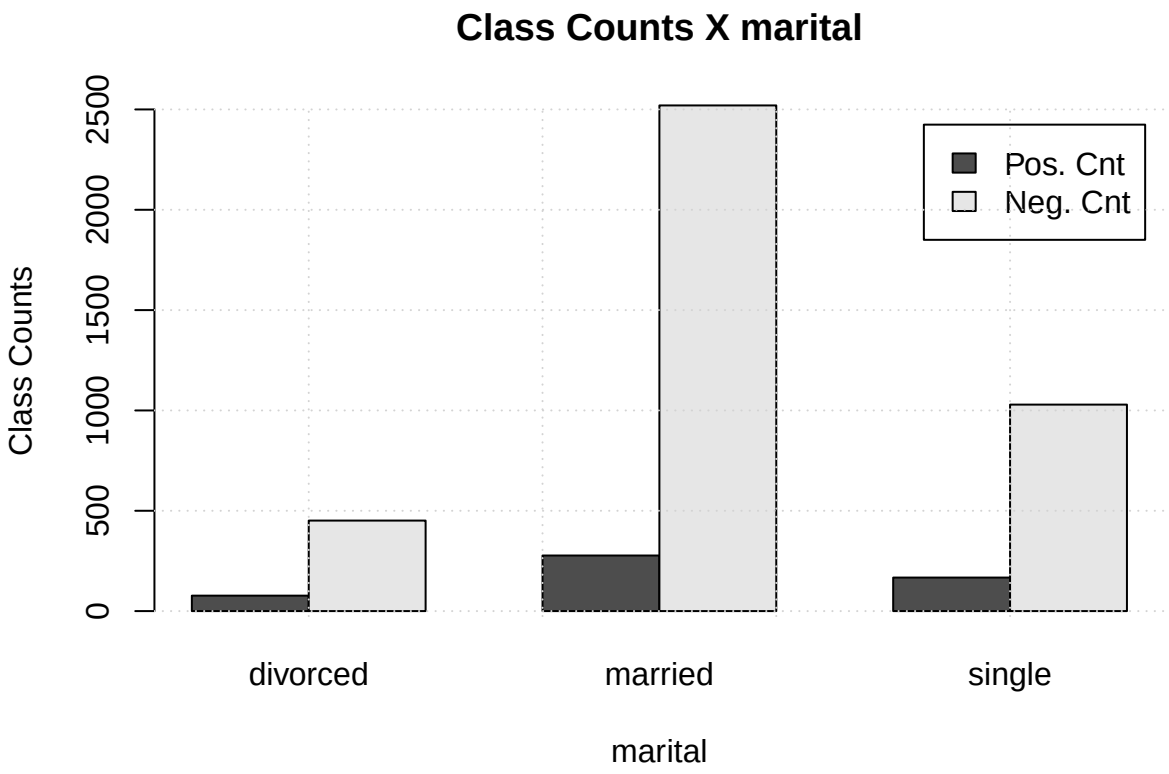

```
plot_rate_bars("job", cex.names = 0.7, las=2)
```



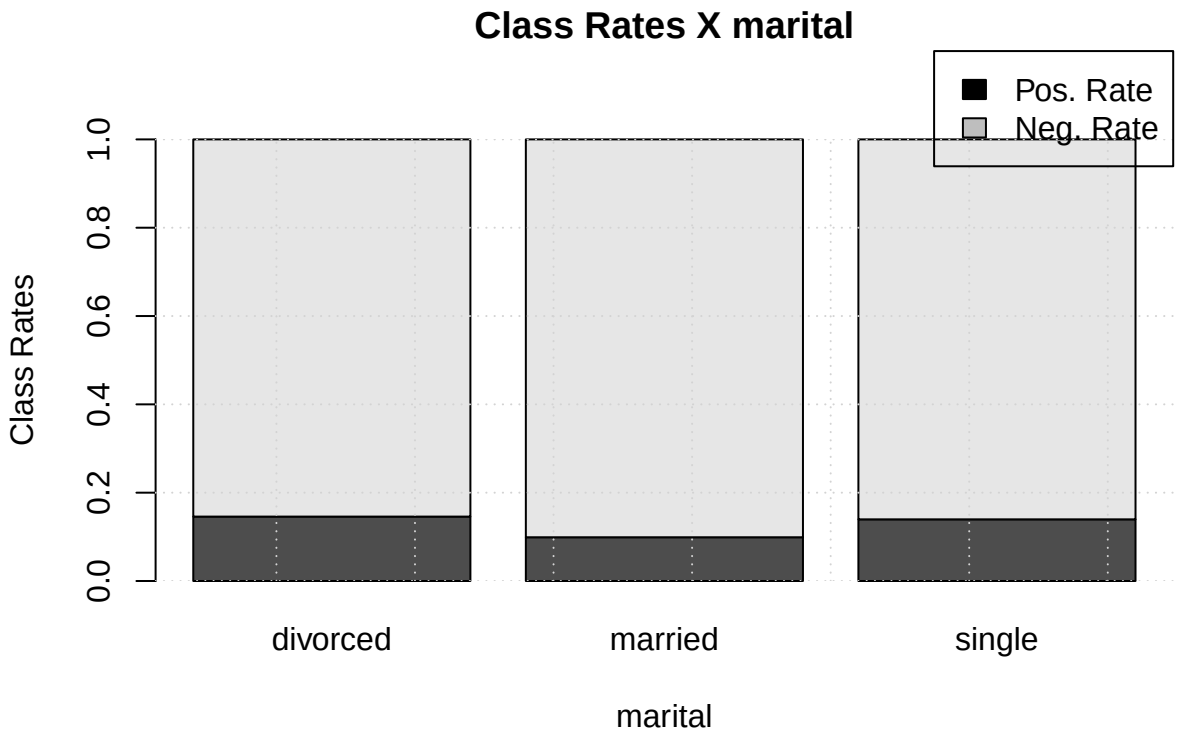
Most of population is employed in management or as a technician and blue-collar workers. Other large categories include administration and service. Every job group shows results, that are similar to the whole data-set, i.e. positive rate close to 10%, except for *retired*, *students* and *unknown*. That is an interesting phenomenon that can be checked later during statistical testing.

1.3.2 Marital Status

```
plot_count_bars("marital")
```



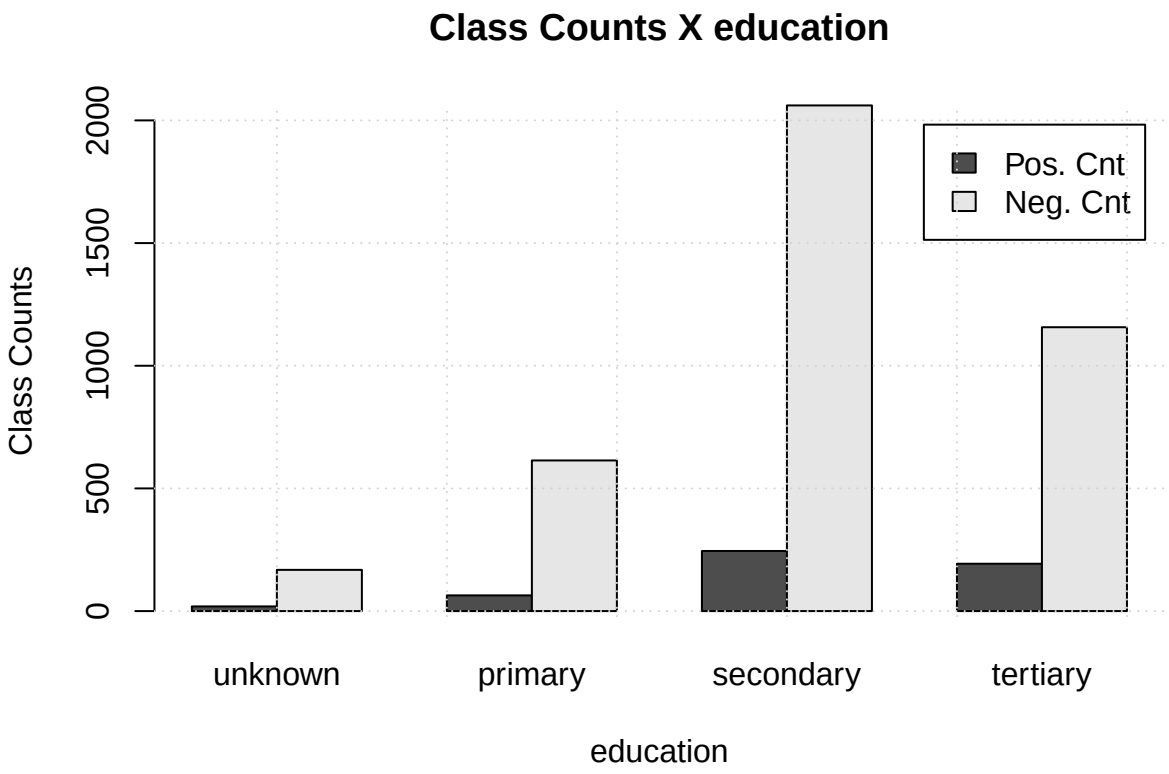

```
plot_rate_bars("marital")
```



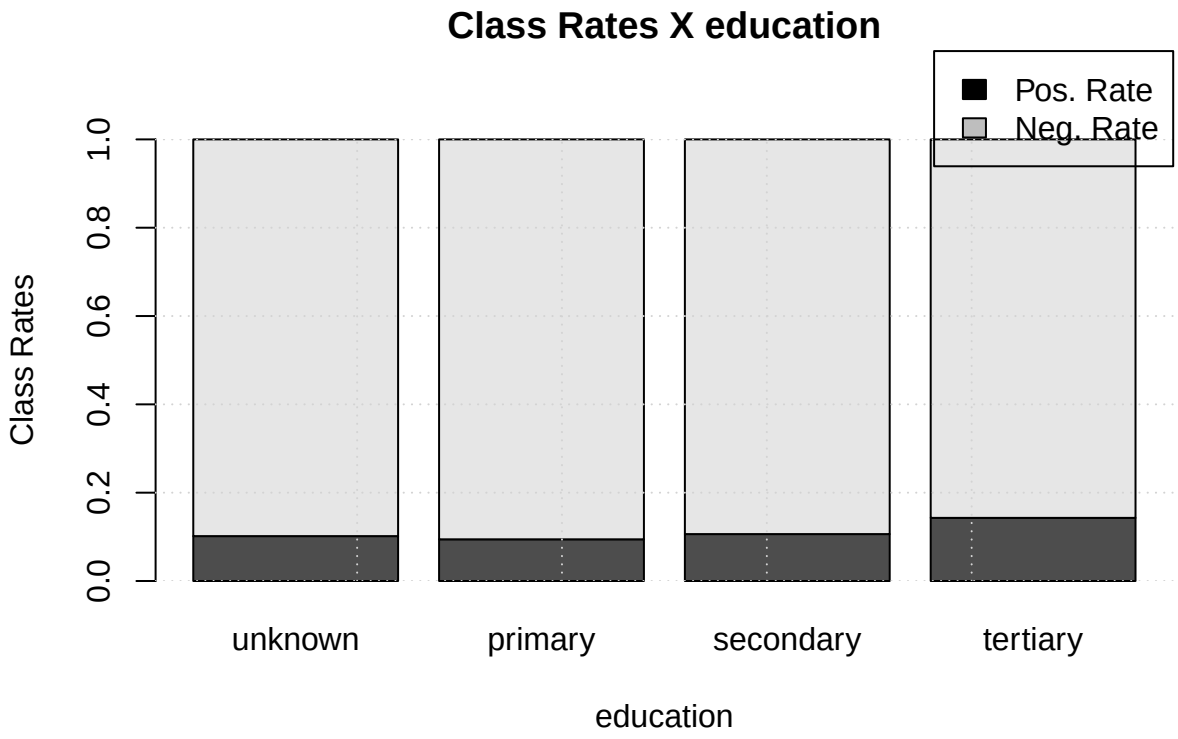
Most of the contacted people were married, but they were the least likely to subscribe to the deposit, though the difference may be insignificant.

1.3.3 Education

```
plot_count_bars("education")
```



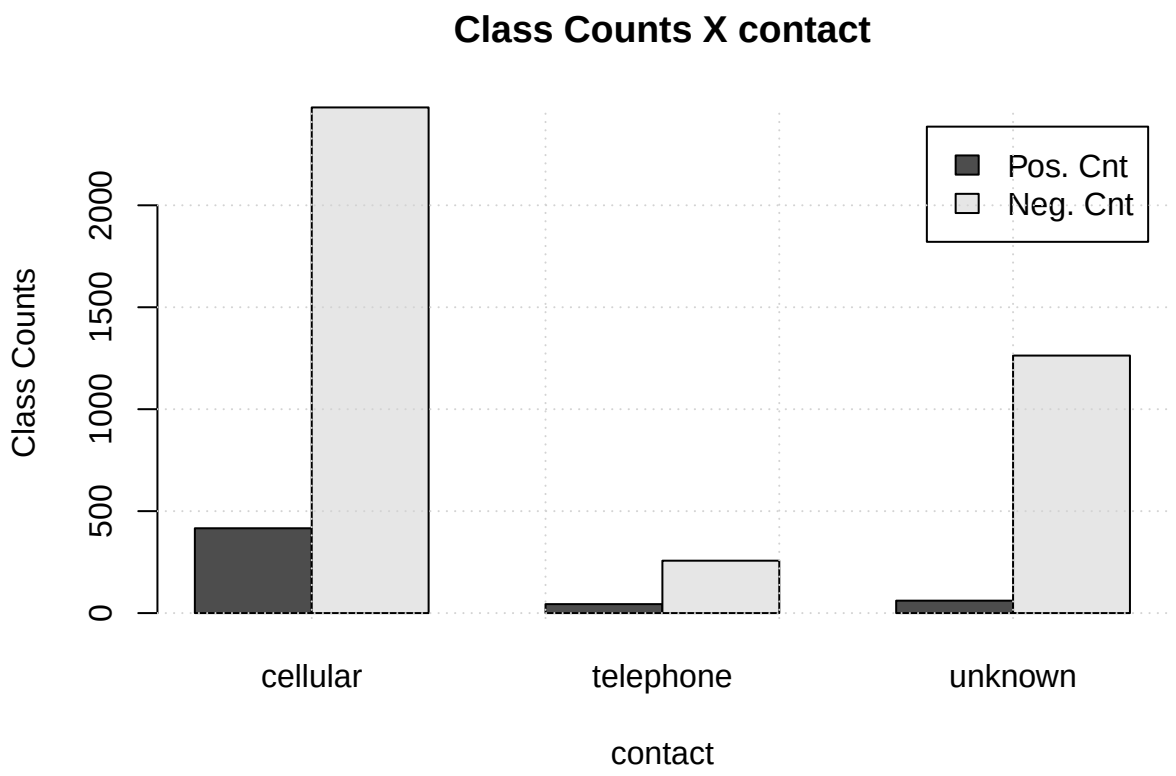

```
plot_rate_bars("education")
```



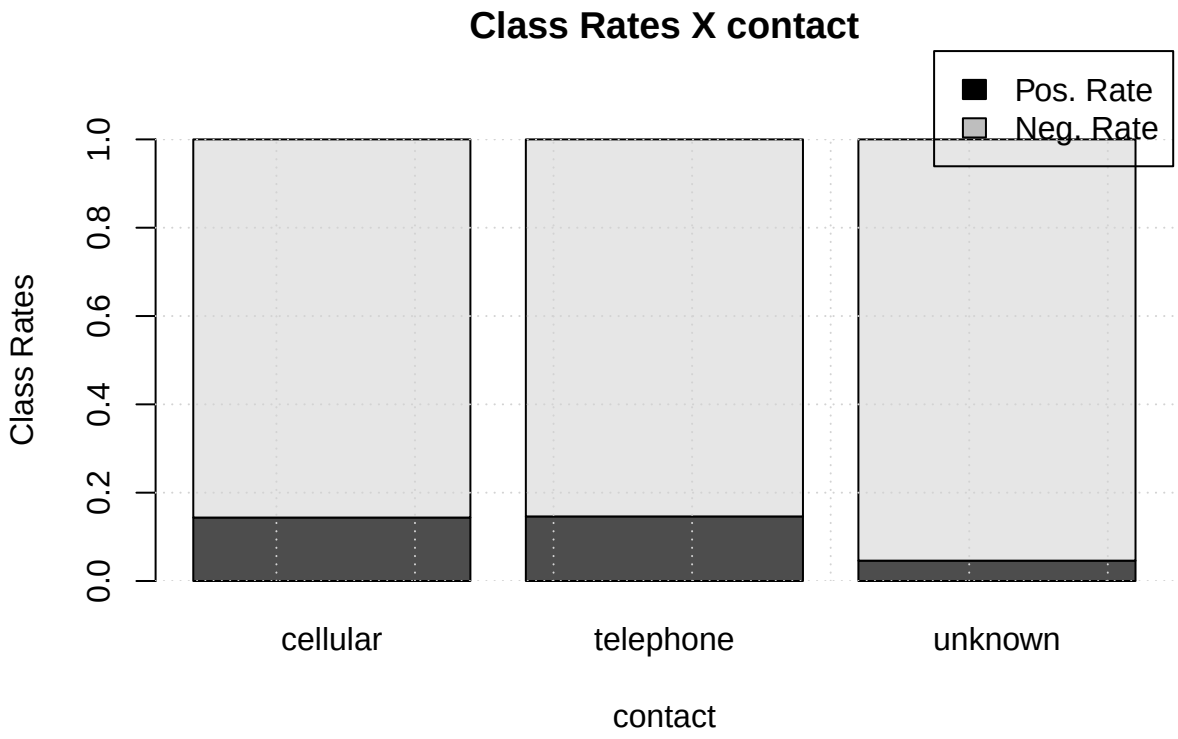
People with secondary education are the biggest fraction of those who took part in the data collection process, it is no surprise, as secondary education is obligatory. Despite the difference in past all three groups agree on data-set wide positive rate of around 10%. Maybe combinations with other categorical and numerical features will be able to help us reveal patterns behind people subscribing, but education alone does not provide with any insights.

1.3.4 Contact

```
plot_count_bars("contact")
```



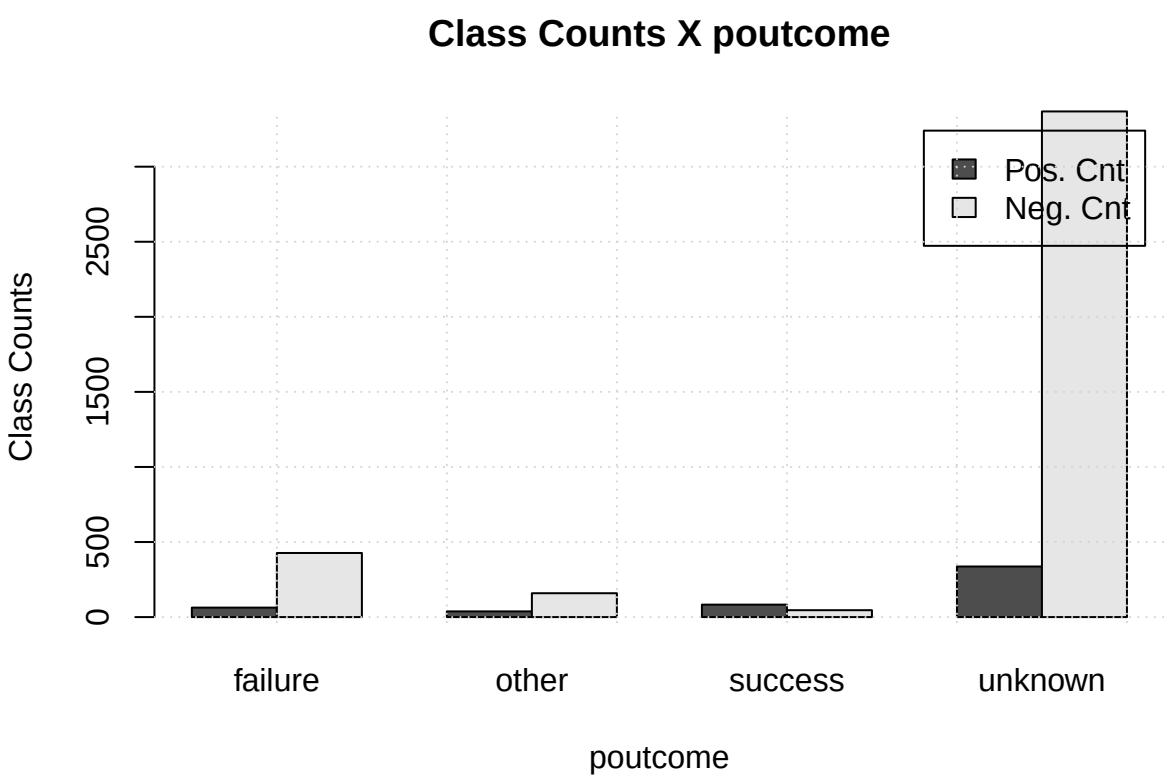

```
plot_rate_bars("contact")
```



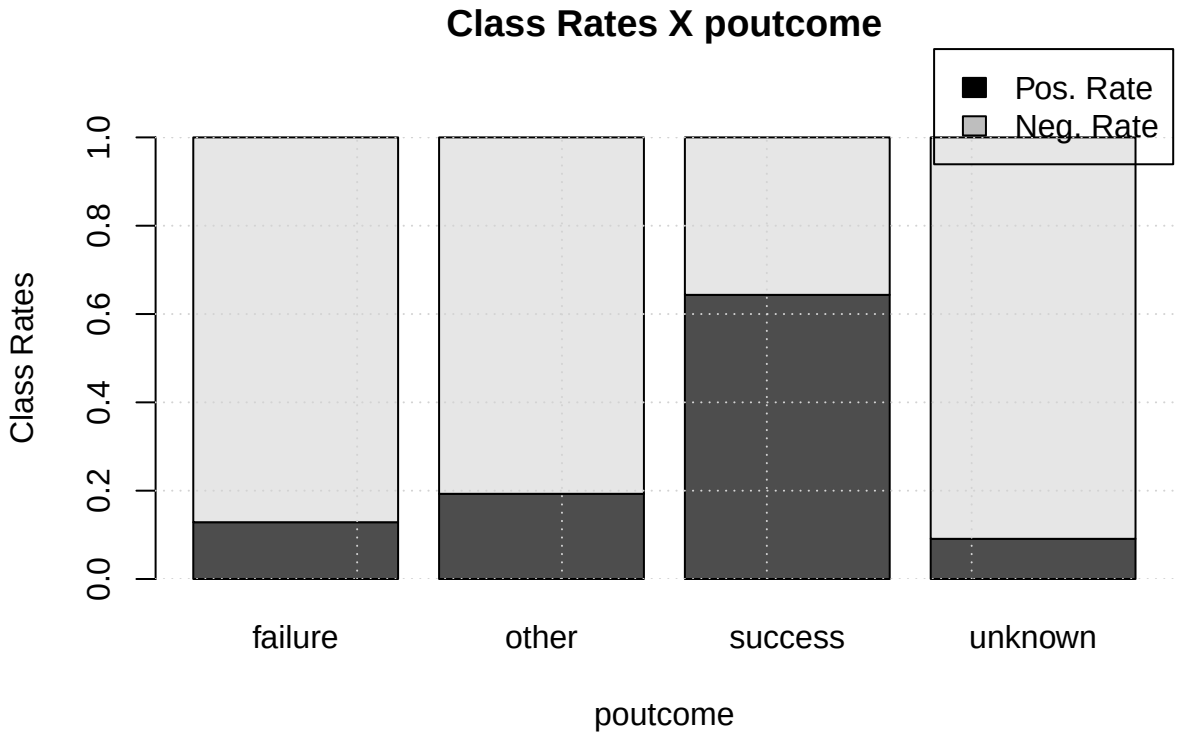
Cellular calls were the most frequent form of communication. Unlabeled records also present a large part of population, while the telephone category is the least prominent. With such distribution of values, we should ask our selves, whether there are any benefits in keeping **contact** feature with almost a third of its records being unknowns, and the rest primarily binned into single category. More over cellular and telephone positive rates are very close and the distinction may in insufficient to provide any knowledge.

1.3.5 Outcome of previous campaign

```
plot_count_bars("poutcome")
```



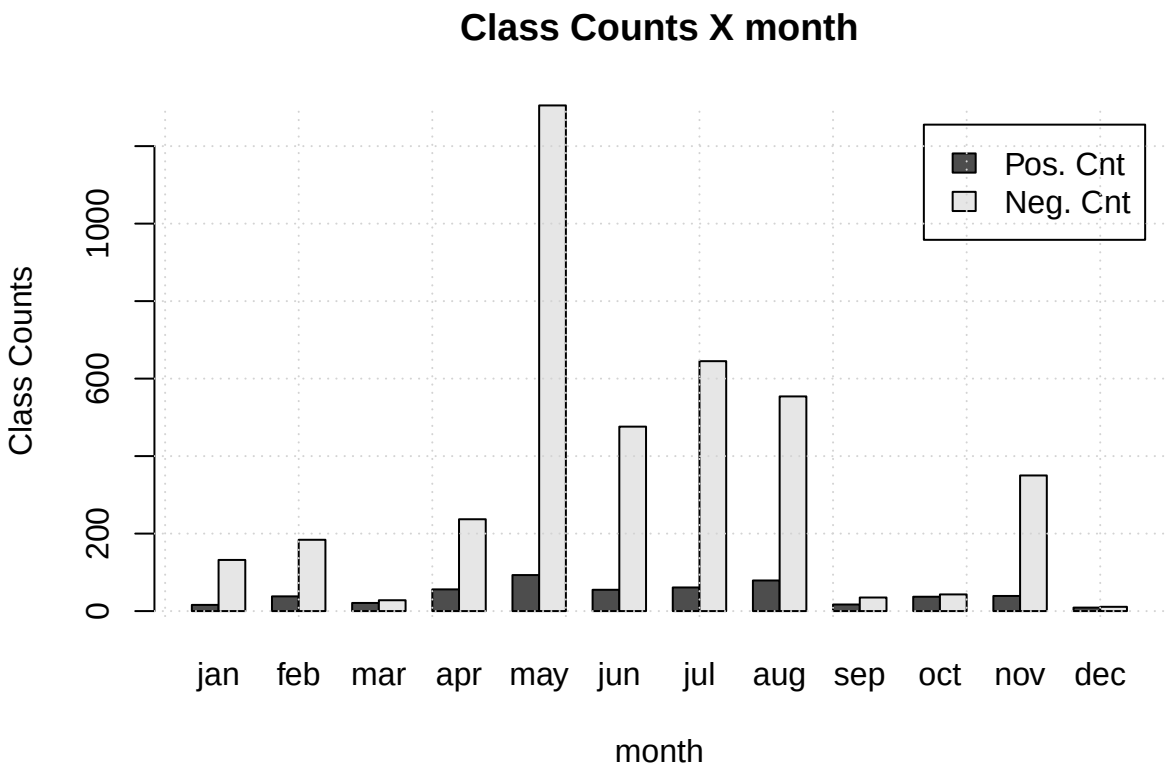

```
plot_rate_bars("poutcome")
```



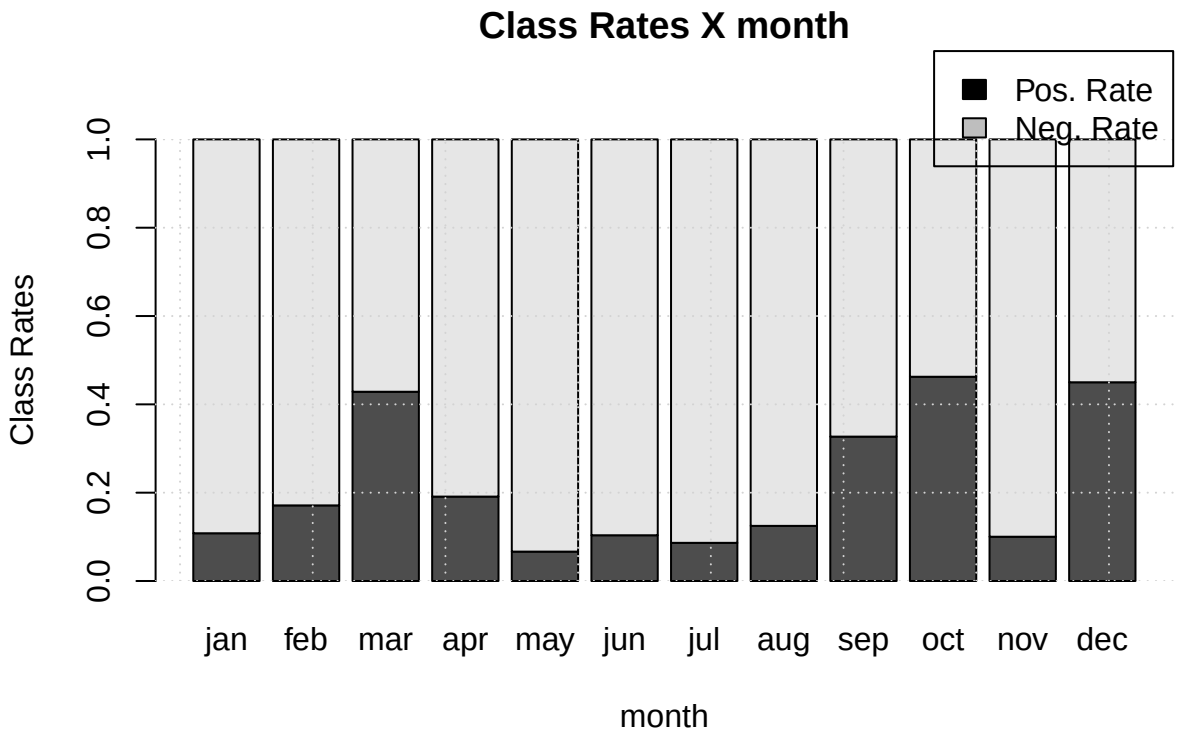
Most of previous outcomes are *unknown*, that is most of people had not been contacted before the campaign. Rates shows, that people who has subscribed during the last campaign, are the most likely to subscribe to the current campaign, while those who was not affected by previous campaign show opposite dynamic.

1.3.6 Month of Call

```
plot_count_bars("month")
```




```
plot_rate_bars("month")
```



Wee that summer monthes (*may* - *aug*) have a lot more data than other. What is ther reason for that is not yet clear, it could be just another data engineered nuance, we could later do a test to check that.

1.4 Numerical Data

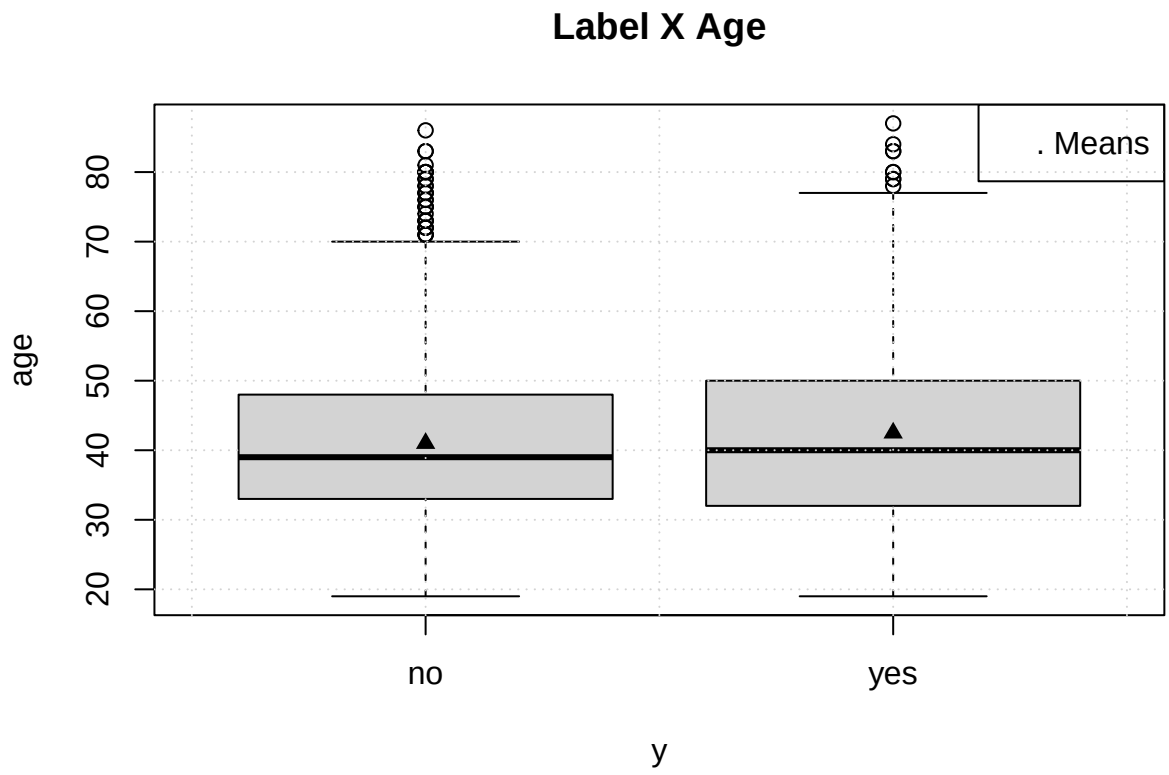
We will plot box-plots of numerical features against target variable overall dataset histogram to study univariate distributions and interactions with target variable.

```
boxplot_mean <- function(col_name, fctr='y') {
    formula = as.formula(paste(col_name, "~", fctr))
    boxplot(formula, data)
    grid()
    means <- aggregate(formula, data, FUN=mean)[,col_name]
    points(
        1:length(unique(data[,fctr])), means, pch=17
    )
    legend("topright", legend="  Means")
}

histogram_desc <- function(col_name, ...) {
    col_data = data[,col_name]
    hist(
        col_data,
        xlab = col_name,
        ...
    )
    abline(v=mean(col_data), col = "red", lwd=3)
```

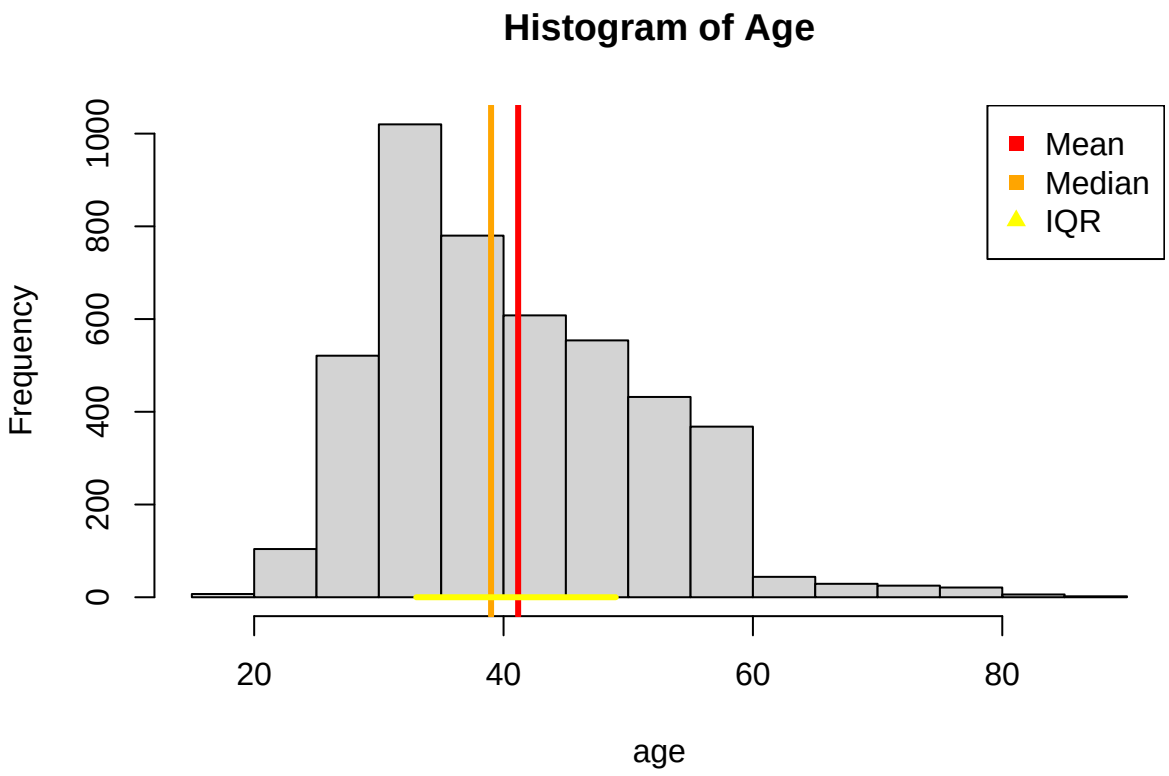
```
    abline(v=median(col_data), col = "orange", lwd=3)
    qs = quantile(col_data, probs=c(0.25, 0.75))
    segments(
        x0=qs["25%"], y0=0, x1=qs["75%"], y1=0, col = "yellow", lwd=3
    )
    legend(
        "topright",
        legend = c("Mean", "Median", "IQR"),
        col = c("red", "orange", "yellow"),
        pch = c(15, 15, 17)
    )
}
```

1.4.1 Age

```
boxplot_mean("age")
title("Label X Age")
```



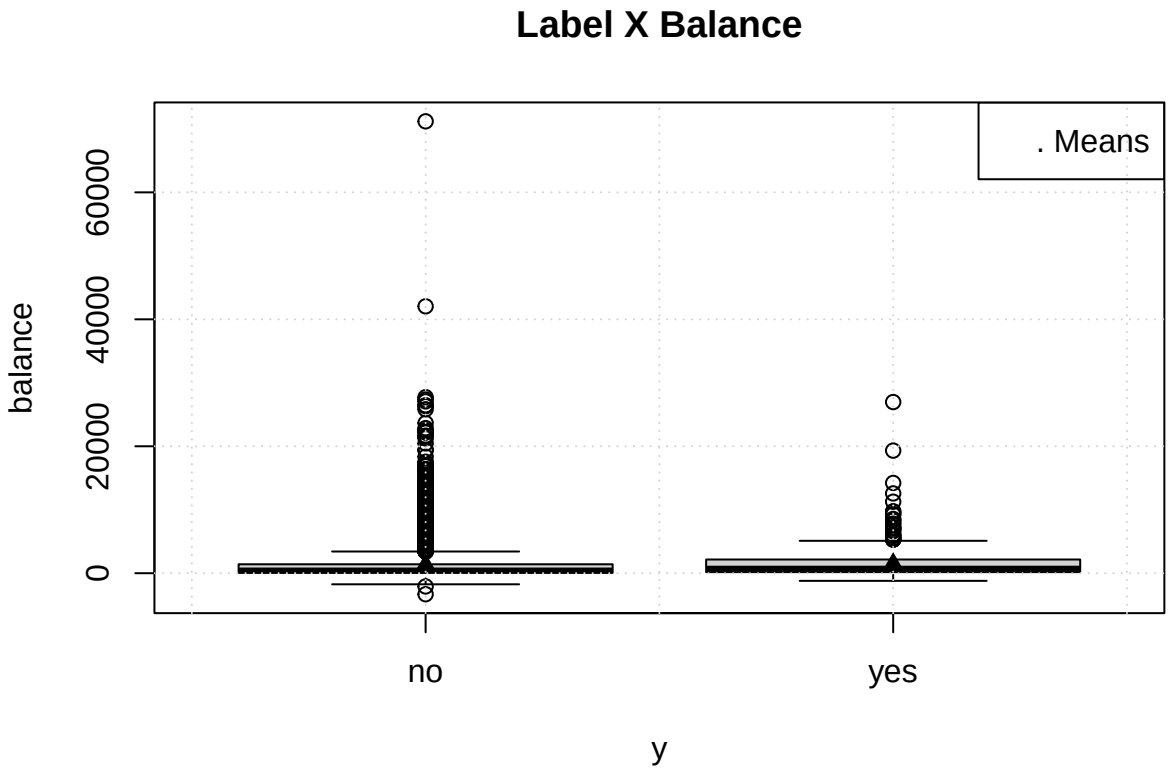

```
histogram_desc("age", main = "Histogram of Age")
```



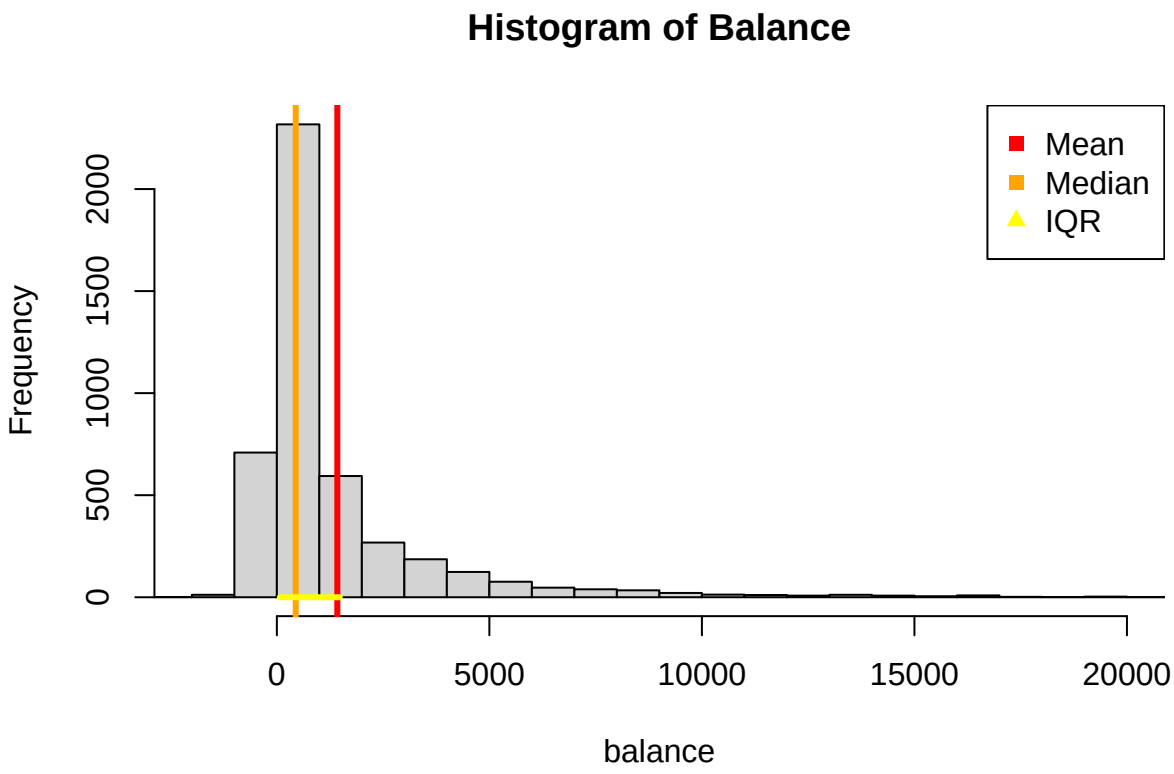
Boxes-and-whispers of both classes are very similar with median 39 and 40 and almost identical means. Population that has not subscribed has slightly lower range. The dataset histograms gravitates towards 30 years, we also see a suffen drop at 60 years, this could be a bias introduced during data engineering process, we should check for that during hypothesis testing.

1.4.2 Balance

```
boxplot_mean("balance")
title("Label X Balance")
```




```
histogram_desc("balance", main = "Histogram of Balance", xlim=c(-2000, 20000), breaks = 100)
```

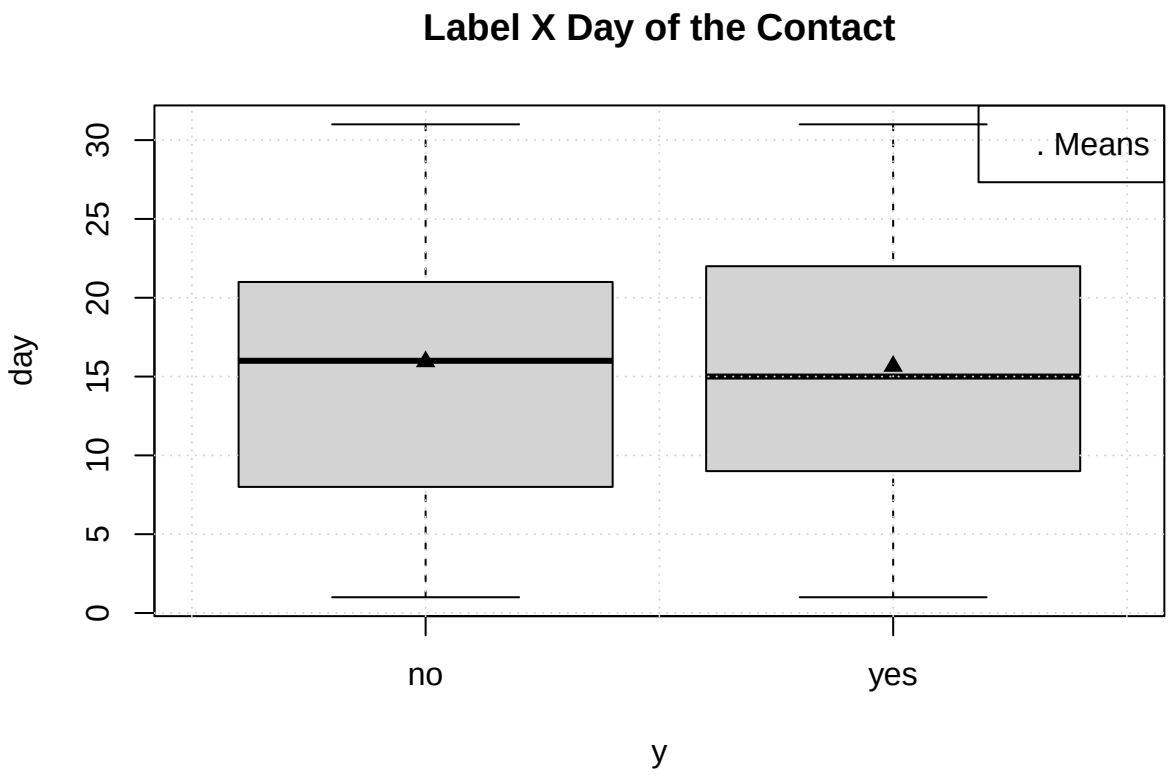


Both ranges are squashed, due to extreme values at the positive tail of distributions. During our modelling step we shall pay great attention to outliers, as they may overinfluence our model.

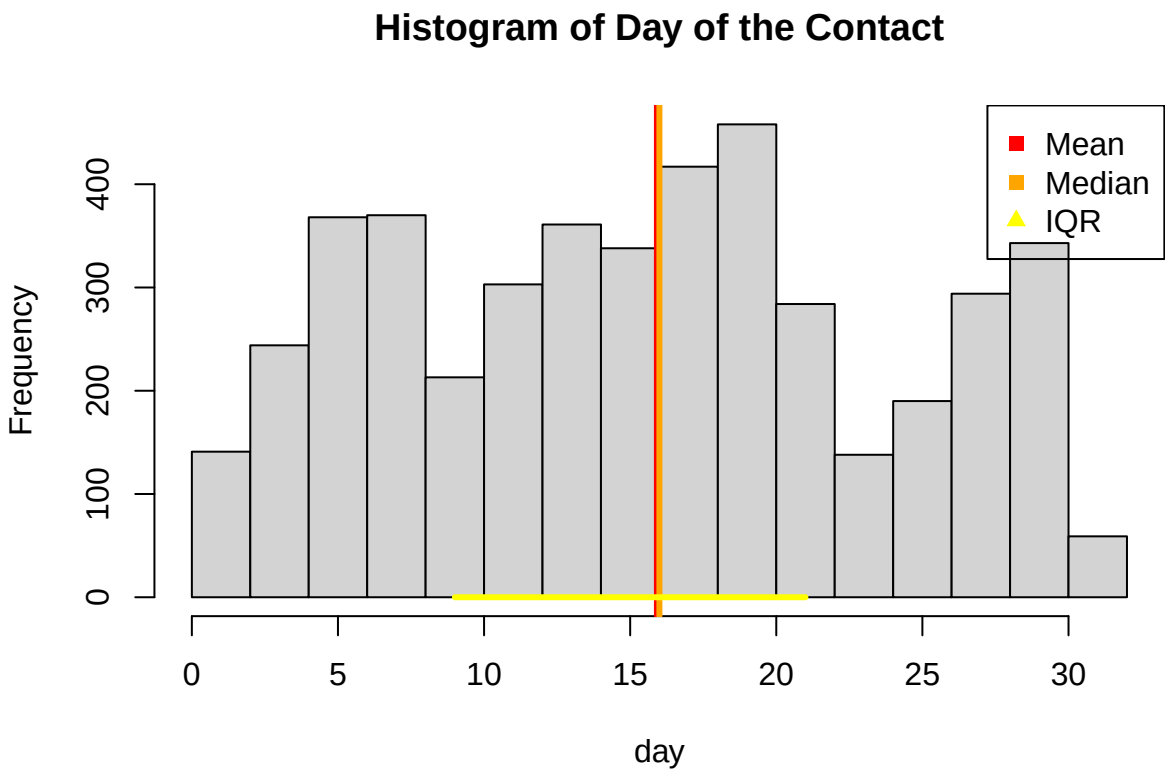
We have clipped the histograms to display greater mass of values, due to the outlier that is easily to spot at the top of boxplot. We see that almost all records are contained within (-1000, 5000) interval and distribution has a spiky bell curve.

1.4.3 Day of the Contact

```
boxplot_mean("day")
title("Label X Day of the Contact")
```



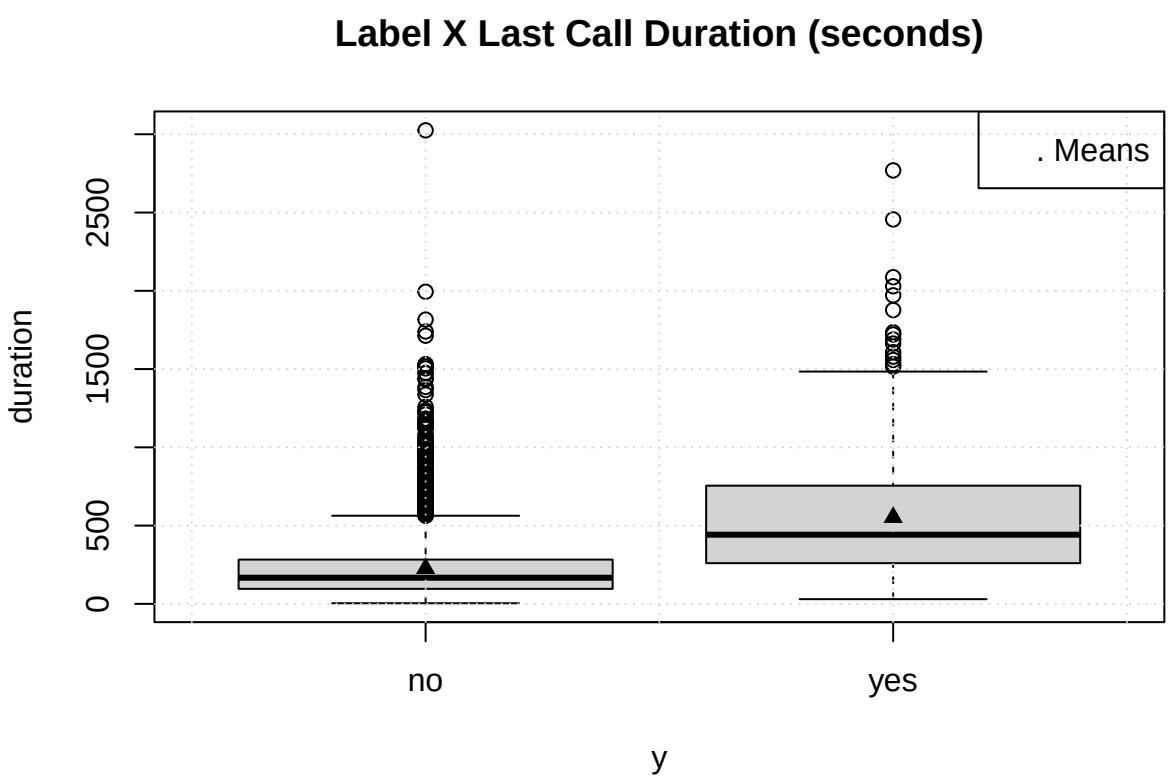

```
histogram_desc("day", main = "Histogram of Day of the Contact")
```



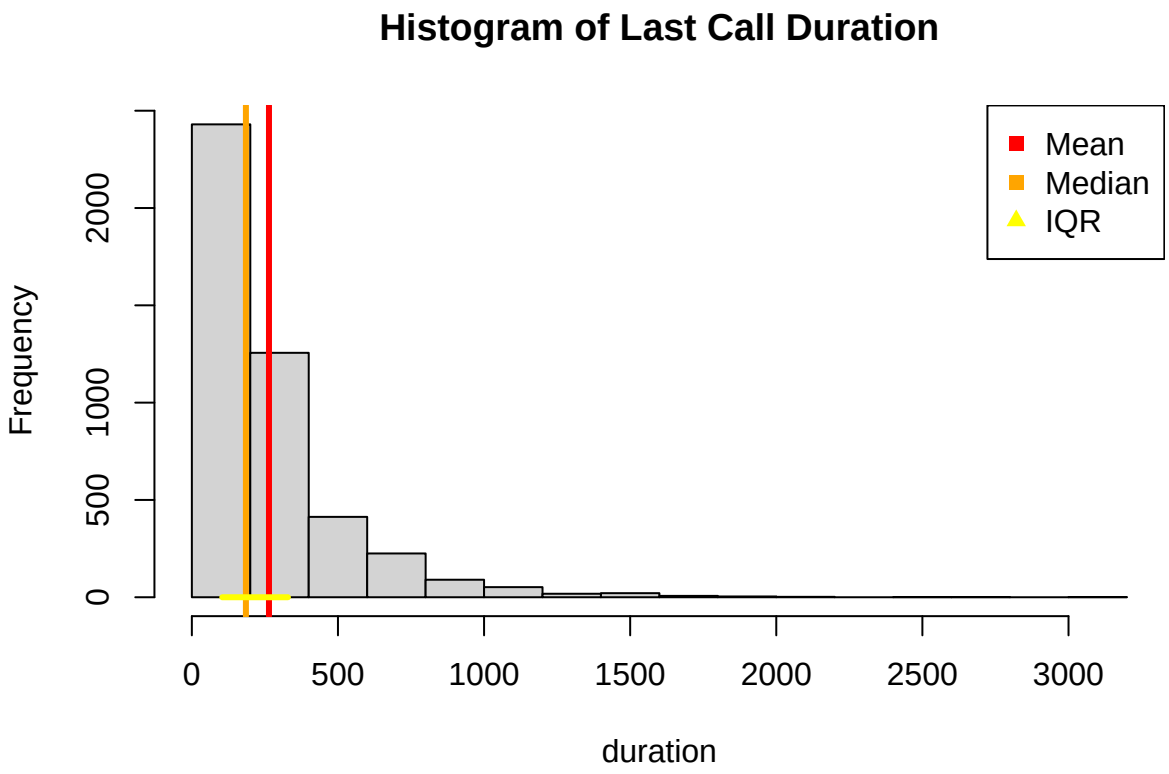
The data follow roughly uniform shape that splits equally into positive and negative population.

1.4.4 Duration of Last Call

```
boxplot_mean("duration")
title("Label X Last Call Duration (seconds)")
```




```
histogram_desc(
    "duration",
    main = "Histogram of Last Call Duration"
)
```

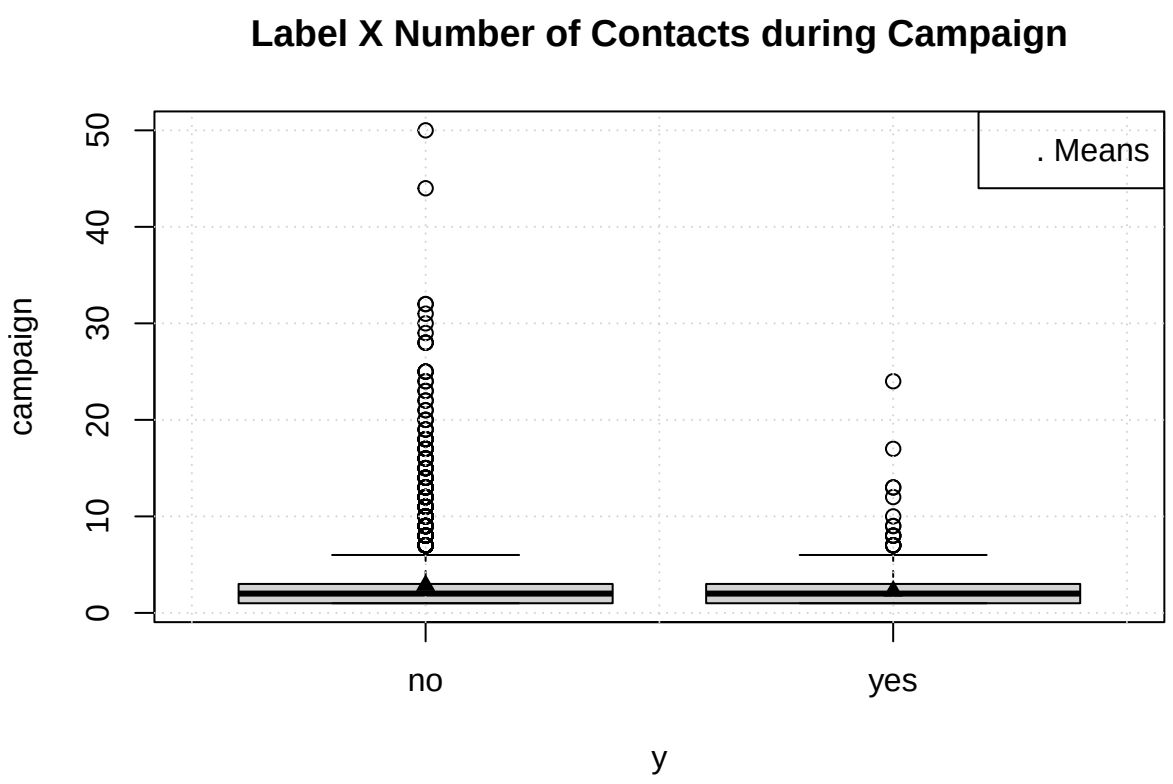


Here we see some real difference: positive population distribution is a lot wider with median 500 seconds = 8.3 minutes, while the other classes range just barely stretches over 500. We also see many outliers in both classes. Such sharp contrast may be instrumental to *yes* population, due to the deposit arranging taking longer time, than simply declining one.

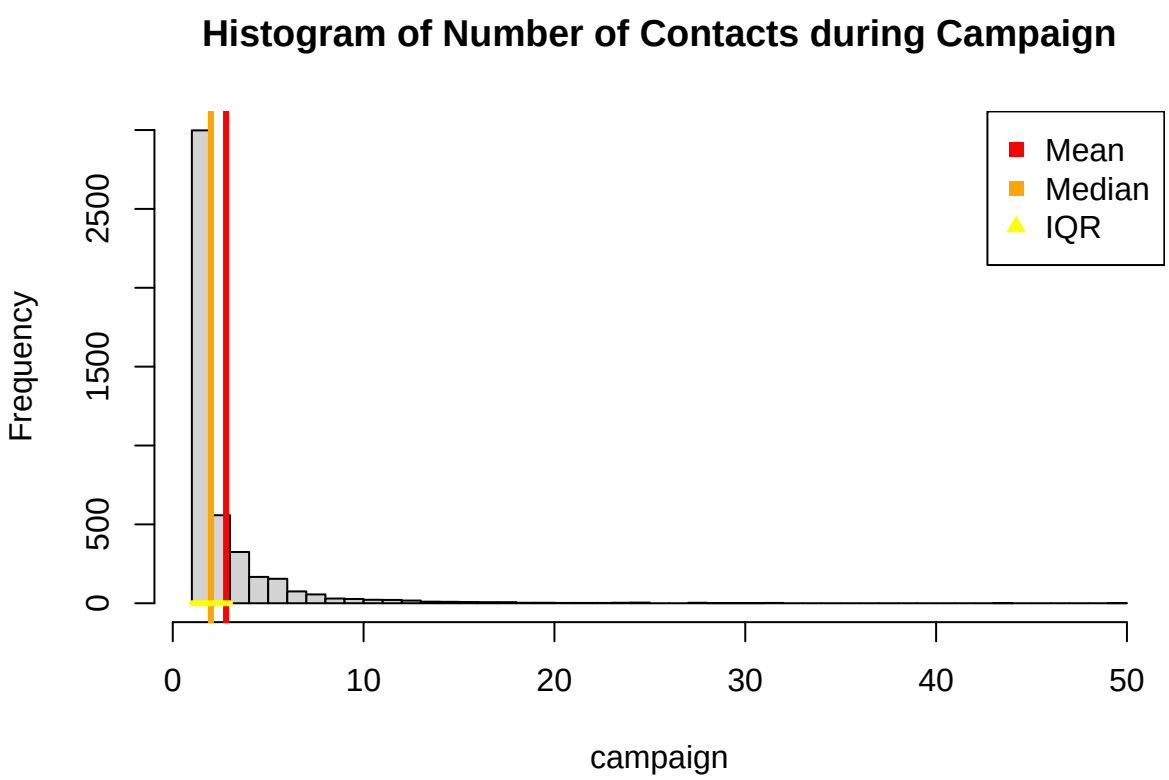
The histogram follows shape of Exponential distribution with a tail quickly vanishing after 1000 seconds.

1.4.5 Number of Contacts during Campaign (campaign)

```
boxplot_mean("campaign")
title("Label X Number of Contacts during Campaign")
```




```
histogram_desc(
    "campaign",
    main = "Histogram of Number of Contacts during Campaign",
    breaks = 50
)
```

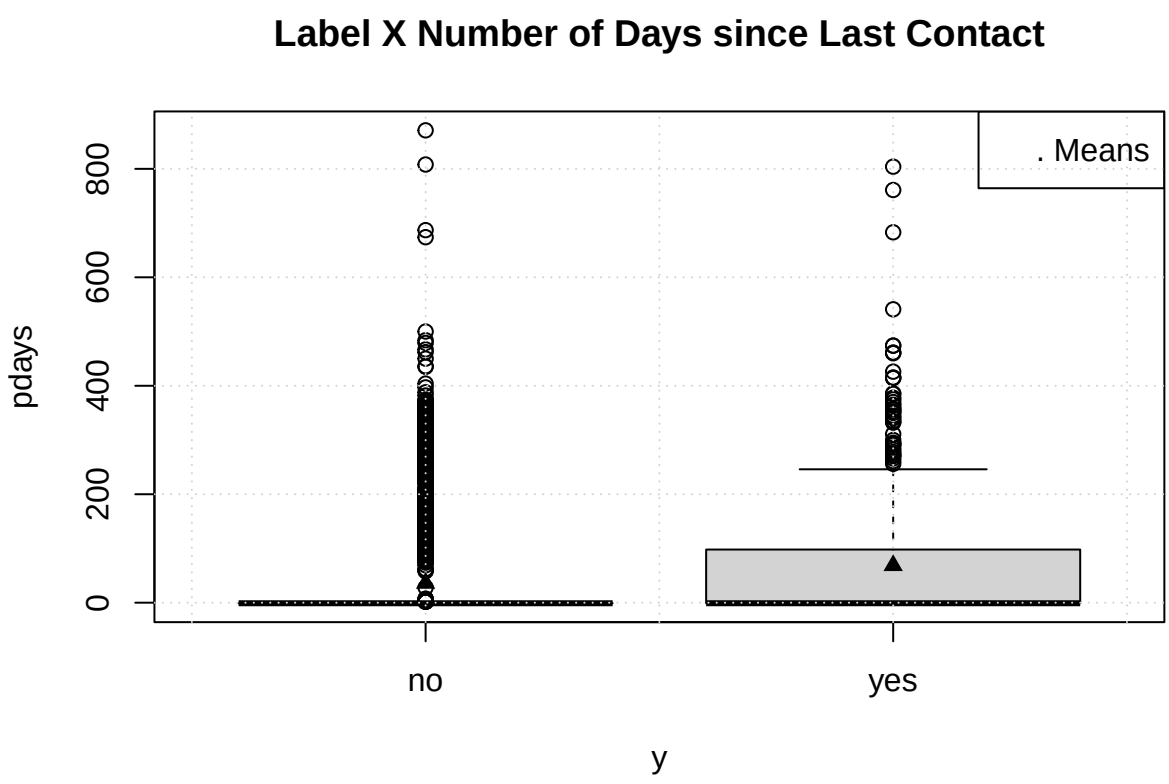


This plot has a lot in common with **balance**. It also shows great number of outliers and range squeezed into bottomline.

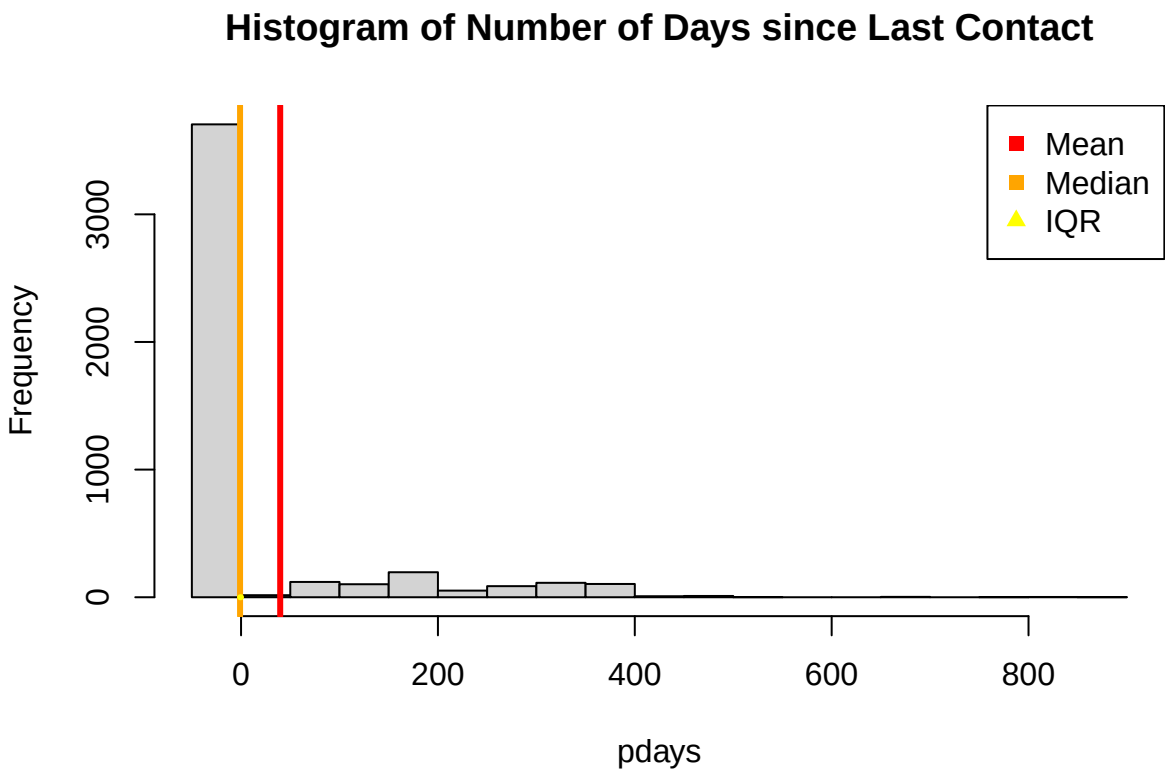
Histograms also shows fast rate of decay after 5 contacts per person.

1.4.6 Number of Days since Last Contact (pdays)

```
boxplot_mean("pdays")
title("Label X Number of Days since Last Contact")
```



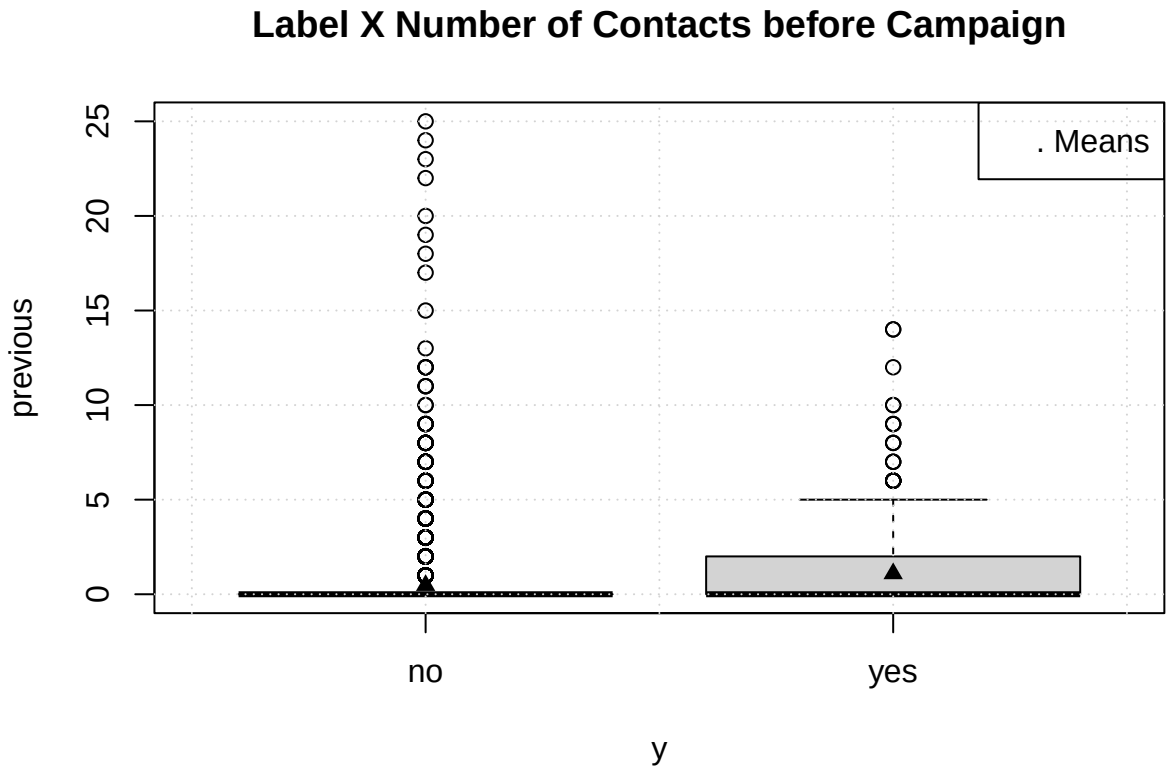

```
histogram_desc(
    "pdays",
    main = "Histogram of Number of Days since Last Contact",
)
```



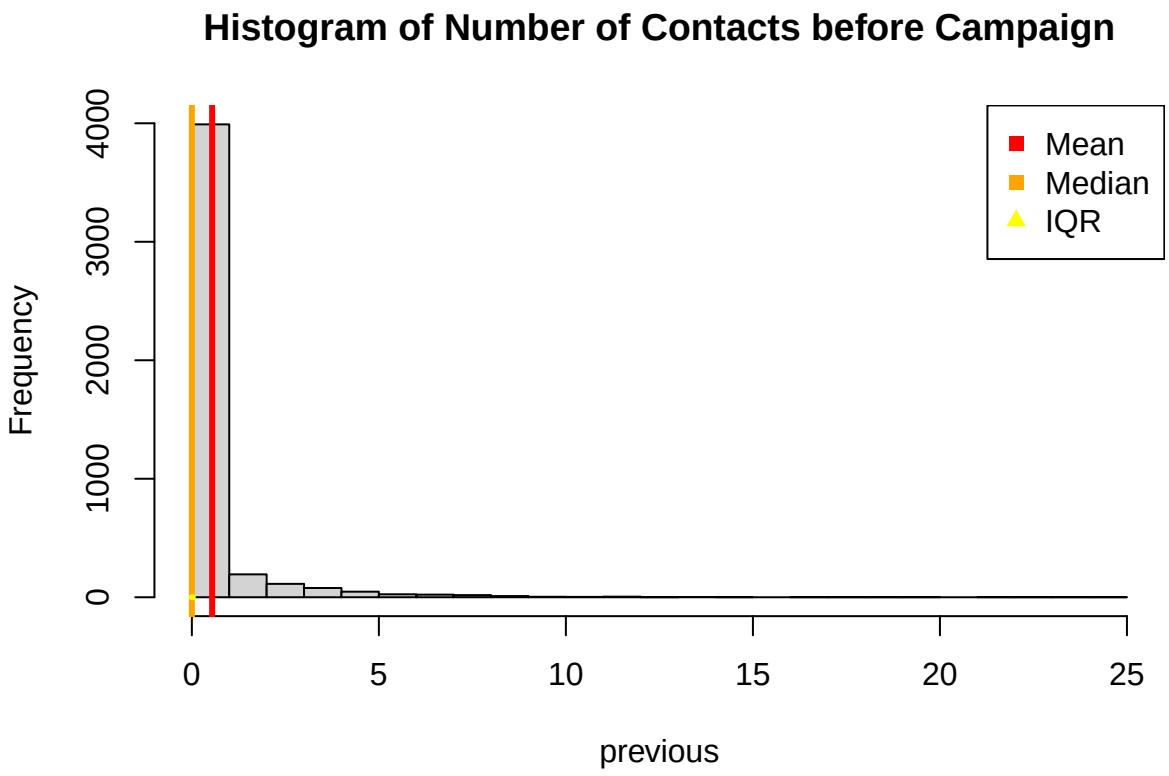
This box-and-whispers graph follows path of **duration** with *yes* being stretched out into a year timespan, while *no* is smashed into zero. An important remark is that value of -1 indicates, that the call made is the first one. We hope that next plots will be able to make it clear, why value -1 is present in most samples.

1.4.7 Number of Contacts before Campaign (previous)

```
boxplot_mean("previous")
title("Label X Number of Contacts before Campaign")
```




```
histogram_desc(
    "previous",
    main = "Histogram of Number of Contacts before Campaign",
    breaks = 30
)
```



This pattern is very similar to the one for **balance** and **pdays**. Here it seems old clients are more eager to subscribe to deposit, than new clients, bases on the chart above.

We can also see that most people, who were contacted during the campaign, had no previous ties with the bank, thus **pdays** number of days since last contact is mostly -1.

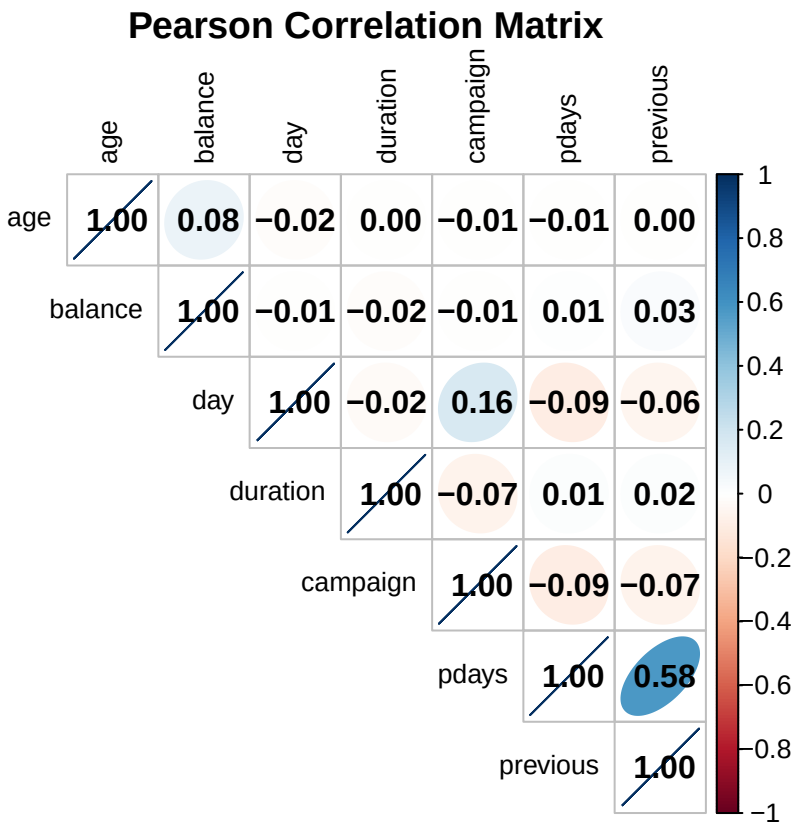
1.5 Feature interactions

We will check bivariate statistics such as correlation coefficient and plot box-plots. We hope to catch connections that are present in data, that omit direct target variable impact.

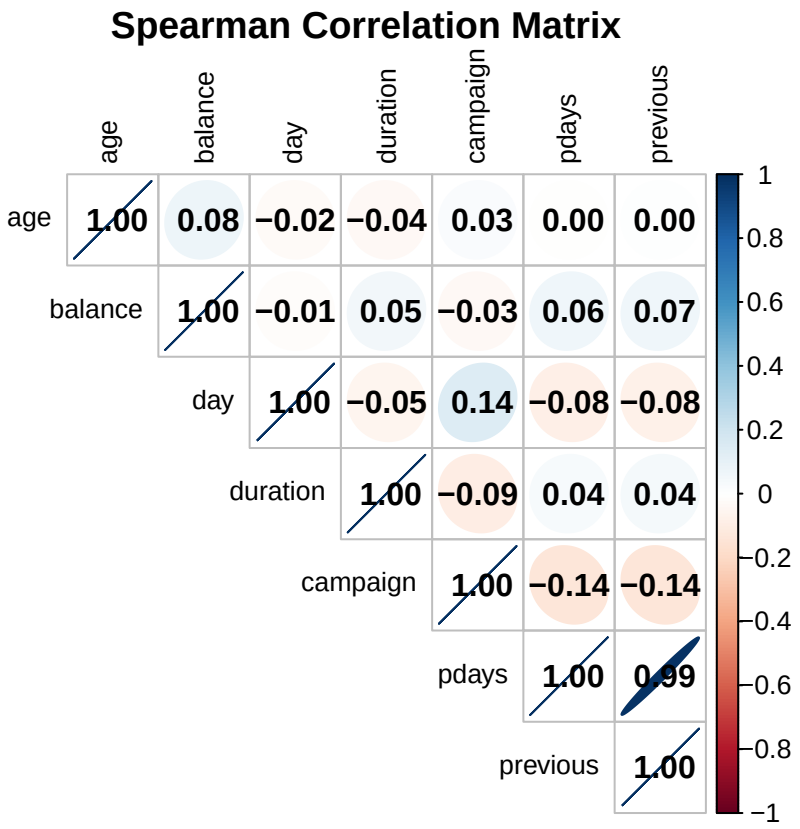
1.5.1 Numerical to Numerical

We look at interactions between features. We will examine Pearson's and Spearman's correlation between every pair of numerical features.

```
numeric_data <- data[sapply(data, is.numeric)]

cor_matrix <- cor(numeric_data, use = "pairwise.complete.obs", method="pearson")
corrplot(
    cor_matrix, method = "ellipse", type = "upper",  tl.col = "black", tl.cex = 0.8, addCoef.col = "bla
    title = "Pearson Correlation Matrix", mar = c(1, 1, 1, 1)
)
```




```r
cor_matrix <- cor(numeric_data, use = "pairwise.complete.obs", method="spearman")
corrplot(
    cor_matrix, method = "ellipse", type = "upper",  tl.col = "black", tl.cex = 0.8, addCoef.col = "bla
    title = "Spearman Correlation Matrix", mar = c(1, 1, 1, 1)
)
```

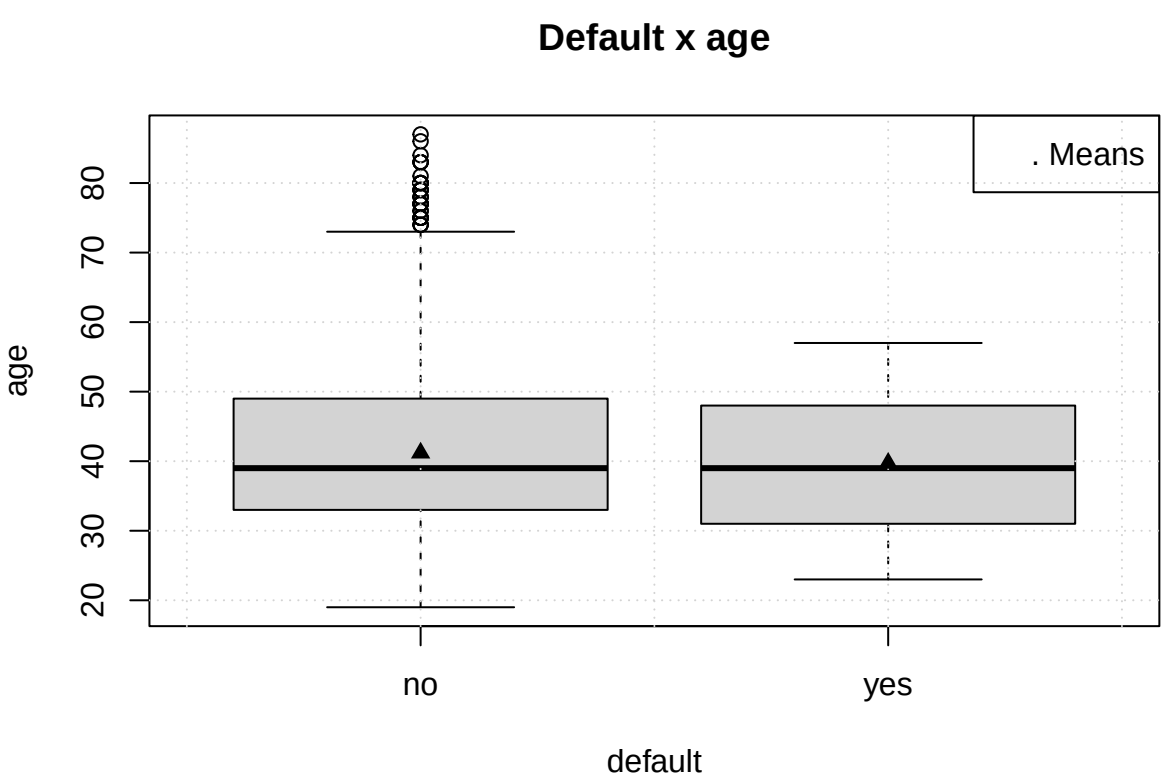


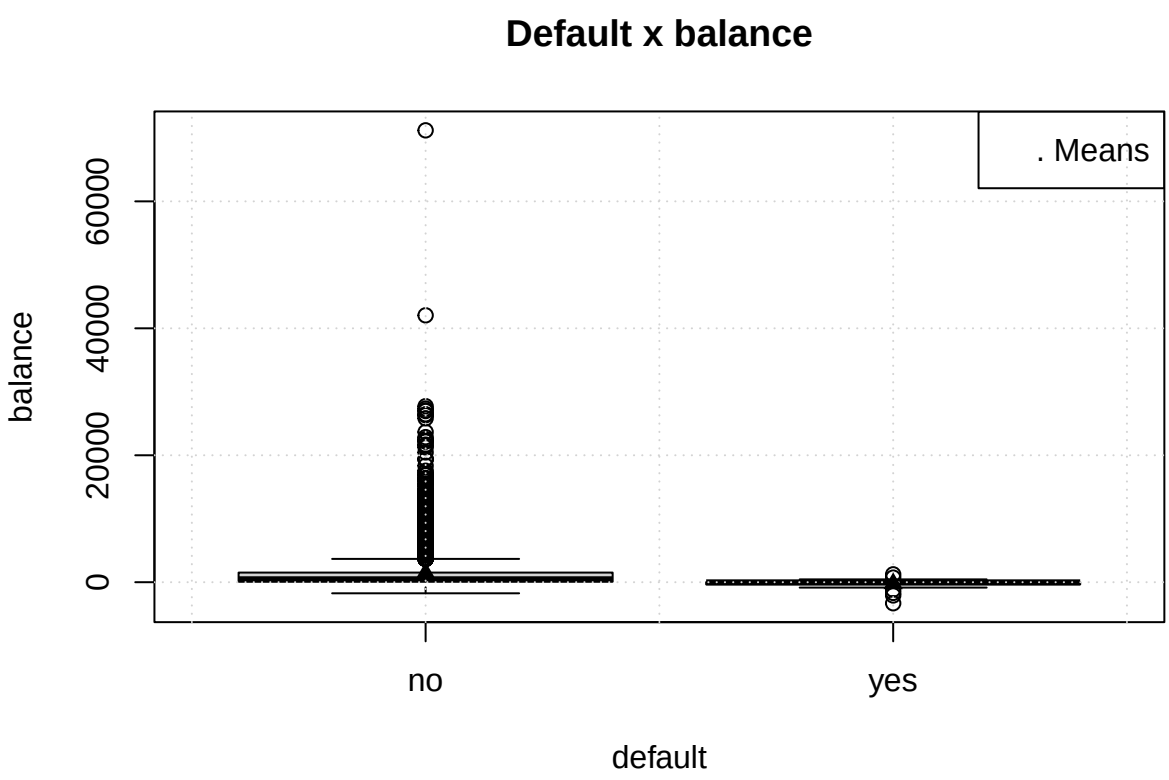
Most features are uncorrelated, except **pdays** and **previous** that have Pearson's $r = 0.58$ and Spearman's $r = 0.98$ coeficients. Other notable, but relatively weak Pearson's correlations are **age-balance**, **day-duration**, **day-pdays**, **campaign-pdays**, **duration-campaign**, **day-previous**, **campaign-previous**. Spearman's correlation shows values of absolute value near 0.05 for most pairs, it is not clear whether it is significant, tests later should show.

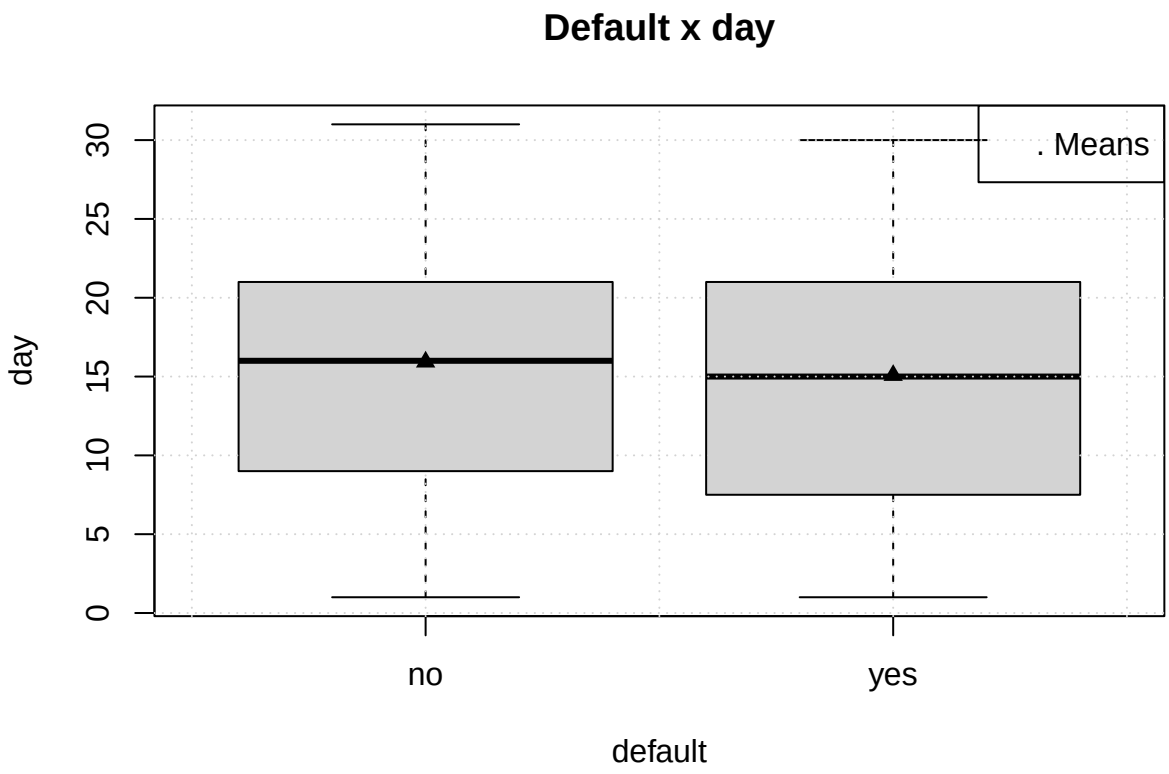
1.5.2 Categorical to Numerical

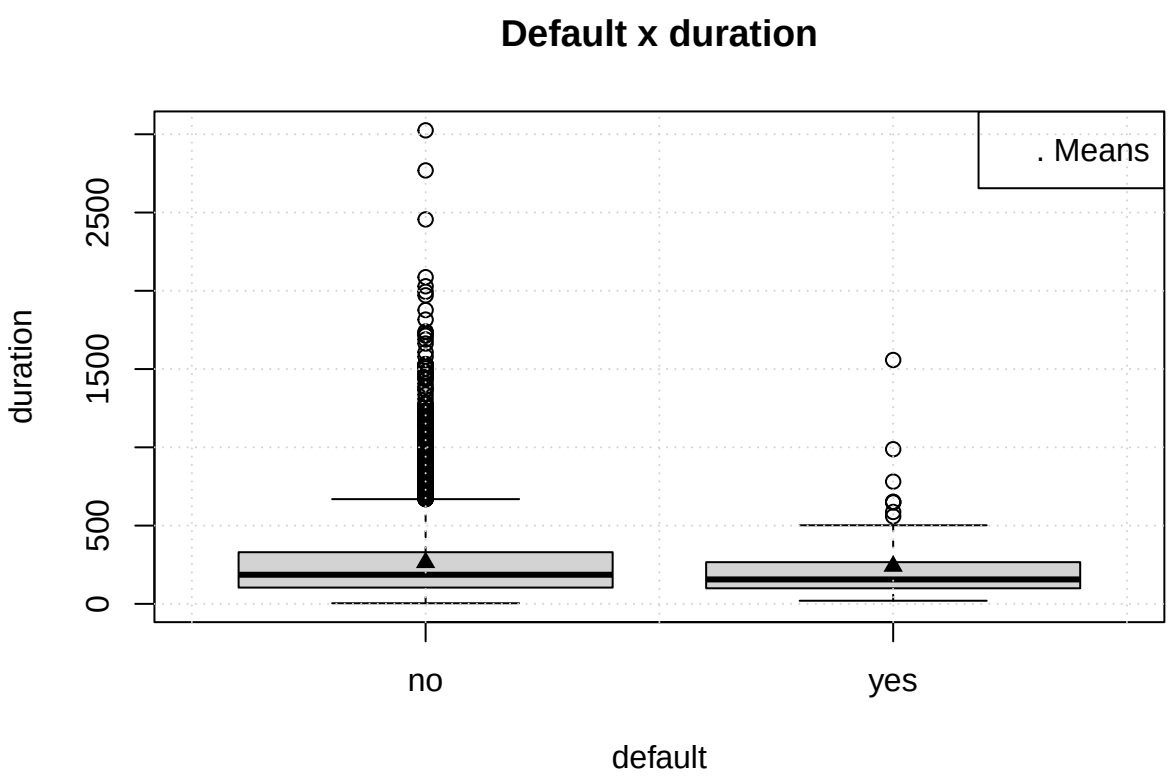
We will plot boxplots between numerical and categorical features. Due to a big number of vizualizations we will focus on peculiarities in data, with lesser comments on patterns.

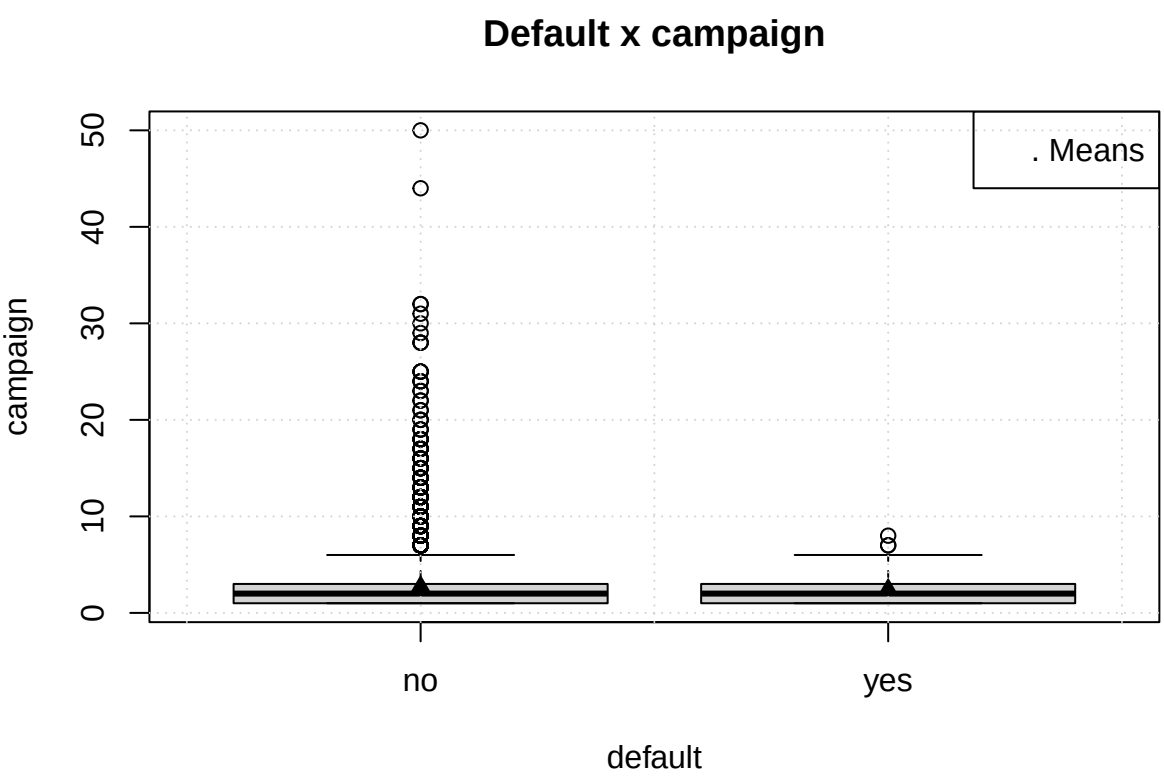
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "default")
    title(sprintf("Default x %s", col))
}
```

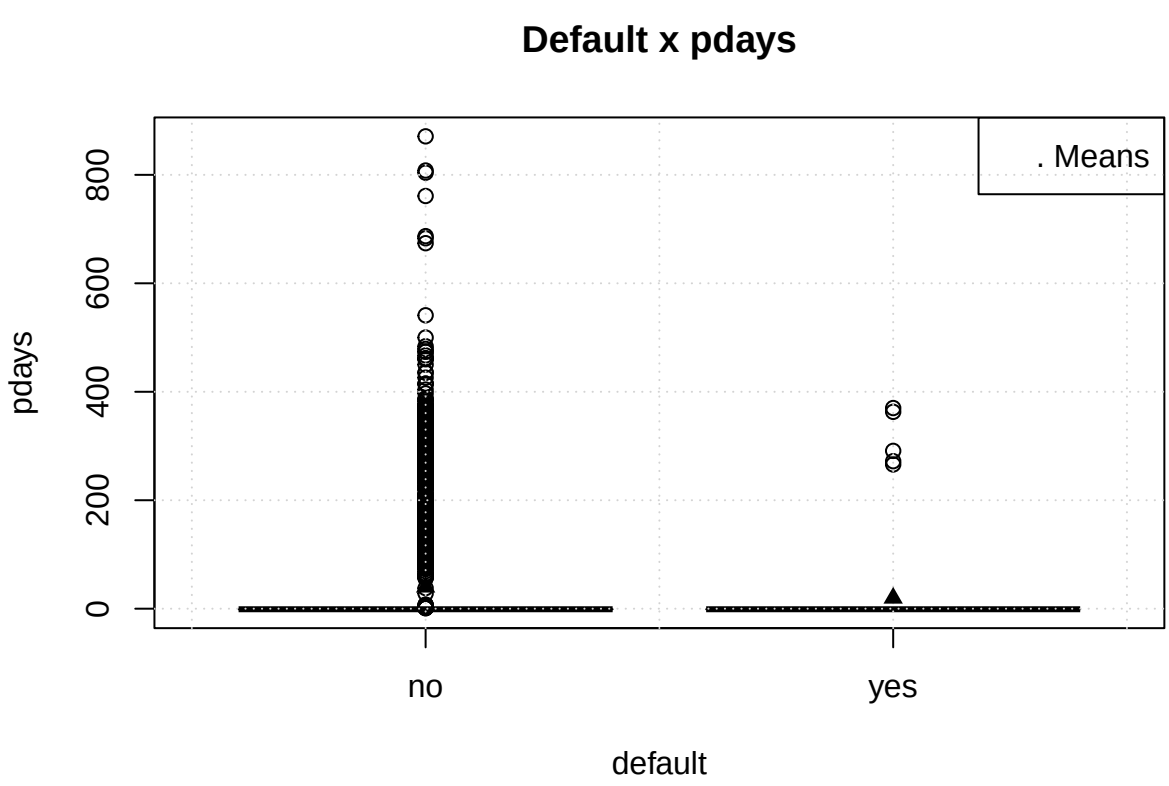

Default x age
. Means
age
20
30
40
50
60
70
80
no
yes
default

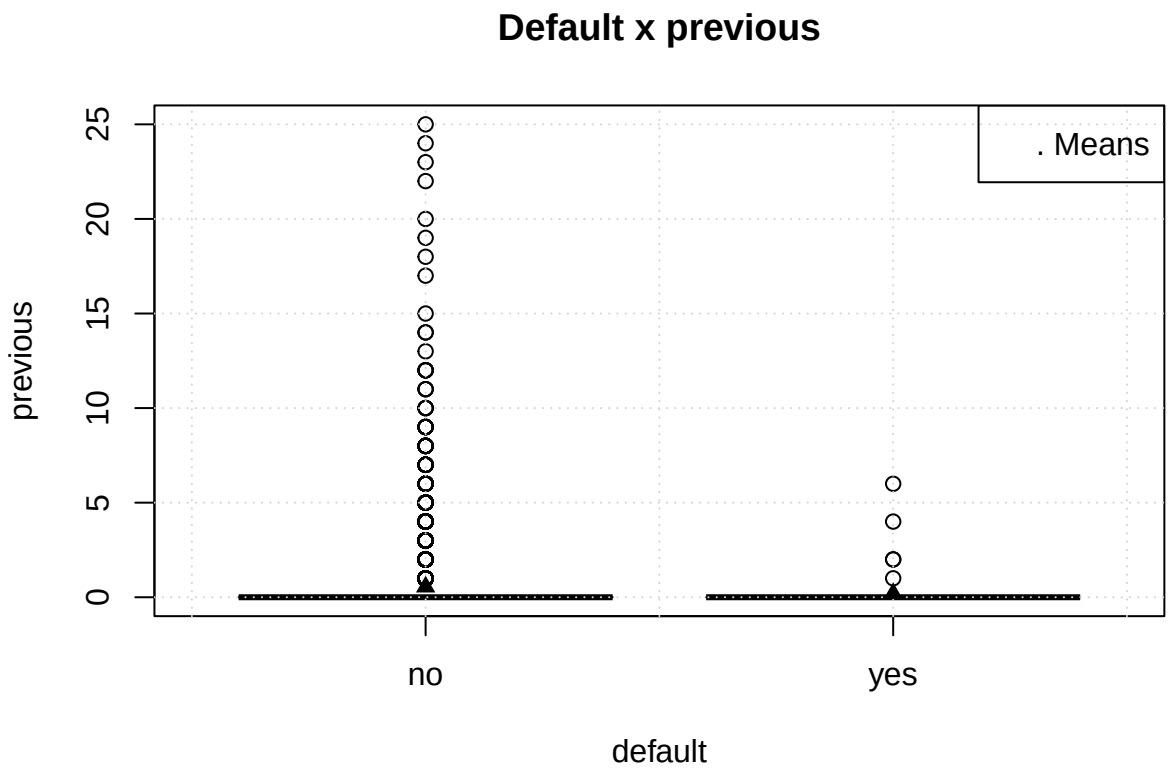

Default x balance
. Means
balance
0
20000
40000
60000
no
yes
default


Default x day
. Means
day
0
5
10
15
20
25
30
no
yes
default


Default x duration
. Means
duration
0
500
1500
2500
no
yes
default

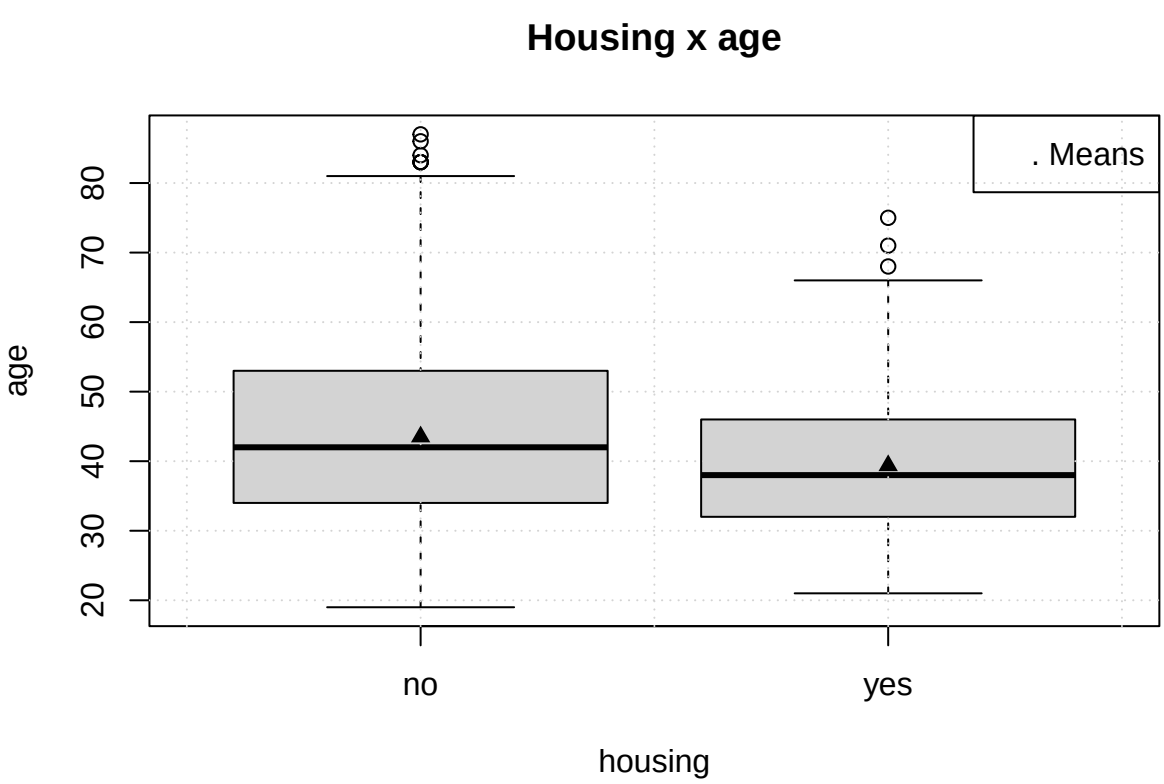

Default x campaign
. Means
campaign
0
10
20
30
40
50
no
yes
default

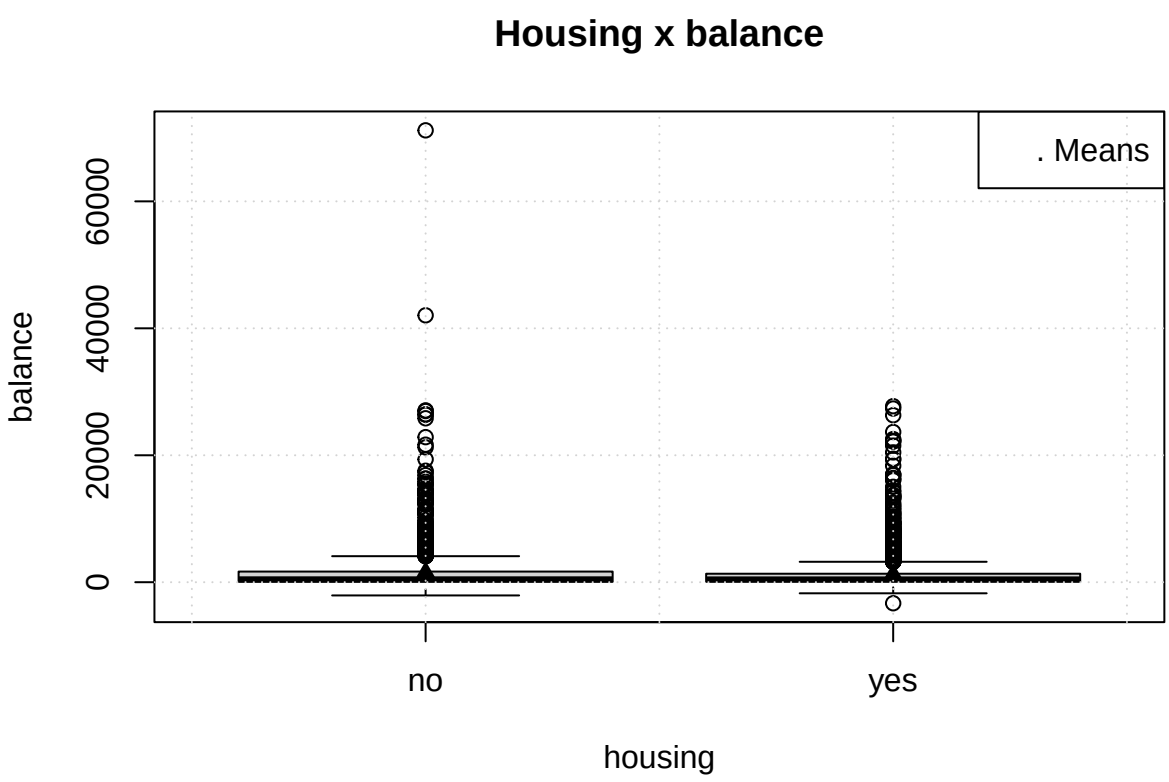

Default x pdays
. Means
pdays
0
200
400
600
800
no
yes
default

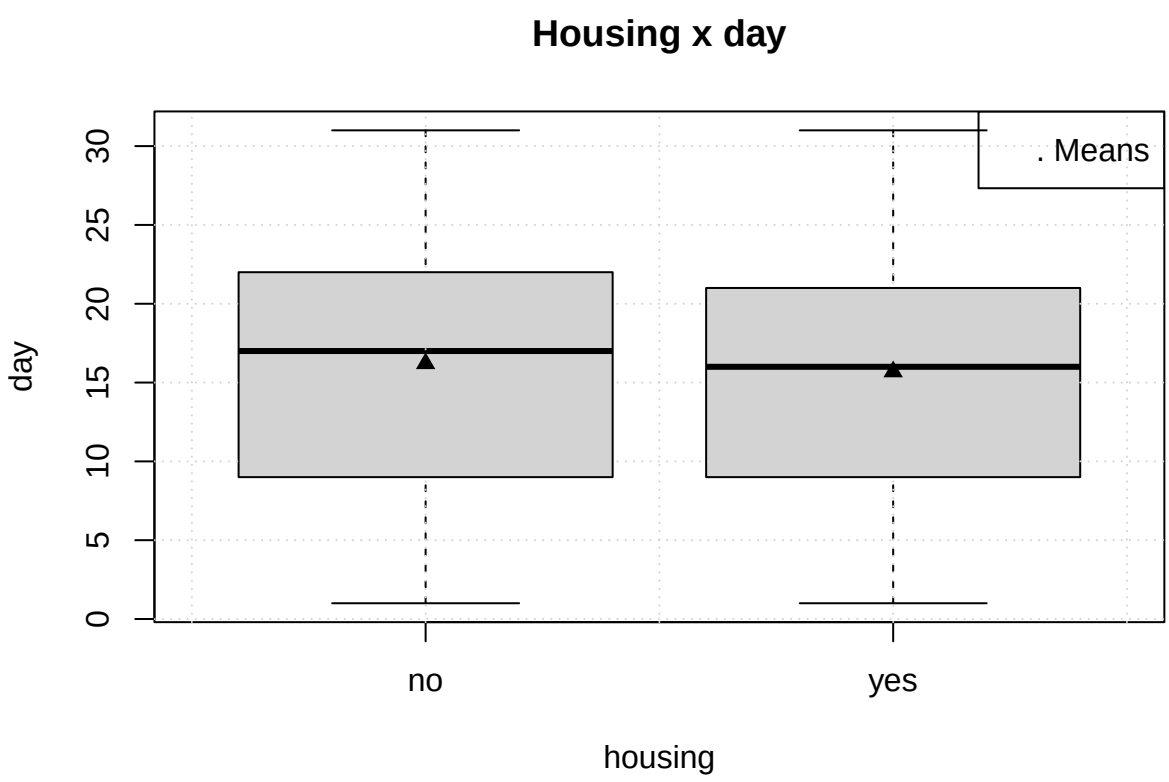


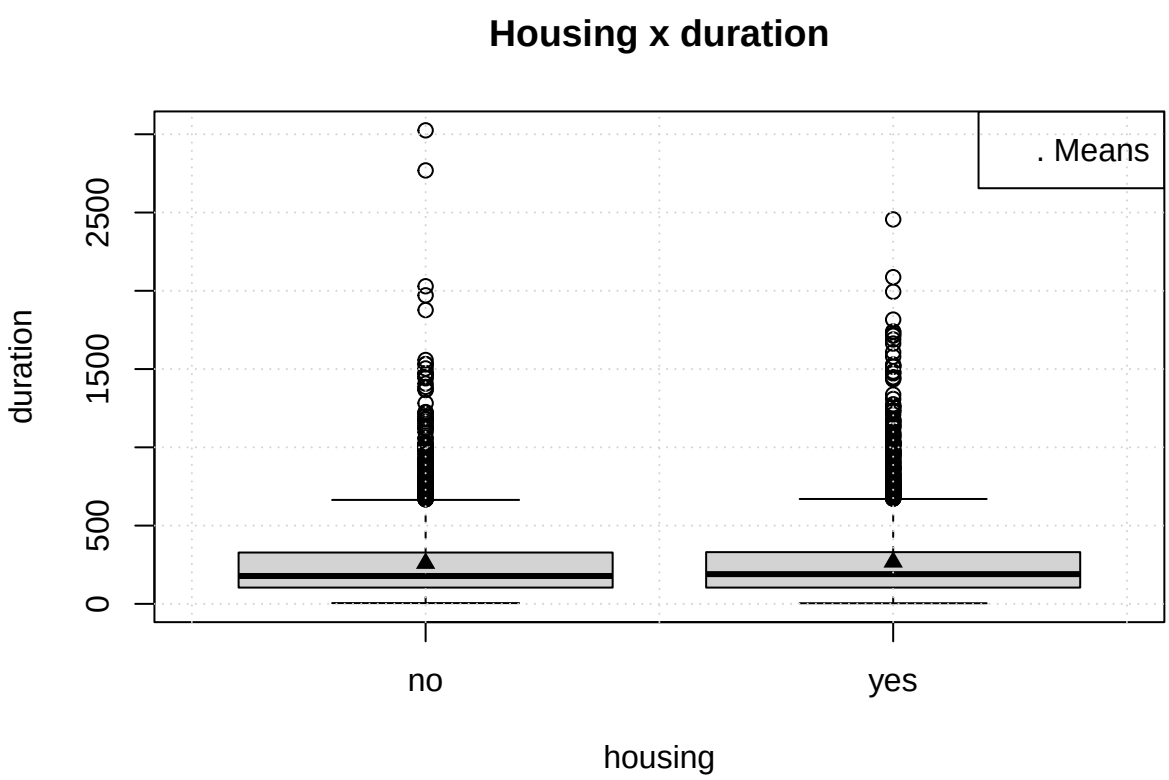
Most boxplots show little to no distinction between **default** classes, but an interesting observation is that calls of **default**-positive population are slightly shorter.

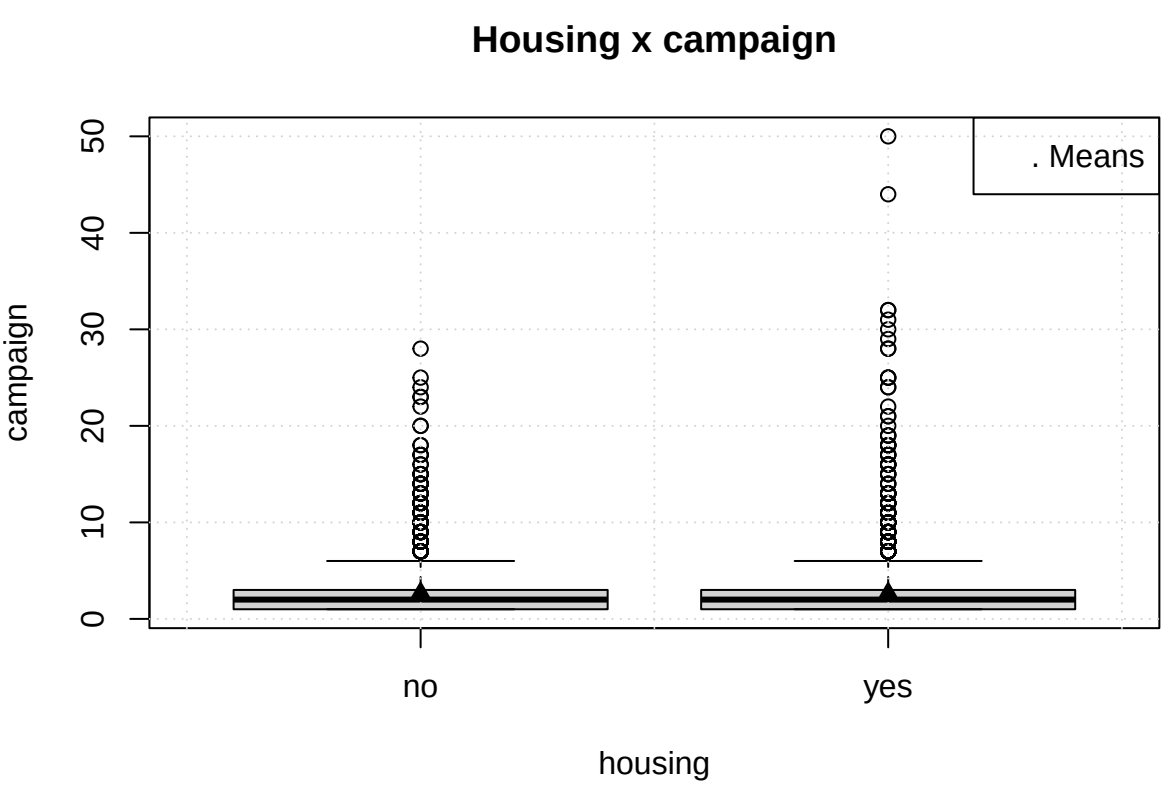
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "housing")
    title(sprintf("Housing x %s", col))
}
```

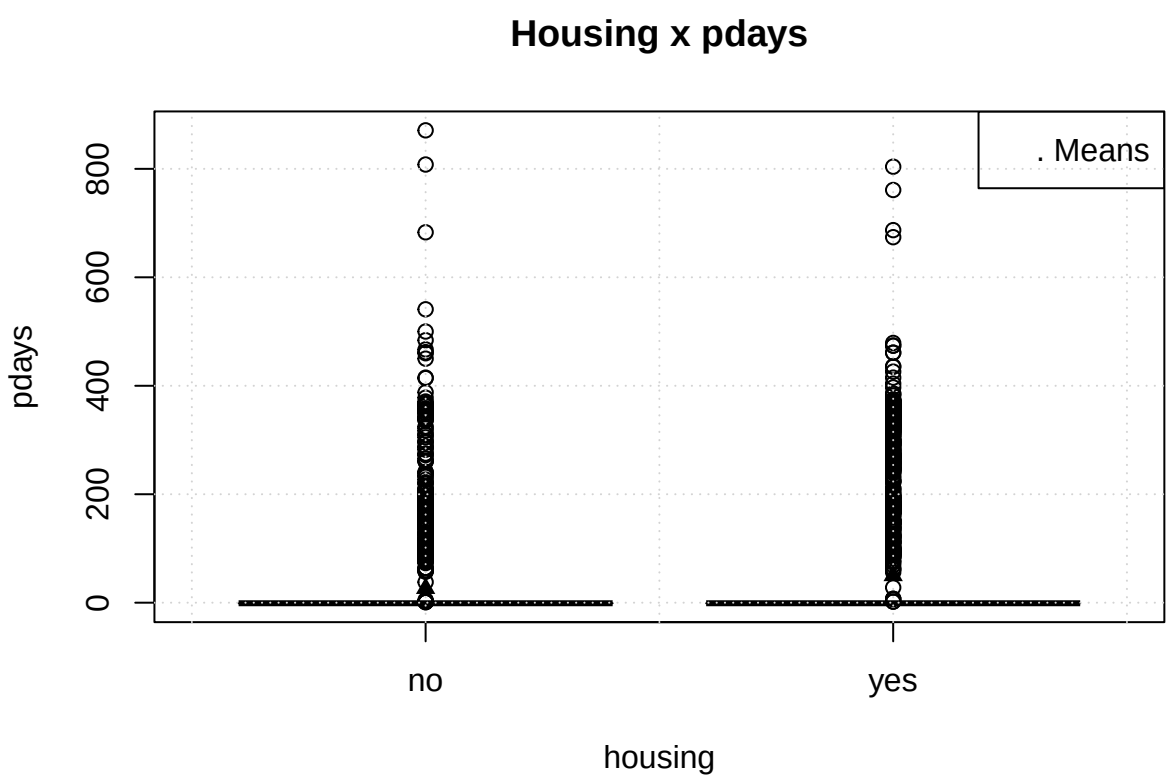

Housing x age
. Means
age
20
30
40
50
60
70
80
no
yes
housing

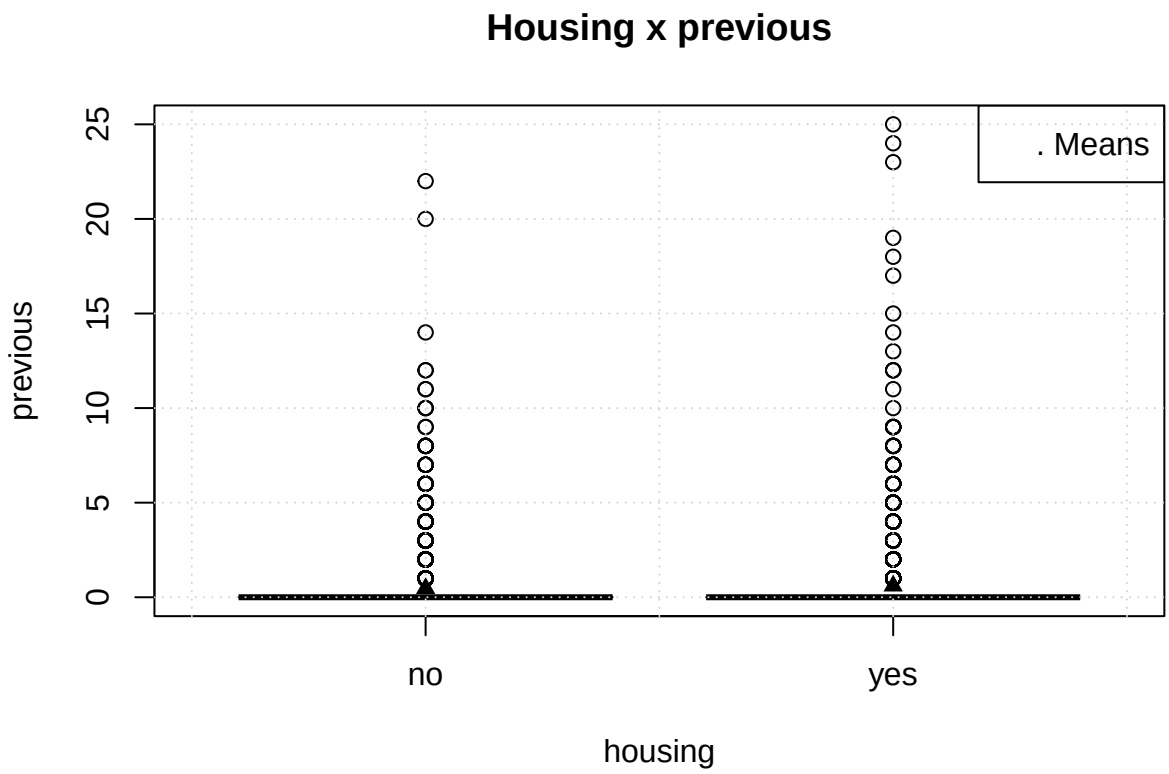

Housing x balance
. Means
balance
0
20000
40000
60000
no
yes
housing


Housing x day
. Means
day
0
5
10
15
20
25
30
no
yes
housing


Housing x duration
. Means
duration
0
500
1500
2500
no
yes
housing

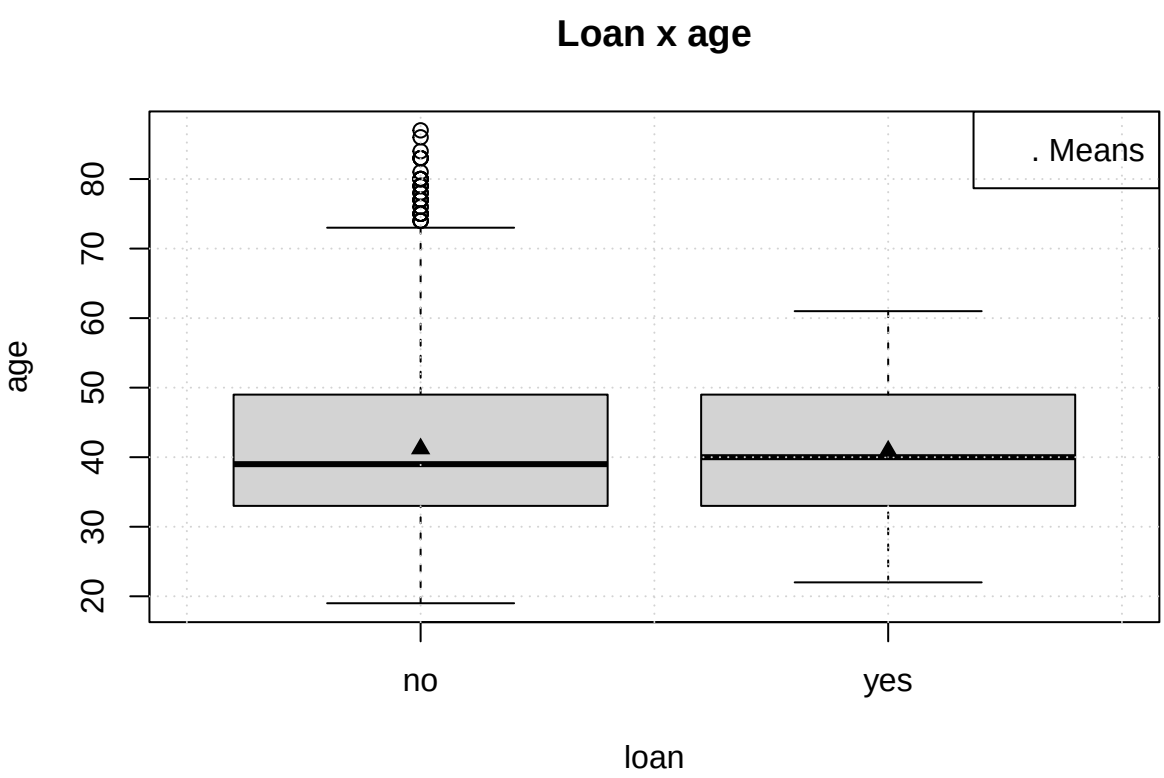

Housing x campaign
. Means
campaign
0
10
20
30
40
50
no
yes
housing

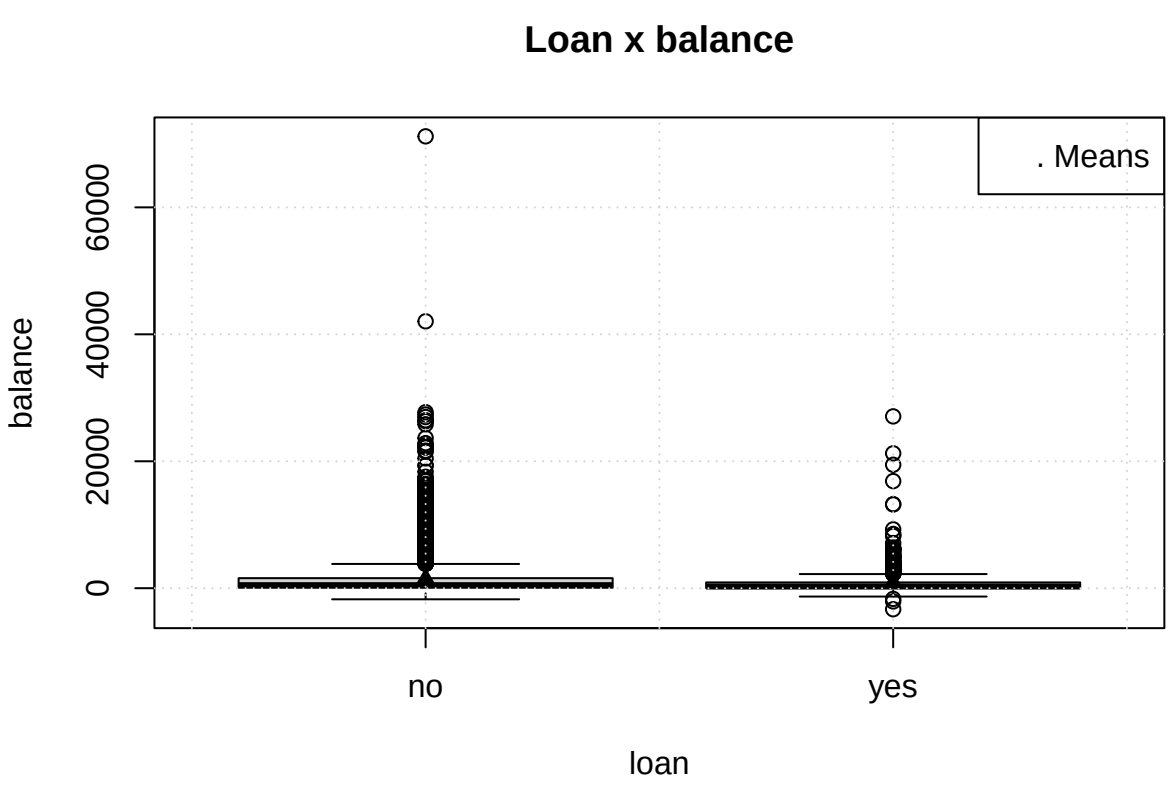

Housing x pdays
. Means
pdays
0
200
400
600
800
no
yes
housing

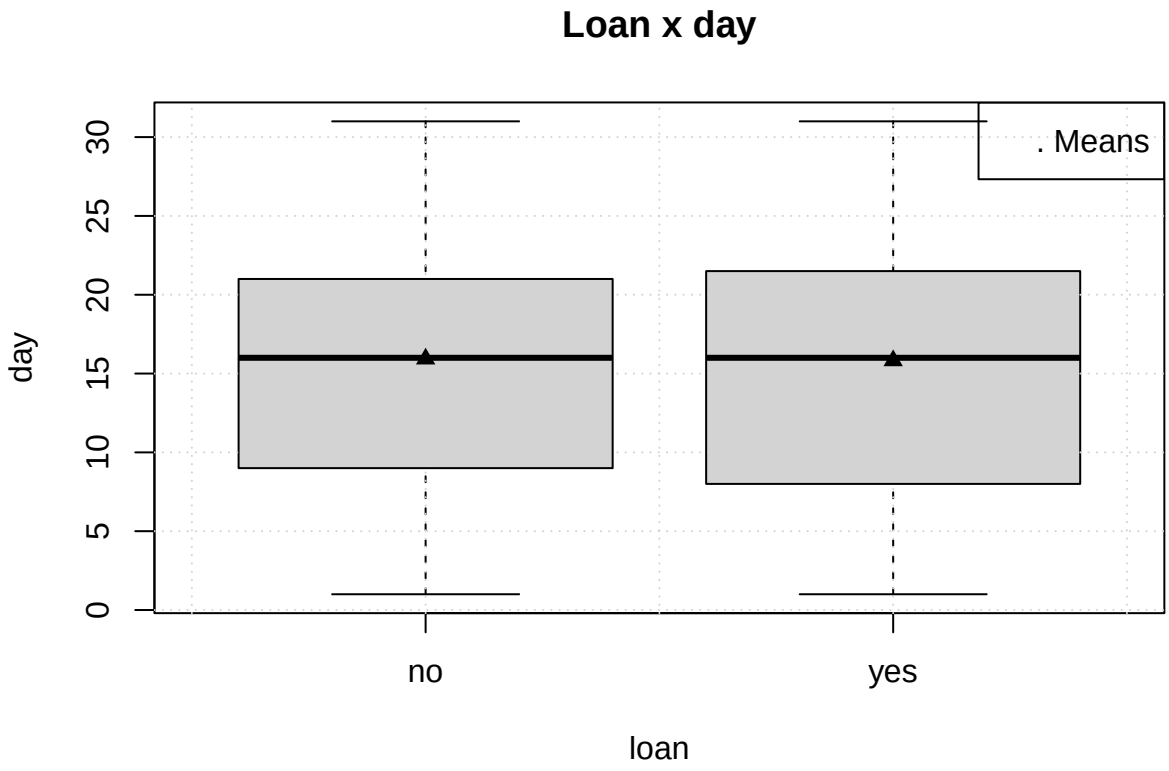


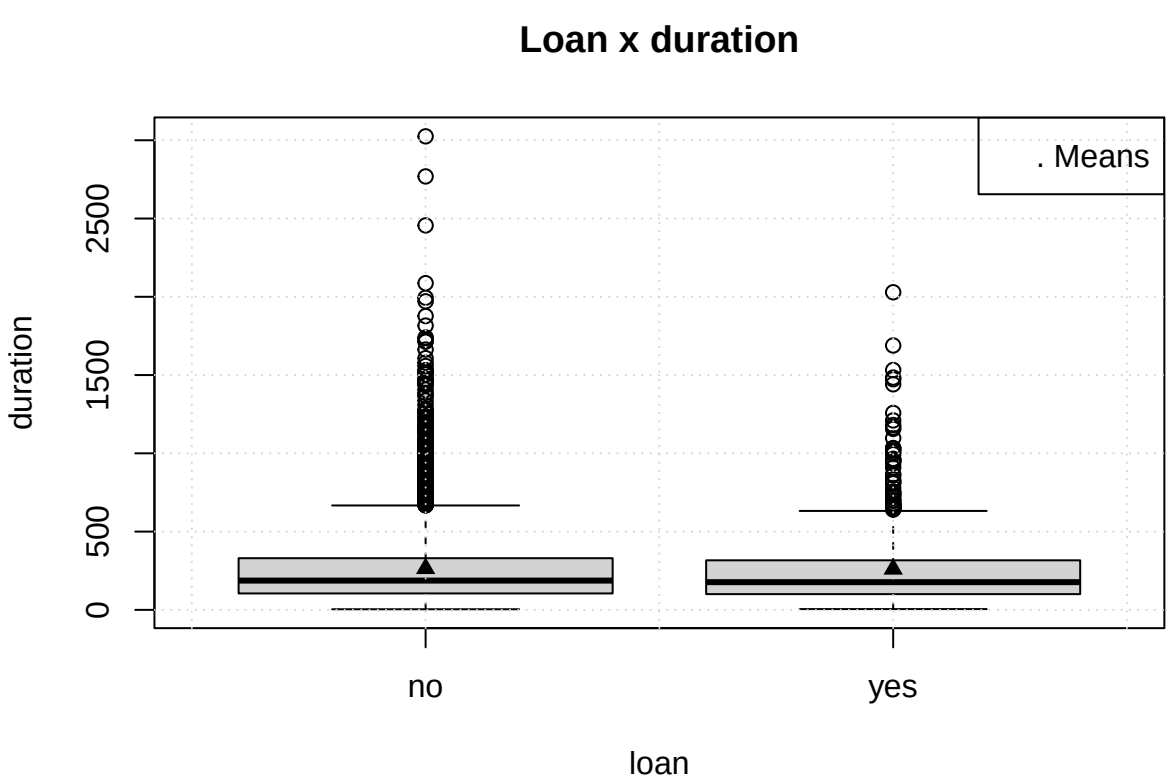
Plots follow whole dataset dynamic, an interesting deviation is that a median and an average people, who have housing loan, are younger, than their counterpart, who does not have the loan.

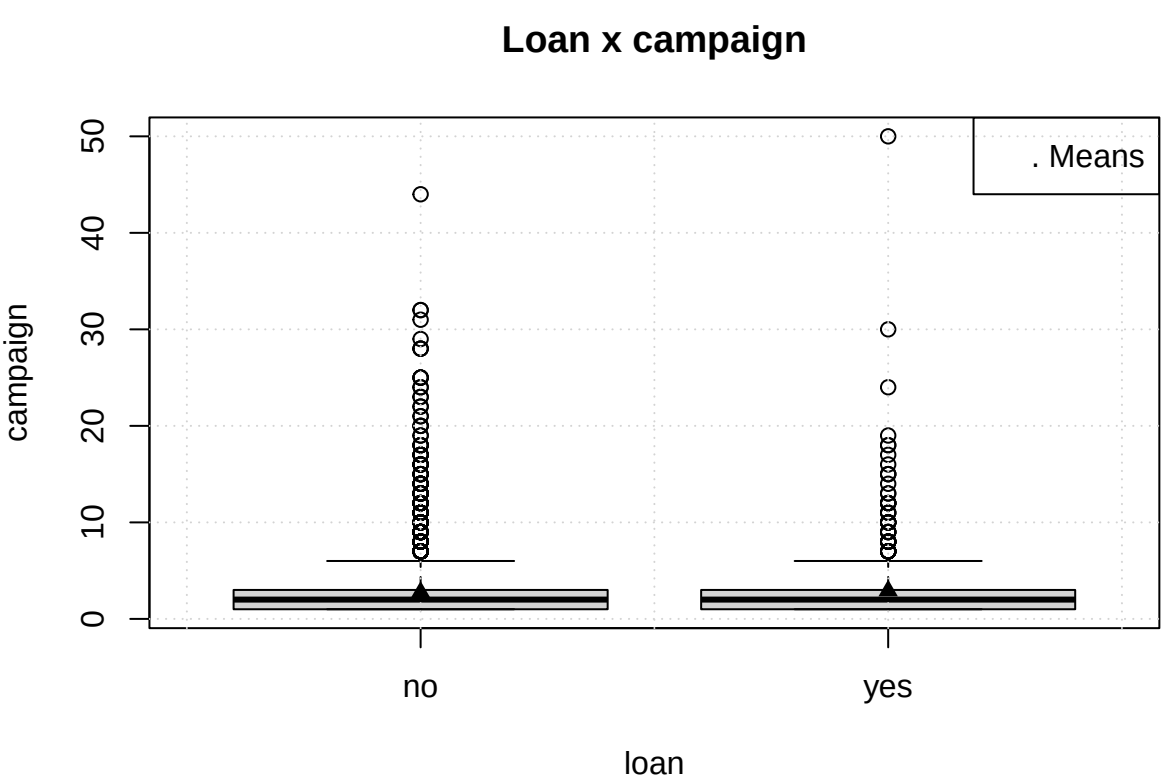
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "loan")
    title(sprintf("Loan x %s", col))
}
```

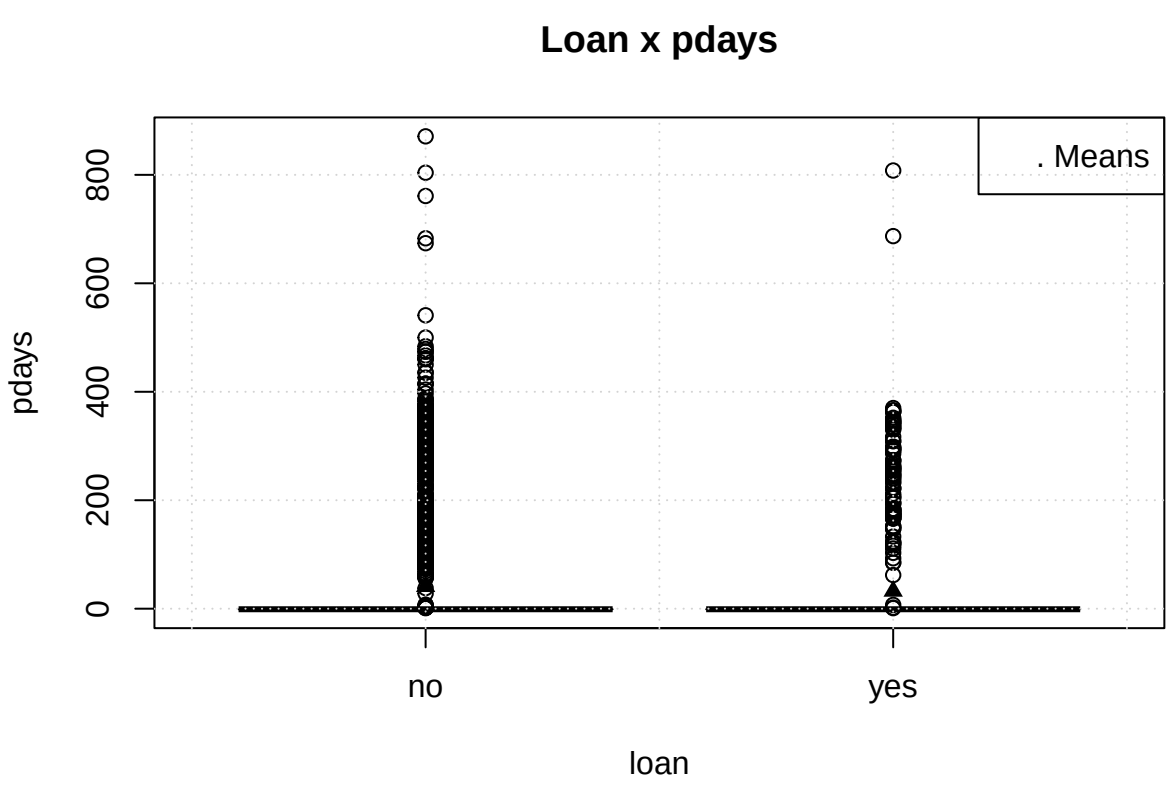

Loan x age
. Means
age
20
30
40
50
60
70
80
no
yes
loan

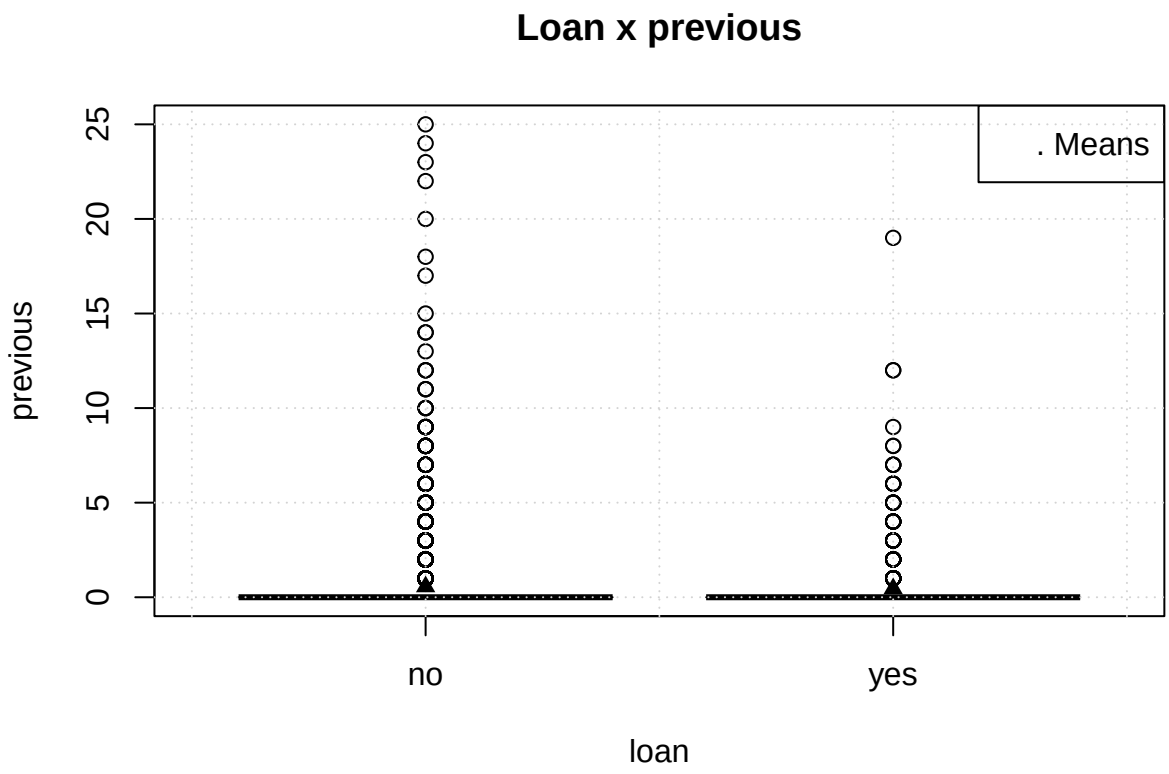

Loan x balance
. Means
balance
0
20000
40000
60000
no
yes
loan


Loan x day
. Means
day
0
5
10
15
20
25
30
no
yes
loan


Loan x duration
. Means
duration
0
500
1500
2500
no
yes
loan

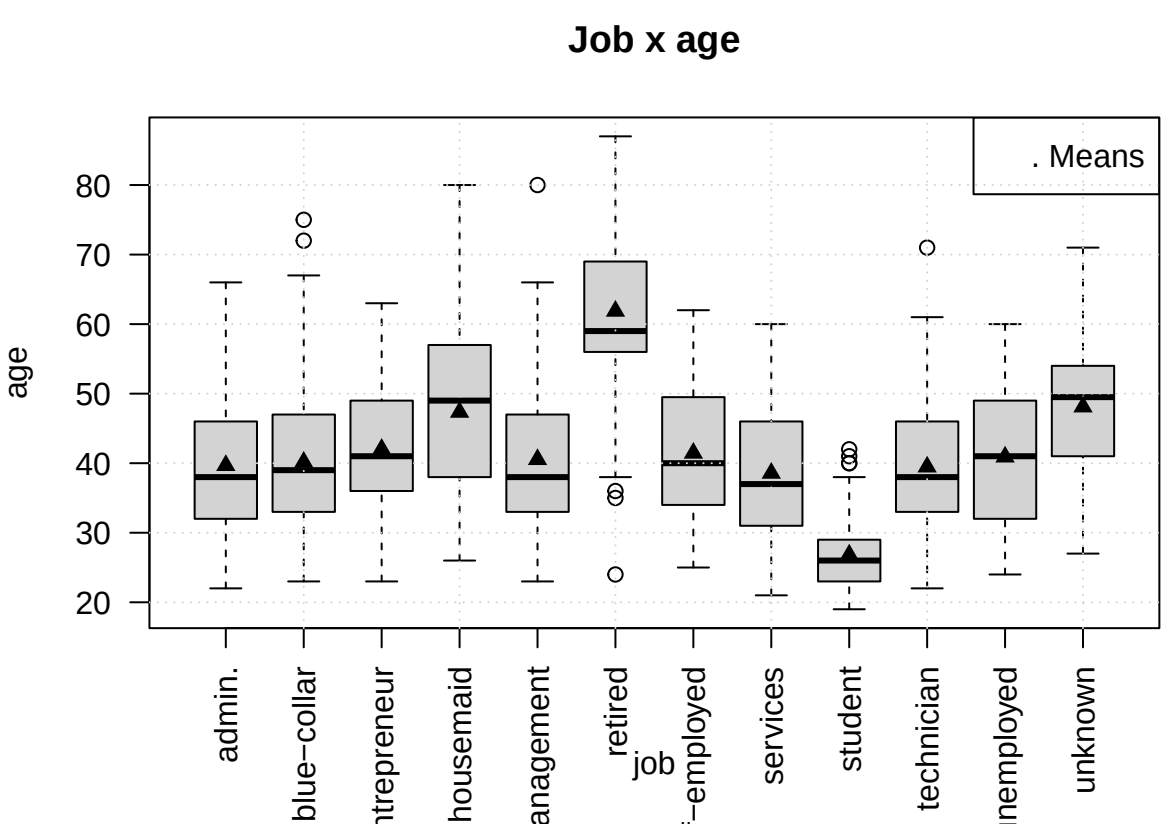

Loan x campaign
. Means
campaign
0
10
20
30
40
50
no
yes
loan

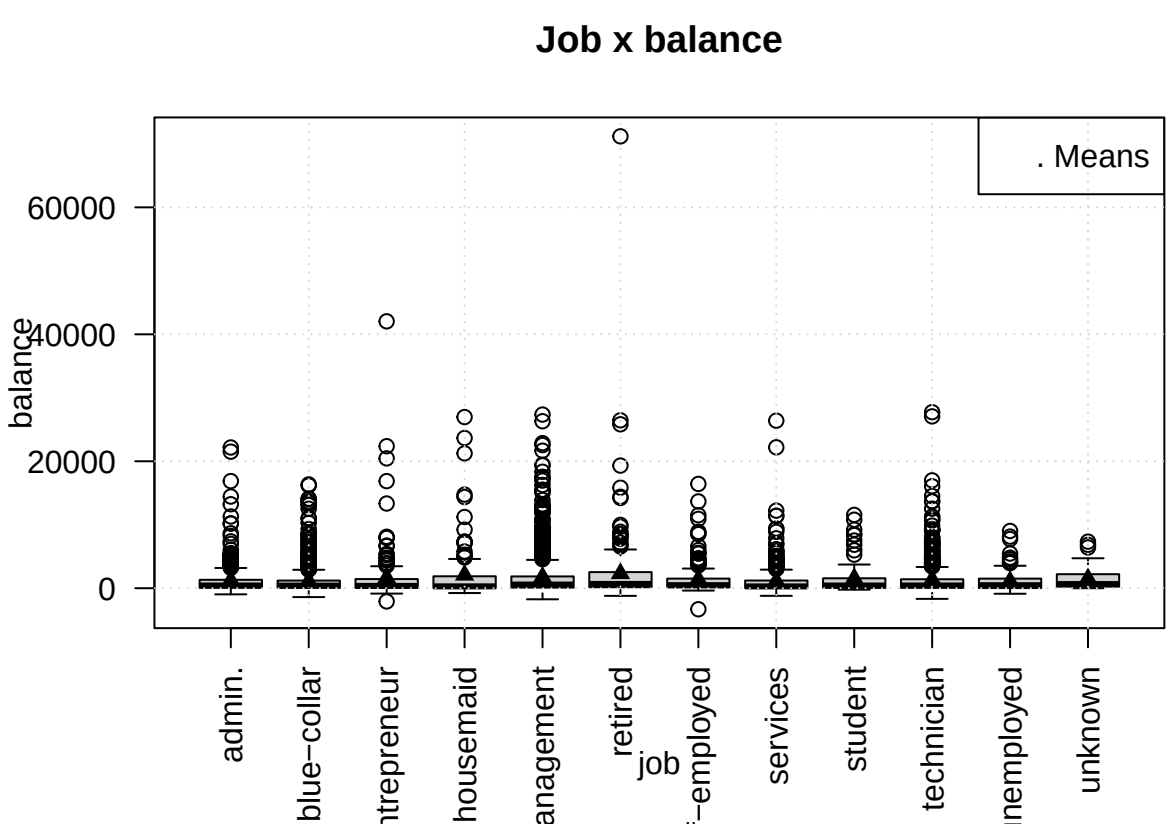

Loan x pdays
. Means
pdays
0
200
400
600
800
no
yes
loan

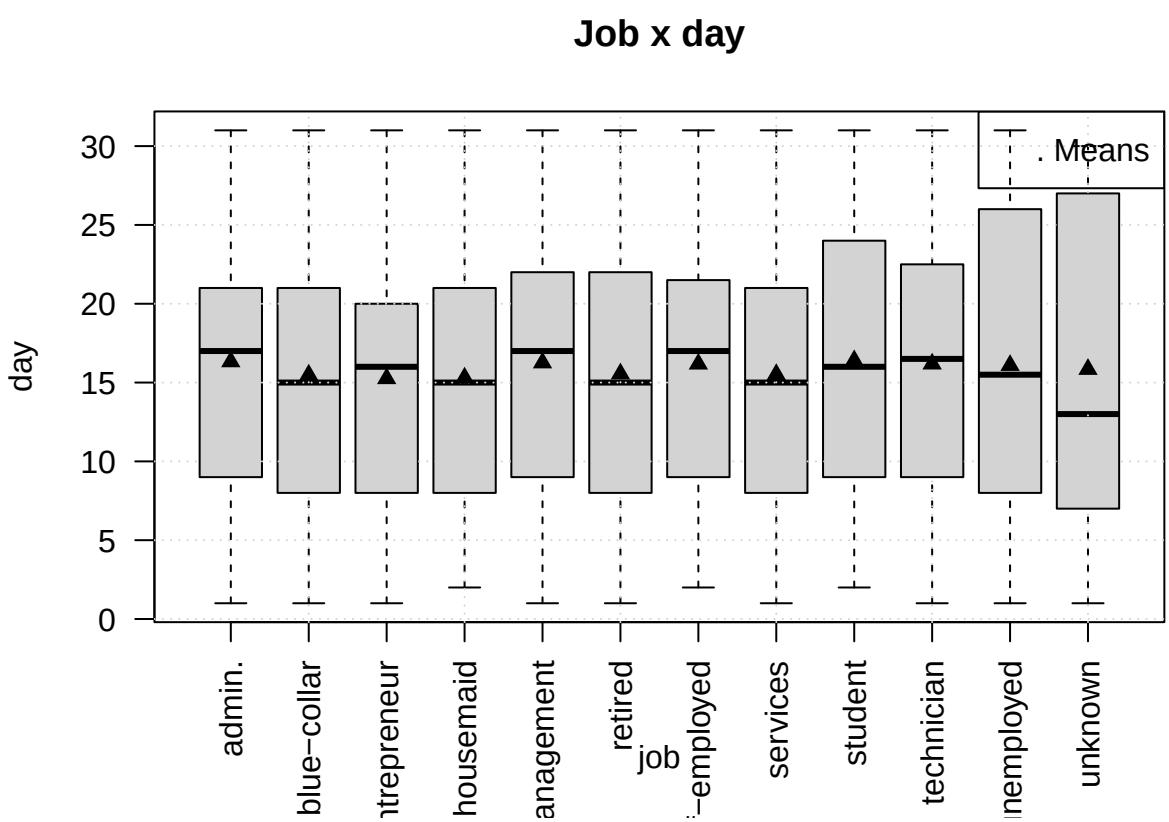


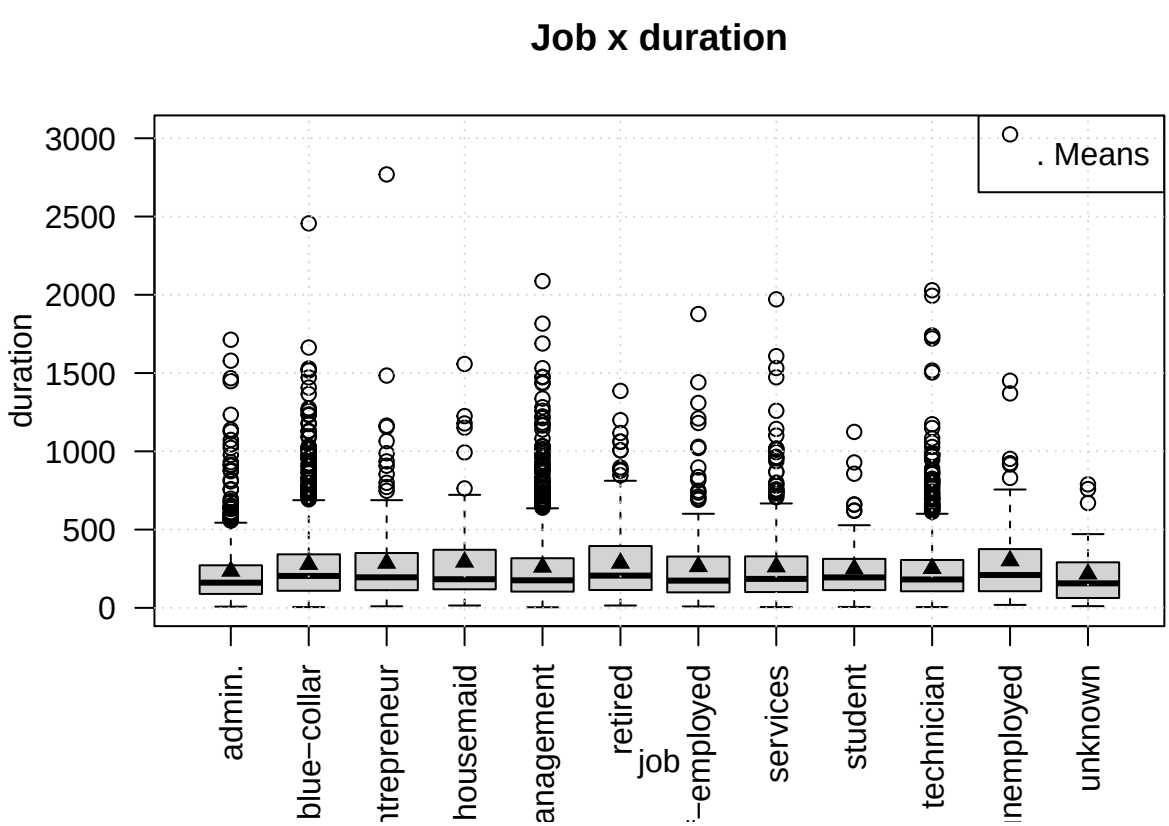
Plots have no new information, one of reasons for that is personal loans are taken by most strats of population and with differing relationship with the bank.

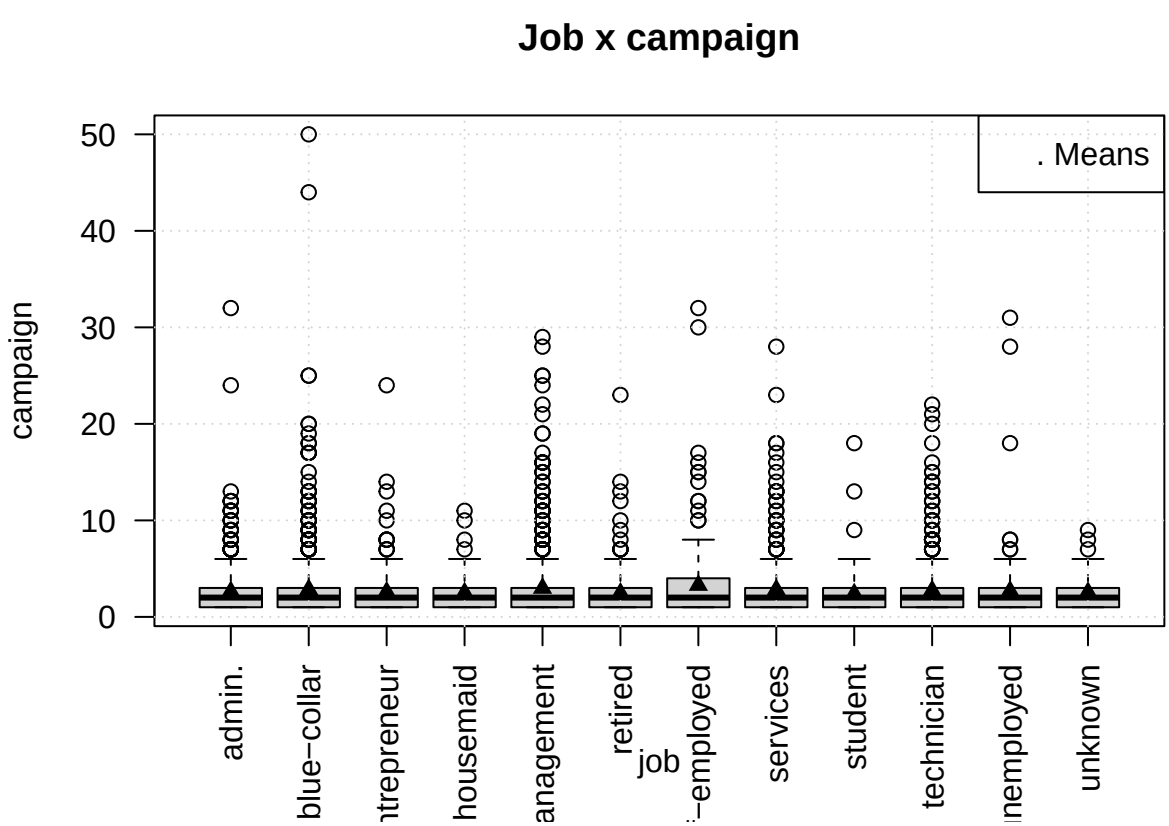
```
n_jobs = length(unique(data$job))
for (col in colnames(numeric_data)) {
    formula = as.formula(paste(col, "~", "job"))
    boxplot(formula, data, las=2)
    grid()
    means <- aggregate(formula, data, FUN=mean)[,col]
    points(
        1:n_jobs, means, pch=17
    )
    legend("topright", legend="  Means")
    title(sprintf("Job x %s", col))
}
```


Job x age
. Means
age
80
70
60
50
40
30
20
admin.
blue−collar
housemaid
retired
services
student
technician
unknown
job

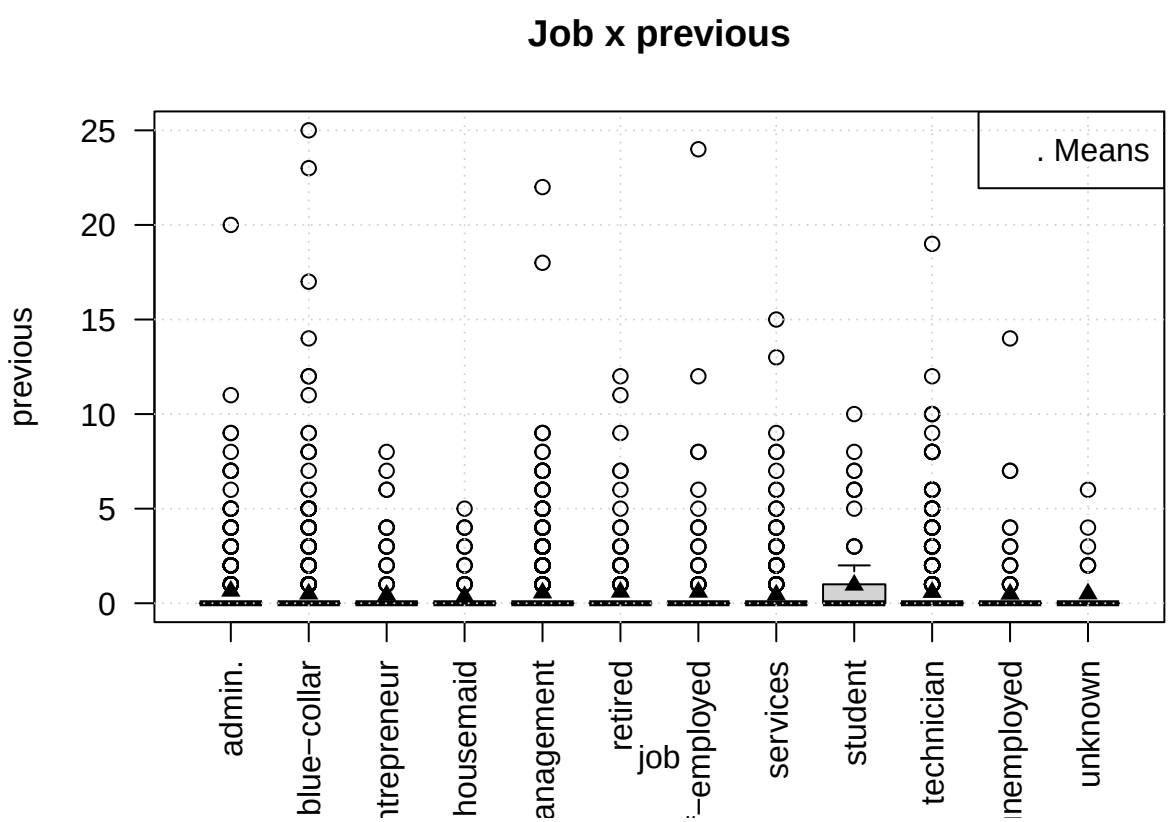

Job x balance
. Means
60000
40000
20000
0
balance
admin.
blue−collar
ntrepreneur
housemaid
anagement
retired
−employed
services
student
technician
nemployed
unknown
job


Job x day
day
30
25
20
15
10
5
0
. Means
admin.
blue−collar
housemaid
retired
services
student
technician
unknown
job


Job x duration
duration
3000
2500
2000
1500
1000
500
0
. Means
admin.
blue−collar
housemaid
retired
services
student
technician
unknown
job

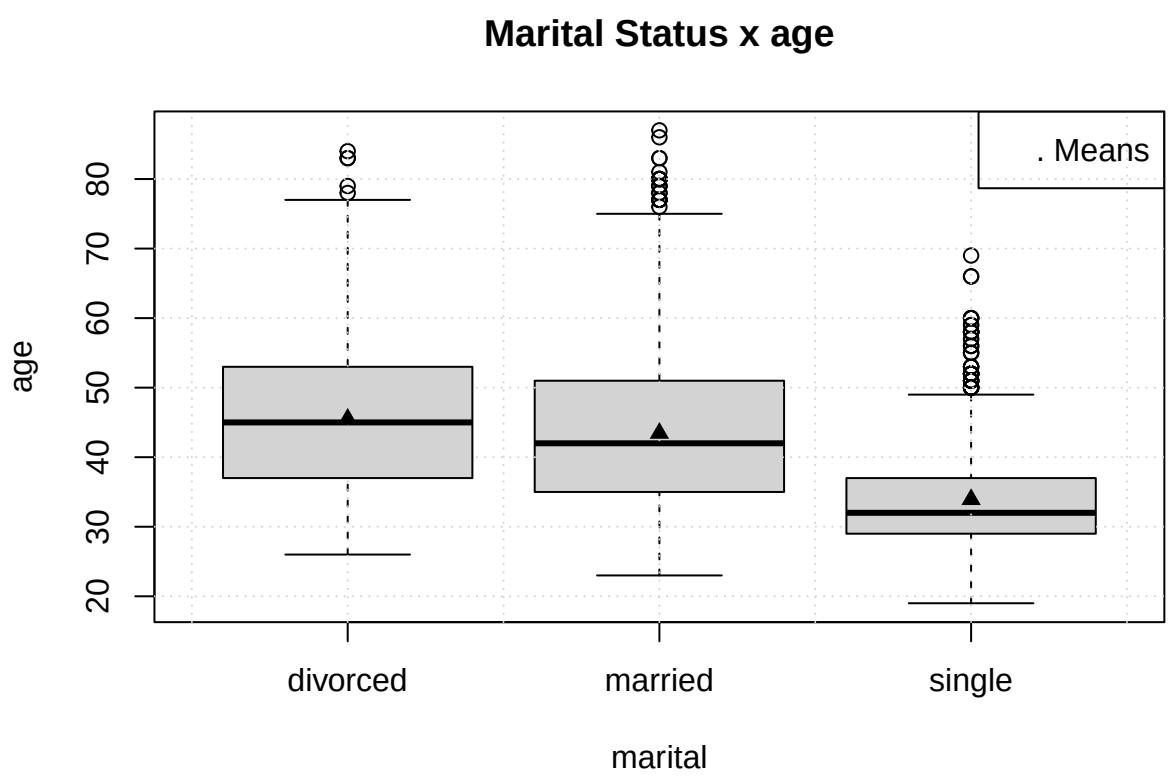

Job x campaign
. Means
campaign
50
40
30
20
10
0
admin.
blue−collar
housemaid
retired
services
student
technician
unknown
job

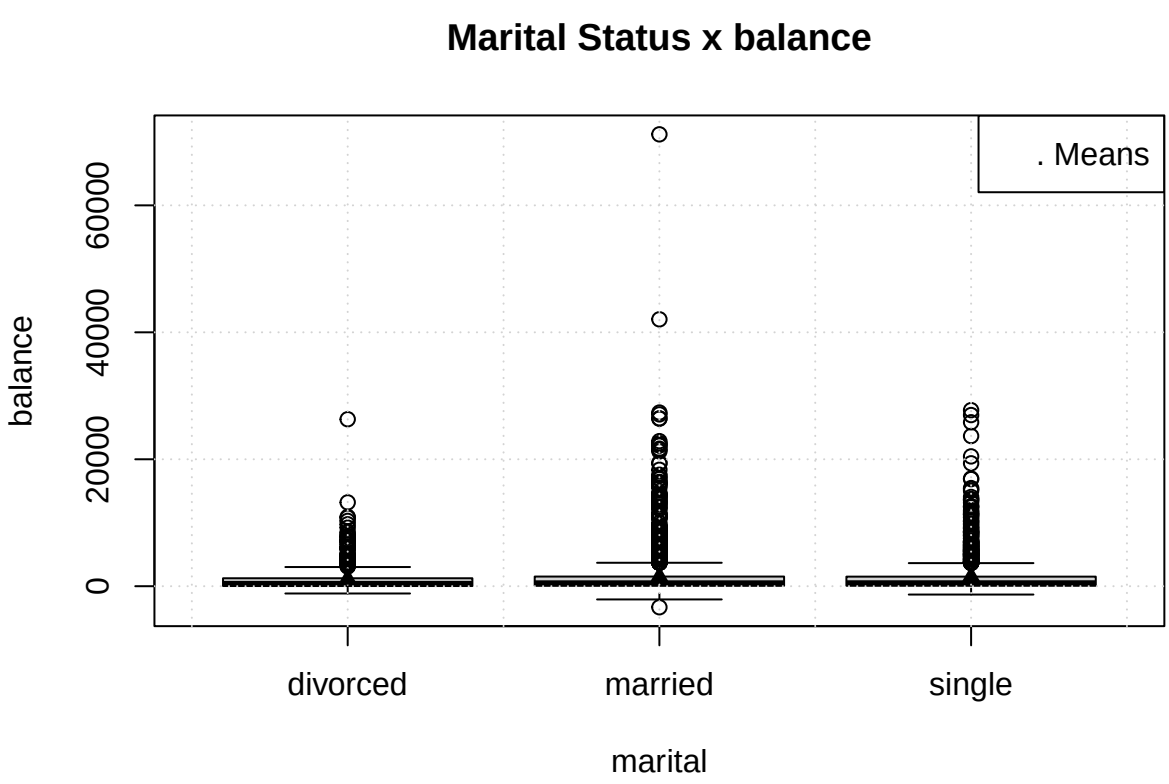

Job x pdays
pdays
0
200
400
600
800
. Means
admin.
blue−collar
housemaid
retired
services
student
technician
unknown
job

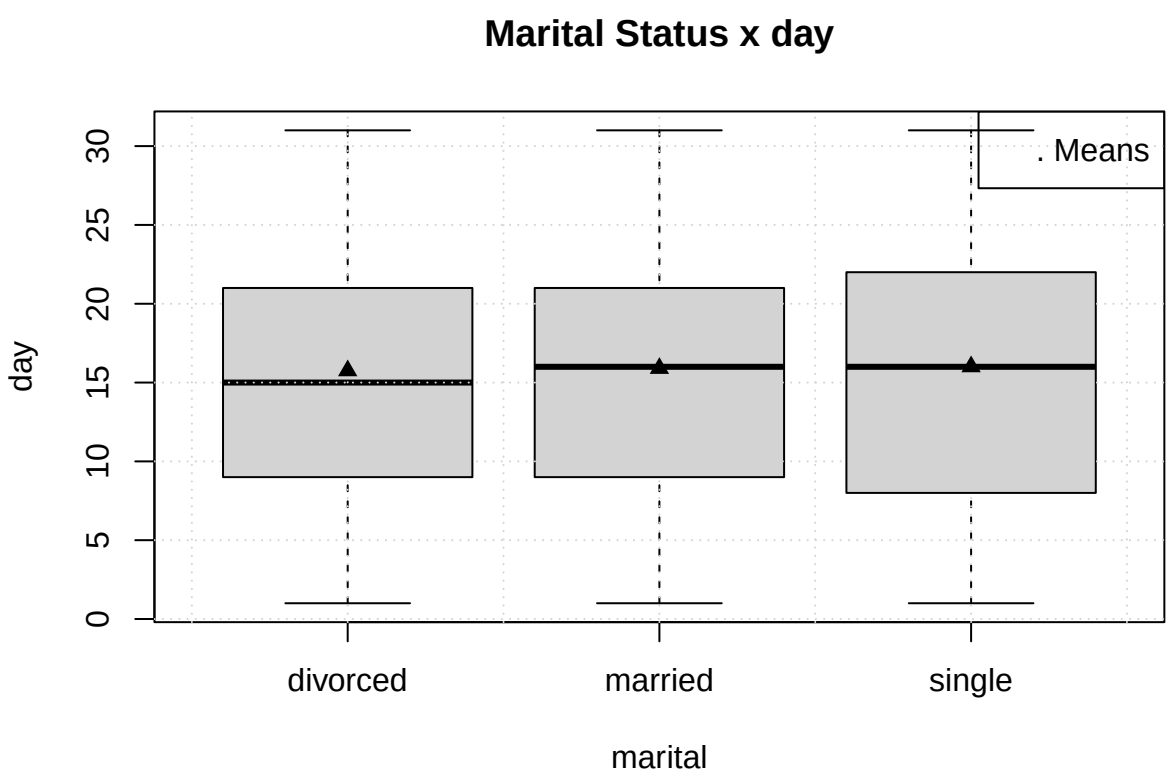


The only point of interest in plots above are higher-than-others values for **previous** in *student* job group. This maybe another data engineering bias, because it is common to use students for data collection campaigns and they might have also been a part of other research or have ties to the bank, nevertheless it is just a hypothesis, that can be checked later.

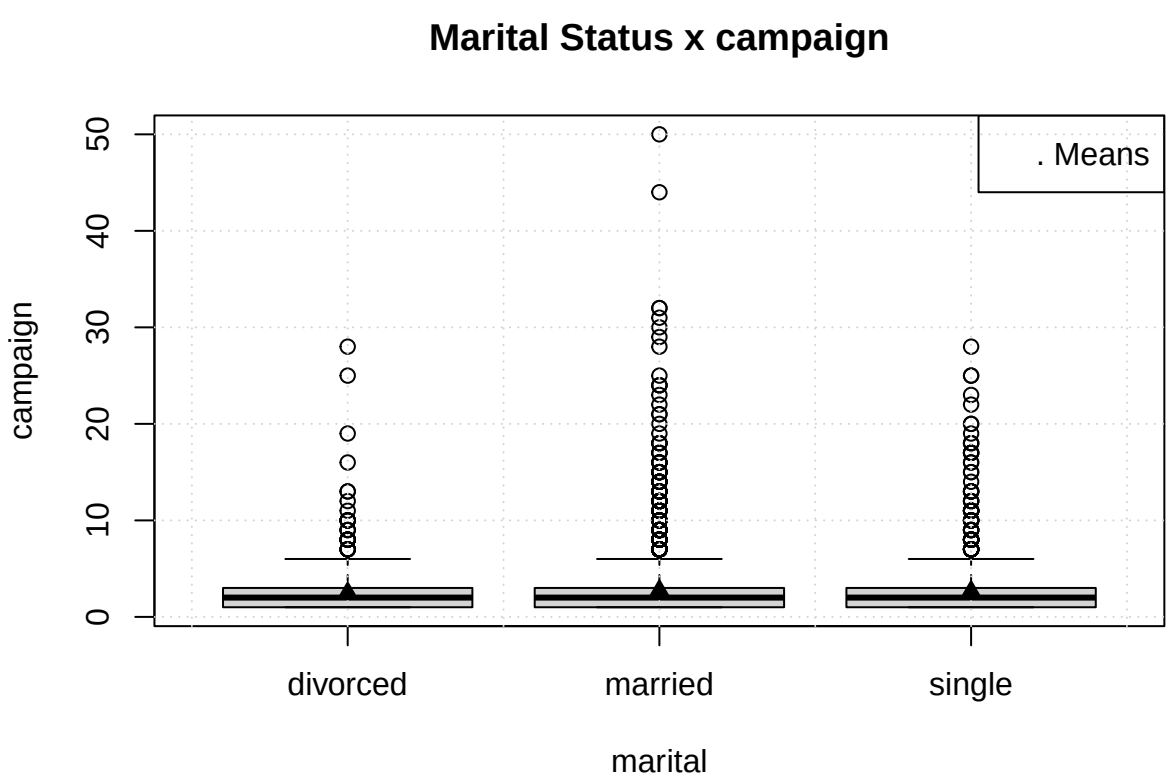
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "marital")
    title(sprintf("Marital Status x %s", col))
}
```

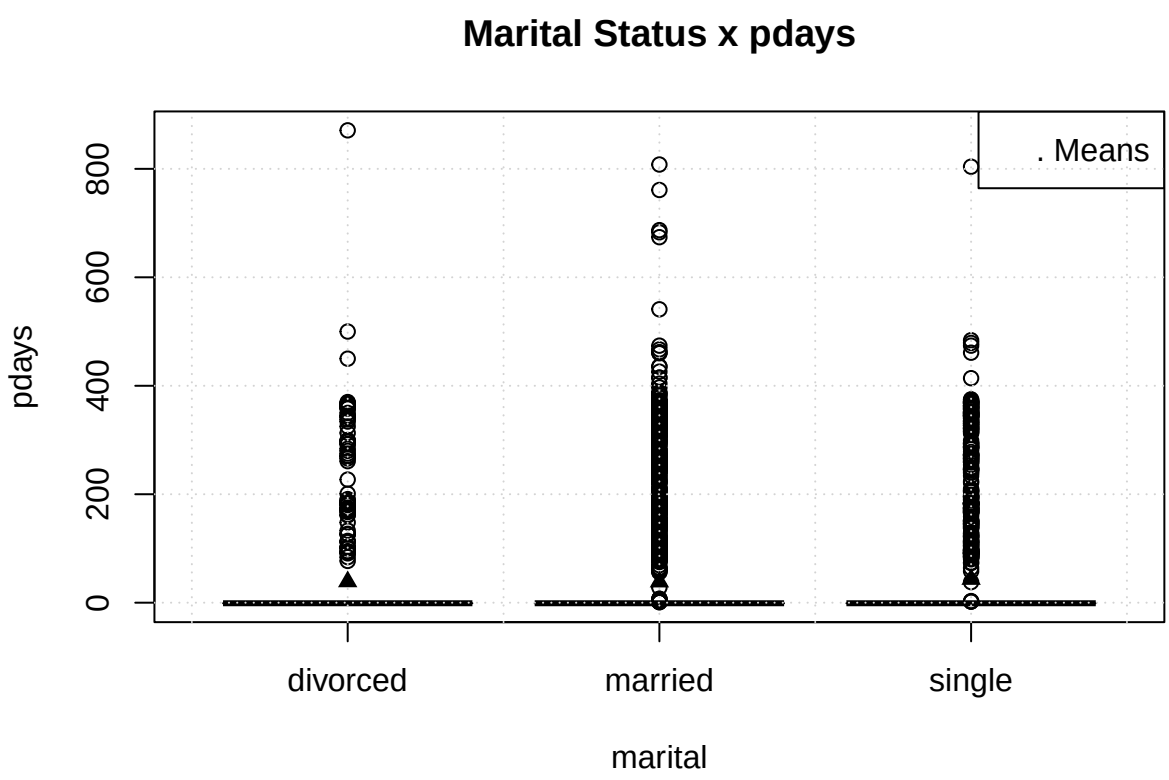

Marital Status x age
. Means
age
20
30
40
50
60
70
80
divorced
married
single
marital

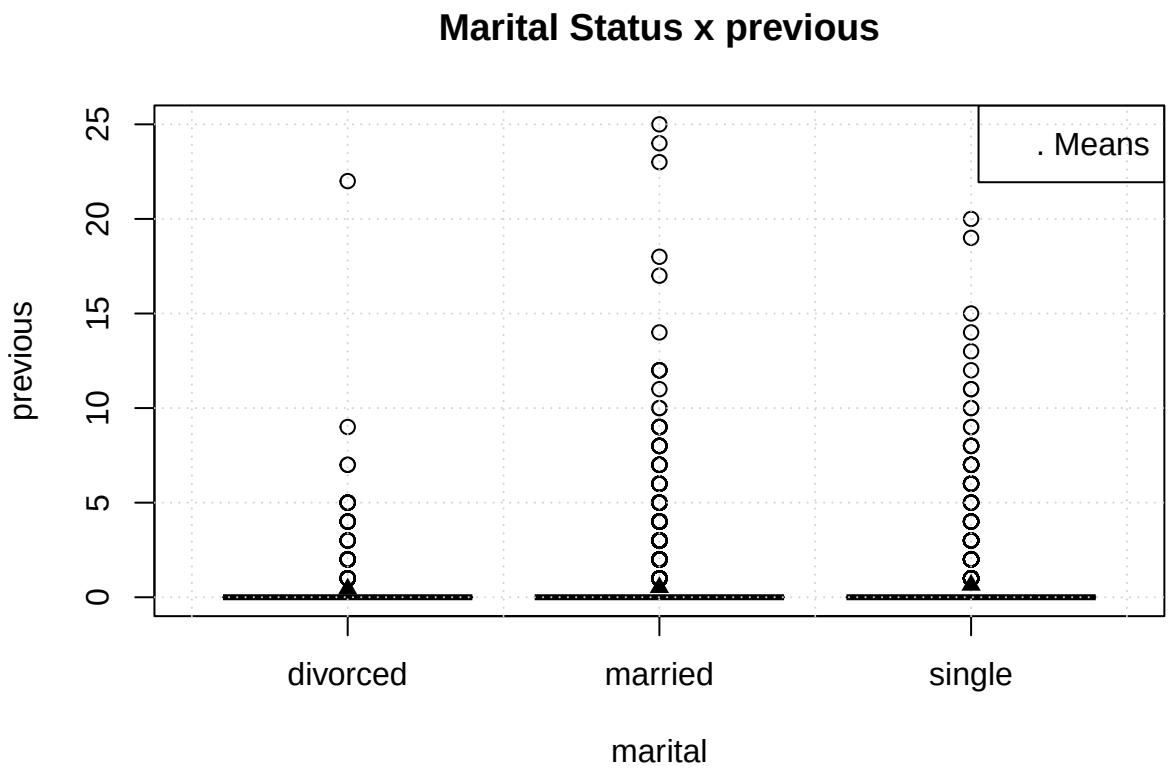

Marital Status x balance
. Means
balance
0
20000
40000
60000
divorced
married
single
marital


Marital Status x day
. Means
day
0
5
10
15
20
25
30
divorced
married
single
marital


Marital Status x duration
. Means
duration
0
500
1500
2500
divorced
married
single
marital

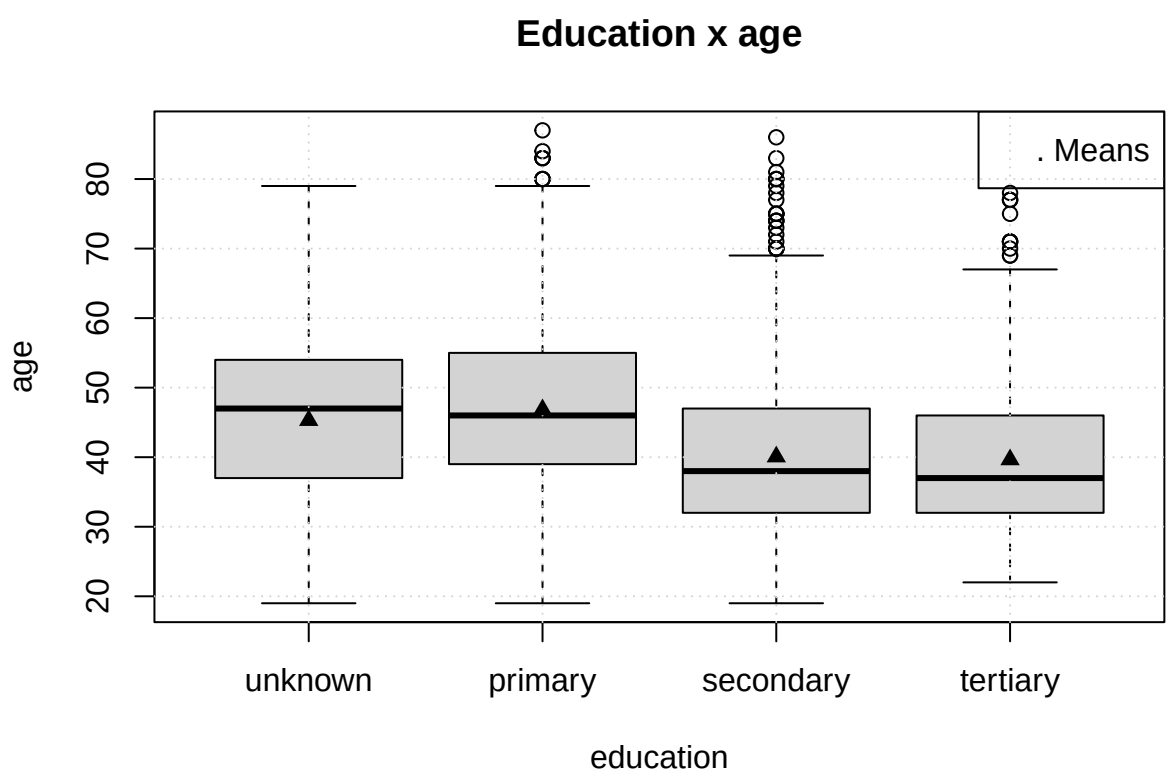

Marital Status x campaign
. Means
campaign
0
10
20
30
40
50
divorced
married
single
marital

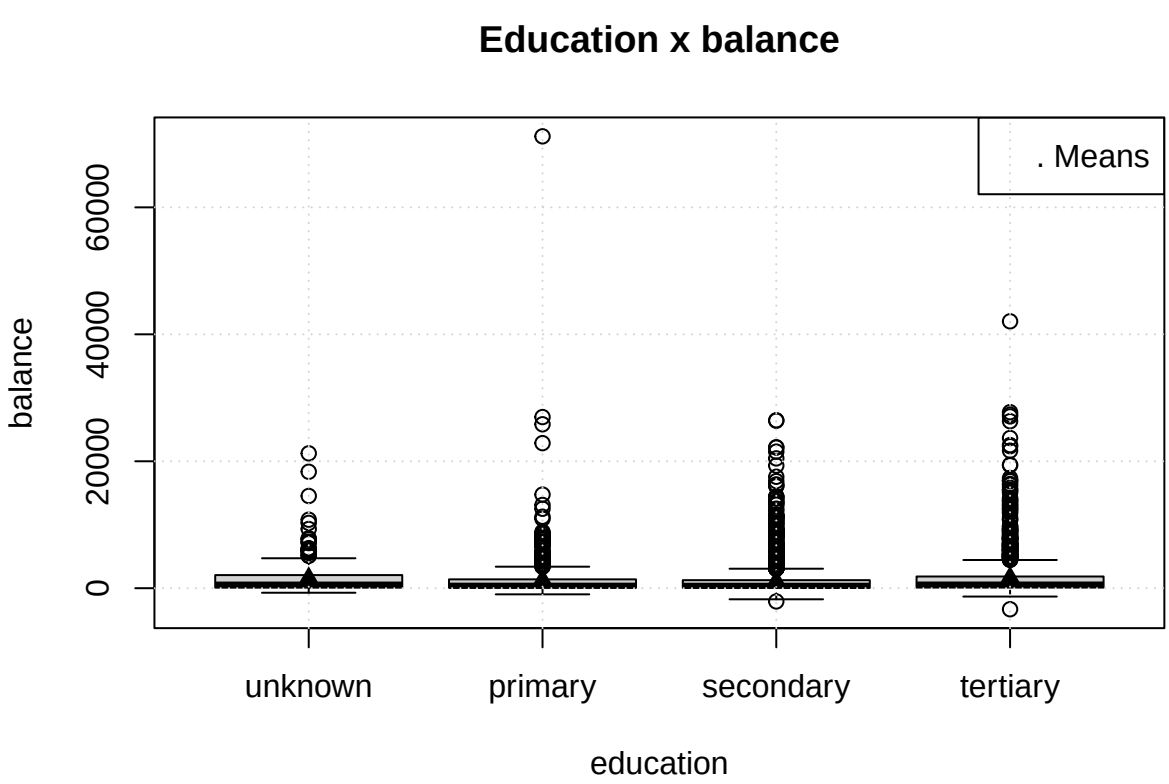

Marital Status x pdays
. Means
pdays
0
200
400
600
800
divorced
married
single
marital

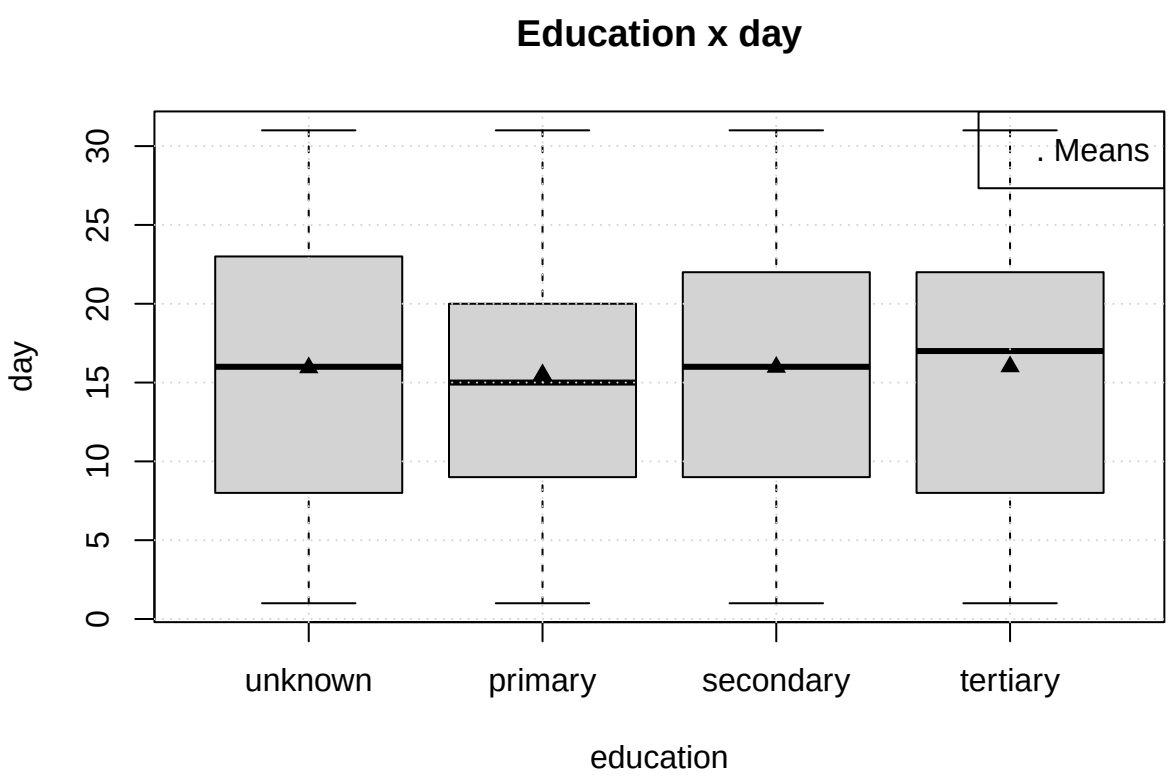


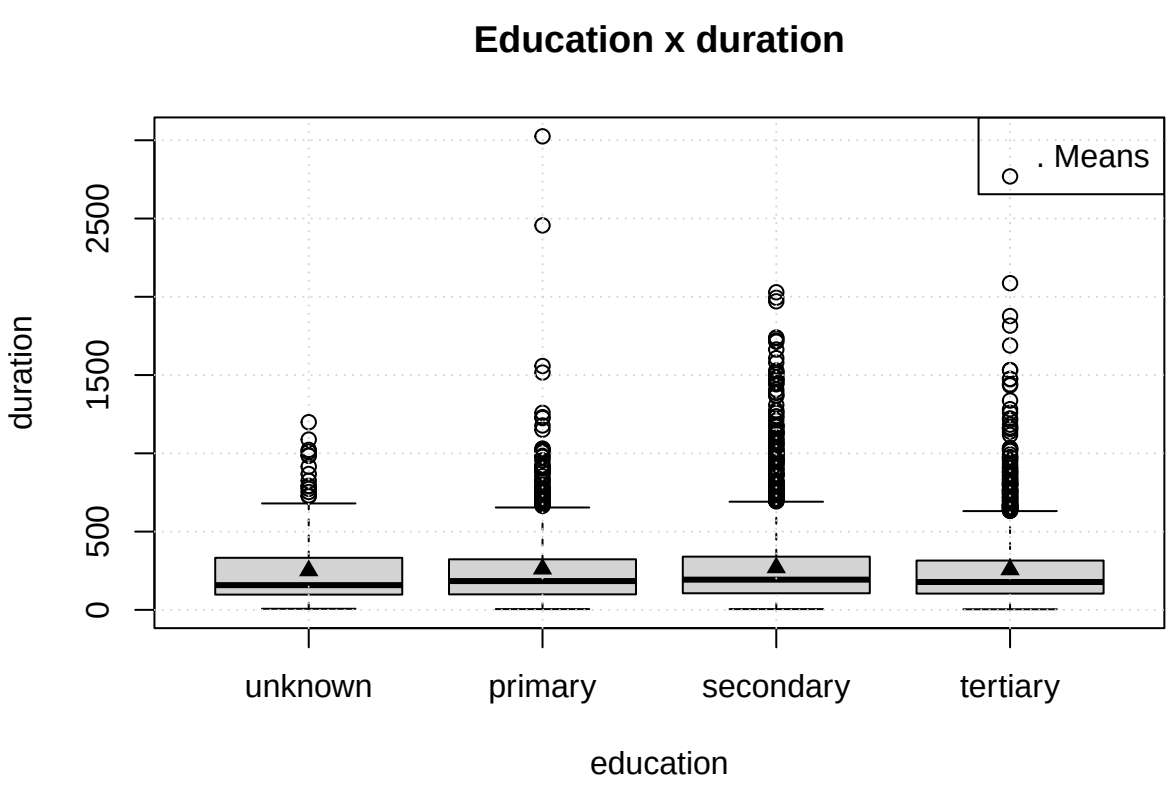
No visible difference between **marital** classes, except for *single* population being a lot younger than *divorced* and *married*.

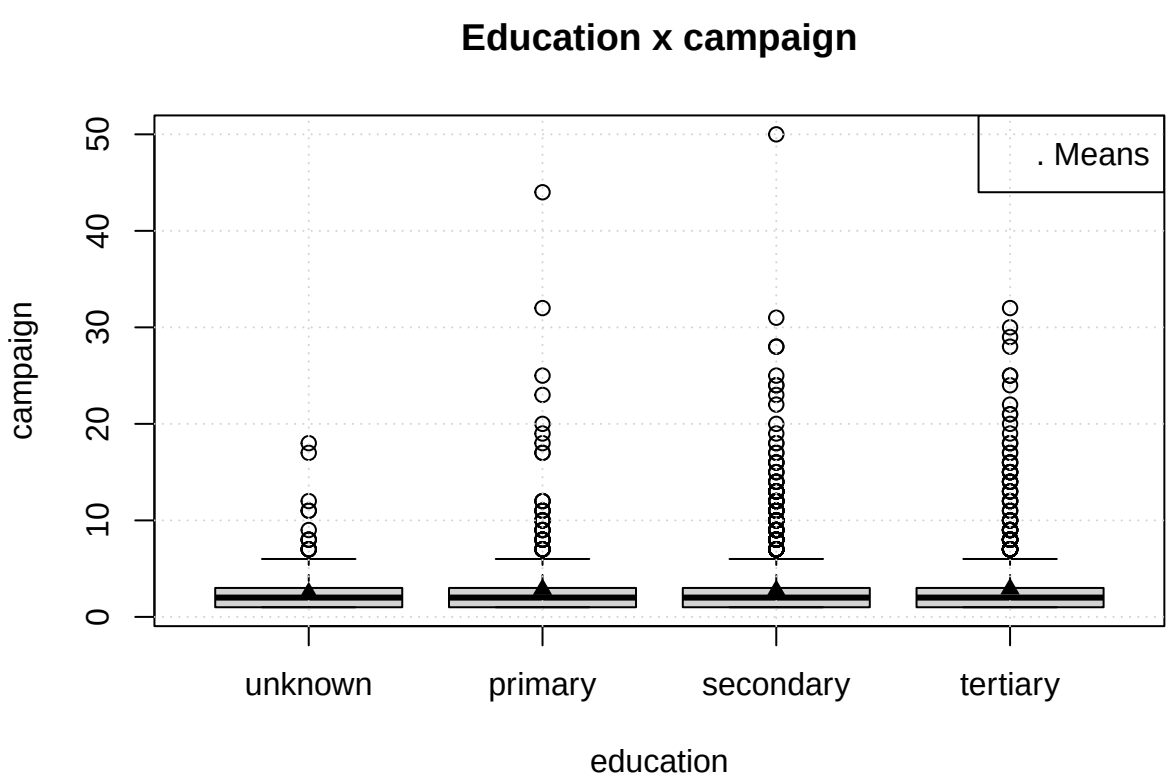
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "education")
    title(sprintf("Education x %s", col))
}
```

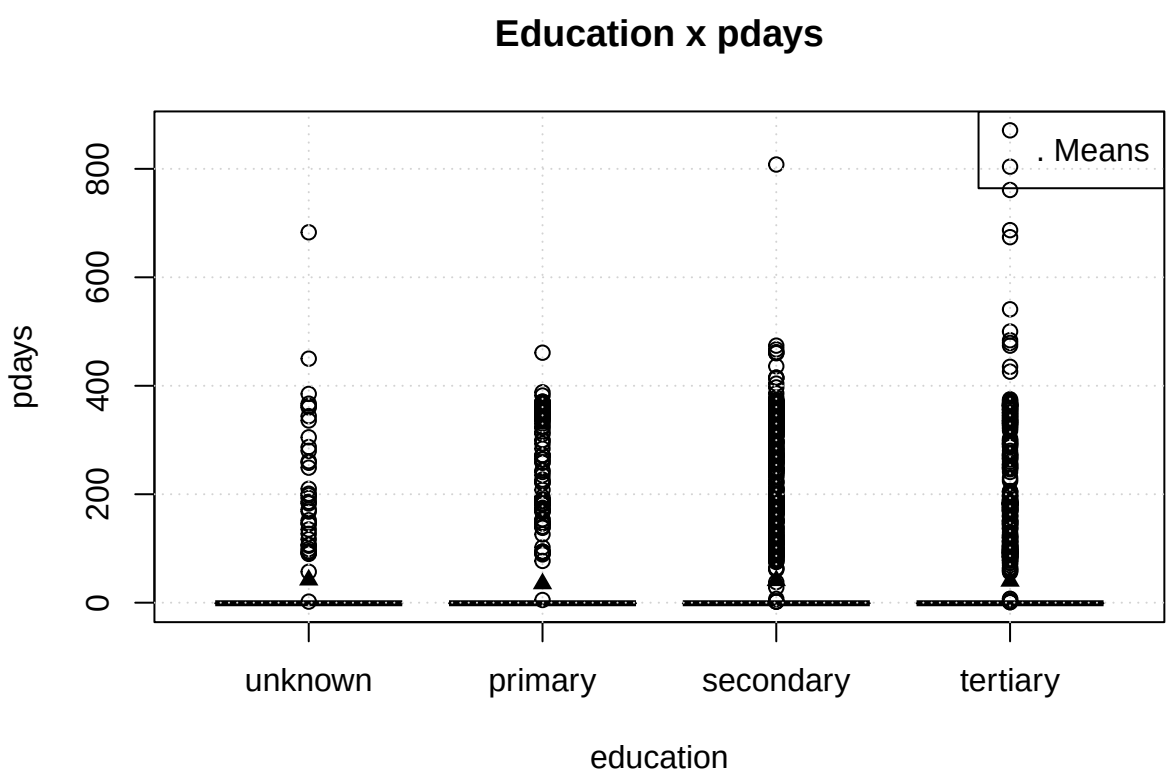

Education x age
. Means
age
20
30
40
50
60
70
80
unknown
primary
secondary
tertiary
education

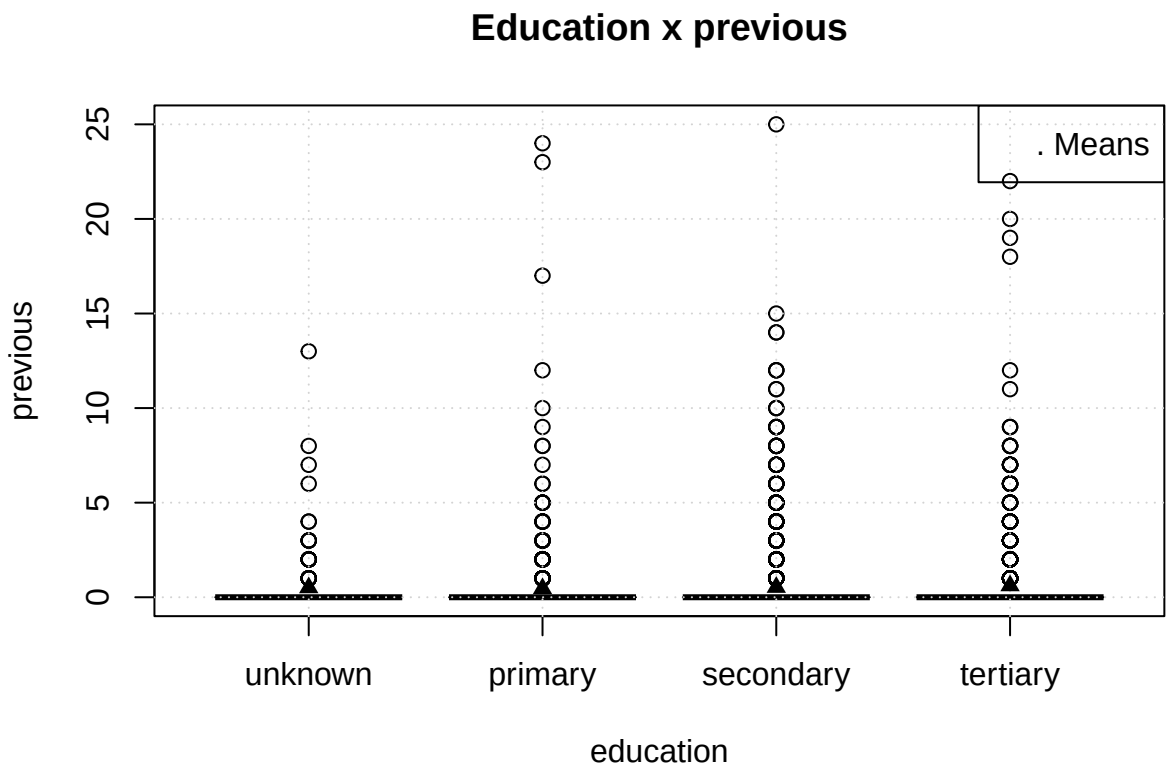

Education x balance
. Means
balance
0
20000
40000
60000
unknown
primary
secondary
tertiary
education


Education x day
. Means
day
0
5
10
15
20
25
30
unknown
primary
secondary
tertiary
education


Education x duration
. Means
duration
0
500
1500
2500
unknown
primary
secondary
tertiary
education

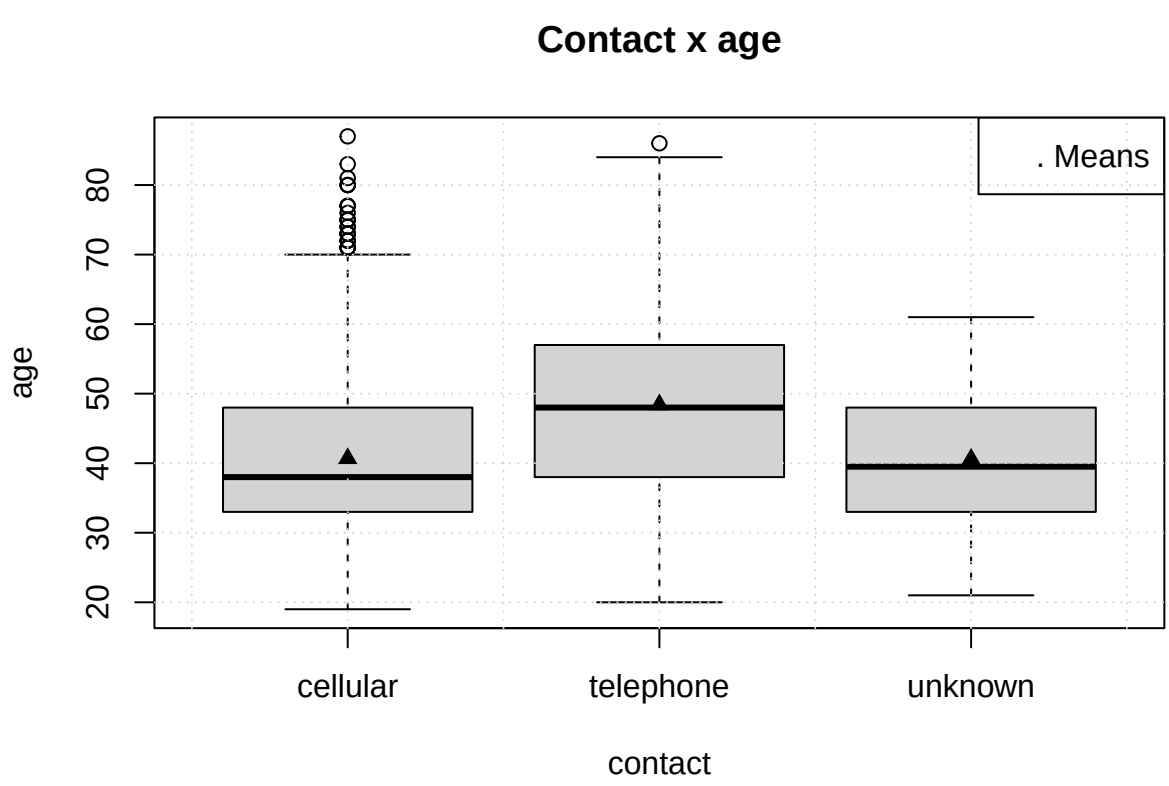

Education x campaign
. Means
campaign
0
10
20
30
40
50
unknown
primary
secondary
tertiary
education

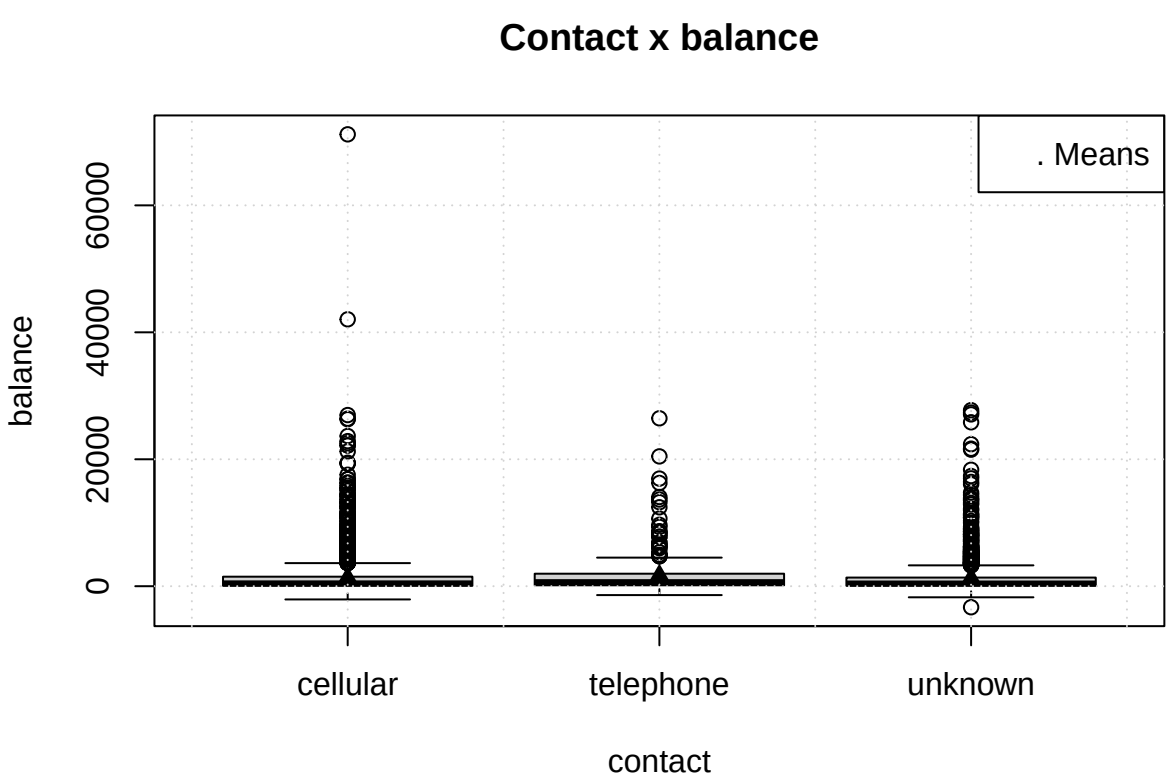

Education x pdays
. Means
pdays
0
200
400
600
800
unknown
primary
secondary
tertiary
education

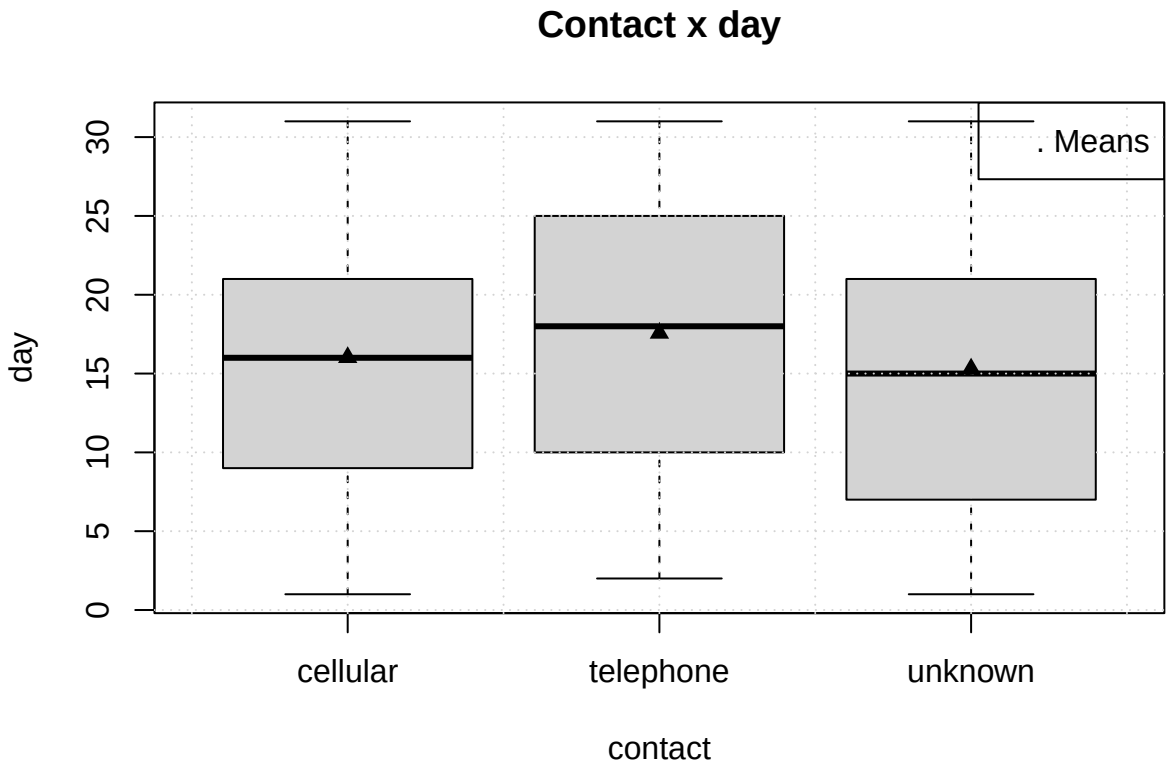


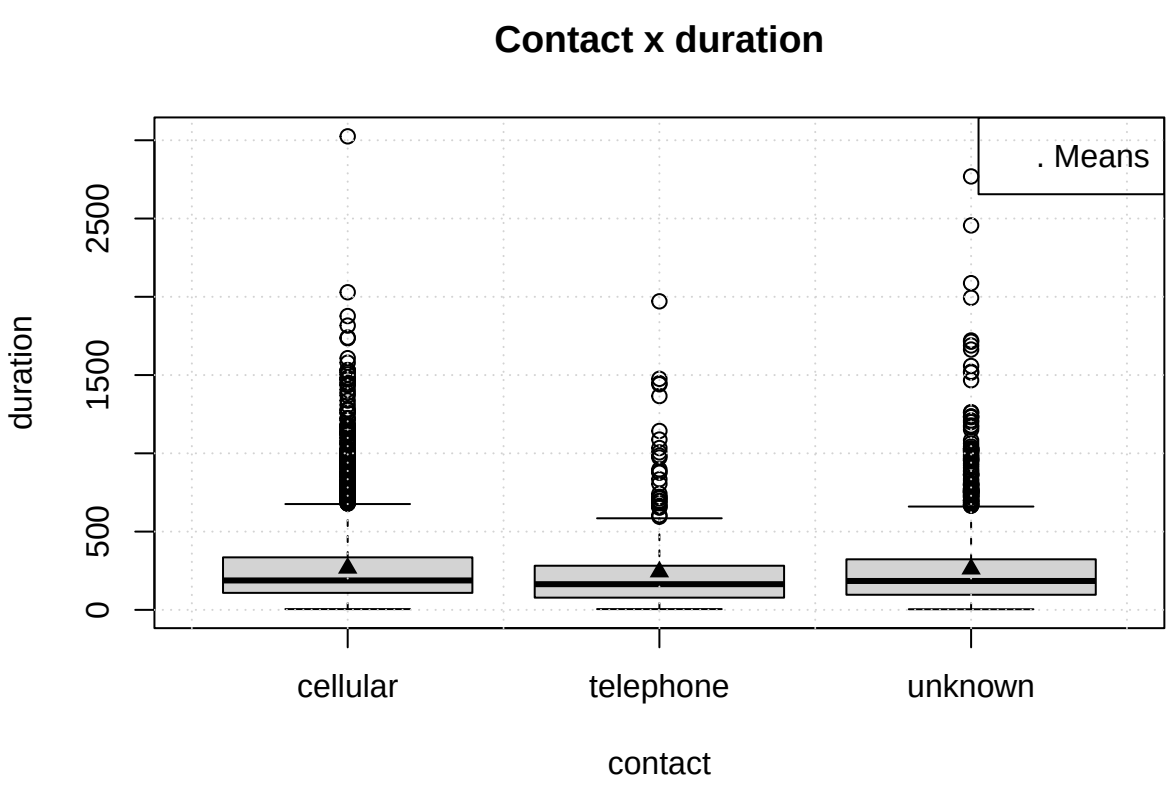
No relations are seen between **education** and numerical values, slight deviations are present in **age** feature, those can be a result of ongoing trend of younger generations being educated more on average.

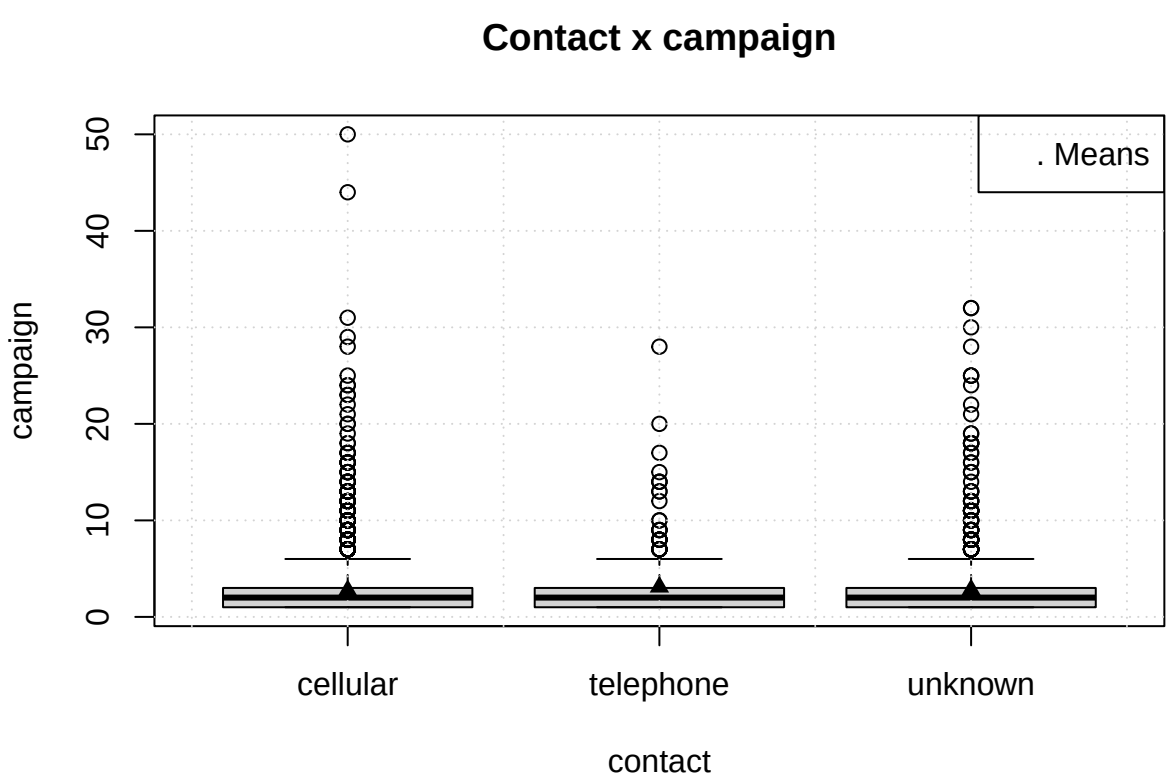
```
for (col in colnames(numeric_data)) {
    boxplot_mean(col, "contact")
    title(sprintf("Contact x %s", col))
}
```

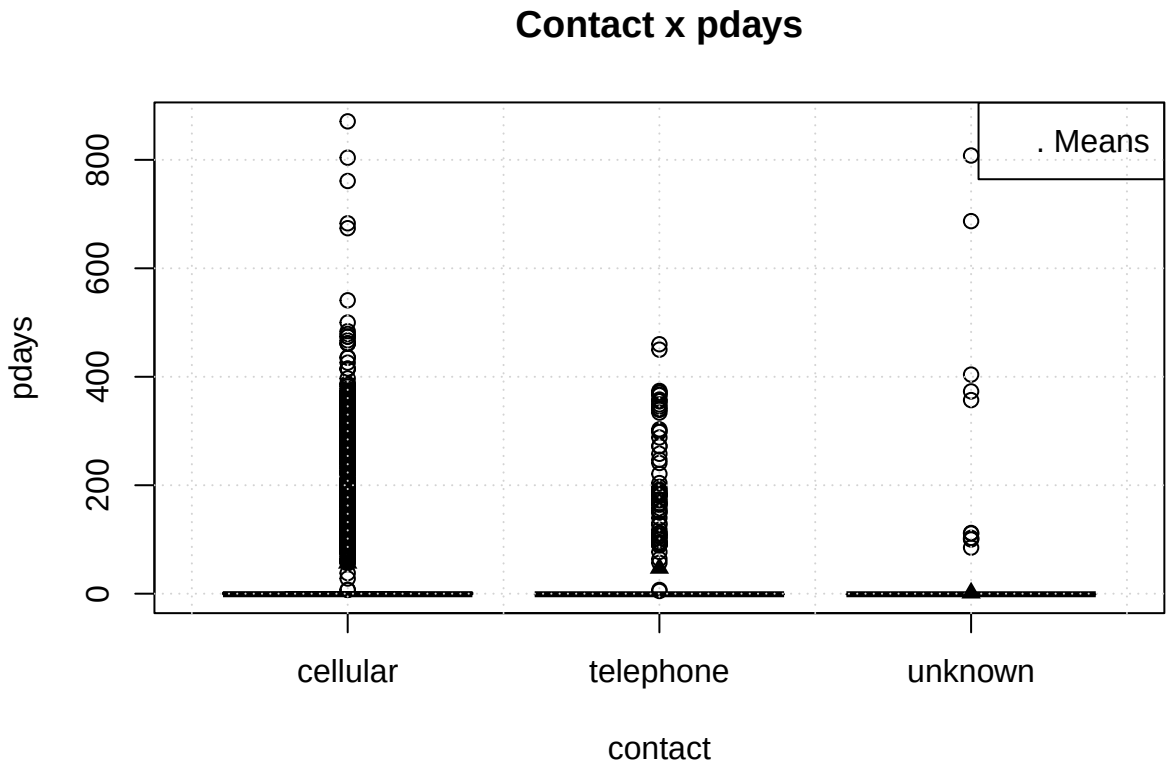

Contact x age
. Means
age
20
30
40
50
60
70
80
cellular
telephone
unknown
contact

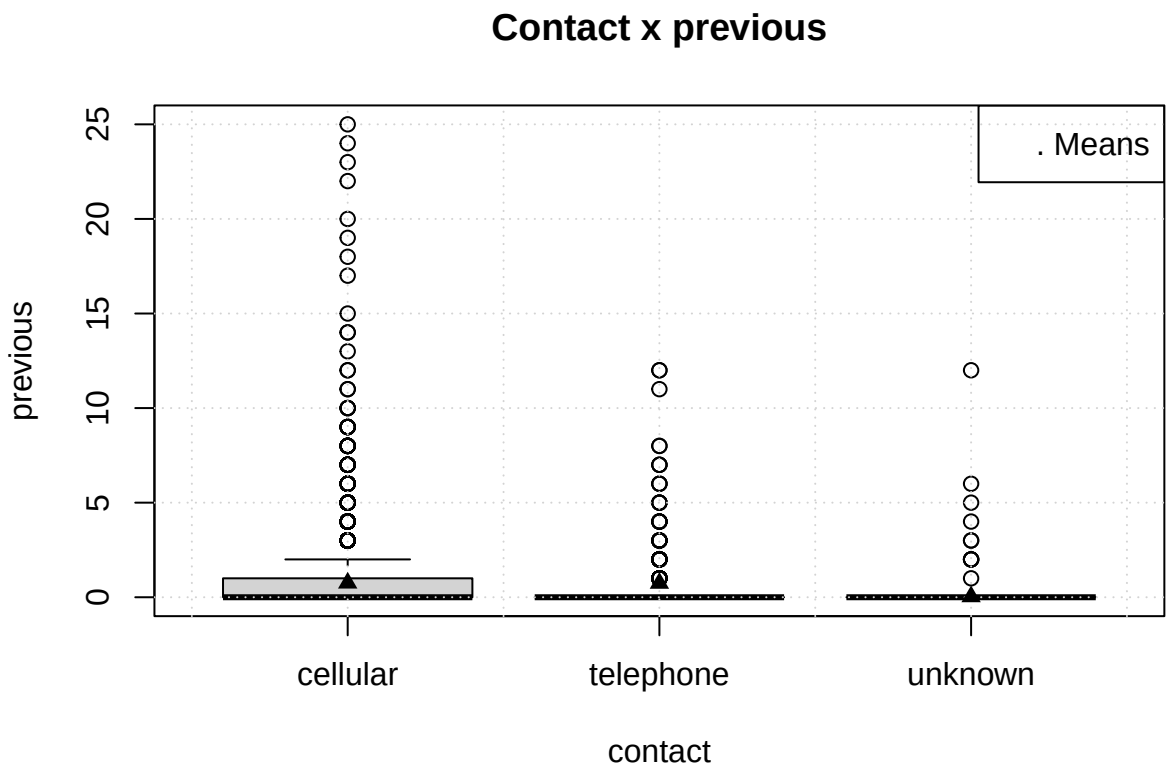

Contact x balance
. Means
balance
0
20000
40000
60000
cellular
telephone
unknown
contact


Contact x day
. Means
day
0
5
10
15
20
25
30
cellular
telephone
unknown
contact


Contact x duration
. Means
duration
0
500
1500
2500
cellular
telephone
unknown
contact


Contact x campaign
. Means
campaign
0
10
20
30
40
50
cellular
telephone
unknown
contact


Contact x pdays
. Means
pdays
0
200
400
600
800
cellular
telephone
unknown
contact



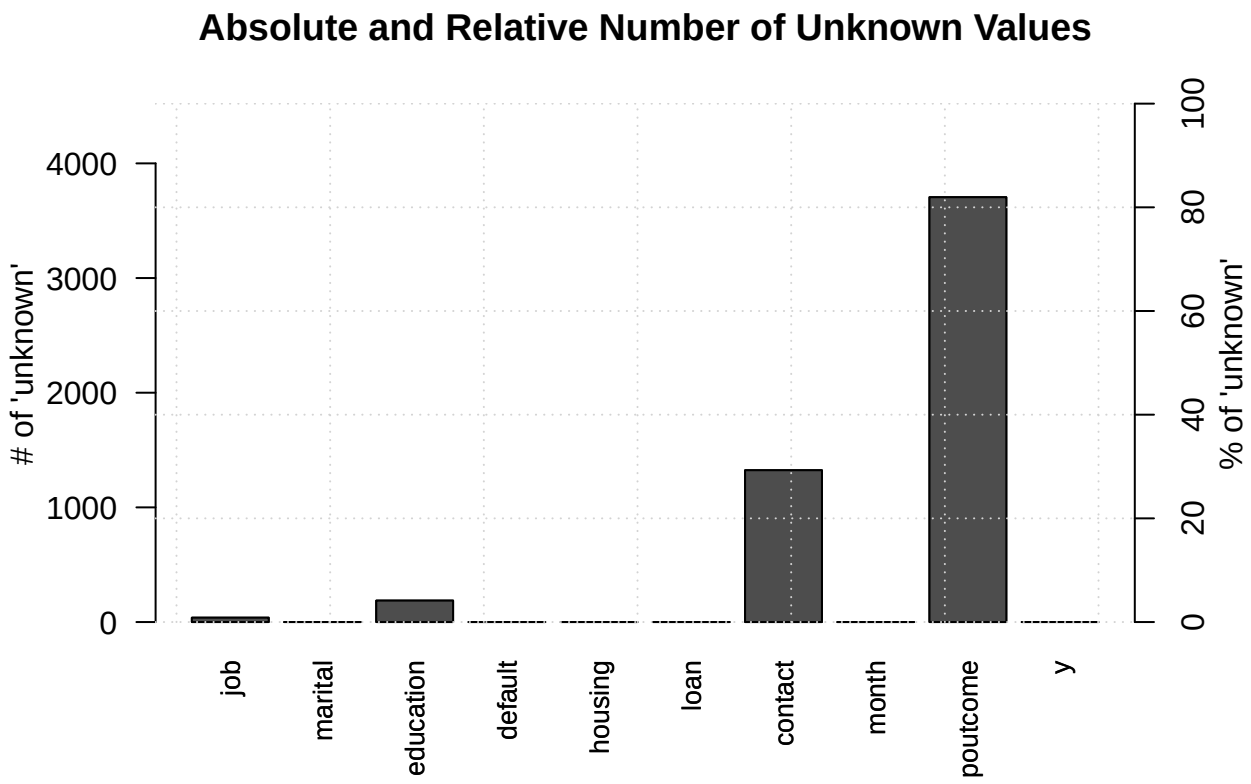
No significant dependancies can be recognized here as well.

1.6 Treatment of 'unknown' values

Throughout this notebook we have seen that some categorical features have *unknown* records. Let us take a look at what fraction of population is affected by them.

```
par(mar=c(5, 4, 4, 3))

categorical_data <- data[sapply(data, is.factor)]
unknown_data <- t(matrix(sapply(categorical_data, function(x) sum(x == "unknown"))))
colnames(unknown_data) <- colnames(categorical_data)
rownames(unknown_data) <- c("# of `unknown`")
barplot(
    unknown_data,
    las=2,
    cex.names = 0.8,
    ylab = "# of 'unknown'", main = "Absolute and Relative Number of Unknown Values",
    ylim = c(0, nrow(data))
)

par(new = T, mar=c(5, 4, 4, 3))
unknown_data_percent <- 100 * unknown_data / nrow(data)
barplot(
    unknown_data_percent,
    las=2,
    cex.names = 0.8,
```

```
    ylim = c(0, 100), axes=F
)
axis(side=4)
mtext("% of 'unknown'", side = 4, line =2)
grid()
```



Most features have no records or neglectible amounts of *unknown*. **education** and **job** have under 10% of their values, while **contact** has ~30% of its values as *unknowns*. The **poutcome** has over 80% of its values as unknowns. During modeling phase we should consider whether we want to use **contact** and **poutcome**.

1.7 Conclusion

After thorough EDA we have collected a battery of knowledge to test against the data. Those include distribution types (normality of age, exponential for duration calls), independence of categorical and numerical features (low correlation scores, similar box-plots over a feature) and their efficacy as predictors for whether a person gets subscribed or not (prevalence of single value through dataset in **pdays**, **poutcome**, **previous**). Specialized treatment should be given to outliers, that arise in every feature and impact it.

2 Conducting Statistical Tests

2.1 Setup

Let us load the data.

```
library(stats)
library(effsize)
```

```
library(goftest)
library(corrplot)

source(here::here("R", "constants.R"))
source(here::here("R", "load_data.R"))

data = load_bank_data()
head(data)
```

```
##   age         job marital education default balance housing loan  contact day
## 1  30  unemployed married   primary      no    1787      no   no cellular  19
## 2  33    services married secondary      no    4789     yes  yes cellular  11
## 3  35  management  single  tertiary      no    1350     yes   no cellular  16
## 4  30  management married  tertiary      no    1476     yes  yes  unknown   3
## 5  59 blue-collar married secondary      no       0     yes   no  unknown   5
## 6  35  management  single  tertiary      no     747      no   no cellular  23
##   month duration campaign pdays previous poutcome  y
## 1   oct       79        1    -1        0  unknown no
## 2   may      220        1   339        4  failure no
## 3   apr      185        1   330        1  failure no
## 4   jun      199        4    -1        0  unknown no
## 5   may      226        1    -1        0  unknown no
## 6   feb      141        2   176        3  failure no
```

We will separate data into numerical and categorical for ease of further analysis.

```
numeric_data <- data[sapply(data, is.numeric)]
categorical_data <- subset(data[sapply(data, is.factor)], select=-y)
```

2.2 Distributions

In this section, we will examine distributions of numerical features and conduct tests to support or reject our hypothesis. We will supplement numerical tests with Q-Q plots with 100 uniformly spaced quantiles. Our data includes 4.5 thousands features, Shapiro-Wilxon and Anderson-Darling tests, that we will be using in this part are notorious for being sensetive to small deviations, so we hope to capture at least some aspects of data distributions. Another reason to use Q-Q plots is, that Anderson-Darling good-fit-test sharply assumes knowledge of theoretical parameters of distributions, which we have no access to, so we will resort to pulled parameters estimated using methods of moments. A computed p-value will be only an approximation, that way. Nevertheless, Q-Q plots are constructed only as a guidance, so they do not provide statistical inference.

We also will not study **previous** and **pdays** as a large portion of their values is *0* or *-1* respectively. Their examination would require removal of those values and further study of residual data.

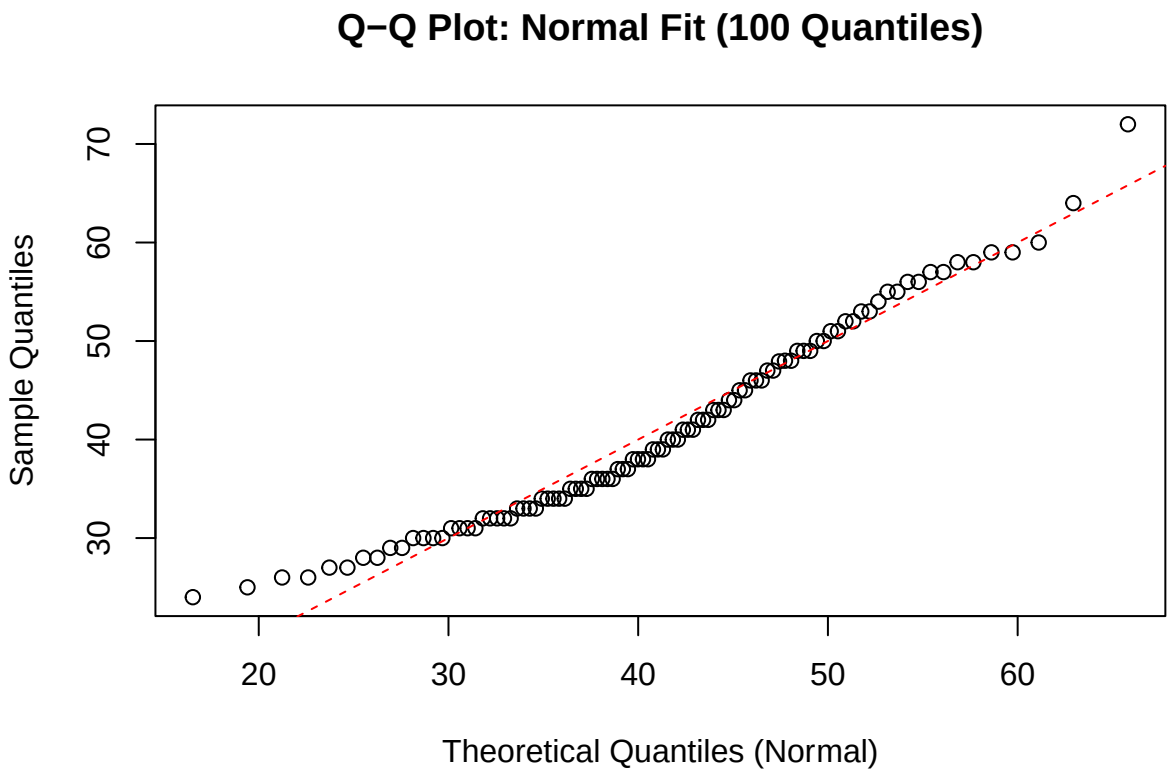
```
qqpercentile_plot <- function(x, dist_name, dist_func, ...) {
    q_n <- 100

    probs <- seq(0, 1, length.out = q_n + 2)[-c(1, q_n + 2)]

    theor_q <- dist_func(probs, ...)

    emp_q <- quantile(x, probs)

    plot(
        theor_q, emp_q,
        main = sprintf("Q-Q Plot: %s Fit (100 Quantiles)", dist_name),
        xlab = sprintf("Theoretical Quantiles (%s)", dist_name),
```

```
        ylab = "Sample Quantiles",
        pch = 1, col = "black"
    )
    abline(0, 1, col='red', lty= 2)
}
```

2.2.1 Age

In the EDA notebook we have discussed that **age** might be normally distributed. Let us test this hypothesis and plot quantiles to quantiles. Our null hypothesis is that **age** variable has normal distribution, alternative is that **age** does not have normal distribution. To test that we will use Shapiro-Wilxon test.

```
x <- data$age

mean_hat <- mean(x)
sd_hat <- sd(x)

qqpercentile_plot(
    x, "Normal", qnorm, mean = mean_hat, sd = sd_hat
)
```



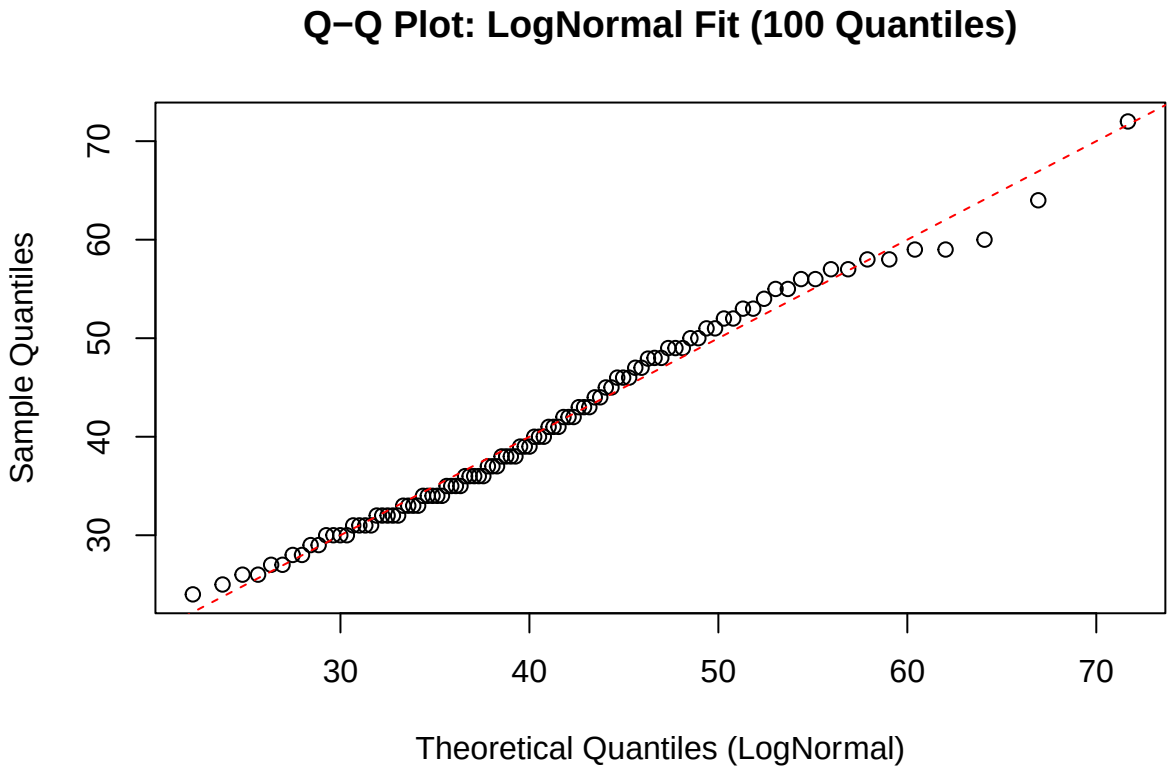

```
shapiro.test(x)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  x
```

```
## W = 0.95951, p-value < 2.2e-16
```

Though Q-Q plot shows quantile points close to the theoretical line for most of quantile range, Shapiro-Wilxon test gives a p-value much lesser than 0.05, and we reject the null hypothesis. If we look closely to the Q-Q plot, we can see great deviation in lower tail. We can also check for the lognormal distribution, because age is strictly positive variable. The null hypothesis now is that **age** is distributed lognormally, alternative is otherwise. We will conduct Anderson-Darling test.

```
x <- data$age

meanlog_hat <- mean(log(x))
sdlog_hat <- sd(log(x))

qqpercentile_plot(
    x, "LogNormal", qlnorm, meanlog = meanlog_hat, sdlog = sdlog_hat
)
```




```
plnorm_fit <- function(x) plnorm(x, meanlog = meanlog_hat, sdlog = sdlog_hat)

ad.test(x, null = plnorm_fit)
```

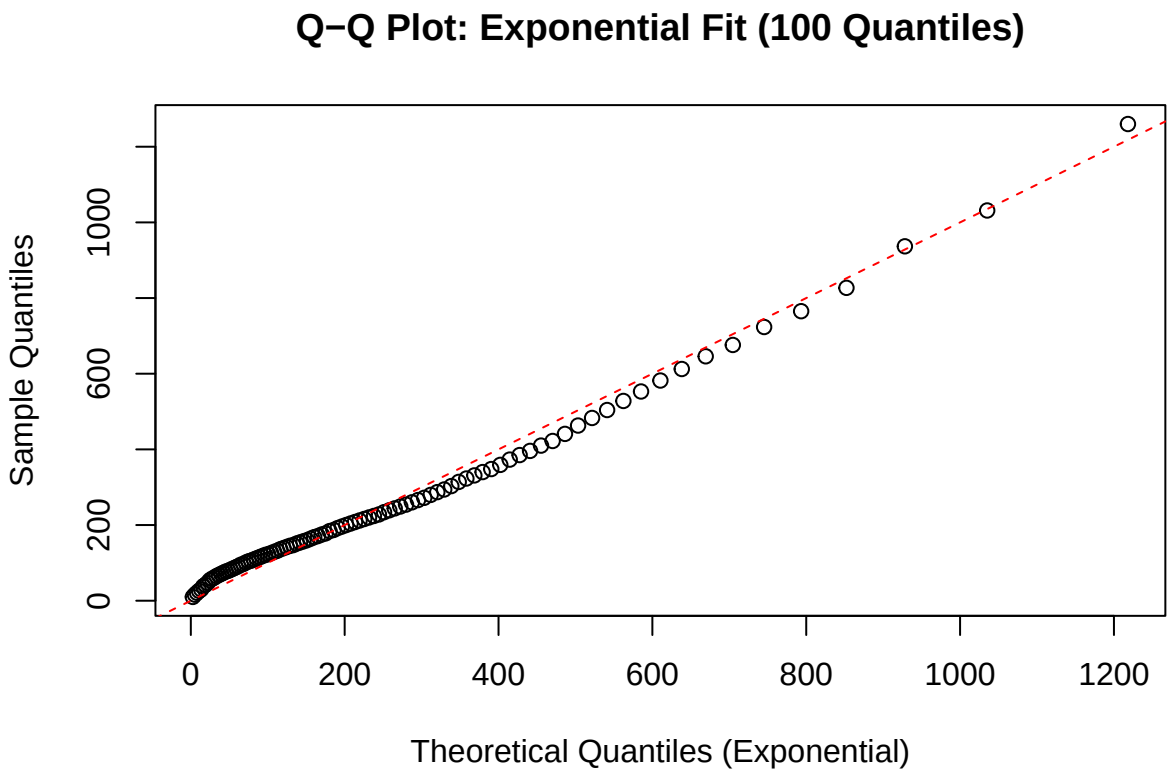
```
##
##  Anderson-Darling test of goodness-of-fit
##  Null hypothesis: distribution 'plnorm_fit'
##  Parameters assumed to be fixed
##
## data:  x
## An = 19.177, p-value = 1.327e-07
```

Quantiles now stick even closer to the line, yet p-value is small enough to reject the null hypothesis. As we have discussed at the beginning of the section, with large sample sizes it is common for distribution shape tests to fail on miniscule deviations.

2.2.2 Duration of Last Call

During EDA we have suggested that data might be exponentially distributed. It makes sense as the call is more likely to end early and whether the call should end depends only on its timespan, but not the starting point, i.e. distribution of its ceasing is memoryless.

```
x <- data$duration

rate_hat <- 1 / mean(x)
qqpercentile_plot(
    x, "Exponential", qexp, rate = rate_hat
)
```




```
pexp_fit <- function(x) pexp(x, rate = rate_hat)

ad.test(x, null = pexp_fit)
```

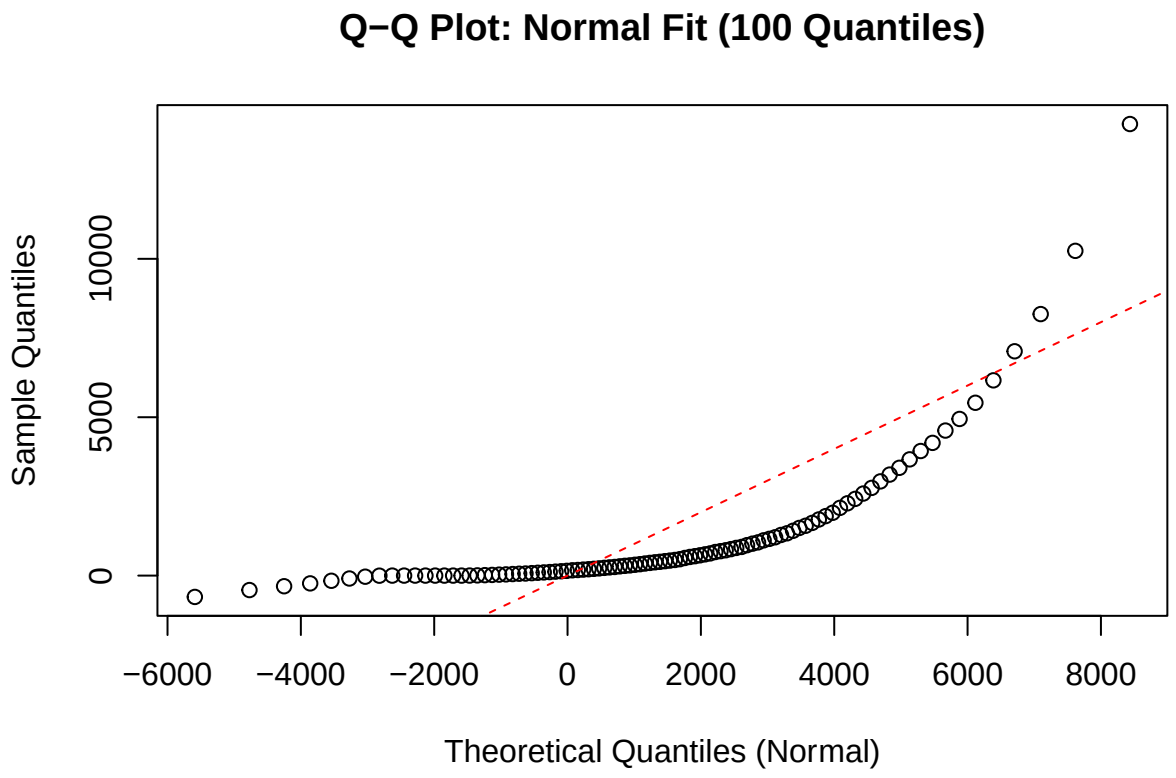
```
##
##  Anderson-Darling test of goodness-of-fit
##  Null hypothesis: distribution 'pexp_fit'
##  Parameters assumed to be fixed
##
## data:  x
## An = 73.114, p-value = 1.327e-07
```

Once again we see a Q-Q plot suggests statment about distribution of data, that **duration** has exponential distribution, yet we must reject the null hypothesis.

2.2.3 Balance

Balance histogram is unimodal and roughly symmetrical, so it would be adequate to think that it has normal distribution.

```
x <- data$balance

mean_hat <- mean(x)
sd_hat <- sd(x)

qqpercentile_plot(
    x, "Normal", qnorm, mean = mean_hat, sd = sd_hat
)
```




```
shapiro.test(x)
```

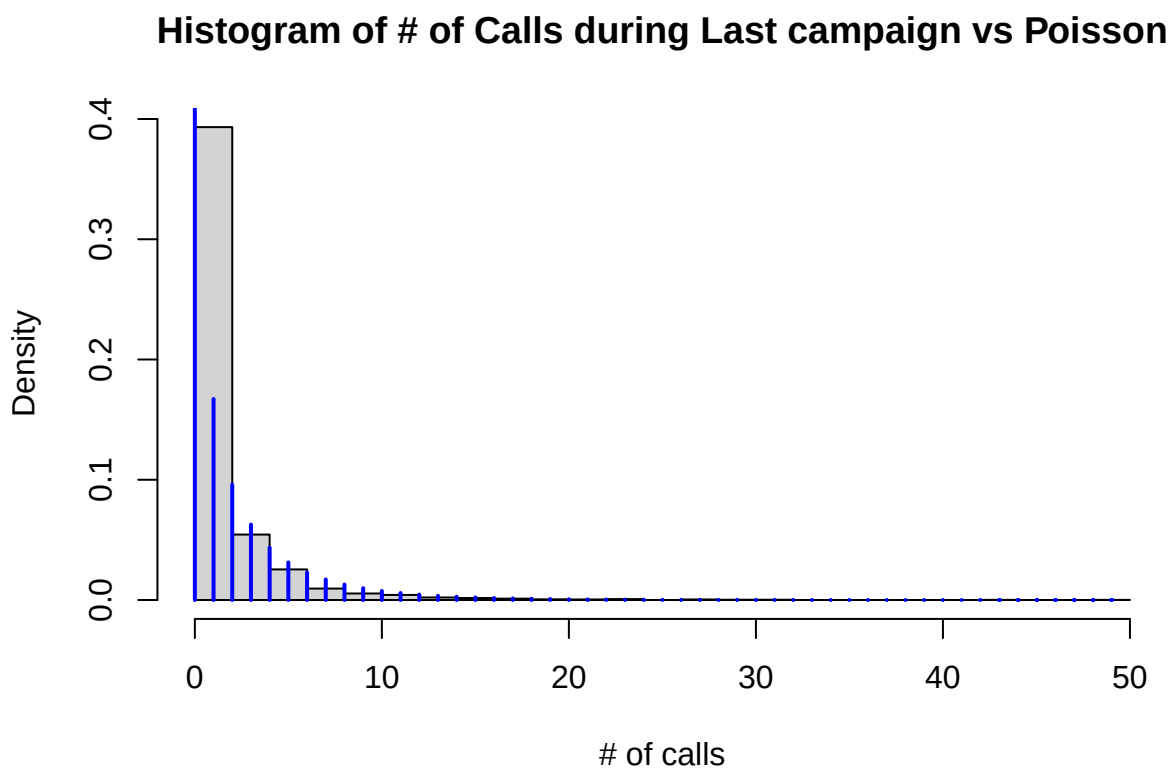
```
##
##  Shapiro-Wilk normality test
##
## data:  x
## W = 0.50151, p-value < 2.2e-16
```

Q-Q plot shows that quantiles do not follow the line even slightly, test also shows that we should reject the null hypothesis of normality of **balance**.

2.2.4 Number of Calls during Campaign

In EDA notebook it was suggested that **campaign** might be negative binomially distributed, due to nature of variable (number of successful trials). It is discrete random variable, so we cannot make a Q-Q plot, instead we will plot poisson mdf over data histogram.

```
x <- data$campaign - 1 # taking part in campaign produces minimal value of 1

hist(
    x,
    main = "Histogram of # of Calls during Last campaign vs Poisson",
    xlab = "# of calls", freq=F, breaks=20
)

mean_hat <- mean(x)
var_hat <- var(x)
n_hat <- (mean_hat^2) / (var_hat - mean_hat)
p_hat <- n_hat / (n_hat + mean_hat)

points(0:max(x), dnbinom(0:max(x), size=n_hat, p=p_hat), type = "h", col = "blue", lwd = 2)
```




```
pnbinom_fit <- function(x) pnbinom(x, size=n_hat, p=p_hat)
ad.test(x, null = pnbinom_fit)
```

```
##
##  Anderson-Darling test of goodness-of-fit
##  Null hypothesis: distribution 'pnbinom_fit'
##  Parameters assumed to be fixed
```

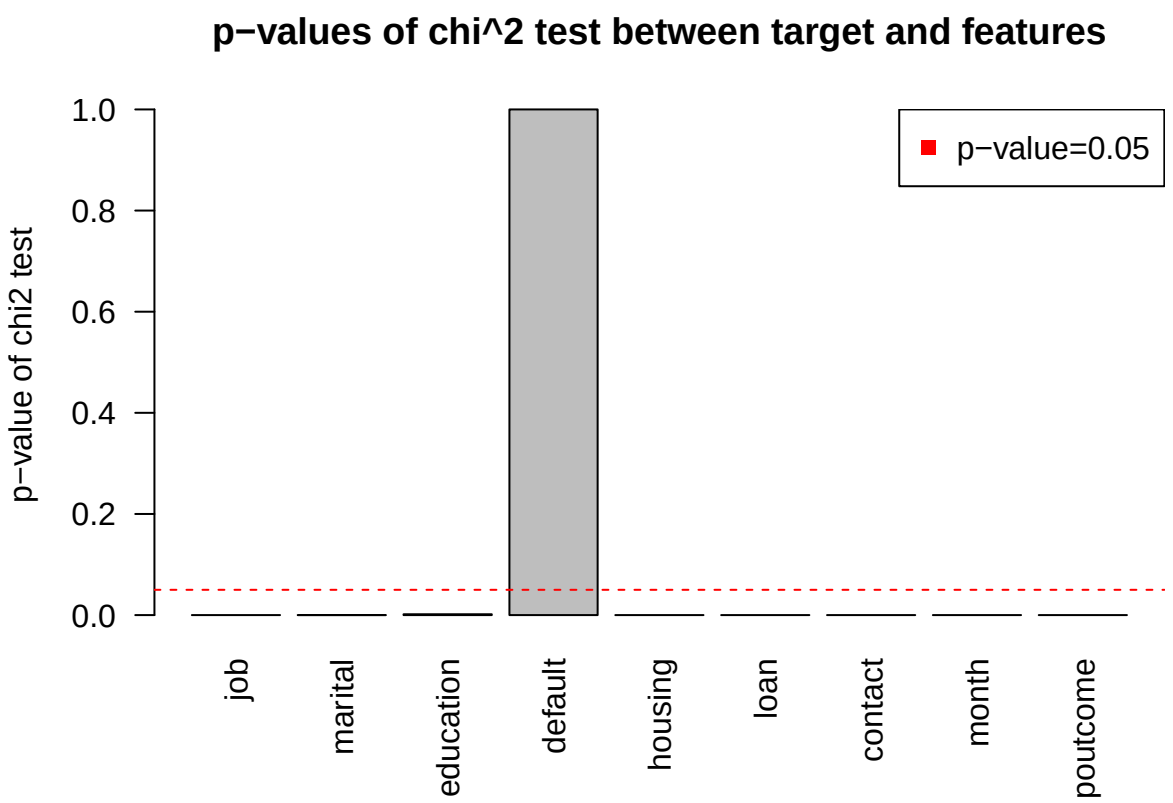
```
##
## data:  x
## An = 1034.1, p-value = 1.327e-07
```

The plot shows that neighborhood of zero is much less probable than fitted negative binomial distribution suggests and after that they behave similarly, but AD test yields too little p-value, so we must reject the null hypothesis.

2.3 Catgorical vs Target

For every categorical feature we want to know whether it has association with target variable. For all categorical features, the null hypothesis is the feature and target variable are independent. Alternative states otherwise. We will do Pearson's chi squared to test these hypothesis.

```
res <- c()
for (col in colnames(categorical_data)) {
    ct <- table(data$y, data[,col])
    res[col] <- chisq.test(ct)$p.value
}
barplot(res, las = 2, ylab = "p-value of chi2 test", main = "p-values of chi^2 test between target and
par(new = T)
abline(h=0.05, col = "red", lty=2)
legend(
    "topright",
    legend = "p-value=0.05",
    pch=15,
    col = 'red'
)
```



We see that default is the only feature for which we fail to reject the null hypothesis and assume that it bears no association with target variable. Other features show extremly low p-values and thus we reject the null hypotheses, that they and target are pairwise independent.

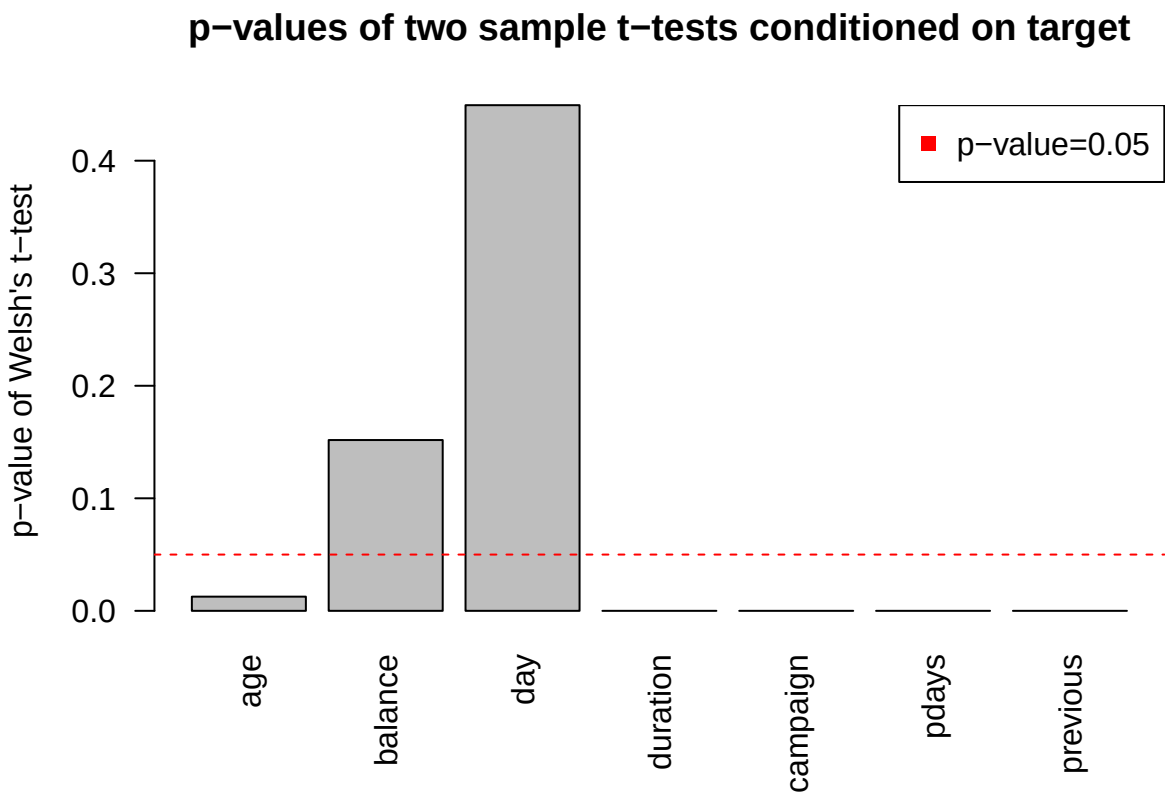
2.4 Numerical vs Target

We would like to know whether standalone numerical features differentiate between target classes. We will check whether means conditioned by target class are different using Welsh's t-test with two-sided null hypothesis, to check how effective is the difference we will also compute Cohen's d.

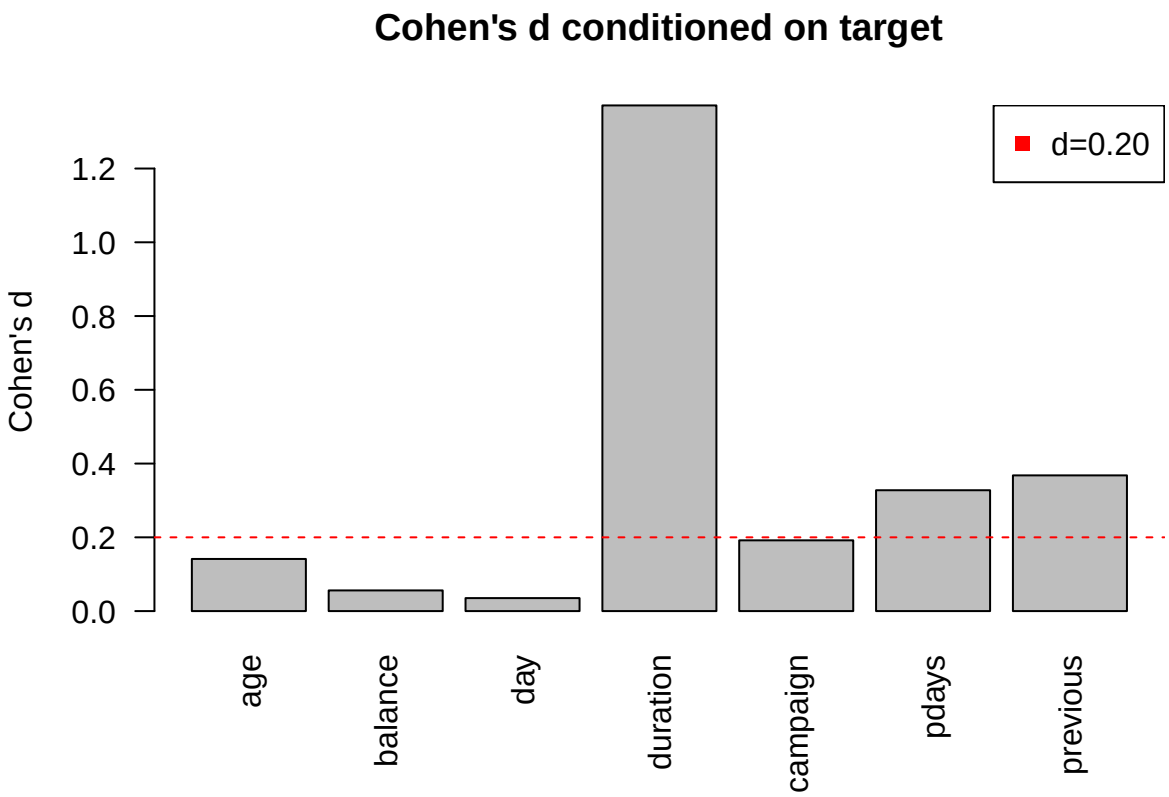
We have 4.5K samples, under CLT normalized mean approaches normal distribution, so we satisfy t-tests assumption.

```
res_t = c()
res_d = c()
for (col_name in colnames(numeric_data)) {
    form <- as.formula(paste(col_name, "~ y"))
    res_t[col_name] <- t.test(form, data=data)$p.value
    res_d[col_name] <- abs(cohen.d(form, data=data)$estimate)
}
barplot(res_t, las = 2, ylab = "p-value of Welsh's t-test", main="p-values of two sample t-tests conditi
par(new = T)
abline(h=0.05, col = "red", lty=2)
legend(
    "topright",
    legend = "p-value=0.05",
    pch=15,
```

```
    col = 'red'
)
```




```
barplot(res_d, las=2, ylab="Cohen's d", main="Cohen's d conditioned on target")
abline(h=0.2, col = "red", lty=2)
legend(
    "topright",
    legend = "d=0.20",
    pch=15,
    col = 'red'
)
```



The only features for which we reject the null hypothesis are **balance** and **day**, for all the other we fail to reject the null hypothesis, that means conditioned by target class are statistically significantly different. Nevertheless only **balance**, **campaign**, **pdays** and **previous** have Cohen's d values that show practical significance of atleast small level (d=0.20).

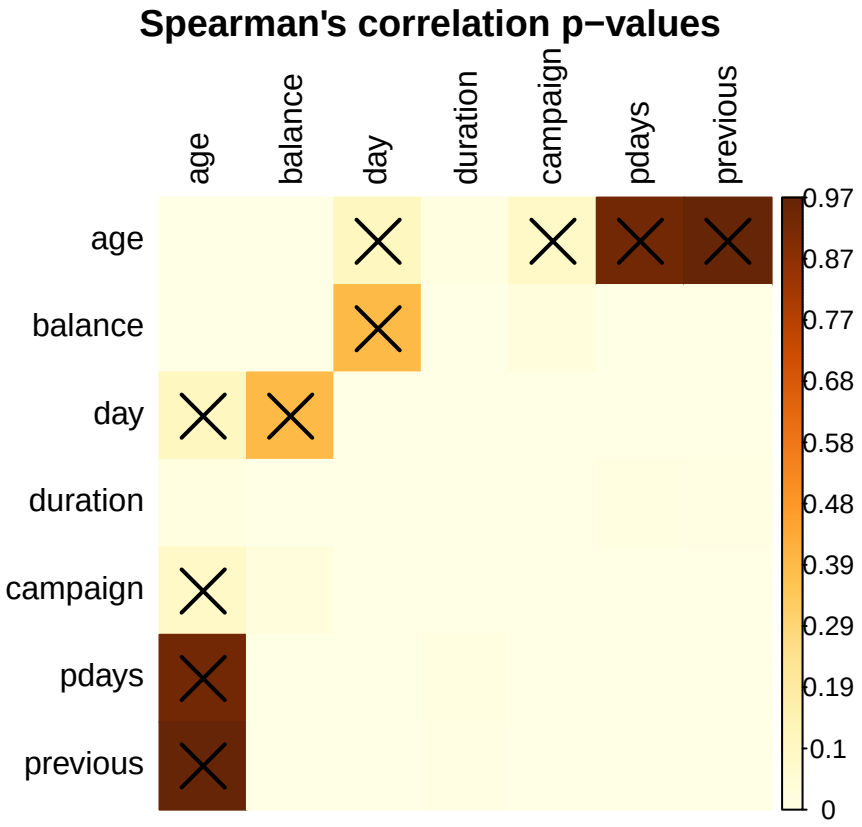
2.5 Numerical vs Numerical

We will conduct correlation tests to check if any numerical variables are related. Due to non-normality of all our features we are left to Spearman's correlation test, so the null hypotheses are that Spearman's correlation coefficients are zeros, in other words, there is no monotonic association between two variables, alternative states otherwise.

We will vizualize p-values obtained for every pair in a heatmap, where cross indicates p-value greater than 0.05, i.e. insignificant evidence in data.

```
mat <- matrix(ncol=ncol(numeric_data), nrow=ncol(numeric_data))
rownames(mat) <- colnames(numeric_data)
colnames(mat) <- colnames(numeric_data)

for (col1 in colnames(numeric_data)) {
    for (col2 in colnames(numeric_data)) {
        mat[col1, col2] = cor.test(data[,col1], data[,col2], method='spearman')$p.value
    }
}
corrplot(
    mat, is.corr = FALSE, method = "color", p.mat = mat,
    tl.col = "black",
```

```
    title = "Spearman's correlation p-values",
    mar = c(1,1,1,1)
)
```



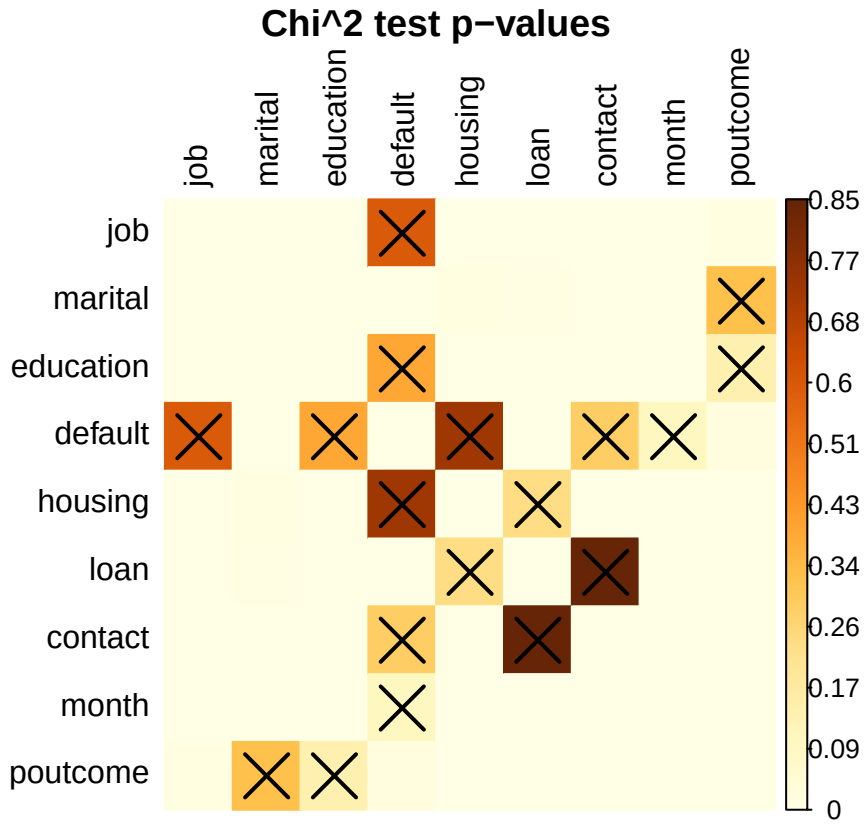
Most of features pairs have low p-values, but almost all pairs containing **age** have large p-values, so we must reject null hypotheses for them. The only variable for which **age** shows significant monotonic relation is **duration**. Another pair for which we fail to reject the null hypothesis is **day**-**balance**.

2.6 Categorical vs Categorical

For every pair of categorical features we will test whether they are independent, this will be the null hypothesis, alternative states that a pair of variables have significant association. We will conduct chi squared tests for this reason.

```
mat <- matrix(ncol=ncol(categorical_data), nrow=ncol(categorical_data))
rownames(mat) <- colnames(categorical_data)
colnames(mat) <- colnames(categorical_data)

for (col1 in colnames(categorical_data)) {
    for (col2 in colnames(categorical_data)) {
        ct <- table(data[,col1], data[,col2])
        mat[col1, col2] = chisq.test(ct)$p.value
    }
}
corrplot(
    mat, is.corr = FALSE, method = "color", p.mat = mat,
    tl.col = "black",
```

```
    title = "Chi^2 test p-values",
    mar = c(1,1,1,1)
)
```



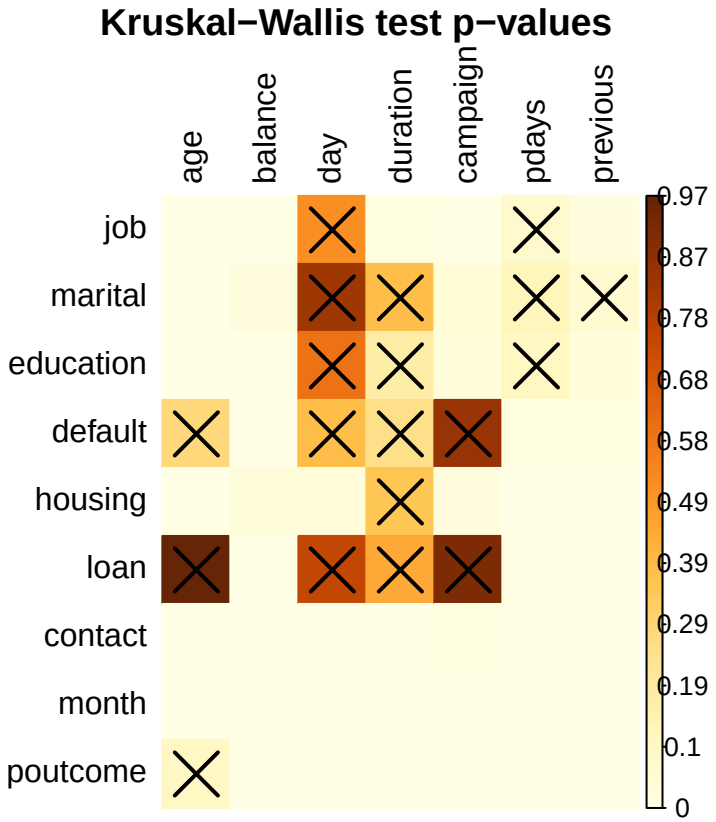
Most pairs show significant association. The odd one is **default** feature, during EDA it was revealed that **default** is almost constant *no*, therefore it is independent from other variables that have more balanced class division. Other independent pairs are **loan-housing**, **loan-contact** and **marital-poutcome**. As we have discussed in EDA personal loans are taken by all groups of population, so its expected that it will behave more independently towards other features.

2.7 Numerical vs Categorical

We would like to check whether there is a connection between feature-classes and numerical variables. One of the way to do it is to check difference between conditioned distributions. Most of our numerical features show scarce signs of normality, so we cannot use ANOVA or AOV. Instead we will use Kruskal-Wallis test. It is similar to ANOVA, but does not require conditioned data to be normal. The null hypotheses are that conditioned distributions are the same with no regard to class, alternatice hypothesis states that otherwise.

```
mat <- matrix(ncol=ncol(numeric_data), nrow=ncol(categorical_data))
rownames(mat) <- colnames(categorical_data)
colnames(mat) <- colnames(numeric_data)

for (col1 in colnames(numeric_data)) {
    for (col2 in colnames(categorical_data)) {
        form <- as.formula(paste(col1, "~", col2))
        kt =  kruskal.test(form, data = data)
        mat[col2, col1] = kt$p.value
```

```
    }
}
corrplot(
    mat, is.corr = FALSE, method = "color", p.mat = mat,
    tl.col = "black",
    title = "Kruskal-Wallis test p-values",
    mar = c(1,2,1,1)
)
```



We see that **day** and **duration** are significantly differentiated by **marital**, **education** and **housing**. **default** being almost constant has great p-values with most numerical features, except **balance**, **pdays** and **previous**. Balance is not differentiated by any categorical variable, while **pdays** and **pvalues** are by **job**, **marital** and **education**. Though we fail to reject the null hypothesis, the p-value for those pairs is visibly less than for other non-rejected pairs.

2.8 Conclusion

We ran a battery of tests exploring univariate and bivariate statistics. We revealed possible strong predictors such as **age** and **duration** from numerical variables and **loan** and **housing** from categorical features. Next will be to create generic and group specific models and profile a target group.

3 Modelling

3.1 Setup

Let us load the data and libraries.

```r
library(stats)
library(glmnet)
library(rpart)
library(rpart.plot)
library(pROC)
library(corrplot)
library(car)

source(here::here("R", "constants.R"))
source(here::here("R", "load_data.R"))

data <- load_bank_data()
head(data)
```

```
##   age         job marital education default balance housing loan  contact day
## 1  30  unemployed married   primary      no    1787      no   no cellular  19
## 2  33    services married secondary      no    4789     yes  yes cellular  11
## 3  35  management  single  tertiary      no    1350     yes   no cellular  16
## 4  30  management married  tertiary      no    1476     yes  yes  unknown   3
## 5  59 blue-collar married secondary      no       0     yes   no  unknown   5
## 6  35  management  single  tertiary      no     747      no   no cellular  23
##   month duration campaign pdays previous poutcome  y
## 1   oct       79        1    -1        0  unknown no
## 2   may      220        1   339        4  failure no
## 3   apr      185        1   330        1  failure no
## 4   jun      199        4    -1        0  unknown no
## 5   may      226        1    -1        0  unknown no
## 6   feb      141        2   176        3  failure no
```

We will also define a generic function to run cross-validation. That way we will estimate accuracy on new data, we also collect all predictions to construct confusion matrix and Receiver Operator Curve. We will display AUC ROC, accuracy, f1 score, precision and recall. We model data to better profile, so we choose the metric of performance to be precision, as it is a measure of confidence of model. Nevertheless precision can be abused, so we will also keep an eye on accuracy and f1 score.

```r
cv_evaluate_model <- function(model_func, predict_func, weights = NULL, k = 5, seed = 42, arg_data=data
      set.seed(seed)

      n <- nrow(data)
      folds <- sample(rep(1:k, length.out = n))

      y_true_all <- c()
      y_pred_all <- c()
      y_prob_all <- c()

      for (i in 1:k) {
          train_idx <- which(folds != i)
          test_idx  <- which(folds == i)

          train_data <- arg_data[train_idx, ]
          test_data  <- arg_data[test_idx, ]
          model <- NULL
          if(!is.null(weights)){
            train_weights <- weights[train_idx]
            model <- model_func(y ~ ., data=train_data, weights=train_weights, ...)
```

```
        }
        else {
          model <- model_func(y ~ ., data=train_data, ...)
        }


        prob <- predict_func(model, test_data)
        pred <- ifelse(prob > 0.5, "yes", "no")

        y_true_all <- c(y_true_all, test_data[['y']])
        y_pred_all <- c(y_pred_all, pred)
        y_prob_all <- c(y_prob_all, prob)
    }

    y_true_all <- factor(ifelse(y_true_all == 1, 'no', 'yes'), levels = c("no", "yes"))
    y_pred_all <- factor(y_pred_all, levels = c("no", "yes"))

    cm <- table(Predicted = y_pred_all, Actual = y_true_all)

    tp <- cm["yes", "yes"]
    tn <- cm["no", "no"]
    fp <- cm["yes", "no"]
    fn <- cm["no", "yes"]

    accuracy <- (tp + tn) / sum(cm)
    precision <- tp / (tp + fp)
    recall <- tp / (tp + fn)
    f1 <- 2 * precision * recall / (precision + recall)


    corrplot(
        cm,
        is.corr = F, method = "color", tl.col = "black", addCoef.col = "black",
        xlab = "Ground Truth", ylab = "Prediction", mar=c(3,0,3,0),
        title = "Confusion Matrix", cl.pos = 'n', col=c('bisque', 'lightblue')
    )
    mtext("Ground Truth", side = 3, line=1, cex = 1.2)
    mtext("Predictions", side = 2, line=0, cex = 1.2)

    roc_obj <- roc(y_true_all, y_prob_all, levels=c("no", "yes"), direction='<')
    plot(roc_obj, col = "red", main = "ROC Curve")

    legend(
        "bottomright",
        legend = c(
            sprintf("AUC = %.3f", auc(roc_obj)),
            sprintf("F1 = %.3f", f1),
            sprintf("Accuracy = %.3f", accuracy),
            sprintf("Precision = %.3f", precision),
            sprintf("Recall = %.3f", recall)
        )
    )
}
```

```
glm_predict_func <- function(model, newdata) predict(model, newdata, type = "response")
rpart_predict_func = function(model, newdata) predict(model, newdata, type = "prob")[, "yes"]
```

Let us define weights as inverse of classes' rates. that way both classes' weights sum up to the same number.

```
weights <- as.double(data$y)
pos_rate <- mean(data$y == 'yes')
weights[data$y == 'yes'] <- 1 / pos_rate
weights[data$y == 'no']  <- 1 / (1 - pos_rate)
```

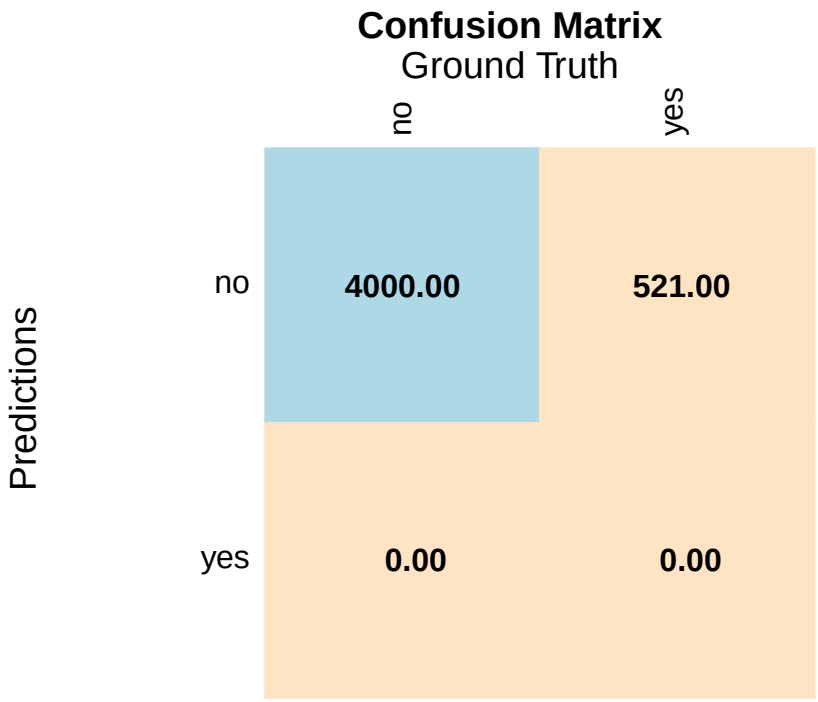
3.2 Baseline Models

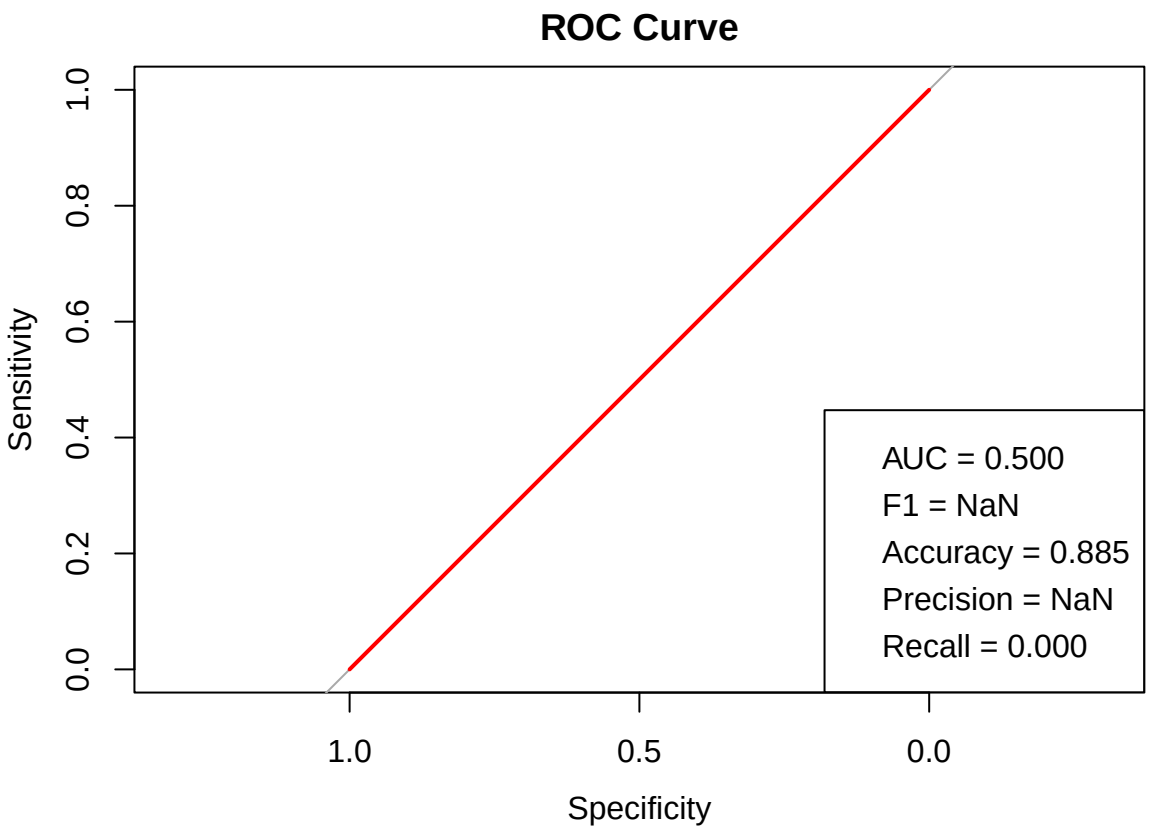
We will create dummy estimator, that predicts the most frequent class, logistic regression and decision tree. Then we will proceed by feature selection and engineering. We hope that we will be able to increase both precision and interpretability of models.

3.2.1 Dummy Estimator

Due to class imbalance dummy estimator will have high accuracy.

```
dummy_model <- function(fml, data, ...) {
    data$y
}

dummy_predict <- function(train_labels, test_data) {
    majority_class <- ifelse(names(sort(table(train_labels), decreasing = TRUE))[1] == 'no', 0, 1)
    rep(majority_class, nrow(test_data))
}
cv_evaluate_model(dummy_model, dummy_predict)
```


Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
4000.00
521.00
0.00
0.00

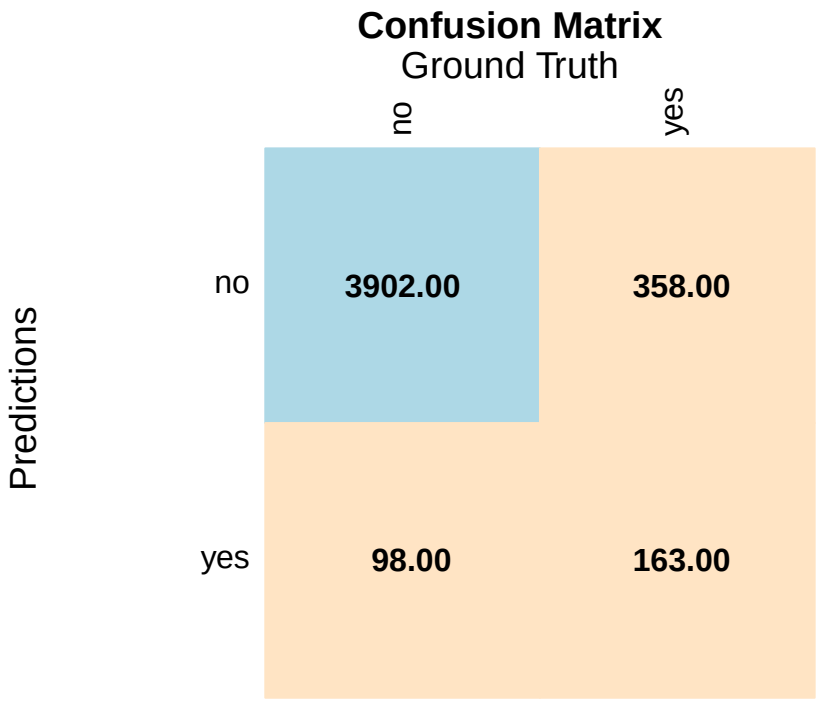


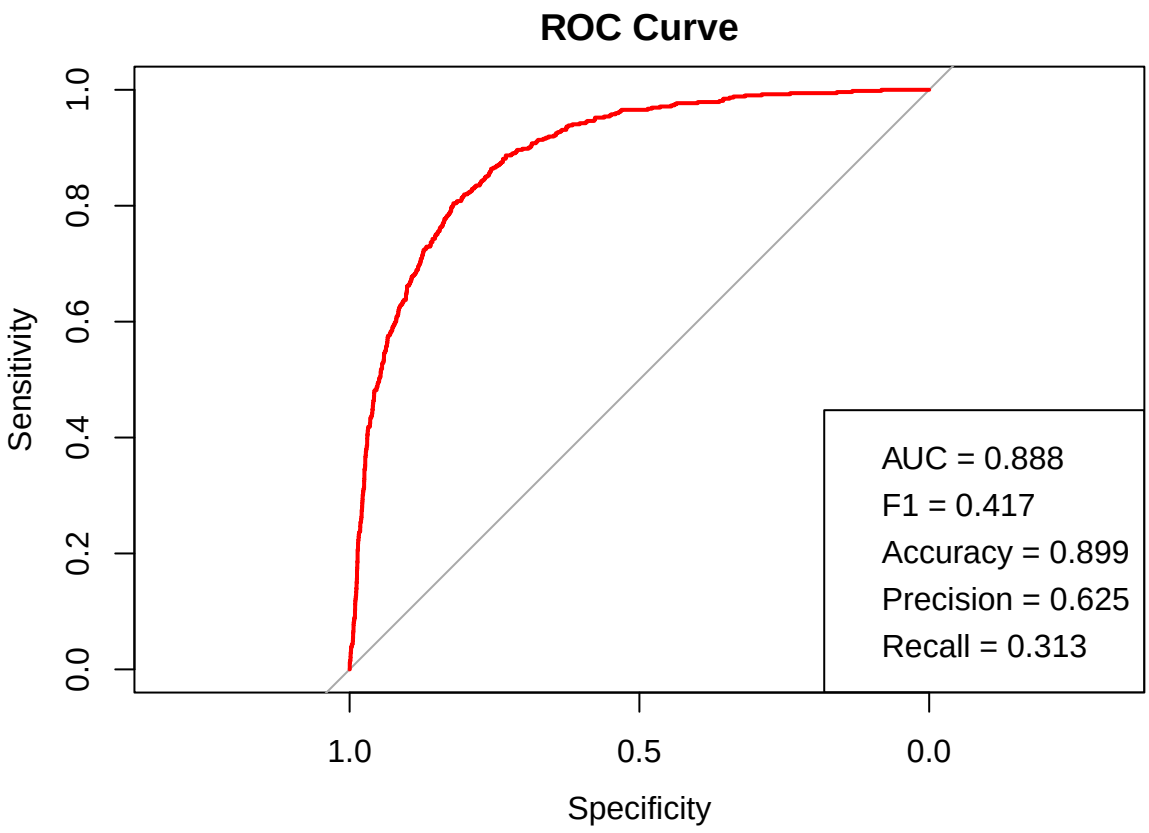
We see that accuracy matches negative rate, and precision and f1 are undefined.

3.2.2 Logistic Regression

We will focus on logistic regression, because it gives a clear interpretability during profiling: positive coefficients correspond to better likelihood of person subscribing.

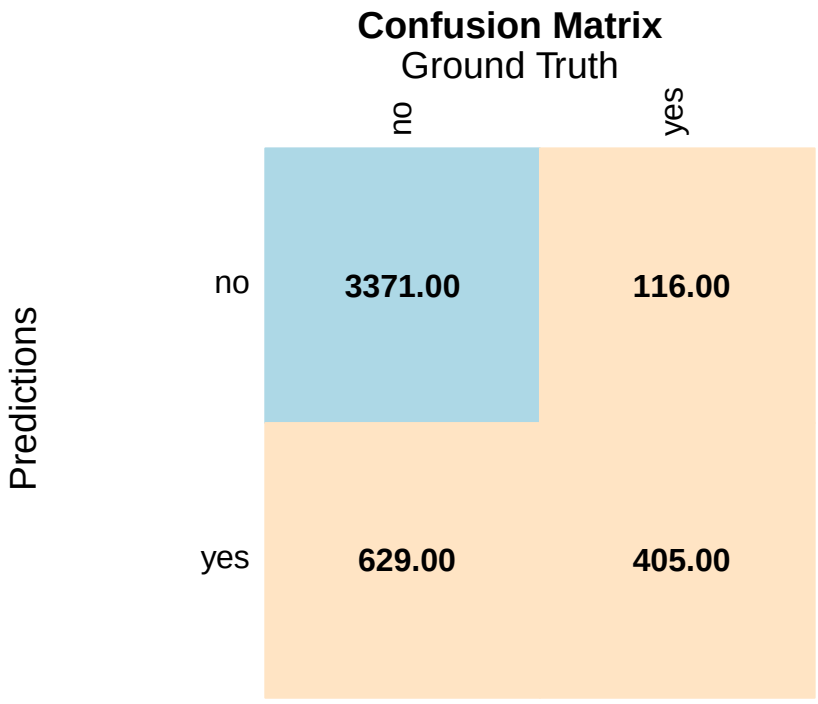
```
cv_evaluate_model(glm, glm_predict_func, family=binomial)
```

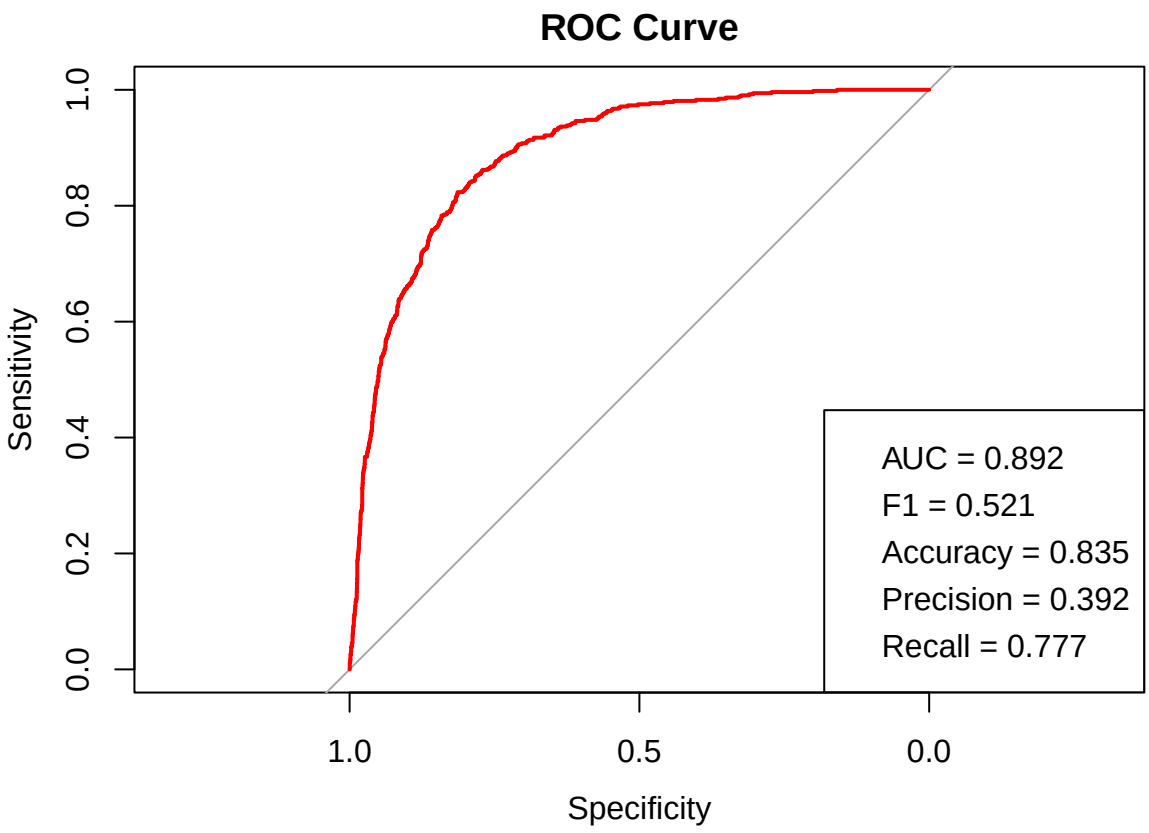

Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
3902.00
358.00
98.00
163.00



Log. regression strongly prefers negative class, which results in recall of 0.313. Nonetheless accuracy is higher negative rate and precision is greater than 0.625. Let us check weighted model.

```
cv_evaluate_model(glm, glm_predict_func, family=binomial, weights=weights)
```


Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
3371.00
116.00
629.00
405.00

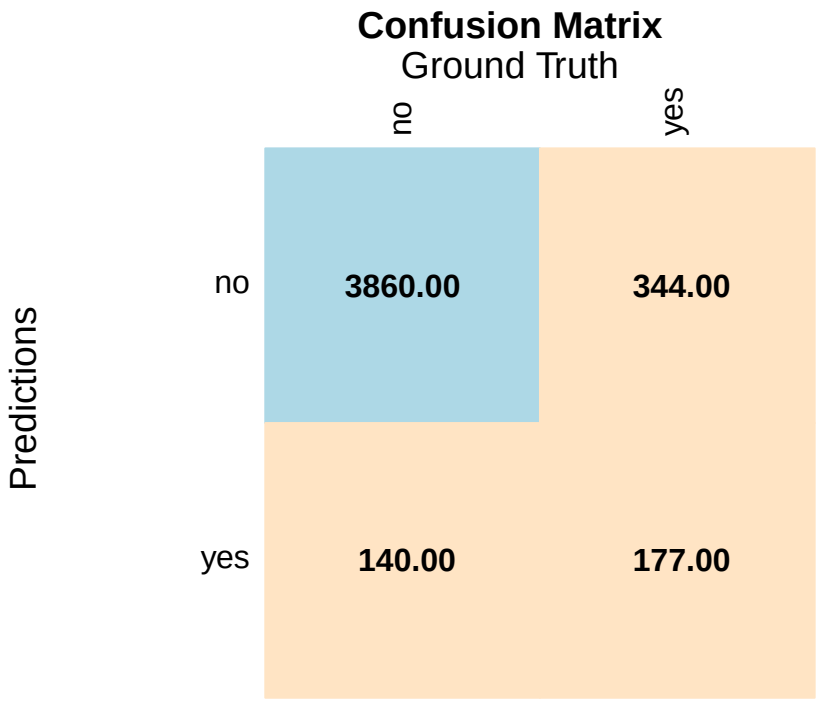


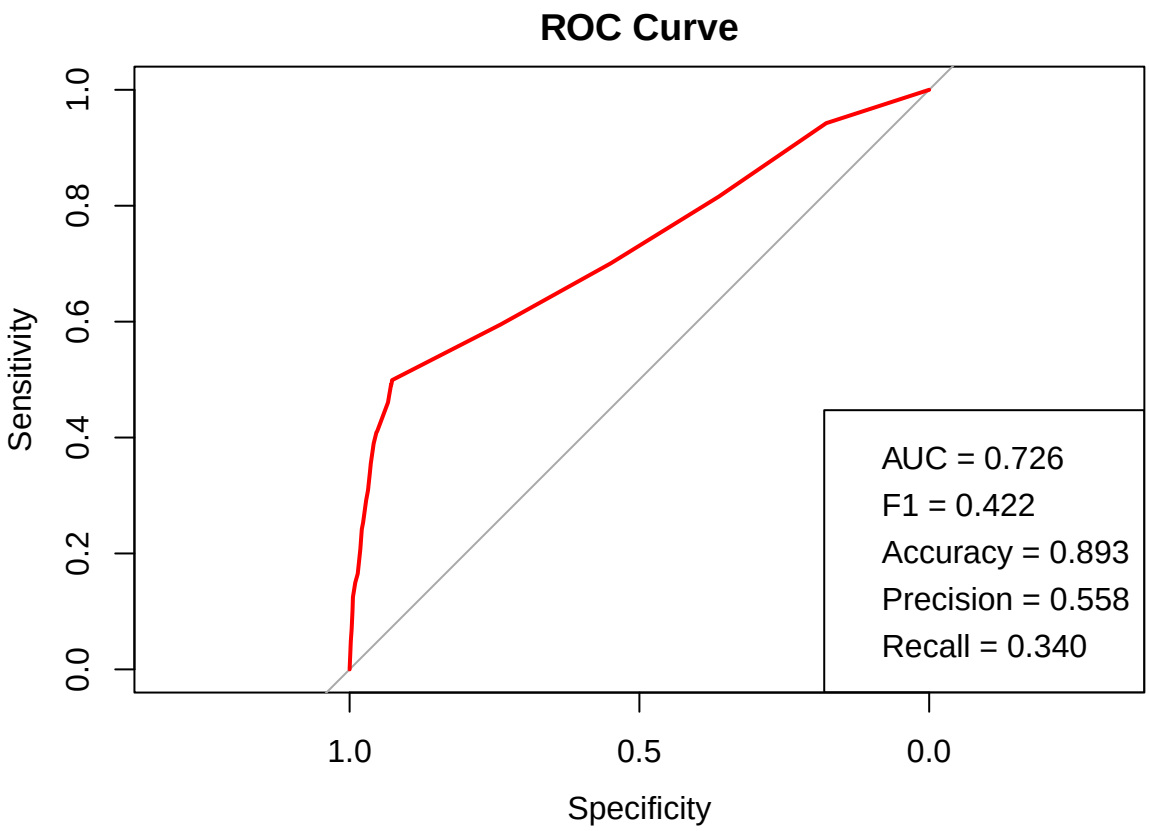
By weighting our samples we have introduce heavy bias towards positive class, so now precision suffers, while recall is up, which results in higher f1 score. This is not the strategy that we try to follow, though it is still important to look at balanced case.

3.2.3 Decision Tree Classifier

Our motivation for decision tree classifier is later rule extraction from it. We do not expect it to perform better than linear model, though we hope to see decent result.

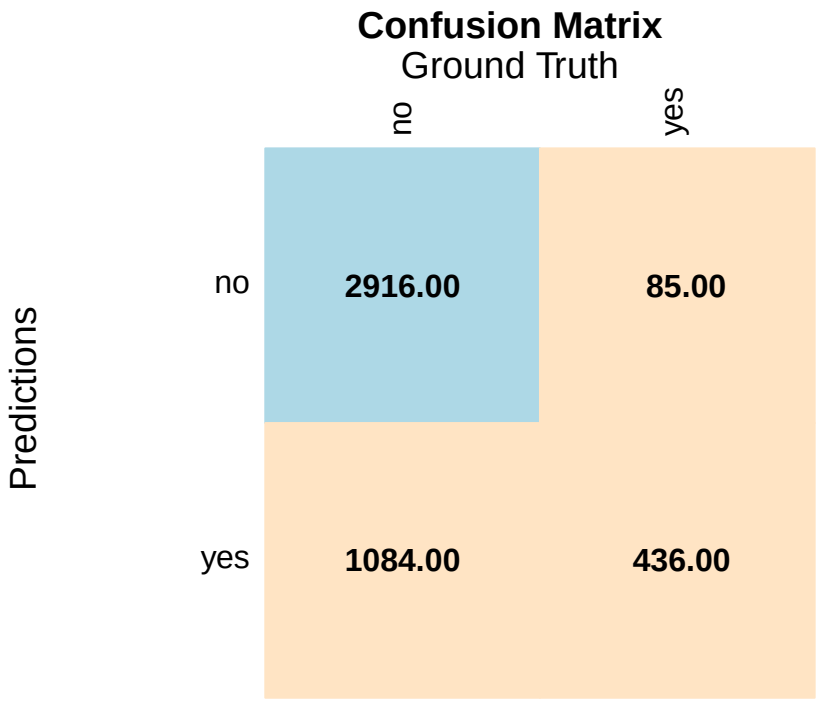
```
cv_evaluate_model(rpart, rpart_predict_func, maxdepth=3, cp=0)
```

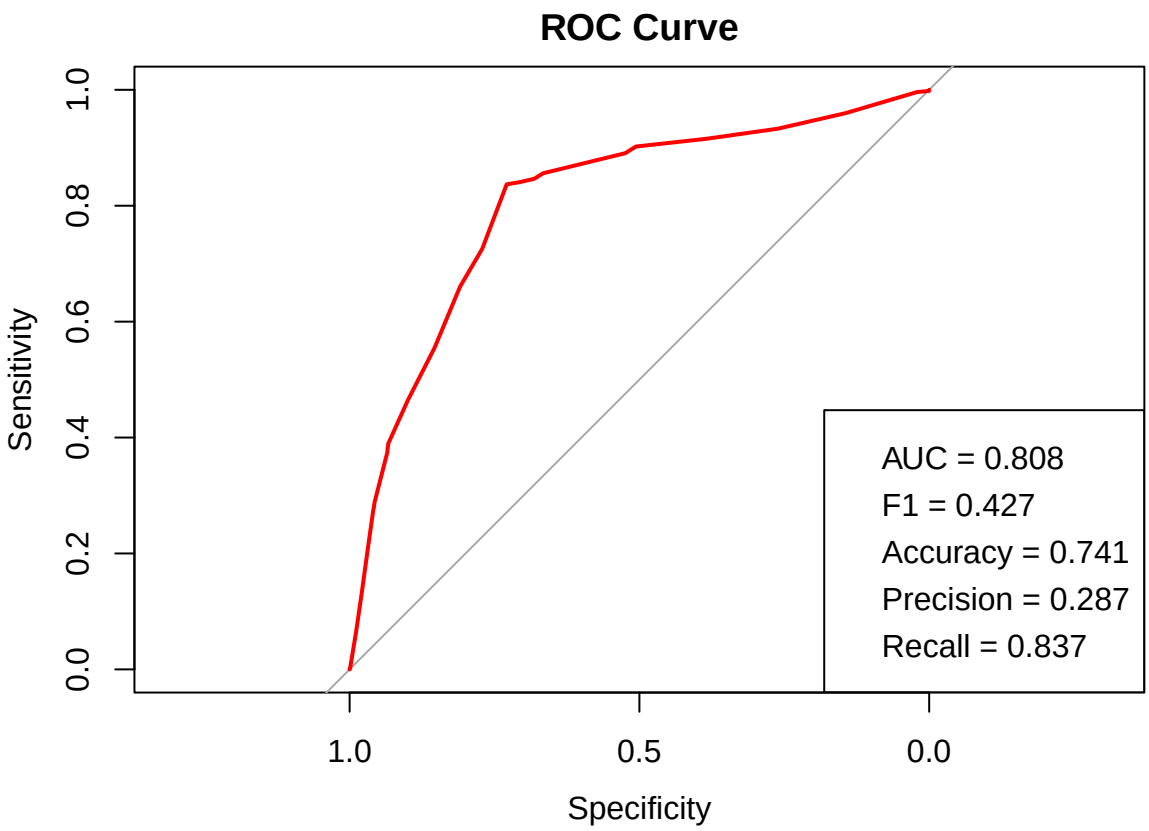

Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
3860.00
344.00
140.00
177.00



ROC has a sharp angle near the bottom, that indicates, that the trees were in favor of negative class. This is in agreement with low recall values. Precision is lower then the one of unweighted logistic regression, though it still presents some confidence.

```
cv_evaluate_model(rpart, rpart_predict_func, weights = weights, maxdepth=3, cp=0)
```


Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
2916.00
85.00
1084.00
436.00



The sharp angle on ROC moved upward, the model prefers positive class, recall is the highest of all the models we have discussed, while precision is the lowest. The result is very similar to the weighted regularized log regression's one.

3.3 Feature Selection and Engineering

During EDA and statistical testing we have confronted several features as being useless (namely **default** and **pdays**). In this section we will use LASSO and greedy step AIC algorithms to find a subset of features that manifests in the best precision. If manual intervention will be needed we will engineer the data ourselves.

3.3.1 LASSO

LASSO is a way to regularize linear models by L1 norm, resulting in sparse solutions. Those features that have zero coefficients can be safely discarded. We will use `glmnet` package LASSO implementation.

```
X <- model.matrix(y ~ ., data = data)[, -1]

y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se')
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")
```

```
print(coefs)
```

```
##                    lambda.1se
## (Intercept)      -2.716503895
## age               0.000000000
## jobblue-collar   -0.082114573
## jobentrepreneur   0.000000000
## jobhousemaid      0.000000000
## jobmanagement     0.000000000
## jobretired        0.352548565
## jobself-employed  0.000000000
## jobservices       0.000000000
## jobstudent        0.125039119
## jobtechnician     0.000000000
## jobunemployed     0.000000000
## jobunknown        0.000000000
## maritalmarried   -0.099122687
## maritalsingle     0.000000000
## education.L       0.103009826
## education.Q       0.000000000
## education.C       0.000000000
## defaultyes        0.000000000
## balance           0.000000000
## housingyes       -0.232198347
## loanyes          -0.229418238
## contacttelephone  0.000000000
## contactunknown   -0.610381526
## day               0.000000000
## month.L           0.000000000
## month.Q           0.000000000
## month.C           0.000000000
## month^4          -0.918196201
## month^5           0.013622038
## month^6           0.370186286
## month^7           0.000000000
## month^8           0.614578758
## month^9           0.000000000
## month^10          0.000000000
## month^11          0.199691467
## duration          0.003405757
## campaign          0.000000000
## pdays             0.000000000
## previous          0.000000000
## poutcomeother     0.068717871
## poutcomesuccess   2.105386702
## poutcomeunknown  -0.178414841
```

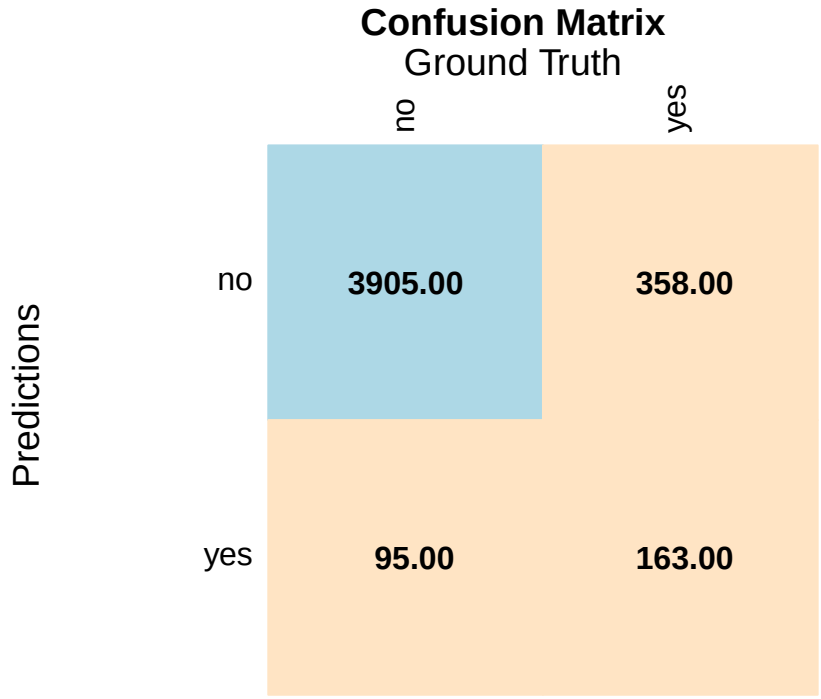
```
non_zero_vars
```

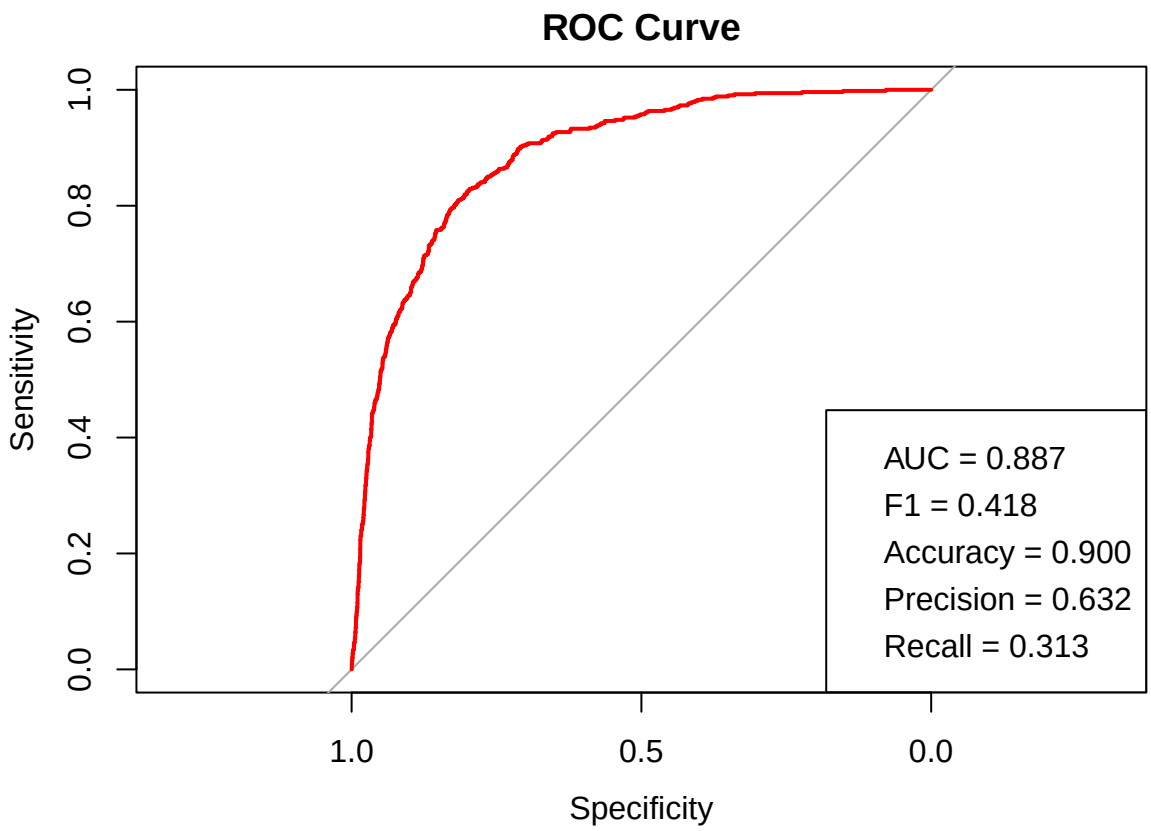
```
##  [1] "jobblue-collar"  "jobretired"      "jobstudent"      "maritalmarried"
##  [5] "education.L"     "housingyes"      "loanyes"         "contactunknown"
##  [9] "month^4"         "month^5"         "month^6"         "month^8"
## [13] "month^11"        "duration"        "poutcomeother"   "poutcomesuccess"
## [17] "poutcomeunknown"
```

Features that we have described as being weak were indeed zeroed down by lasso, while features like **housing** and **duration** remained. The **month** feature got expanded in 12 features (corresponding to 12 degrees of freedom) and 5 of those subfeatures have not only non-zero coef. but they have high absolute value.

Let us test selected features using CV.

```
lasso_data <- data[,c("y", "job", "marital", "education", "housing", "loan", "contact", "month", "durati
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```





We have been able to perform better with 90% accuracy and 63.2% precision. Thus we can say that our model overfitted the data.

Now we will try to the model trained without **month** feature.

```
build_formula <- function(arg_data, target = 'y', exclude = NULL) {
  predictors <- setdiff(names(arg_data), c(target, exclude))
  as.formula(paste(target, "~", paste(predictors, collapse = " + ")))
}

X <- model.matrix(build_formula(data, exclude = 'month'), data = data)[, -1]  # remove intercept

y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se')  # or use s = "lambda.min" if using cv.glmnet
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")


print(coefs)
```

```
##                   lambda.1se
## (Intercept)      -2.806758311
```

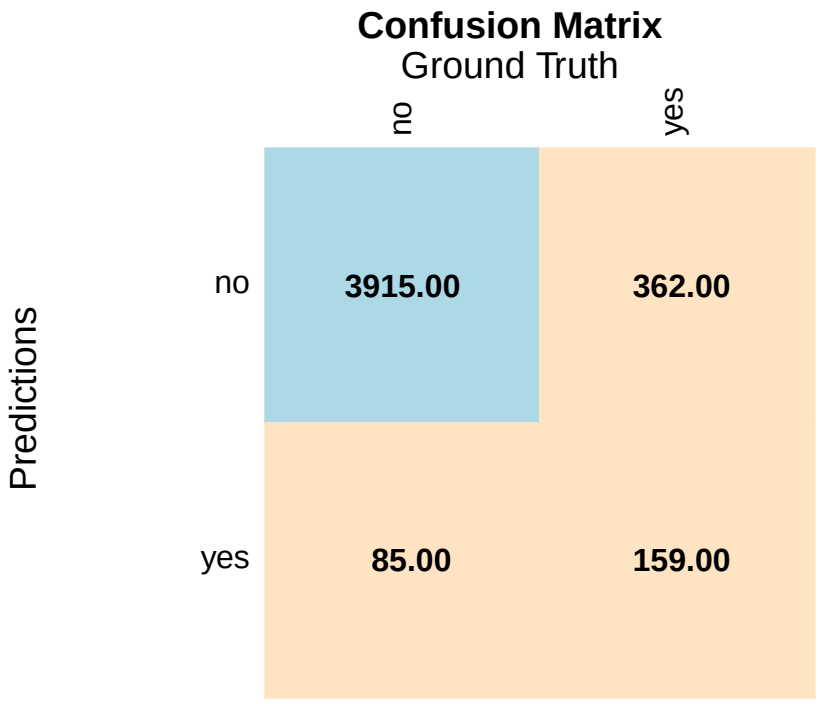
```
## age               0.000000000
## jobblue-collar   -0.034282217
## jobentrepreneur   0.000000000
## jobhousemaid      0.000000000
## jobmanagement     0.000000000
## jobretired        0.227313388
## jobself-employed  0.000000000
## jobservices       0.000000000
## jobstudent        0.000000000
## jobtechnician     0.000000000
## jobunemployed     0.000000000
## jobunknown        0.000000000
## maritalmarried   -0.003312757
## maritalsingle     0.000000000
## education.L       0.000000000
## education.Q       0.000000000
## education.C       0.000000000
## defaultyes        0.000000000
## balance           0.000000000
## housingyes       -0.234323072
## loanyes          -0.120390968
## contacttelephone  0.000000000
## contactunknown   -0.581197216
## day               0.000000000
## duration          0.003165426
## campaign          0.000000000
## pdays             0.000000000
## previous          0.000000000
## poutcomeother     0.000000000
## poutcomesuccess   2.082604619
## poutcomeunknown  -0.192419186
```

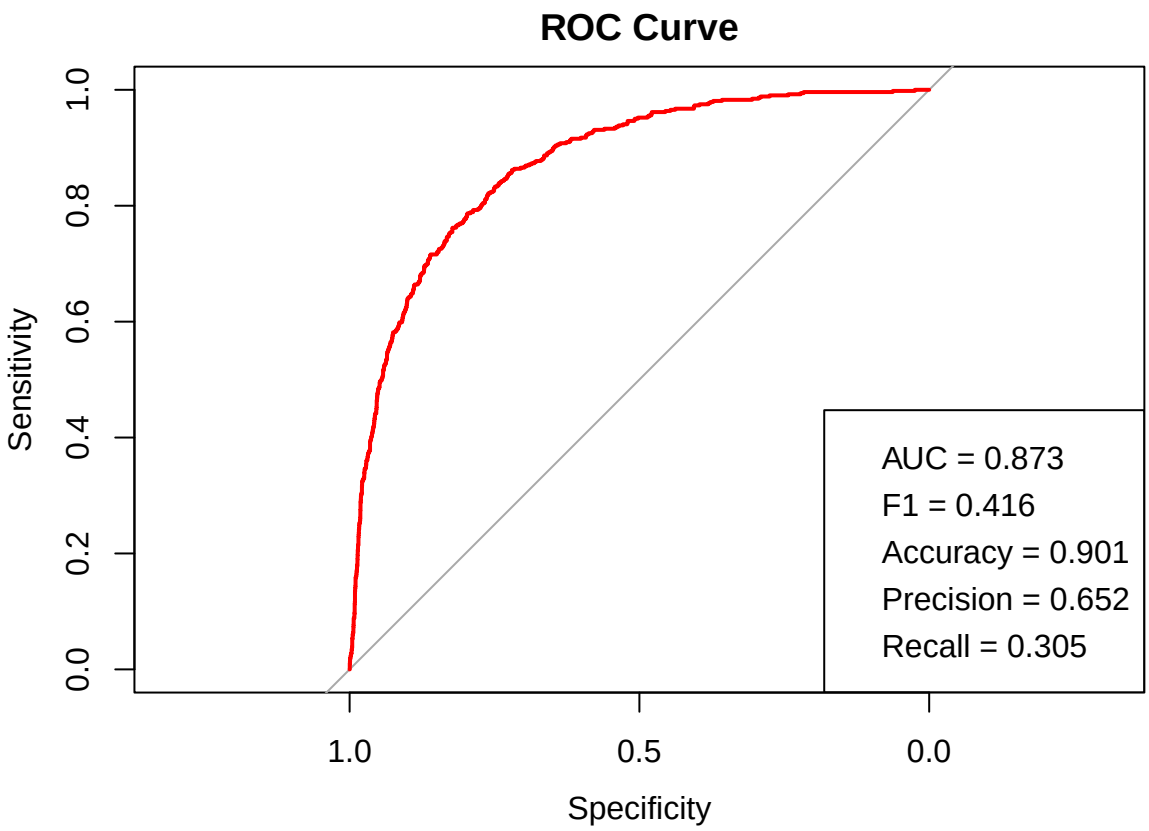
```
non_zero_vars
```

```
## [1] "jobblue-collar"  "jobretired"      "maritalmarried"  "housingyes"
## [5] "loanyes"         "contactunknown"  "duration"        "poutcomesuccess"
## [9] "poutcomeunknown"
```

An interesting observation is that without **month**, **education** and **day** features also disappeared.

```
lasso_data <- data[,c("y", "job", "marital", "housing", "loan", "contact", "duration", "poutcome")]
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```


Confusion Matrix
Ground Truth
no
yes
Predictions
no
yes
3915.00
362.00
85.00
159.00



We were able to bring accuracy and precision up once more. This time it was miniscule step, but nevertheless we obtained a better model.

3.3.2 Step with AIC heuristics

Next we will try to fined the best subset using greedy search with AIC heuristics.

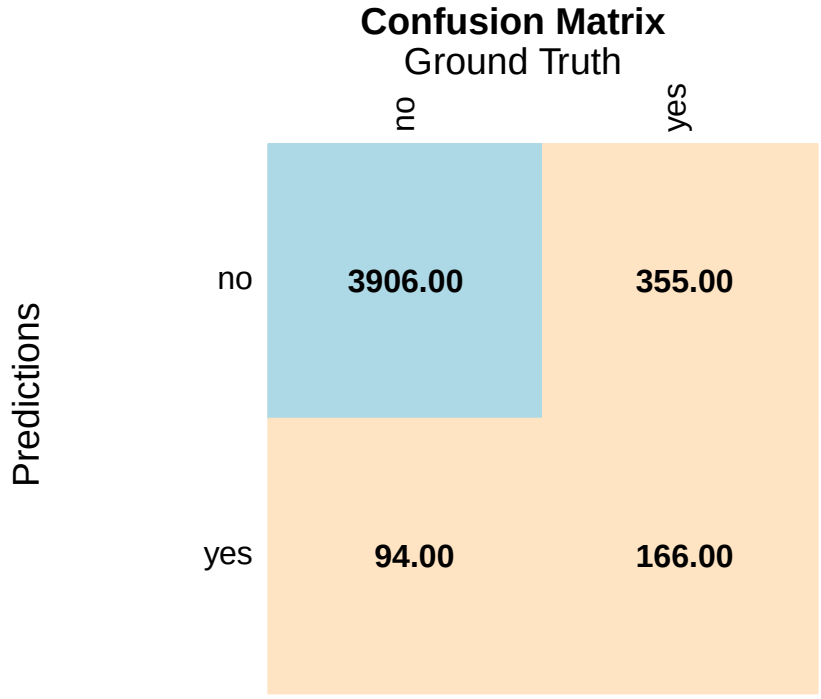
```
step(glm(y ~ ., data, family='binomial'), direction='both', trace=0)
```

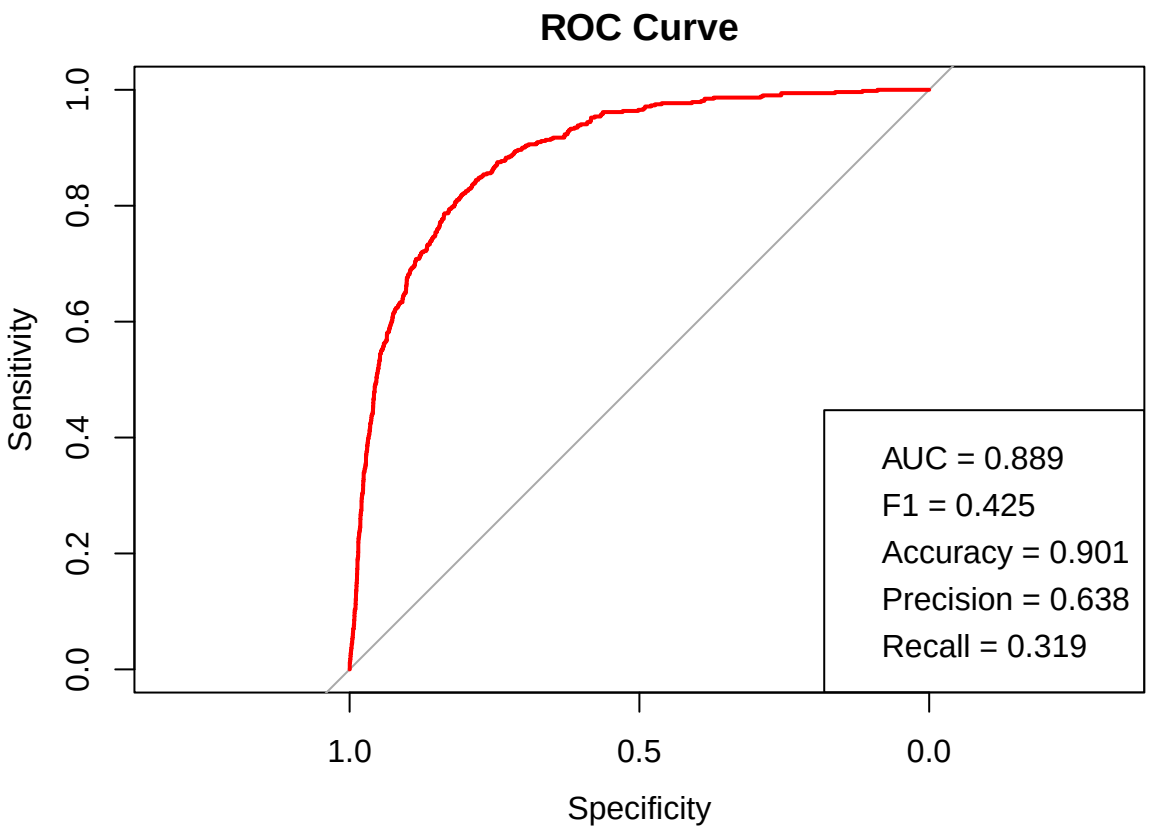
```
##
## Call:  glm(formula = y ~ marital + education + housing + loan + contact +
##     day + month + duration + campaign + poutcome, family = "binomial",
##     data = data)
##
## Coefficients:
##      (Intercept)    maritalmarried     maritalsingle       education.L
##        -2.573557         -0.510829         -0.318135          0.473929
##      education.Q       education.C       housingyes           loanyes
##        -0.014415          0.063460         -0.343417         -0.643090
## contacttelephone    contactunknown               day           month.L
##        -0.007710         -1.457777          0.016295          0.275732
##           month.Q           month.C           month^4           month^5
##        -0.432947          0.518895         -1.496841          0.848790
##           month^6           month^7           month^8           month^9
##         0.973761          0.746550          1.539579         -0.688630
##          month^10          month^11          duration          campaign
##        -0.048261          0.623386          0.004184         -0.072216
##    poutcomeother   poutcomesuccess   poutcomeunknown
```

```
##         0.481385          2.414545         -0.093110
##
## Degrees of Freedom: 4520 Total (i.e. Null);  4494 Residual
## Null Deviance:       3231
## Residual Deviance: 2196  AIC: 2250
```

Once again we see that **month**'s subfeatures get high coefficients. Step found a larger subset, than LASSO. Step is greedy, so it is hardered to find extreme solutions such as LASSO.

```
step_data <- data[,c("y", "marital", "education", "housing", "loan", "contact", "day", "month", "duratio
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





We were able to find a good model. It has high precision and accuracy. Let us get rid of **month** to compare to engineered LASSO model.

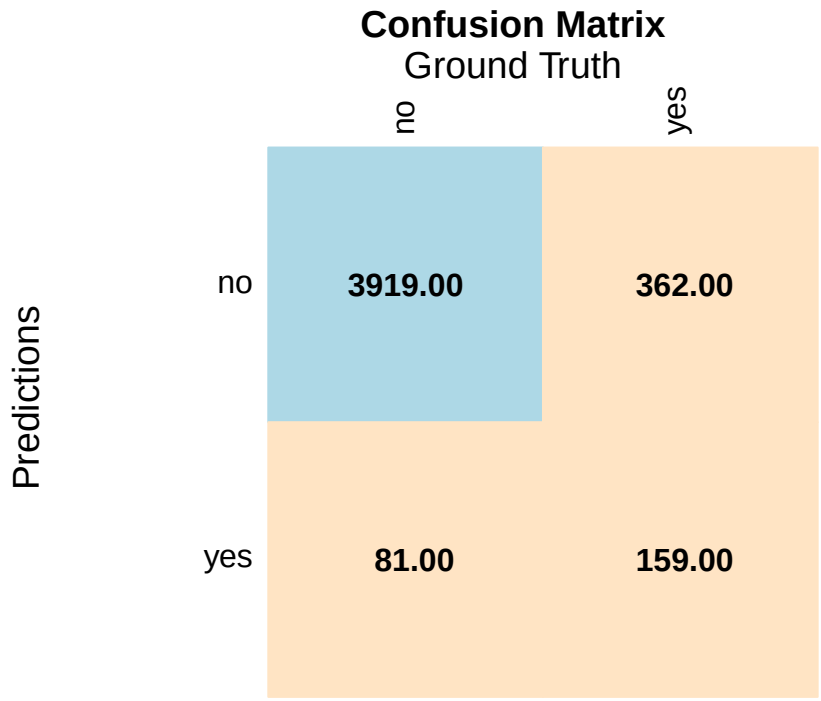
```
step(glm(y ~ ., data[,setdiff(names(data), c('month'))], family='binomial'), direction='both', trace=0)
```

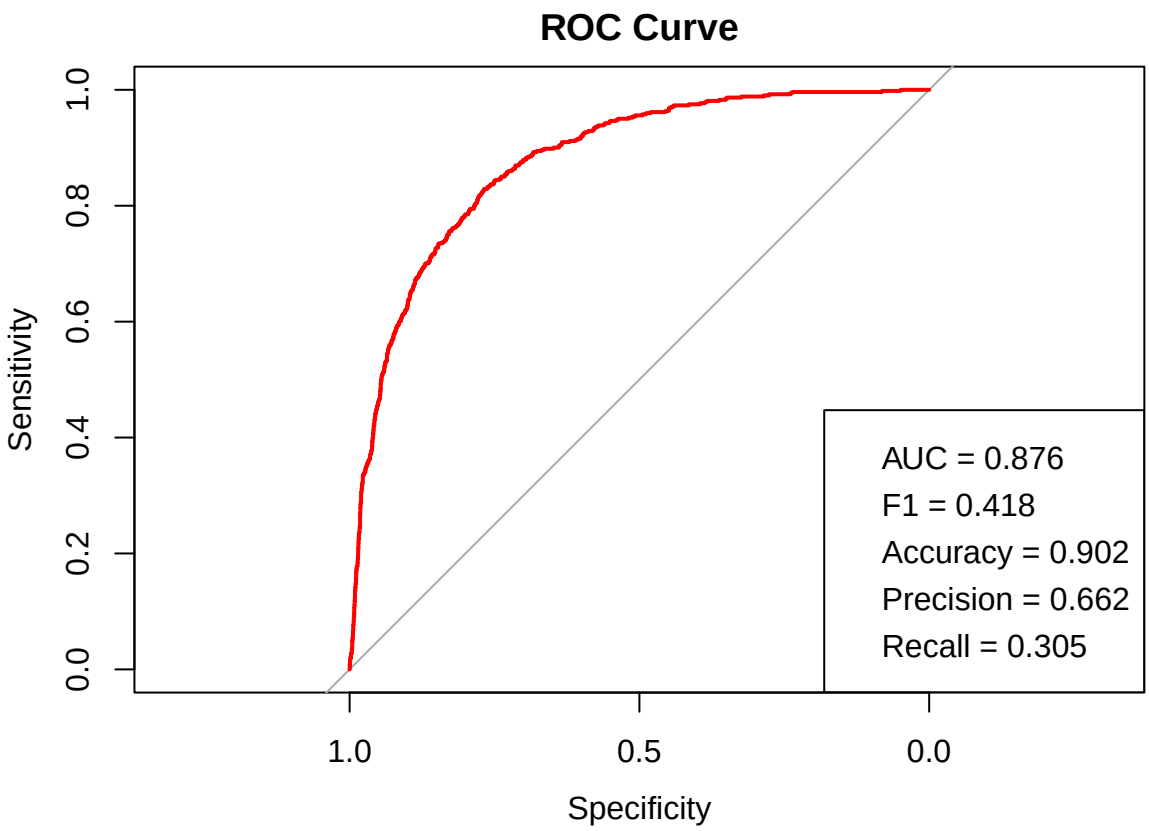
```
##
## Call:  glm(formula = y ~ job + marital + education + housing + loan +
##     contact + duration + campaign + poutcome, family = "binomial",
##     data = data[, setdiff(names(data), c("month"))])
##
## Coefficients:
##      (Intercept)    jobblue-collar   jobentrepreneur      jobhousemaid
##        -2.377736         -0.449450         -0.430007         -0.303290
##    jobmanagement        jobretired  jobself-employed       jobservices
##        -0.103964          0.605486         -0.234316         -0.228378
##       jobstudent     jobtechnician     jobunemployed        jobunknown
##         0.513997         -0.250985         -0.694997          0.467686
##   maritalmarried     maritalsingle       education.L       education.Q
##        -0.439144         -0.209409          0.487303         -0.030979
##      education.C        housingyes           loanyes  contacttelephone
##         0.099334         -0.480192         -0.757944         -0.041167
##   contactunknown          duration          campaign     poutcomeother
##        -1.096033          0.004039         -0.074999          0.477239
##  poutcomesuccess   poutcomeunknown
##         2.410096         -0.227528
##
## Degrees of Freedom: 4520 Total (i.e. Null);  4495 Residual
```

```
## Null Deviance:       3231
## Residual Deviance: 2278  AIC: 2330
```

This time only **day** was zeroed out, while **education** remained.

```
step_data <- data[,c("y", "job", "marital", "education", "housing", "loan", "contact", "duration", "camp
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





We obtain the best model yet with precision of 66%, though It has more features than LASSO selected.

3.3.3 Job Aggregation

All methods that we have used before showed that all **job** categories except *blue-collar* and *retired* have low or zero coefficients, so it will be meaningful to test whether we can use smaller model without loss of precision.

```
data.specific_job <- data
data.specific_job[!(data$job == 'blue-collar' | data$job == 'retired'),"job"] <- "unknown"
head(data.specific_job)
```

```
##   age         job marital education default balance housing loan  contact day
## 1  30     unknown married   primary      no    1787      no   no cellular  19
## 2  33     unknown married secondary      no    4789     yes  yes cellular  11
## 3  35     unknown  single  tertiary      no    1350     yes   no cellular  16
## 4  30     unknown married  tertiary      no    1476     yes  yes  unknown   3
## 5  59 blue-collar married secondary      no       0     yes   no  unknown   5
## 6  35     unknown  single  tertiary      no     747      no   no cellular  23
##   month duration campaign pdays previous poutcome  y
## 1   oct       79        1    -1        0  unknown no
## 2   may      220        1   339        4  failure no
## 3   apr      185        1   330        1  failure no
## 4   jun      199        4    -1        0  unknown no
## 5   may      226        1    -1        0  unknown no
## 6   feb      141        2   176        3  failure no
```

Firstly we will run LASSO with joint jobs categories.

```r
X <- model.matrix(build_formula(data.specific_job, exclude = 'month'), data = data)[, -1]  # remove int

y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se')  # or use s = "lambda.min" if using cv.glmnet
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")


print(coefs)
```

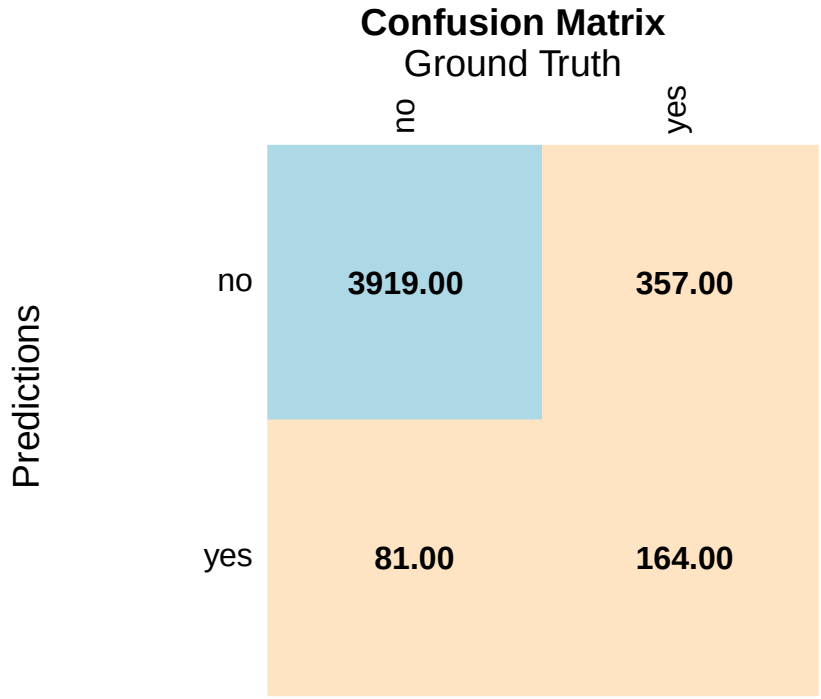
```
##                    lambda.1se
## (Intercept)      -2.806758311
## age               0.000000000
## jobblue-collar   -0.034282217
## jobentrepreneur   0.000000000
## jobhousemaid      0.000000000
## jobmanagement     0.000000000
## jobretired        0.227313388
## jobself-employed  0.000000000
## jobservices       0.000000000
## jobstudent        0.000000000
## jobtechnician     0.000000000
## jobunemployed     0.000000000
## jobunknown        0.000000000
## maritalmarried   -0.003312757
## maritalsingle     0.000000000
## education.L       0.000000000
## education.Q       0.000000000
## education.C       0.000000000
## defaultyes        0.000000000
## balance           0.000000000
## housingyes       -0.234323072
## loanyes          -0.120390968
## contacttelephone  0.000000000
## contactunknown   -0.581197216
## day               0.000000000
## duration          0.003165426
## campaign          0.000000000
## pdays             0.000000000
## previous          0.000000000
## poutcomeother     0.000000000
## poutcomesuccess   2.082604619
## poutcomeunknown  -0.192419186
```

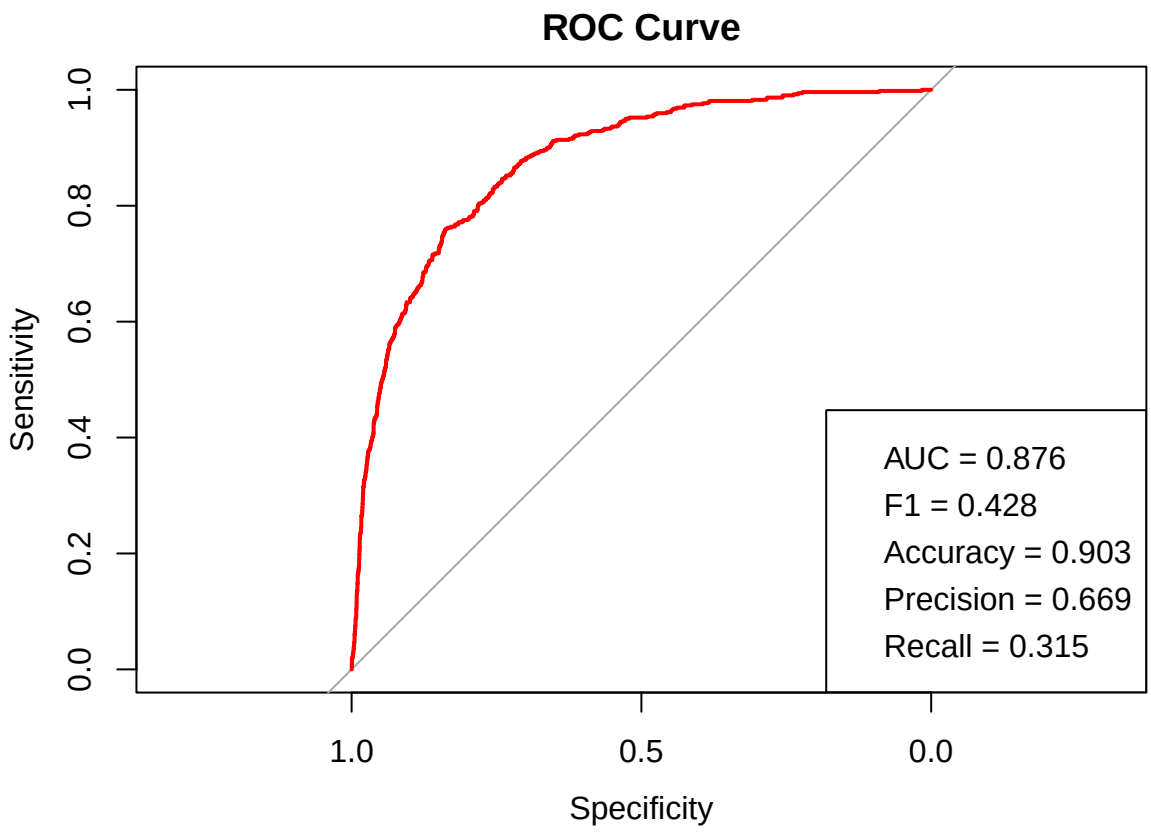
```r
non_zero_vars
```

```
## [1] "jobblue-collar"  "jobretired"      "maritalmarried"  "housingyes"
## [5] "loanyes"         "contactunknown"  "duration"        "poutcomesuccess"
## [9] "poutcomeunknown"
```

The feature selection resulted in the very same features as before, let us run CV.

```
lasso_data <- data.specific_job[,c("y", "job", "marital", "housing", "loan", "contact", "duration", "pou
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```





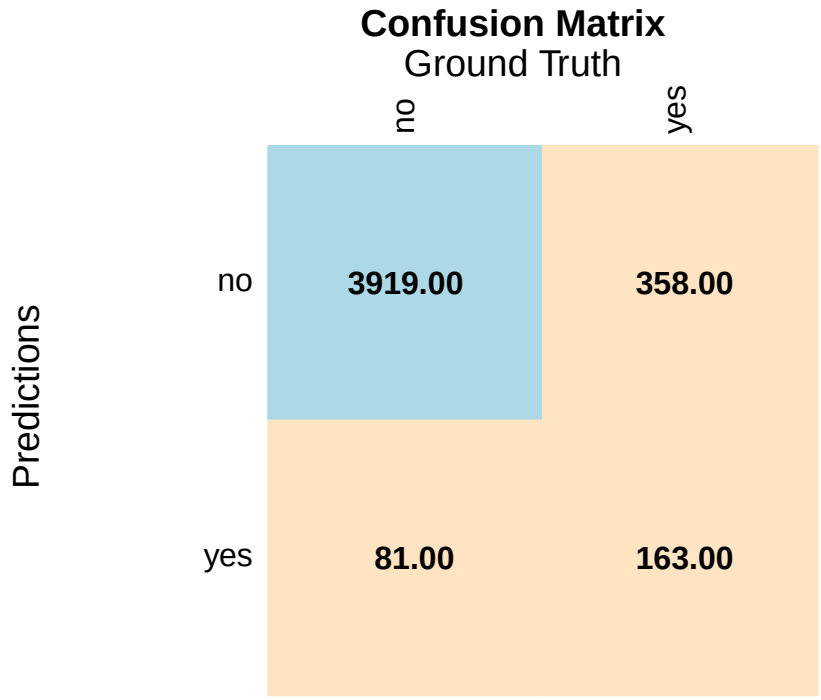
We have also obtained better results regarding precision. Let us run the same procedure with step function.

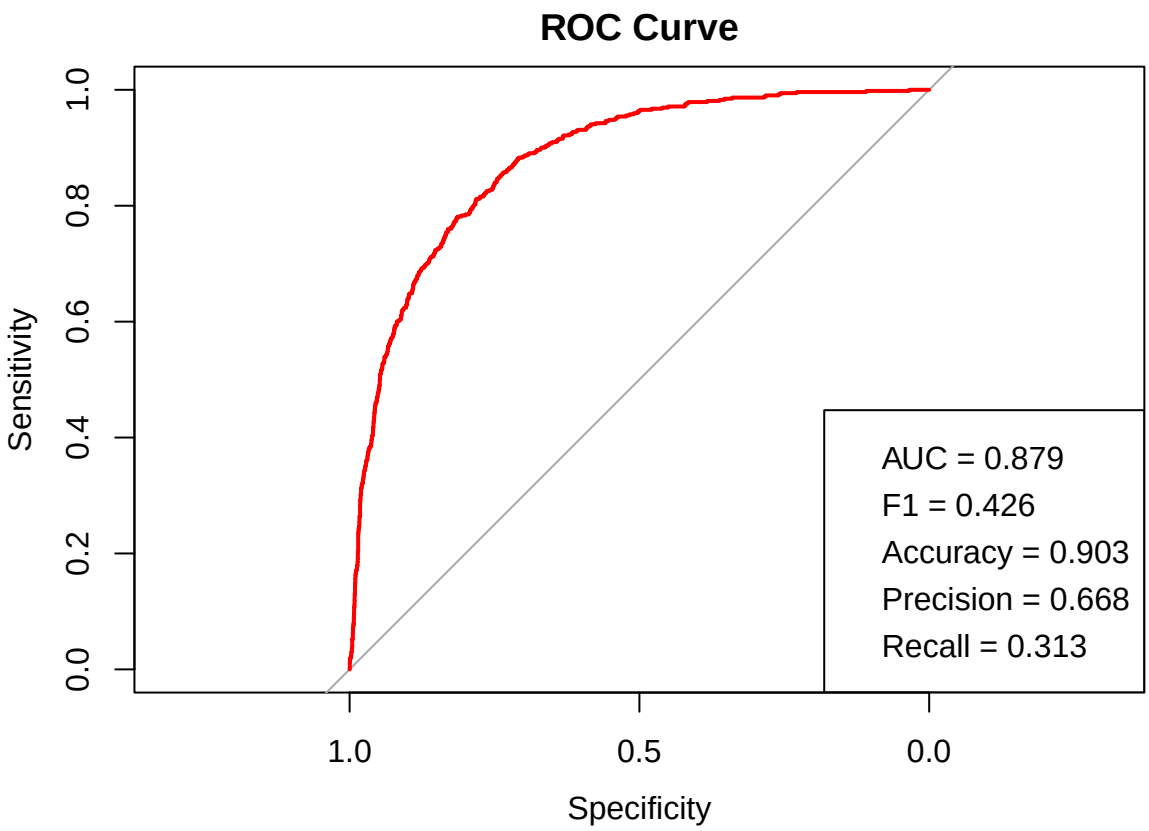
```
step(glm(y ~ ., data=data.specific_job[,setdiff(names(data), c('month'))] , family = binomial), directi
```

```
##
## Call:  glm(formula = y ~ job + marital + education + housing + loan +
##     contact + duration + campaign + poutcome, family = binomial,
##     data = data.specific_job[, setdiff(names(data), c("month"))])
##
## Coefficients:
##      (Intercept)        jobretired        jobunknown    maritalmarried
##        -2.789462          1.052547          0.274764         -0.431151
##    maritalsingle       education.L       education.Q       education.C
##        -0.149293          0.428147          0.038811          0.057484
##       housingyes           loanyes  contacttelephone    contactunknown
##        -0.493092         -0.763278         -0.042073         -1.104398
##         duration          campaign     poutcomeother   poutcomesuccess
##         0.004008         -0.074595          0.495816          2.416749
##  poutcomeunknown
##        -0.237666
##
## Degrees of Freedom: 4520 Total (i.e. Null);  4504 Residual
## Null Deviance:       3231
## Residual Deviance: 2289  AIC: 2323
```

The situation remains the same, the features chosen by step has not changed.

```
step_data <- data.specific_job[,c("y", "job", "marital", "education", "housing", "loan", "contact", "du
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





This model also beats its previous version, though precision is slightly lower.

3.4 Final model

After all feaute selection and engineering we choose LASSO model with joint **job** and excluded **month** feature.

```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + duration, data=data.specifi
summary(final_model)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##     poutcome + duration, family = binomial, data = data.specific_job)
##
## Coefficients:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -2.9046849  0.2794213 -10.395  < 2e-16 ***
## jobretired        1.1080491  0.2479450   4.469 7.86e-06 ***
## jobunknown        0.4148859  0.1630157   2.545  0.01093 *
## maritalmarried   -0.4384189  0.1666483  -2.631  0.00852 **
## maritalsingle    -0.1201637  0.1808508  -0.664  0.50641
## housingyes       -0.4952104  0.1190575  -4.159 3.19e-05 ***
## loanyes          -0.7790165  0.1914404  -4.069 4.72e-05 ***
## contacttelephone -0.1048519  0.2169909  -0.483  0.62895
## contactunknown   -1.1135127  0.1741227  -6.395 1.61e-10 ***
## poutcomeother     0.4736040  0.2553980   1.854  0.06369 .
## poutcomesuccess   2.3926894  0.2530984   9.454  < 2e-16 ***
```

```
## poutcomeunknown  -0.3034844  0.1707815  -1.777  0.07556 .
## duration          0.0039657  0.0001917  20.683  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3231.0  on 4520  degrees of freedom
## Residual deviance: 2305.4  on 4508  degrees of freedom
## AIC: 2331.4
##
## Number of Fisher Scoring iterations: 6
```

We see that *single* marital status, *telephone* contact and *other*, *unknown* previous outcome are unstable, their effect on the model is not too large, due to their small coefficients, so we will not confront this.

3.5 Assumption Verification

Logistic regression assumes (besides i.i.d. and categorical target) linearity of continuous logits, no outliers and no multicollinearity. We will check those assumptions with Box-Tidwell test, Cook's distance and GVIF metric.

3.5.1 Linearity of Continuous Variable Logits

We will run Box-Tidwell test that determines what power transformation makes logit linear, we test that power transformation has exponent of 1. The only continuous feature that we have is **duration**.

```
data.specific_job.numeric <- data.specific_job
data.specific_job.numeric$y <- ifelse(data.specific_job$y == "yes", 1, 0)
boxTidwell(y ~ duration, data = data.specific_job.numeric)
```

```
##  MLE of lambda Score Statistic (t)  Pr(>|t|)
##        0.79334             -4.4352 9.419e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## iterations =  5
```

We must reject the null hypothesis, we will apply the power transformation with MLE estimated exponent to **duration**.

```
data.specific_job.numeric$duration_transformed <- data.specific_job$duration ^ lambda
car::boxTidwell(y ~ duration_transformed, data = data.specific_job.numeric)
```

```
##  MLE of lambda Score Statistic (t) Pr(>|t|)
##        0.99984             -0.0022   0.9982
##
## iterations =  0
```

We have passed the test, let us examine fixed model.

```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration ^ lambda), data=
summary(final_model)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##     poutcome + I(duration^lambda), family = binomial, data = data.specific_job)
```

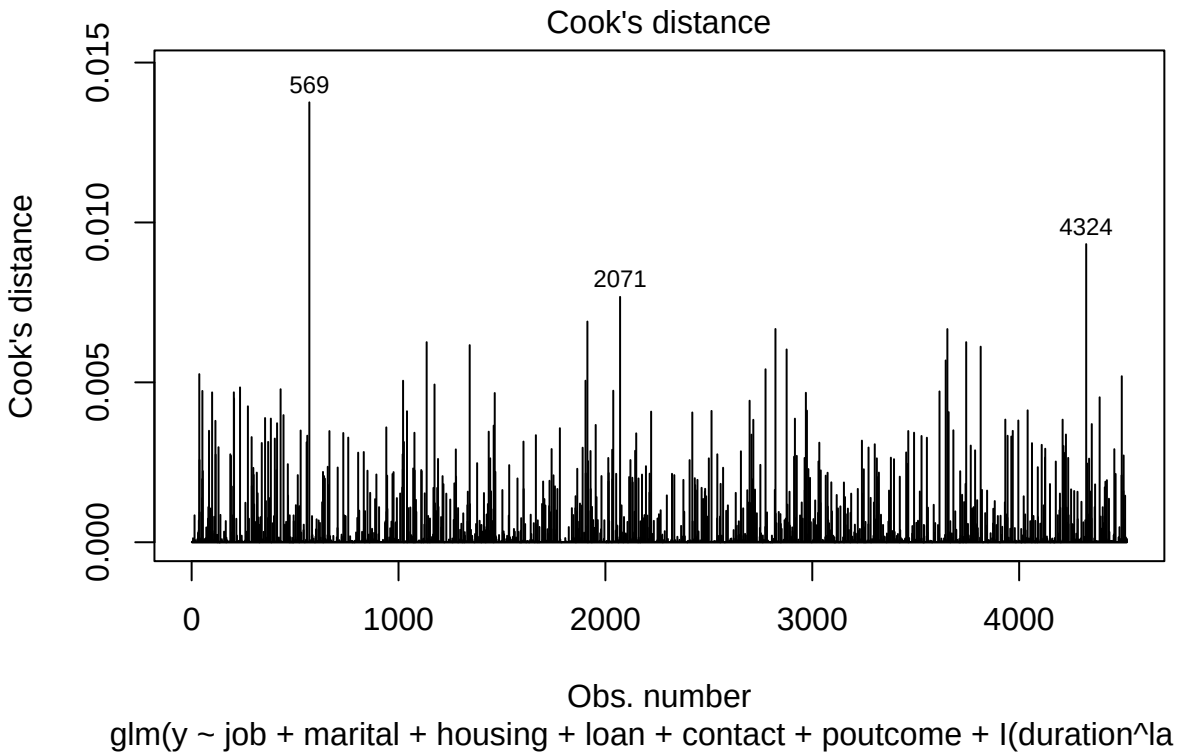
```
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -3.3835606  0.2872977 -11.777  < 2e-16 ***
## jobretired          1.1048991  0.2495903   4.427 9.56e-06 ***
## jobunknown          0.4246159  0.1629615   2.606  0.00917 **
## maritalmarried     -0.4343403  0.1678107  -2.588  0.00965 **
## maritalsingle      -0.1134125  0.1821536  -0.623  0.53353
## housingyes         -0.4986848  0.1196789  -4.167 3.09e-05 ***
## loanyes            -0.7780515  0.1913010  -4.067 4.76e-05 ***
## contacttelephone   -0.0828720  0.2191188  -0.378  0.70528
## contactunknown     -1.1037494  0.1734015  -6.365 1.95e-10 ***
## poutcomeother       0.4755413  0.2577277   1.845  0.06502 .
## poutcomesuccess     2.3874747  0.2555046   9.344  < 2e-16 ***
## poutcomeunknown    -0.3073699  0.1719614  -1.787  0.07387 .
## I(duration^lambda)  0.0185148  0.0008628  21.459  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3231.0  on 4520  degrees of freedom
## Residual deviance: 2270.4  on 4508  degrees of freedom
## AIC: 2296.4
##
## Number of Fisher Scoring iterations: 6
```

The coefficients did not change, but AIC decreased as well as deviances.

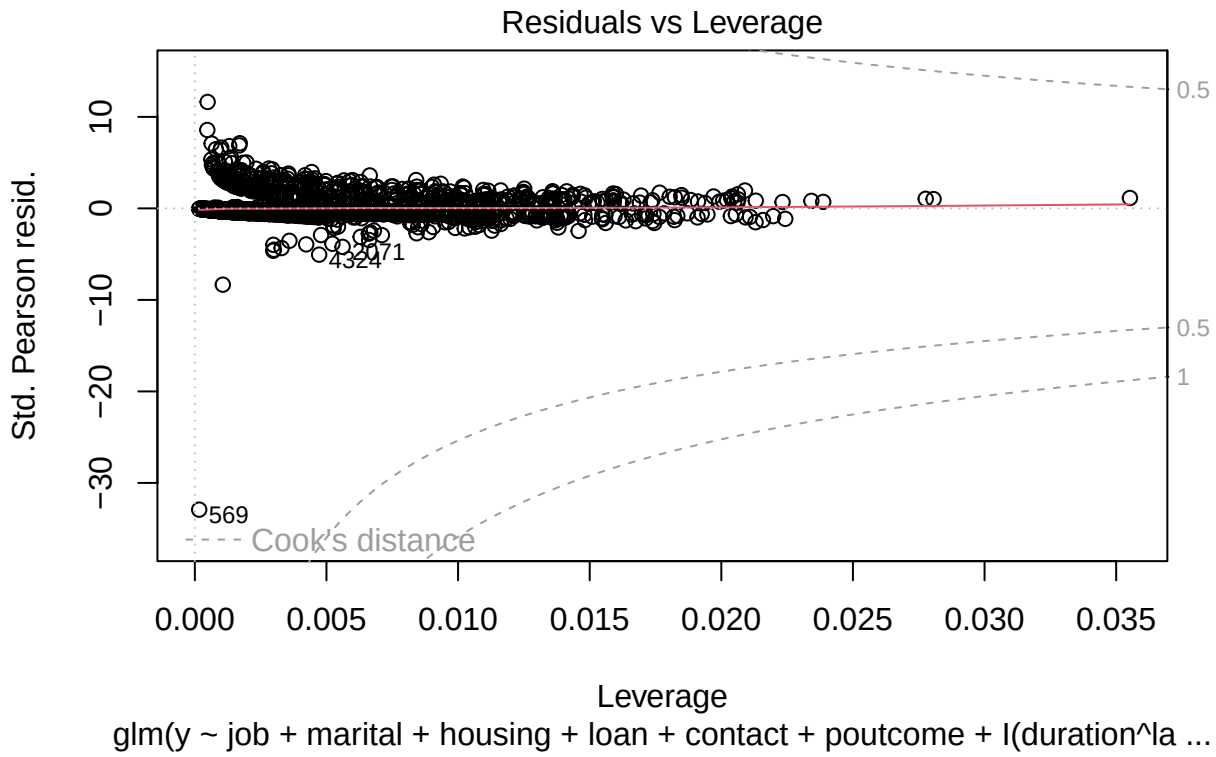
3.5.2 Outliers

We will check for outliers in data set using Cook's d.

```
plot(final_model, which=4)
```




```
plot(final_model, which=5)
```



None of the datapoints lie outside of Cook's d range, so formally we have no outliers. On the other hand sample 569 has the largest distance, though it has low leverage we shall take a look at it.

```
data[569,]
```

```
##     age        job marital education default balance housing loan  contact day
## 569  59 unemployed married   primary      no       0      no   no cellular  30
##     month duration campaign pdays previous poutcome  y
## 569   jan     3025        2    -1        0  unknown no
```

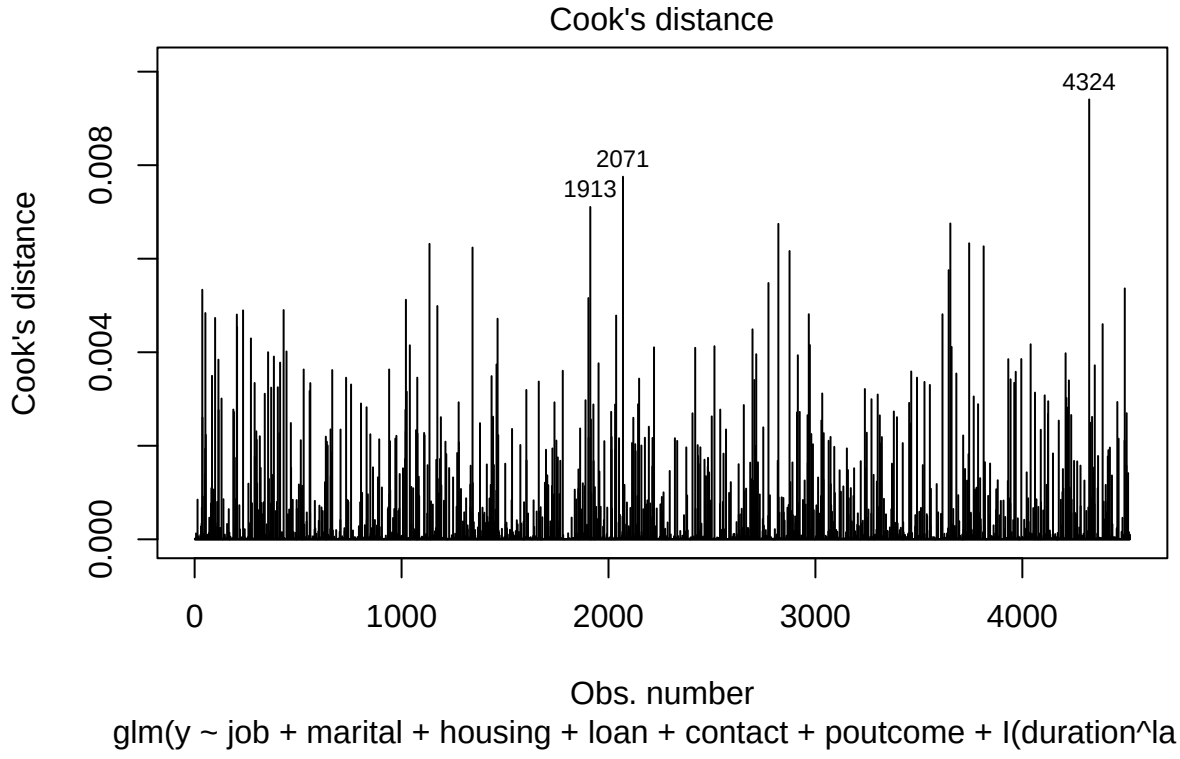
The biggest quirk of this sample is long call **duration** (3000s=50 minutes), yet it still has not subscribed to the deposit. We will remove this sample, because we have enough data and its low leverage guarantee that it will not hurt our model.

```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration ^ lambda), data=
summary(final_model)
```

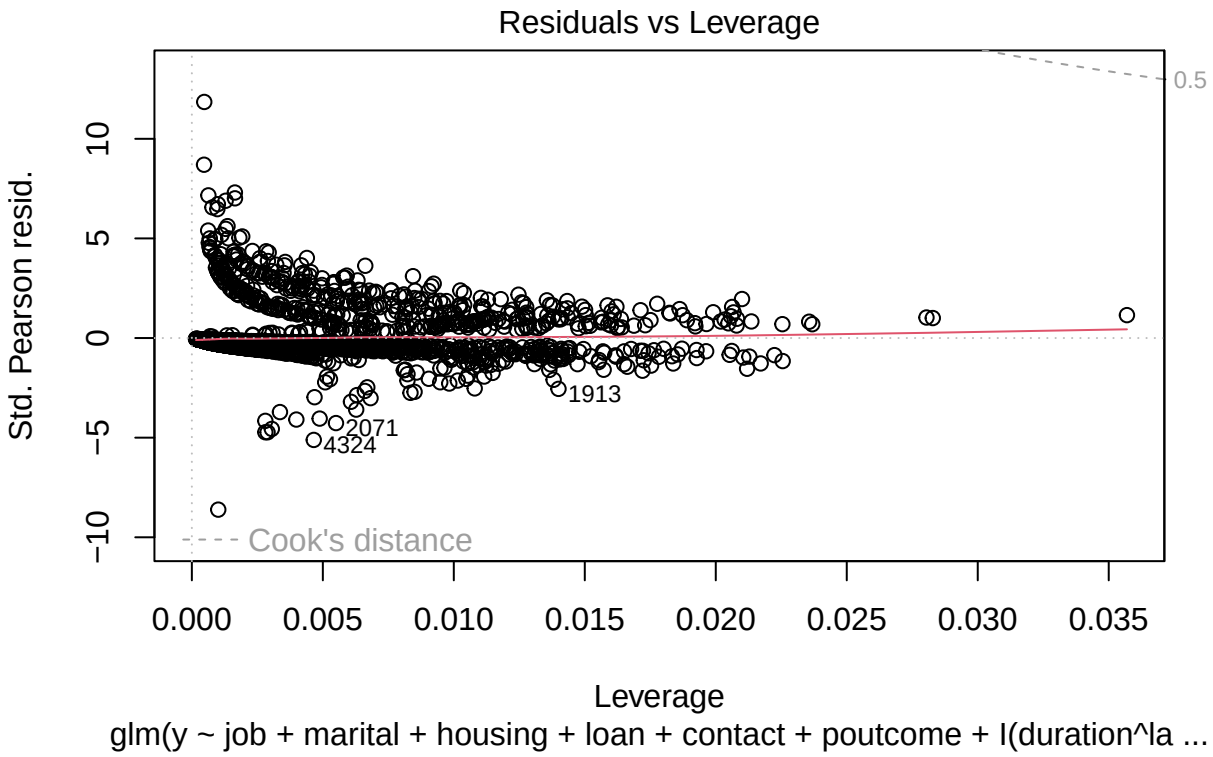
```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##     poutcome + I(duration^lambda), family = binomial, data = data.specific_job[-569,
##     ])
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -3.4234144  0.2887112 -11.858  < 2e-16 ***
## jobretired          1.1079931  0.2505248   4.423 9.75e-06 ***
## jobunknown          0.4337103  0.1636434   2.650  0.00804 **
## maritalmarried     -0.4286682  0.1684699  -2.544  0.01094 *
```

```
## maritalsingle      -0.1144606  0.1829358  -0.626  0.53152
## housingyes         -0.5092094  0.1200770  -4.241 2.23e-05 ***
## loanyes            -0.7889140  0.1921775  -4.105 4.04e-05 ***
## contacttelephone   -0.0867686  0.2201306  -0.394  0.69346
## contactunknown     -1.1175292  0.1742463  -6.414 1.42e-10 ***
## poutcomeother       0.4754379  0.2587739   1.837  0.06617 .
## poutcomesuccess     2.3901548  0.2561809   9.330  < 2e-16 ***
## poutcomeunknown    -0.3070400  0.1725872  -1.779  0.07523 .
## I(duration^lambda)  0.0188705  0.0008714  21.656  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3230.8  on 4519  degrees of freedom
## Residual deviance: 2256.2  on 4507  degrees of freedom
## AIC: 2282.2
##
## Number of Fisher Scoring iterations: 6
```

```
plot(final_model, which=4)
```




```
plot(final_model, which=5)
```



With no strong outliers we can proceed to the last assumption.

3.5.3 Multicollinearity

Logistic regression is a linear model and can suffer a lot from multicollinearity. Base R implementation does not use any regularization, so it can destroy our model. Luckily LASSO usually removes collinear features, so we should be good to go.

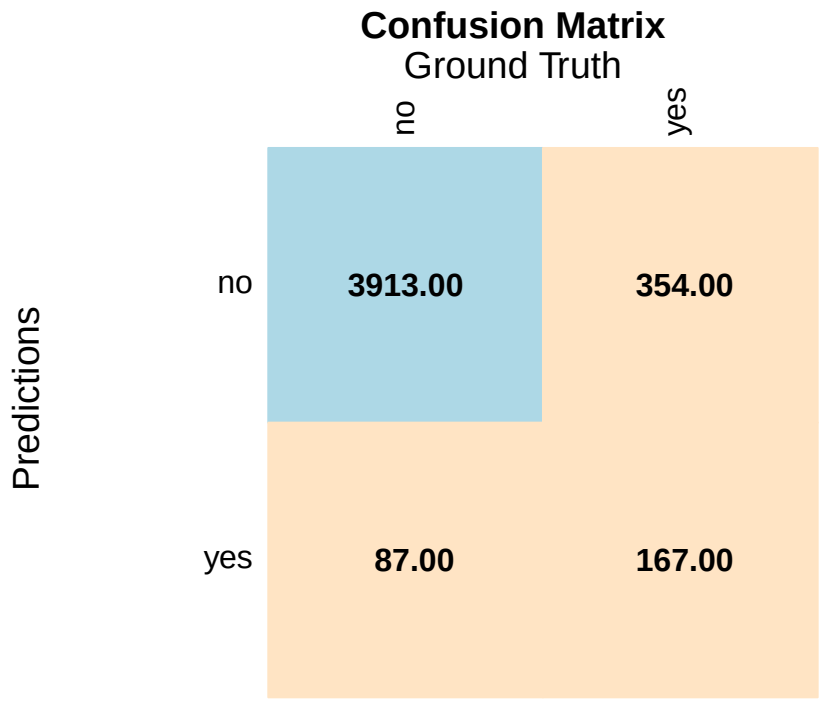
```
vif(final_model)
```

```
##                          GVIF Df GVIF^(1/(2*Df))
## job                  1.162905  2        1.038451
## marital              1.059598  2        1.014578
## housing              1.147895  1        1.071399
## loan                 1.018021  1        1.008970
## contact              1.169395  2        1.039897
## poutcome             1.131937  3        1.020870
## I(duration^lambda) 1.087601  1        1.042881
```

All adjusted GVIF values are way lower than 2, so we can assume that no multicollinearity is present in the data. Let us plot last ROC curve and confusion matrix.

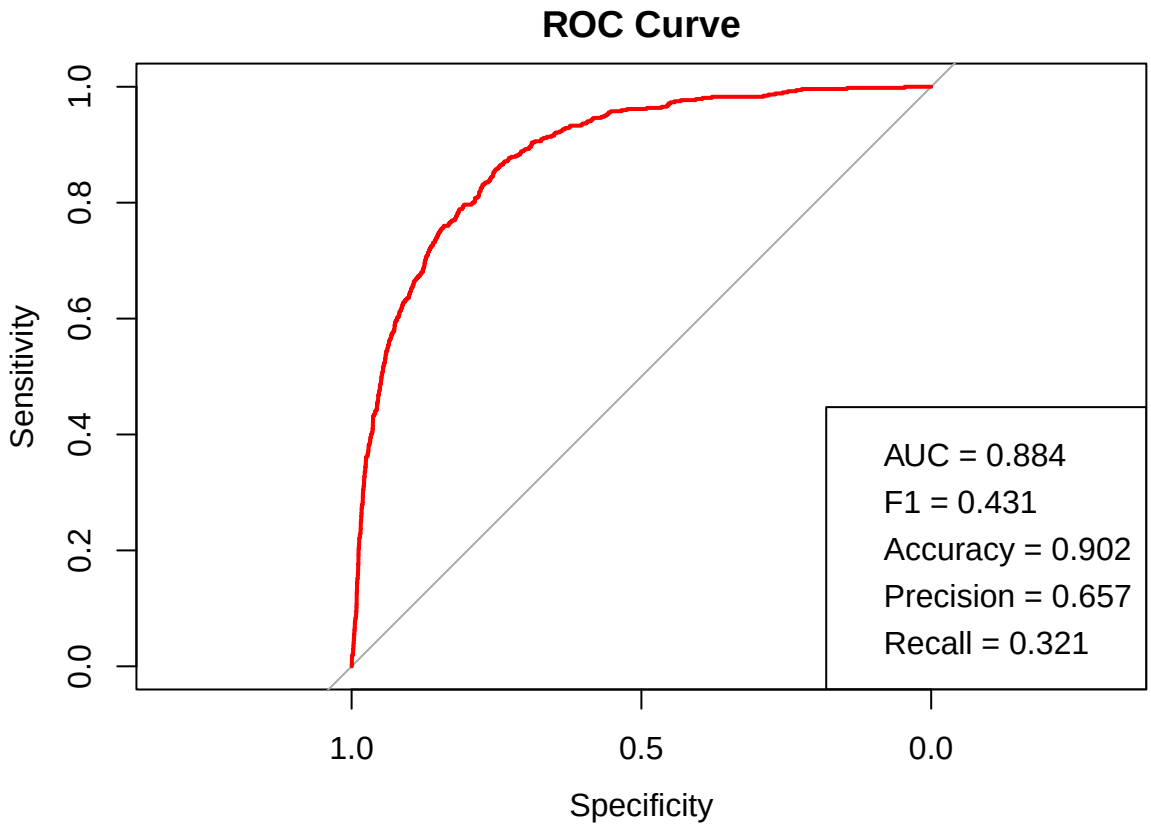
```
y_true_all <- data.specific_job$y
y_prob_all <- glm_predict_func(final_model, data.specific_job)
y_pred_all <- ifelse(y_prob_all > 0.5, "yes", "no")
y_pred_all <- factor(y_pred_all, levels = c("no", "yes"))

cm <- table(Predicted = y_pred_all, Actual = y_true_all)
```

```
tp <- cm["yes", "yes"]
tn <- cm["no", "no"]
fp <- cm["yes", "no"]
fn <- cm["no", "yes"]

accuracy <- (tp + tn) / sum(cm)
precision <- tp / (tp + fp)
recall <- tp / (tp + fn)
f1 <- 2 * precision * recall / (precision + recall)


corrplot(
  cm,
  is.corr = F, method = "color", tl.col = "black", addCoef.col = "black",
  xlab = "Ground Truth", ylab = "Prediction", mar=c(3,0,3,0),
  title = "Confusion Matrix", cl.pos = 'n', col=c('bisque', 'lightblue')
)
mtext("Ground Truth", side = 3, line=1, cex = 1.2)
mtext("Predictions", side = 2, line=0, cex = 1.2)
```




```
roc_obj <- roc(y_true_all, y_prob_all, levels=c("no", "yes"), direction='<')
plot(roc_obj, col = "red", main = "ROC Curve")

legend(
  "bottomright",
```

```
  legend = c(
      sprintf("AUC = %.3f", auc(roc_obj)),
      sprintf("F1 = %.3f", f1),
      sprintf("Accuracy = %.3f", accuracy),
      sprintf("Precision = %.3f", precision),
      sprintf("Recall = %.3f", recall)
  )
)
```

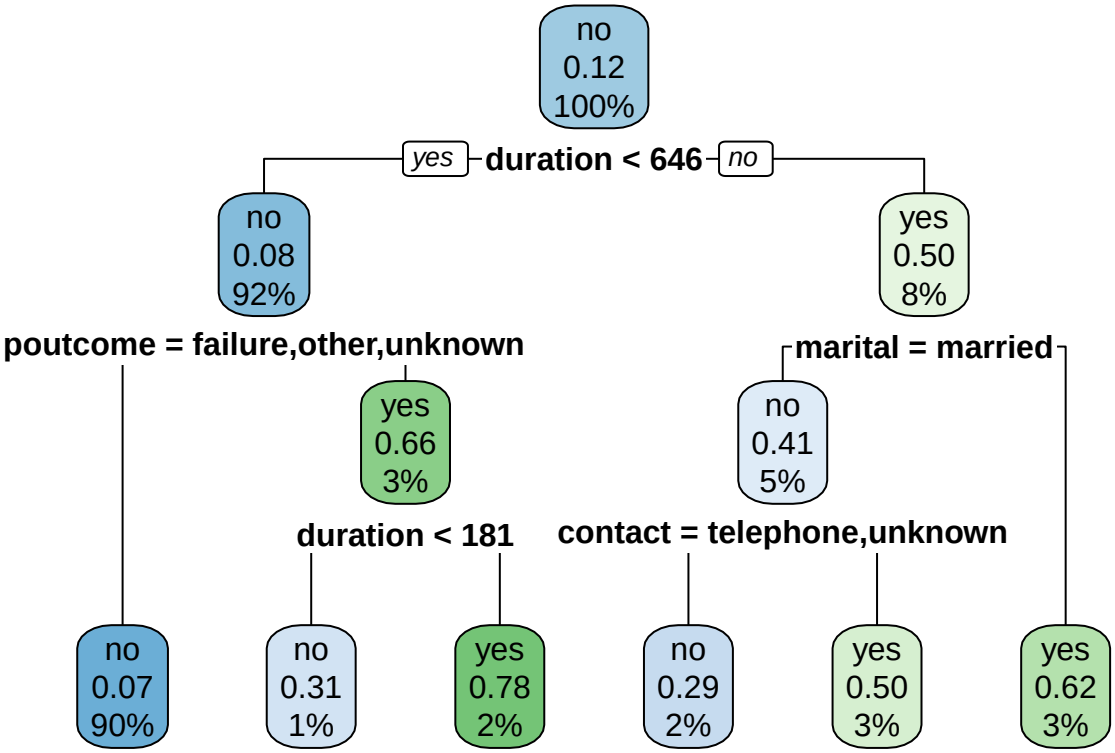


These are purely training values, though we still have large enough precision and good accuracy. The model still favors negative class, though not as sharply as before.

3.6 Final Decision Tree

Lastly we will train decision tree for further rule extraction.

```
data.lasso_specific_job <- data.specific_job[,c("y", "job", "marital", "housing", "loan", "contact", "du
tree_model <- rpart(y ~ ., data.lasso_specific_job, maxdepth=3, cp=0)
rpart.plot(tree_model)
```



Most of the leaf nodes that predict positive class have close to 50% positive population, so the rules extracted from this tree should be taken with a grain of salt.

3.7 Wrap up

Now we will save our models.

```
saveRDS(final_model, file=here::here("models", "log_regression.rds"))
saveRDS(tree_model,  file=here::here("models", "decision_tree.rds"))
```

In this notebook we have done feature selection both manual and automatic. Next step is to use all the collected knowledge to design target profile.

4 Profiling

4.1 Setup

Let us load the data and libraries.

```
library(stats)
library(rpart)
library(rpart.plot)
library(corrplot)
library(car)
library(vioplot)
library(effsize)
```

```r
source(here::here("R", "constants.R"))
source(here::here("R", "load_data.R"))

data <- load_bank_data()
logistic_regression <- readRDS(here::here("models", "log_regression.rds"))
decision_tree <- readRDS(here::here("models", "decision_tree.rds"))
head(data)
```

```
##   age         job marital education default balance housing loan  contact day
## 1  30  unemployed married   primary      no    1787      no   no cellular  19
## 2  33    services married secondary      no    4789     yes  yes cellular  11
## 3  35  management  single  tertiary      no    1350     yes   no cellular  16
## 4  30  management married  tertiary      no    1476     yes  yes  unknown   3
## 5  59 blue-collar married secondary      no       0     yes   no  unknown   5
## 6  35  management  single  tertiary      no     747      no   no cellular  23
##   month duration campaign pdays previous poutcome  y
## 1   oct       79        1    -1        0  unknown no
## 2   may      220        1   339        4  failure no
## 3   apr      185        1   330        1  failure no
## 4   jun      199        4    -1        0  unknown no
## 5   may      226        1    -1        0  unknown no
## 6   feb      141        2   176        3  failure no
```

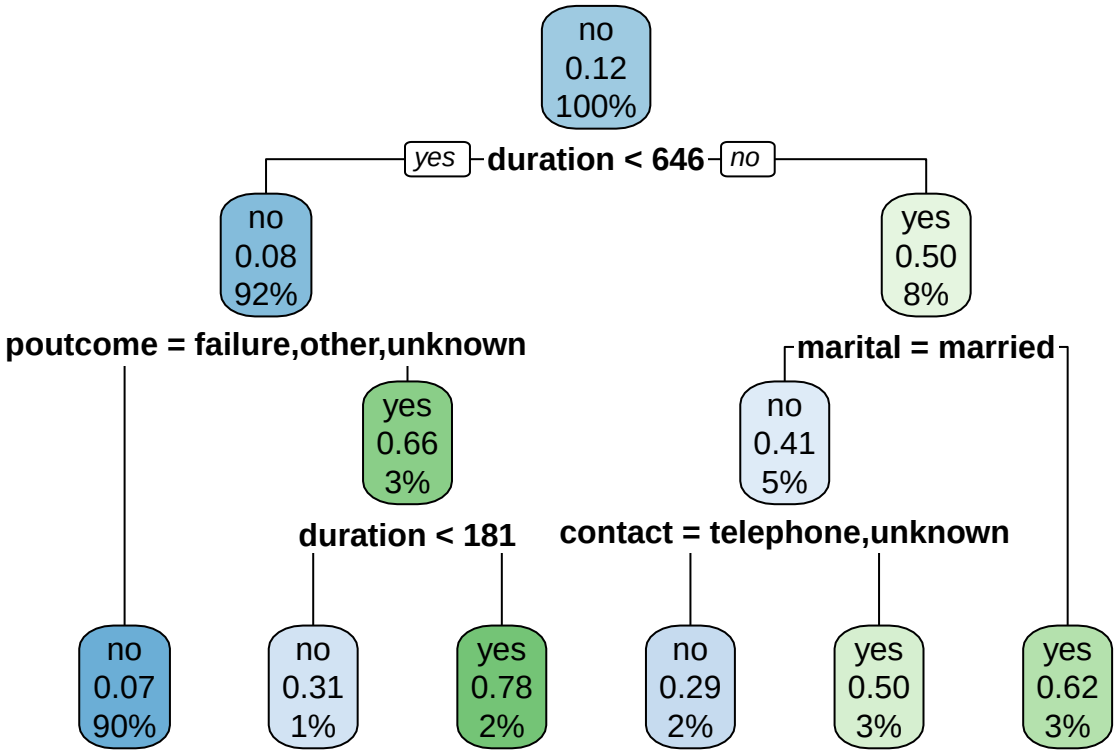
We will also transform the data the same way that we have done during modeling.

```r
data.joint_job <- data
data.joint_job[!(data$job == 'blue-collar' | data$job == 'retired'),"job"] <- "unknown"
data.pos <- data[data$y == 'yes',]
data.joint_job.pos <- data.joint_job[data.joint_job$y == 'yes',]

summary(logistic_regression)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##     poutcome + I(duration^lambda), family = binomial, data = data.specific_job[-569,
##     ])
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -3.4234144  0.2887112 -11.858  < 2e-16 ***
## jobretired          1.1079931  0.2505248   4.423 9.75e-06 ***
## jobunknown          0.4337103  0.1636434   2.650  0.00804 **
## maritalmarried     -0.4286682  0.1684699  -2.544  0.01094 *
## maritalsingle      -0.1144606  0.1829358  -0.626  0.53152
## housingyes         -0.5092094  0.1200770  -4.241 2.23e-05 ***
## loanyes            -0.7889140  0.1921775  -4.105 4.04e-05 ***
## contacttelephone   -0.0867686  0.2201306  -0.394  0.69346
## contactunknown     -1.1175292  0.1742463  -6.414 1.42e-10 ***
## poutcomeother       0.4754379  0.2587739   1.837  0.06617 .
## poutcomesuccess     2.3901548  0.2561809   9.330  < 2e-16 ***
## poutcomeunknown    -0.3070400  0.1725872  -1.779  0.07523 .
## I(duration^lambda)  0.0188705  0.0008714  21.656  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3230.8  on 4519  degrees of freedom
## Residual deviance: 2256.2  on 4507  degrees of freedom
## AIC: 2282.2
##
## Number of Fisher Scoring iterations: 6
```

```
rpart.plot(decision_tree)
```



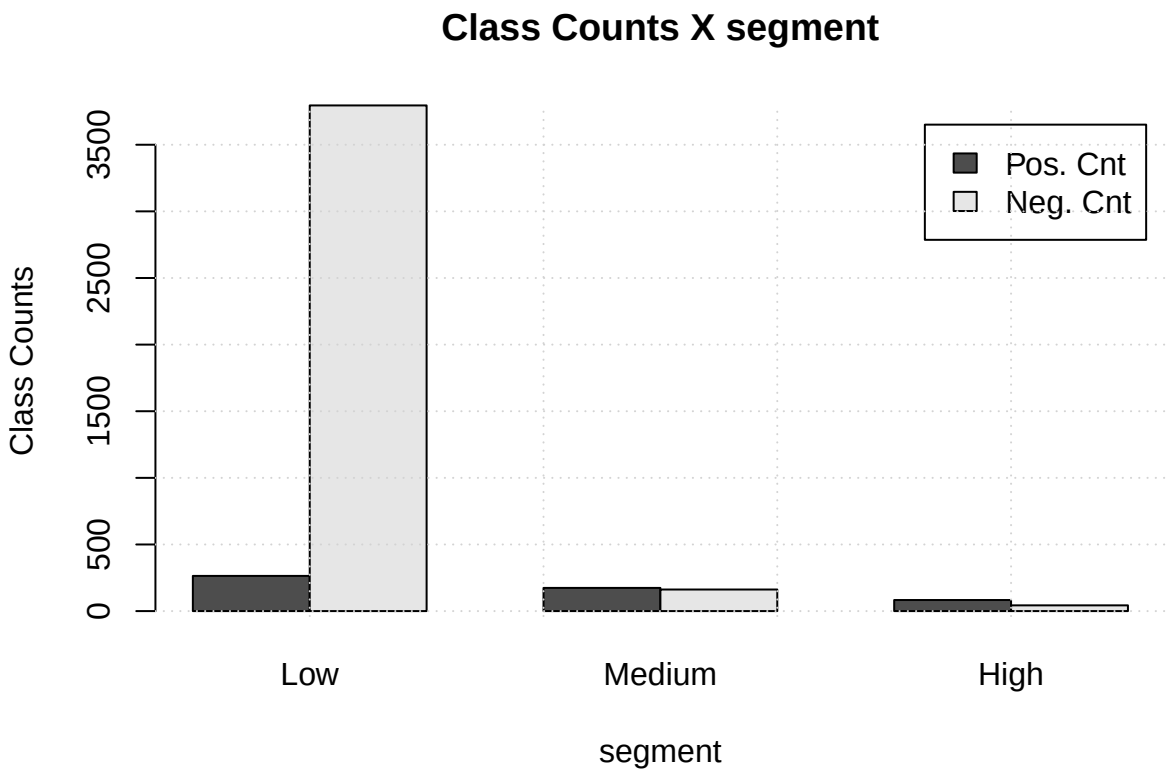
4.1.1 Data segmentation

In this subsection we will segment the dataset into three parts, based on logistic regression predictions. The predicted probability lying in interval (0, 0.3) will correspond to *Low*, (0.3, 0.7) to *Medium* and (0.7, 1.0) to *High* segments.

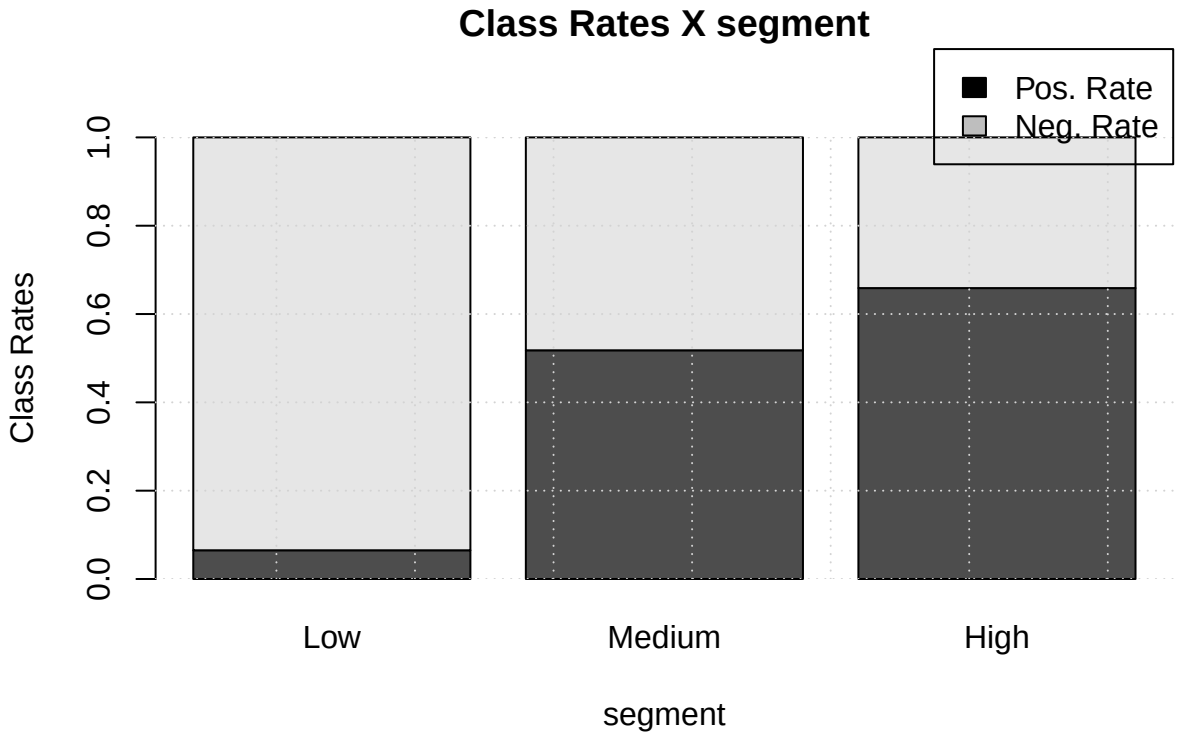
```
plot_count_bars <- function(col, arg_data, ...) {
    pos_labeled = arg_data[arg_data$y == "yes",]
    neg_labeled = arg_data[arg_data$y == "no",]
    pos <- aggregate(x = list(count = pos_labeled$y), by = list(column = pos_labeled[,col]), FUN = leng
    neg <- aggregate(x = list(count = neg_labeled$y), by = list(column = neg_labeled[,col]), FUN = leng
    data_matrix <- matrix(
        c( pos$count,  neg$count), nrow = 2, byrow = TRUE
    )
    colnames(data_matrix) <- pos$column
    rownames(data_matrix) <- c("Pos. Cnt", "Neg. Cnt")
```

```
    barplot(
        data_matrix,
        beside = TRUE,
        legend = rownames(data_matrix),
        main = sprintf("Class Counts X %s", col),
        xlab = col, ylab = "Class Counts",
        ...
    )
    grid()
}

plot_rate_bars <- function(col, arg_data, ...) {
    pos_labeled = arg_data[arg_data$y == "yes",]
    neg_labeled = arg_data[arg_data$y == "no",]
    par(mar=c(5, 4, 6, 2))
    total_cnt <- aggregate(x = list(count = arg_data$y), by = list(column = arg_data[,col]), FUN = leng
    pos <- aggregate(x = list(count = pos_labeled$y), by = list(column = pos_labeled[,col]), FUN = leng
    neg <- aggregate(x = list(count = neg_labeled$y), by = list(column = neg_labeled[,col]), FUN = leng
    data_matrix <- matrix(
        c( pos$count / total_cnt,  neg$count / total_cnt), nrow = 2, byrow = TRUE
    )
    colnames(data_matrix) <- pos$column
    rownames(data_matrix) <- c("Pos. Rate", "Neg. Rate")
    barplot(
        data_matrix,
        main = sprintf("Class Rates X %s", col),
        ylim=c(0,1),
        xlab = col, ylab = "Class Rates",
        ...
    )
    grid()
    legend(
        "topright",
        legend = rownames(data_matrix),
        xpd = T,
        fill=c("black", "grey"),
        inset=c(0, -0.2)
    )
}
```

```
data.joint_job$score <- predict(logistic_regression, newdata=data.joint_job, type='response')
data.joint_job$segment <- cut(data.joint_job$score, breaks = c(-Inf, 0.3, 0.7, Inf), labels = c("Low",

data.joint_job.pos$score <- predict(logistic_regression, newdata = data.joint_job.pos, type='response')
data.joint_job.pos$segment <- cut(data.joint_job.pos$score, breaks = c(-Inf, 0.3, 0.7, Inf), labels = c

plot_count_bars("segment", data.joint_job)
```




```
plot_rate_bars( "segment", data.joint_job)
```



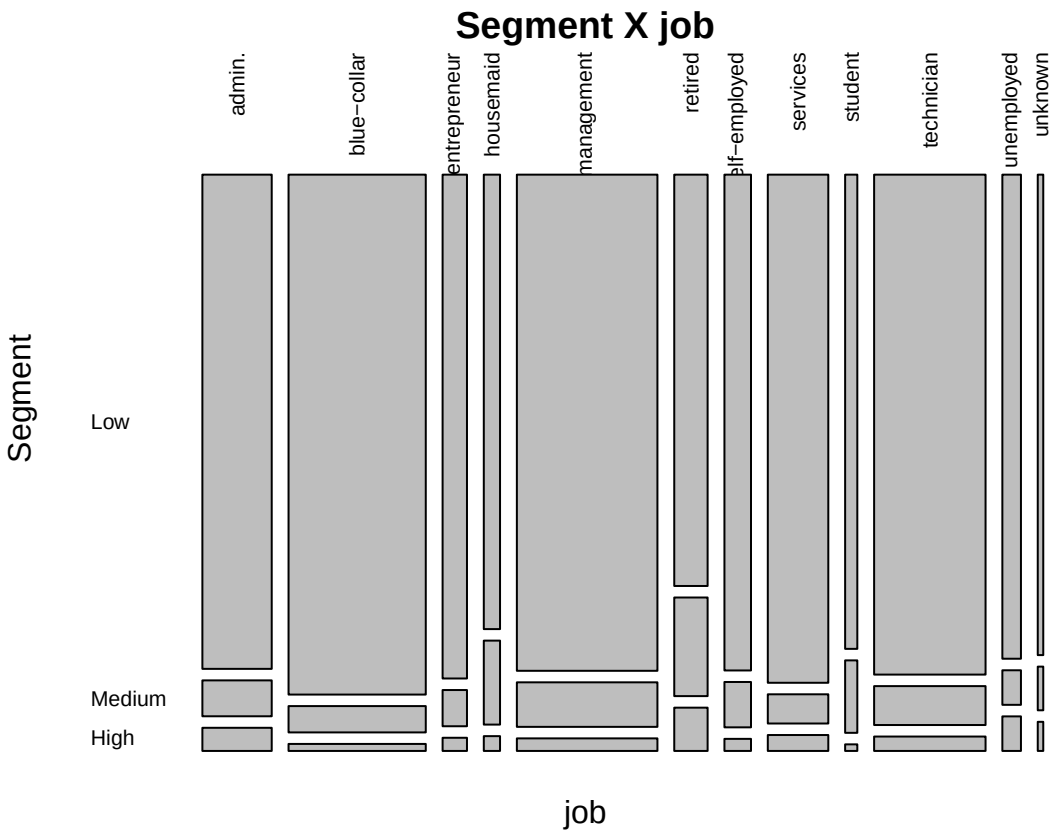
The population is primarily in *Low* segment, while *Medium* and *High* segments have a lot less samples, but more than a half of them are labelled positively.

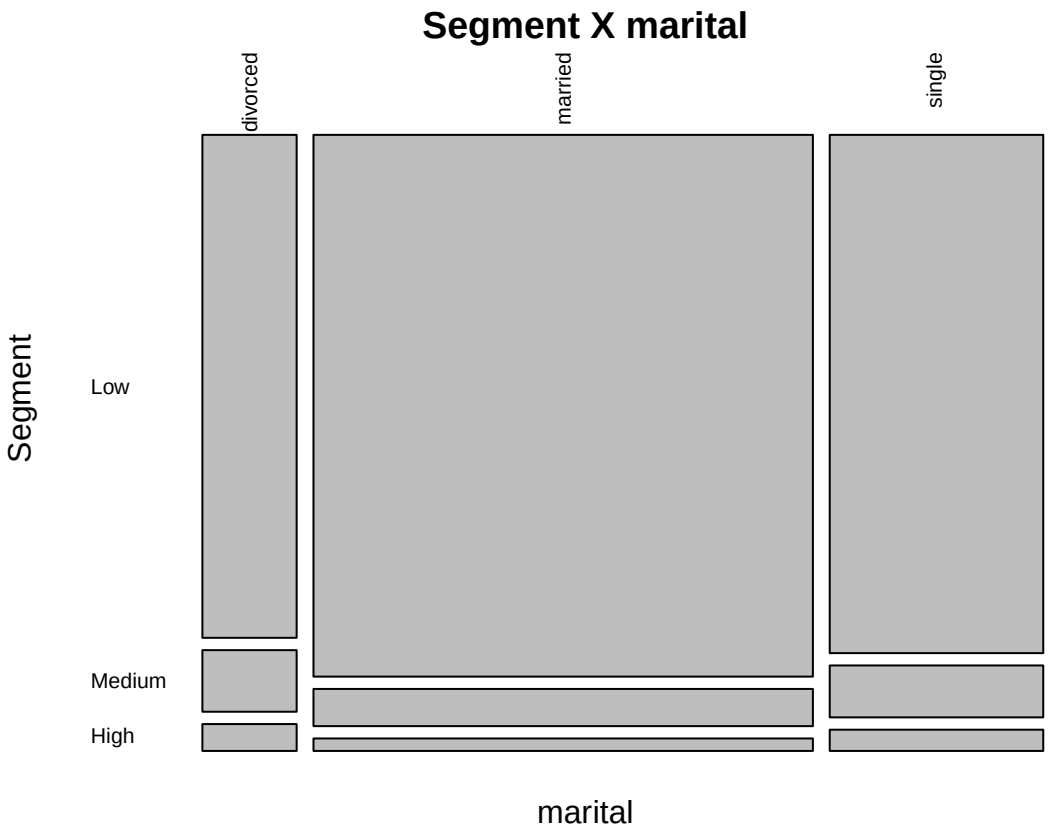
4.2 Data analysis

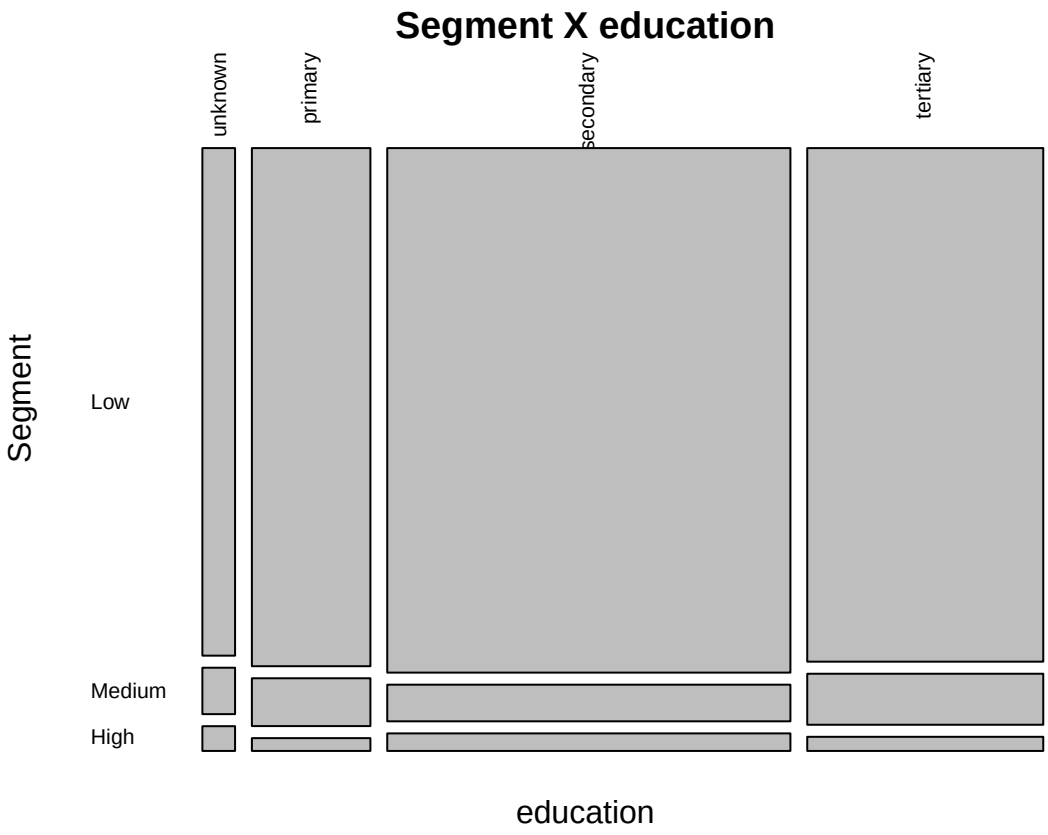
In this section we will conduct an examination of interactions between features and **segment**. It must generally follow structure of target variables interactions, though we expect to understand structure of data better.

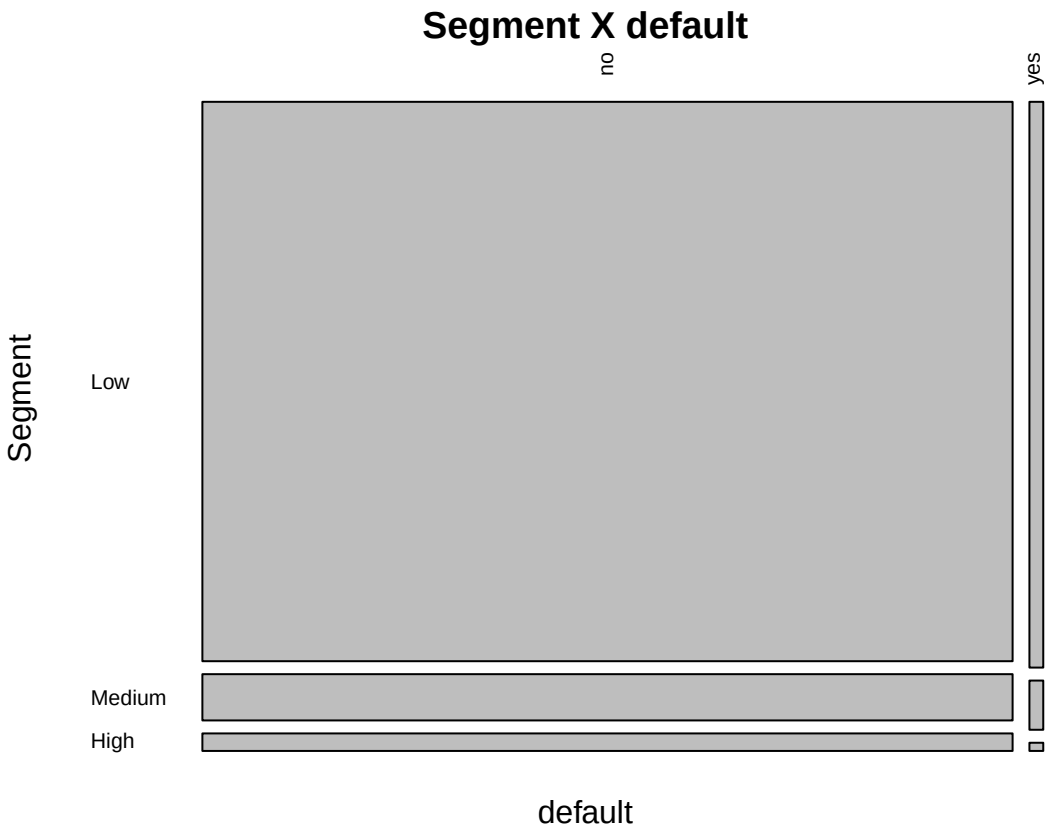
We will start by plotting mosaic plots of contigency tables segment versus categorical features.

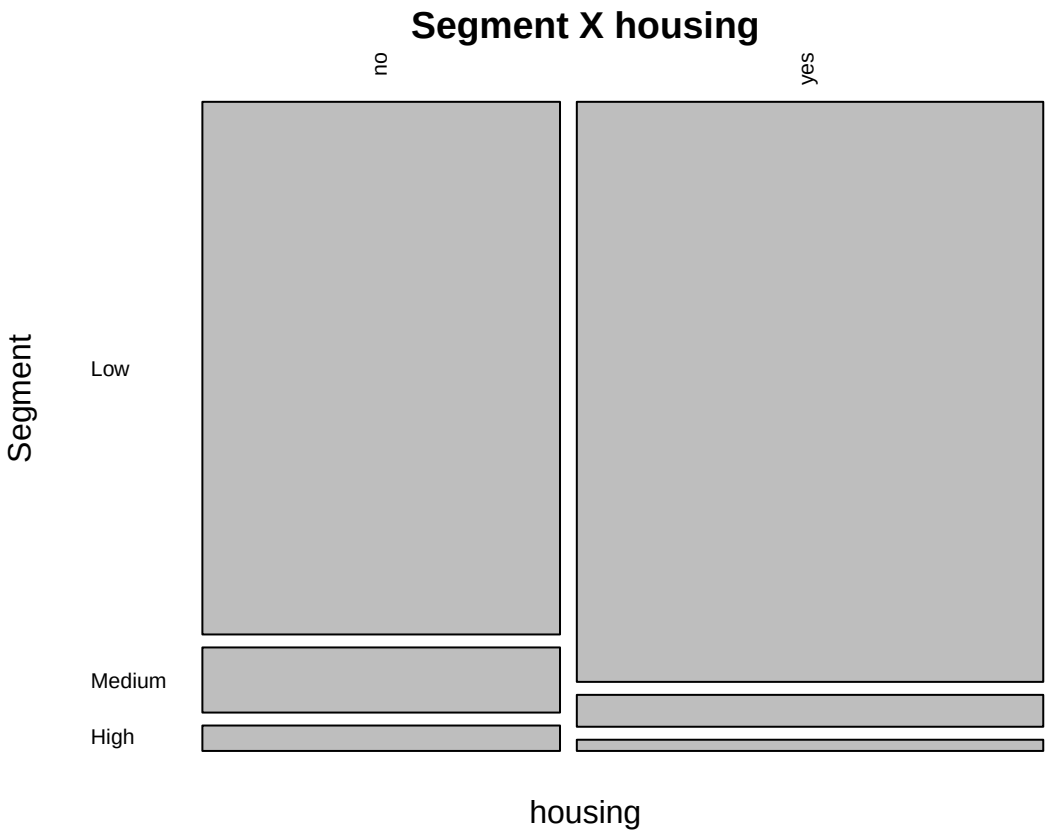
```
for(col in colnames(data[sapply(data, is.factor)])) {
    if(col == "segment" || col == "y"){ next}
    ct = table(data[,col], data.joint_job$segment)
    par(mar=c(3,4,1,2))
    mosaicplot(
        ct, ylab="Segment", xlab=col,
        main=sprintf("Segment X %s", col),
        las = 2
    )
}
```

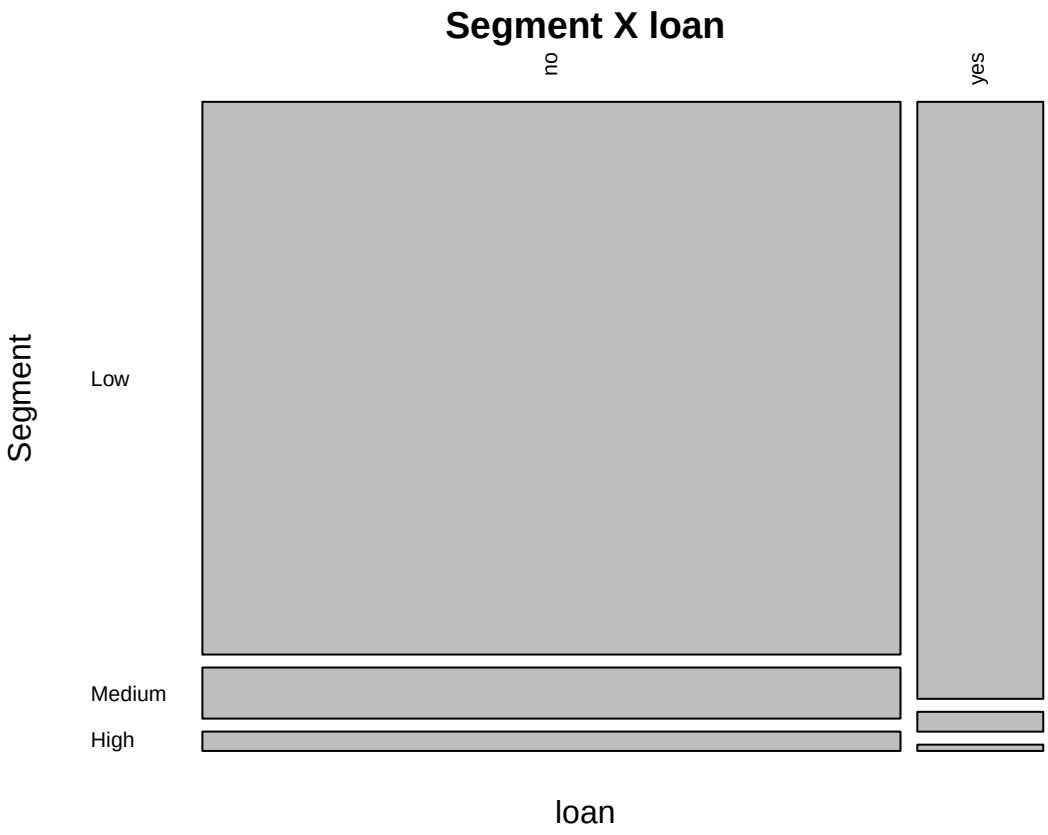

Segment X job
admin.
blue−collar
entrepreneur
housemaid
management
retired
self−employed
services
student
technician
unemployed
unknown
Segment
Low
Medium
High
job

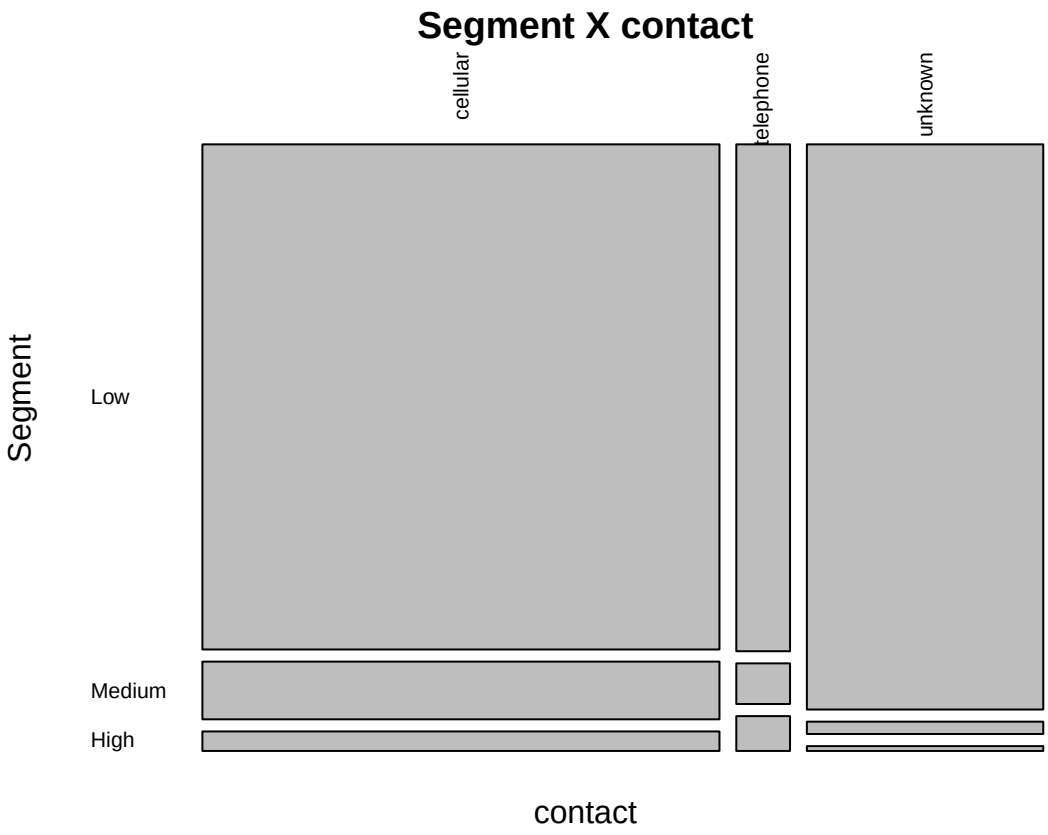

Segment X marital
divorced
married
single
Segment
Low
Medium
High
marital

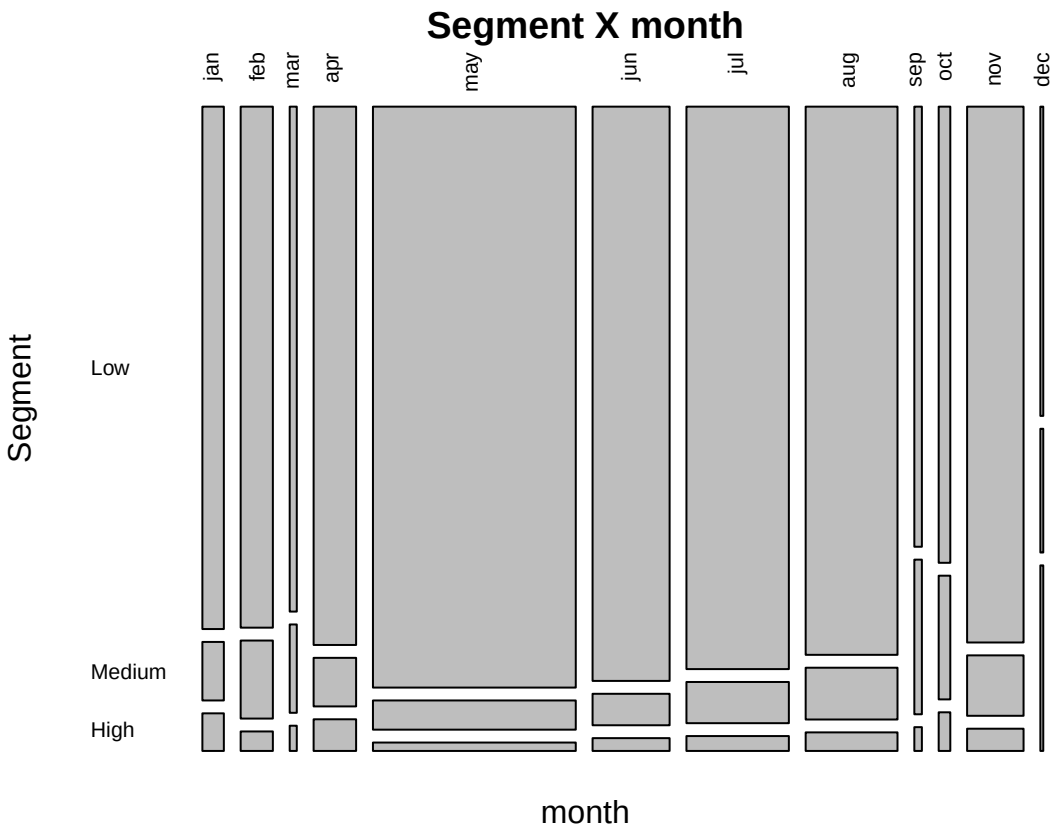

Segment X education
unknown
primary
secondary
tertiary
Segment
Low
Medium
High
education

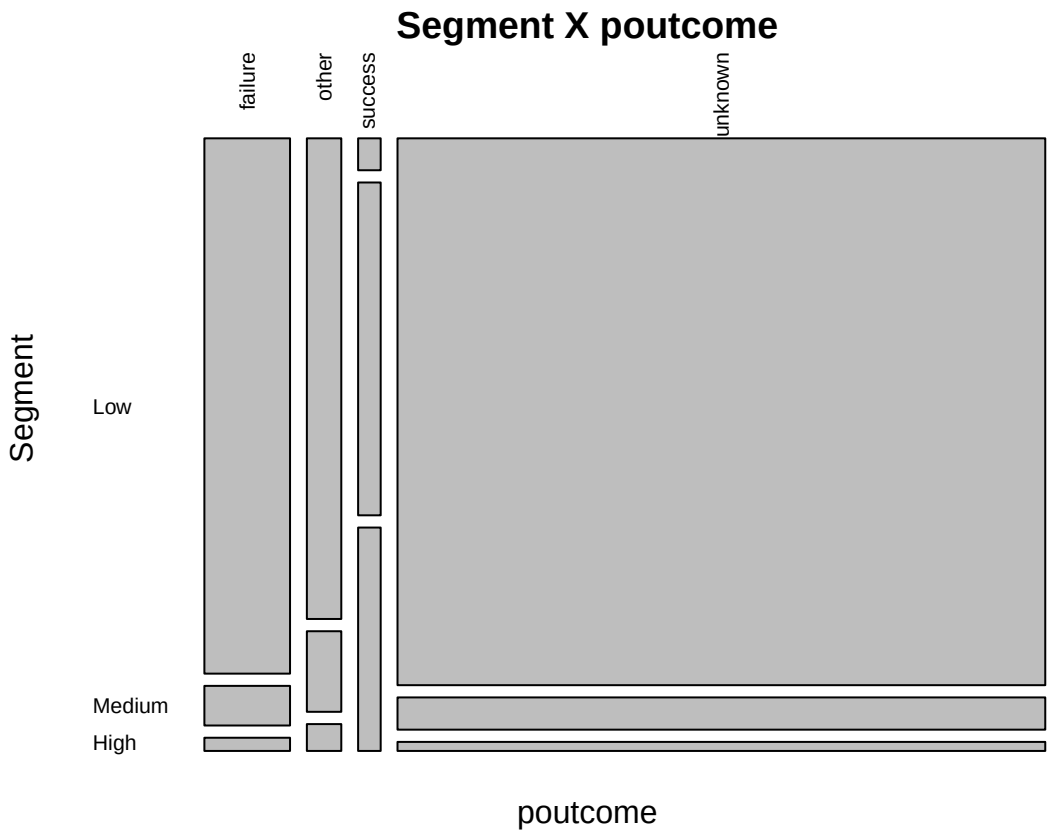

Segment X default
no
yes
Segment
Low
Medium
High
default


Segment X housing
no
yes
Segment
Low
Medium
High
housing


Segment X loan
no
yes
Segment
Low
Medium
High
loan


Segment X contact
cellular
telephone
unknown
Segment
Low
Medium
High
contact

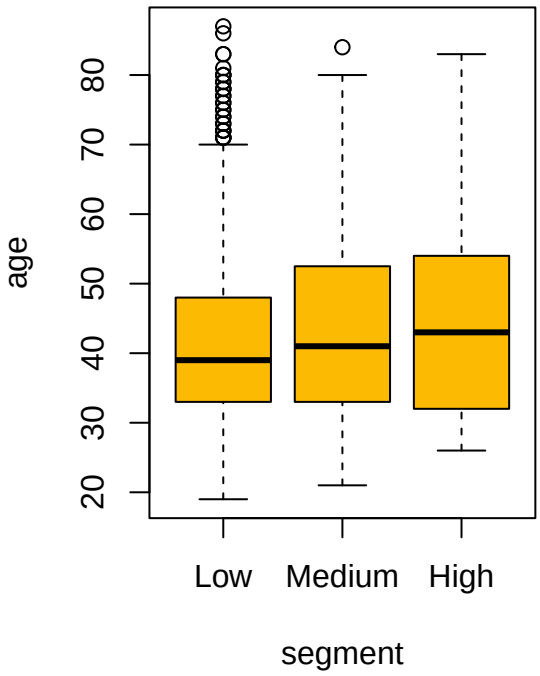

Segment X month
jan
feb
mar
apr
may
jun
jul
aug
sep
oct
nov
dec
Segment
Low
Medium
High
month

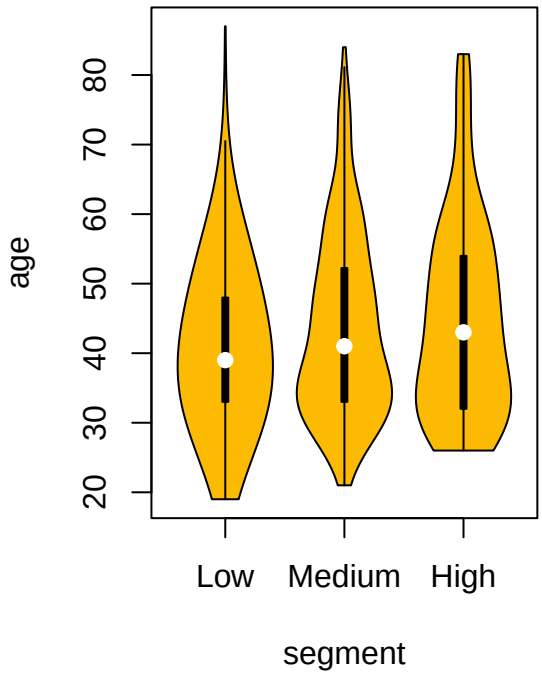


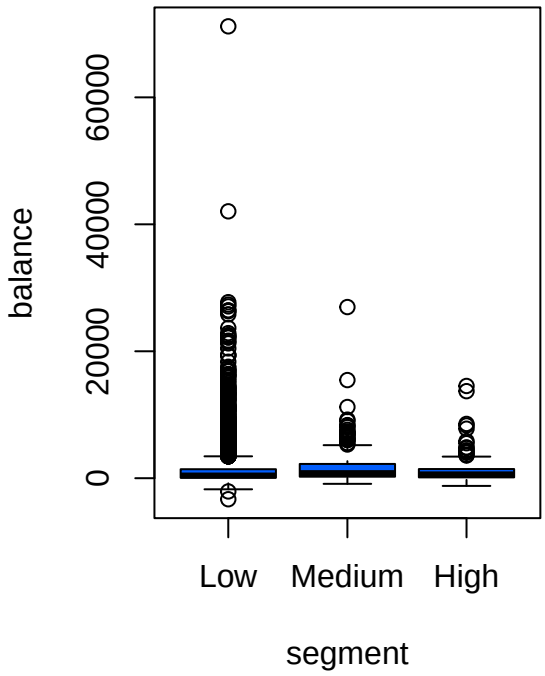
Most features do not differentiate between segments, the ones that do are **job** (specifically *retired*, *student* and *housemaid* are more likely to subscribe to a deposit), **housing** and **loan** (in both categories *no **is preferred by *Medium* segment) and** poutcome** with *success* and *other* showing great proportion in *Medium* and *High*.

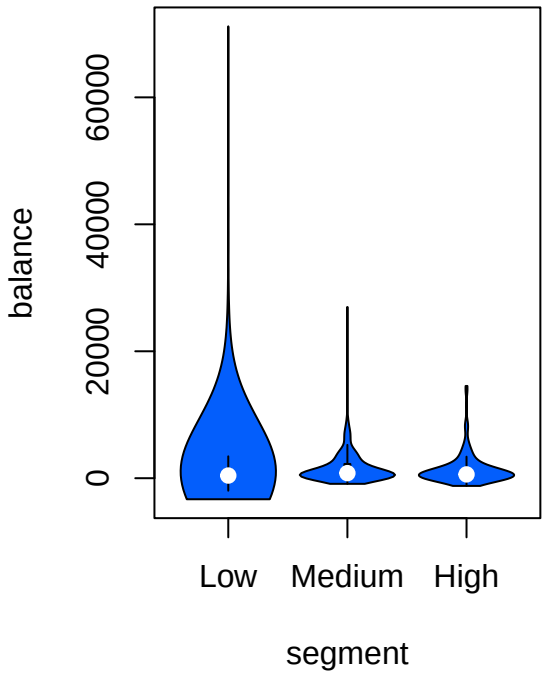
Now we will investigate relationship between **segment** and numerical features. We will use boxplot and violinplot.

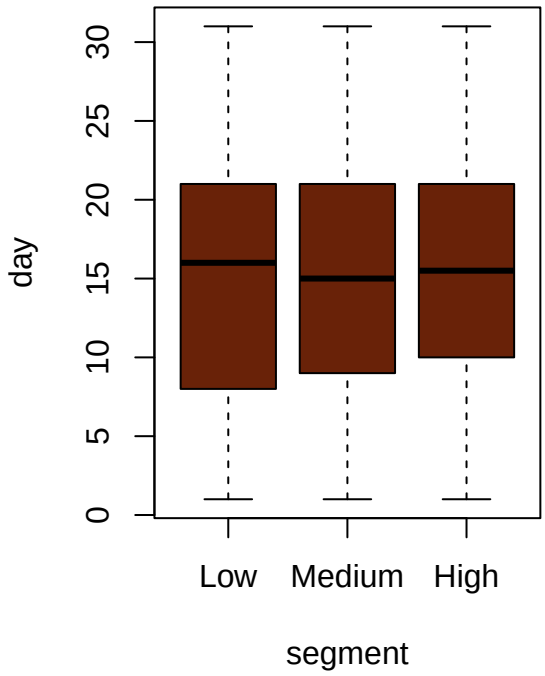
```
ylim=c(0, 1)
for(col in colnames(data.joint_job[sapply(data.joint_job, is.numeric)])) {
    if(col == "score") {next}
    fml <- as.formula(paste(col, " ~ segment"))
    par(mfrow=c(1, 2))
    boxplot(fml, data.joint_job, col=COLORS[col])
    vioplot(fml, data.joint_job, col=COLORS[col])
}
```

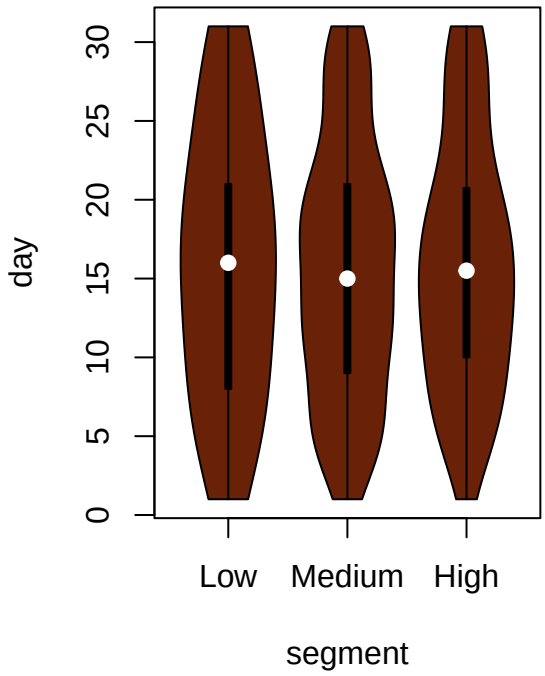

age
20
30
40
50
60
70
80
Low
Medium
High
segment

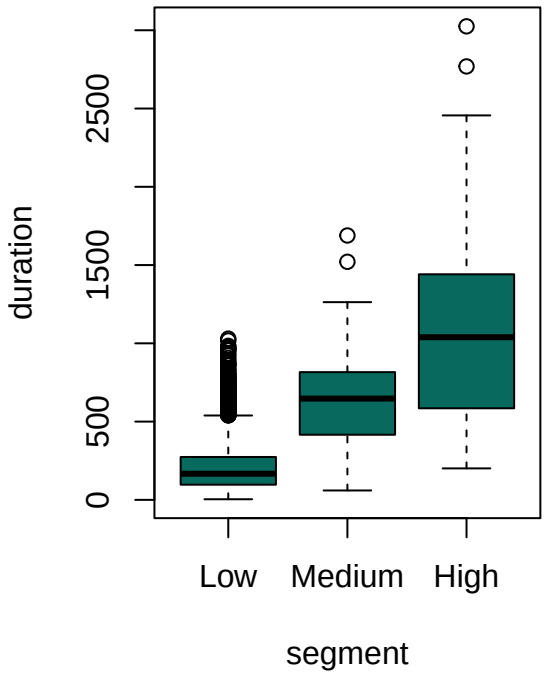

age
20
30
40
50
60
70
80
Low
Medium
High
segment

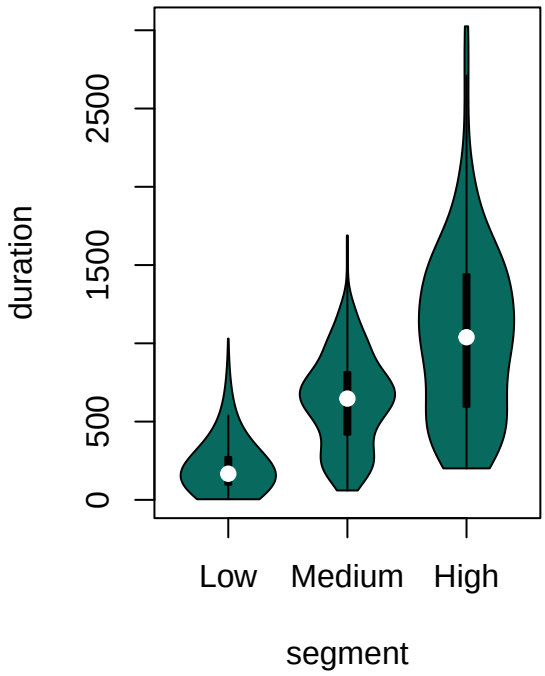

balance
0
20000
40000
60000
Low
Medium
High
segment

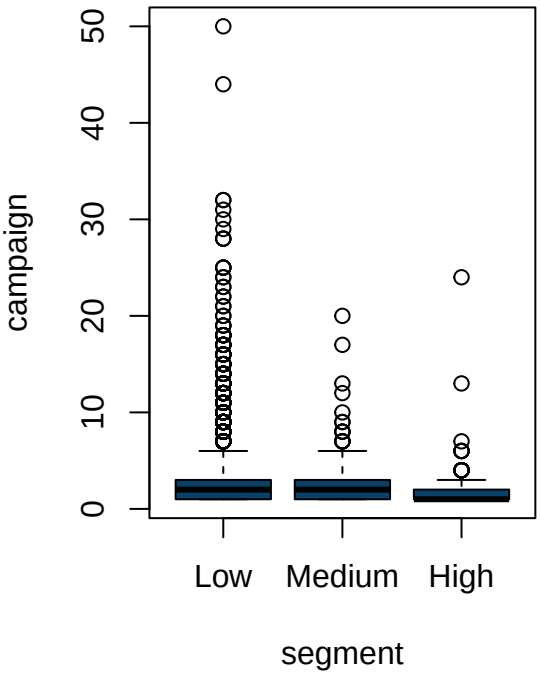

balance
0
20000
40000
60000
Low
Medium
High
segment

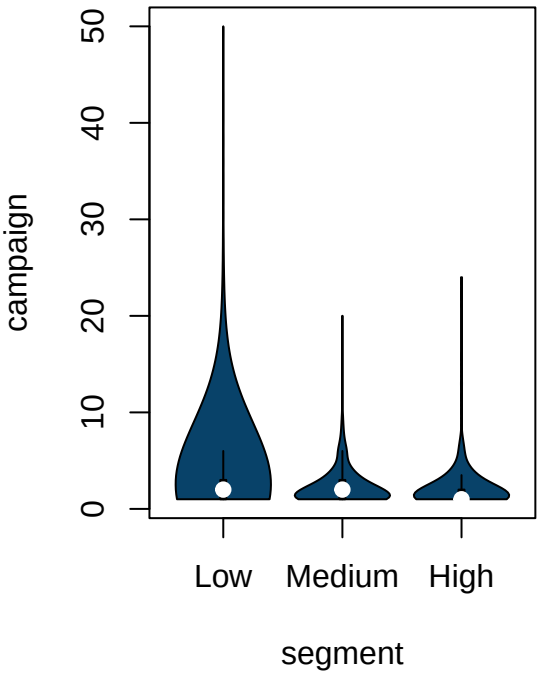

day
0
5
10
15
20
25
30
Low
Medium
High
segment

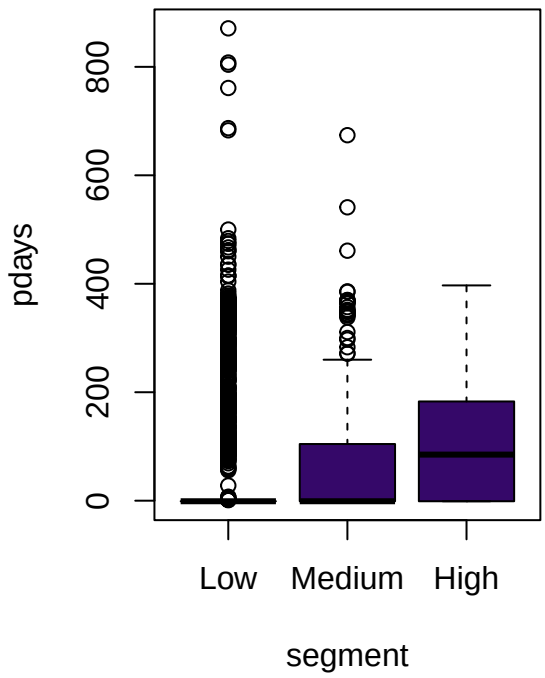

day
0
5
10
15
20
25
30
Low
Medium
High
segment

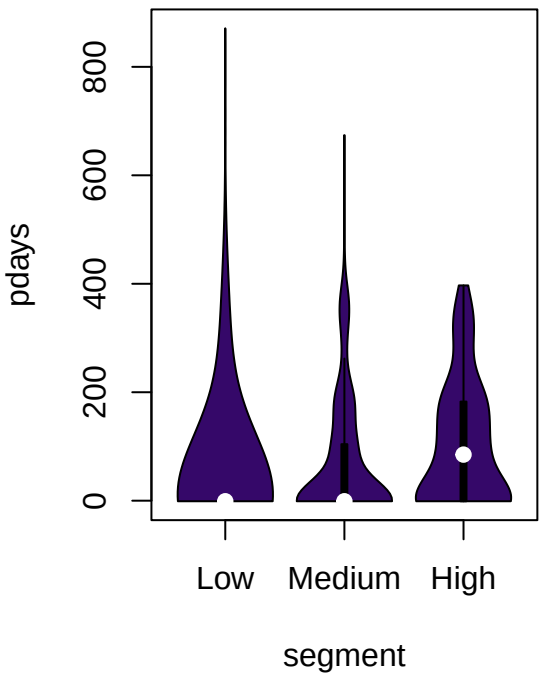

duration
0
500
1500
2500
Low
Medium
High
segment

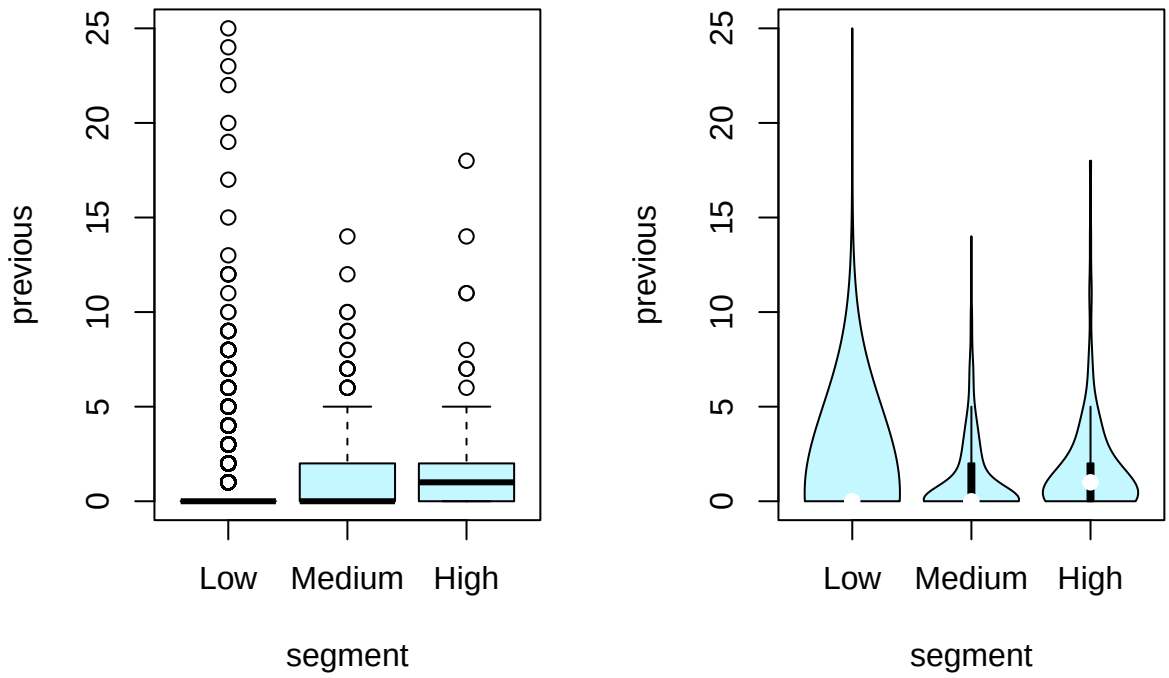

duration
0
500
1500
2500
Low
Medium
High
segment


campaign
0
10
20
30
40
50
Low
Medium
High
segment


campaign
0
10
20
30
40
50
Low
Medium
High
segment


pdays
0
200
400
600
800
Low
Medium
High
segment


pdays
0
200
400
600
800
Low
Medium
High
segment



The greatest distinction is in **duration**, though as we have discussed it must be instrumental for subscribing to deposits. Minor interactions are present in **pdays**, **previous** and **balance** with *Low* values being concentrated in the neighborhood of zero. We will check those features for profiling.

4.3 Creating Profiles

We have gathered a packet of features and values to check: **housing** and **loan** with *no*; **job** with *retired*, *student* and *housemaid*; **pdays**, **previous** and **balance** with strictly positive values. We will create univariate and bivariate profiles, that balance between positive rate and coverage.

We will compute size of profiles, positive proportion. We will also conduct proportion test to check that deposit subscribers are more likely in the profile, than in its compliment, as well as a binomial test to check profile against the whole dataset. To measure how practically significant is profile we will compute Cohen's h, Risk Difference and Odds Ratio.

```
num_y <- ifelse(data$y == "yes", 1, 0)
cohen.h <- function(p1, p2) {
  2 * asin(sqrt(p1)) - 2 * asin(sqrt(p2))
}
analyse_profile <- function(profile_bool, baseline.data = num_y) {
    num_profile <- ifelse(baseline.data[profile_bool], 1, 0)
    num_comp_profile <- ifelse(baseline.data[!profile_bool], 1, 0)

    print("----------------------------------------------------", quote=F)
    print(sprintf("Coverage:        %d samples = %.1f%% of dataset", length(num_profile), 100 * length(
    print(
        sprintf(
            "Positives:       %d samples = %.1f%% of profile = %.1f%% of all positives",
```

```
            sum(num_profile), 100 * mean(num_profile), 100 * sum(num_profile) / sum(baseline.data)
        ),
        quote=F
    )
    print("-----------------------------------------------------", quote=F)
    if (length(num_profile) < 1 || sum(num_profile) < 1) return()


    p_profile <- mean(num_profile)
    p_baseline <- mean(baseline.data)

    prop_pvalue <- prop.test(x = c(sum(num_profile), sum(num_comp_profile)), n = c(length(num_profile),
    binom_pvalue <- binom.test(sum(num_profile), length(num_profile), p=p_baseline, alternative = "less
    print(sprintf("Proportion test p-value against profile's compliment:     %.3f", prop_pvalue), quote=
    print(sprintf("Exact Binomial test p-value:                              %.3f", binom_pvalue), quot

    print("-----------------------------------------------------", quote=F)
    cohen_h <- cohen.h(p_profile, p_baseline)
    risk_difference <- p_profile - p_baseline
    odds_ratio <- p_profile * (1 - p_profile) / (p_baseline * (1 - p_baseline))


    print(sprintf("Cohen's h value: %.3f", cohen_h), quote=F)
    print(sprintf("Risk difference: %.3f", risk_difference), quote=F)
    print(sprintf("Odds ratio:      %.3f", odds_ratio), quote=F)
    print("-----------------------------------------------------", quote=F)
    print("", quote=F)
}
```

4.3.1 Univariate

During exploration of single variable we would like to find profiles that have both wide coverage, have greater proportion of positive samples than dataset and have Cohen's h greater than 0.1.

```
print("Housing = no")
```

```
## [1] "Housing = no"
```

```
analyse_profile(data$housing == "no")
```

```
## [1] -----------------------------------------------------
## [1] Coverage:        1962 samples = 43.4% of dataset
## [1] Positives:       301 samples = 15.3% of profile = 57.8% of all positives
## [1] -----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -----------------------------------------------------
## [1] Cohen's h value: 0.112
## [1] Risk difference: 0.038
## [1] Odds ratio:      1.274
## [1] -----------------------------------------------------
## [1]
```

```
print("Loan = no")
```

```
## [1] "Loan = no"
```

```r
analyse_profile(data$loan == "no")
```

```
## [1] -------------------------------------------------
## [1] Coverage:        3830 samples = 84.7% of dataset
## [1] Positives:       478 samples = 12.5% of profile = 91.7% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              0.969
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.029
## [1] Risk difference: 0.010
## [1] Odds ratio:      1.071
## [1] -------------------------------------------------
## [1]
```

Both binary features show very wide coverage (housing - 43%, loan - 85%), though effect size metrics show low values. We will use these features in combinations with other features to try to imptove positive rate.

```r
print("Job = student")
```

```
## [1] "Job = student"
```

```r
analyse_profile(data$job == "student")
```

```
## [1] -------------------------------------------------
## [1] Coverage:        84 samples = 1.9% of dataset
## [1] Positives:       19 samples = 22.6% of profile = 3.6% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.999
## [1] Exact Binomial test p-value:                              0.999
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.299
## [1] Risk difference: 0.111
## [1] Odds ratio:      1.717
## [1] -------------------------------------------------
## [1]
```

```r
print("Job = housemaid")
```

```
## [1] "Job = housemaid"
```

```r
analyse_profile(data$job == "housemaid")
```

```
## [1] -------------------------------------------------
## [1] Coverage:        112 samples = 2.5% of dataset
## [1] Positives:       14 samples = 12.5% of profile = 2.7% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.571
## [1] Exact Binomial test p-value:                              0.692
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.030
## [1] Risk difference: 0.010
## [1] Odds ratio:      1.073
## [1] -------------------------------------------------
## [1]
```

```r
print("Job = retired")
```

```
## [1] "Job = retired"
```

```r
analyse_profile(data$job == "retired")
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        230 samples = 5.1% of dataset
## [1] Positives:       54 samples = 23.5% of profile = 10.4% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ----------------------------------------------------
## [1] Cohen's h value: 0.319
## [1] Risk difference: 0.120
## [1] Odds ratio:      1.762
## [1] ----------------------------------------------------
## [1]
```

Both *student* and *housemaid* have awful coverage under 2.5%. *Housemaid* also has tiny improvment, so we discard it right away. *Retired* is a relatively large population with good positive rate, so we expect it to behave great in interaction profiles.

```r
print("Previous outcome = success")
```

```
## [1] "Previous outcome = success"
```

```r
analyse_profile(data$poutcome == "success")
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        129 samples = 2.9% of dataset
## [1] Positives:       83 samples = 64.3% of profile = 15.9% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ----------------------------------------------------
## [1] Cohen's h value: 1.169
## [1] Risk difference: 0.528
## [1] Odds ratio:      2.250
## [1] ----------------------------------------------------
## [1]
```

```r
print("Previous outcome = other")
```

```
## [1] "Previous outcome = other"
```

```r
analyse_profile(data$poutcome == "other")
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        197 samples = 4.4% of dataset
## [1] Positives:       38 samples = 19.3% of profile = 7.3% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              0.999
## [1] ----------------------------------------------------
## [1] Cohen's h value: 0.217
## [1] Risk difference: 0.078
## [1] Odds ratio:      1.527
```

```
## [1] ------------------------------------------------
## [1]
```

```
print("Previous outcome = other or success")
```

```
## [1] "Previous outcome = other or success"
```

```
analyse_profile((data$poutcome == "success") | (data$poutcome == "other"))
```

```
## [1] ------------------------------------------------
## [1] Coverage:        326 samples = 7.2% of dataset
## [1] Positives:       121 samples = 37.1% of profile = 23.2% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.617
## [1] Risk difference: 0.256
## [1] Odds ratio:      2.289
## [1] ------------------------------------------------
## [1]
```

Previous outcome indeed shows great results with unified group covering 7.2% dataset and more than doubling the odds. The question is whether it is just a connection with a bank, or the result of previous campaign that let people to subscribe to a deposit.

```
print("Contacts during previous campaign > 0")
```

```
## [1] "Contacts during previous campaign > 0"
```

```
analyse_profile(data$previous > 0)
```

```
## [1] ------------------------------------------------
## [1] Coverage:        816 samples = 18.0% of dataset
## [1] Positives:       184 samples = 22.5% of profile = 35.3% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.297
## [1] Risk difference: 0.110
## [1] Odds ratio:      1.713
## [1] ------------------------------------------------
## [1]
```

```
print("Contacts during previous campaign > 2")
```

```
## [1] "Contacts during previous campaign > 2"
```

```
analyse_profile(data$previous > 1)
```

```
## [1] ------------------------------------------------
## [1] Coverage:        530 samples = 11.7% of dataset
## [1] Positives:       133 samples = 25.1% of profile = 25.5% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.357
```

```
## [1] Risk difference: 0.136
## [1] Odds ratio:      1.844
## [1] --------------------------------------------------
## [1]
```

```
print("Contacts during previous campaign  > 5")
```

```
## [1] "Contacts during previous campaign  > 5"
```

```
analyse_profile(data$pdays > 5)
```

```
## [1] --------------------------------------------------
## [1] Coverage:        805 samples = 17.8% of dataset
## [1] Positives:       182 samples = 22.6% of profile = 34.9% of all positives
## [1] --------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] --------------------------------------------------
## [1] Cohen's h value: 0.298
## [1] Risk difference: 0.111
## [1] Odds ratio:      1.716
## [1] --------------------------------------------------
## [1]
```

Shrinking allowed window only made the profile narrow, without improving positive rate, so we will stick with any strictly positive number of contacts during previous campaign.

```
print("Days since last concact during previous campaign > 0")
```

```
## [1] "Days since last concact during previous campaign > 0"
```

```
analyse_profile(data$pdays > 0)
```

```
## [1] --------------------------------------------------
## [1] Coverage:        816 samples = 18.0% of dataset
## [1] Positives:       184 samples = 22.5% of profile = 35.3% of all positives
## [1] --------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] --------------------------------------------------
## [1] Cohen's h value: 0.297
## [1] Risk difference: 0.110
## [1] Odds ratio:      1.713
## [1] --------------------------------------------------
## [1]
```

```
print("Days since last concact during previous campaign > 100")
```

```
## [1] "Days since last concact during previous campaign > 100"
```

```
analyse_profile(data$pdays > 100)
```

```
## [1] --------------------------------------------------
## [1] Coverage:        680 samples = 15.0% of dataset
## [1] Positives:       122 samples = 17.9% of profile = 23.4% of all positives
## [1] --------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] --------------------------------------------------
```

```
## [1] Cohen's h value: 0.182
## [1] Risk difference: 0.064
## [1] Odds ratio:      1.444
## [1] ----------------------------------------------------
## [1]
```

```
print("Days since last concact during previous campaign > 200")
```

```
## [1] "Days since last concact during previous campaign > 200"
```

```
analyse_profile(data$pdays > 200)
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        382 samples = 8.4% of dataset
## [1] Positives:       53 samples = 13.9% of profile = 10.2% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.922
## [1] Exact Binomial test p-value:                              0.933
## [1] ----------------------------------------------------
## [1] Cohen's h value: 0.071
## [1] Risk difference: 0.024
## [1] Odds ratio:      1.172
## [1] ----------------------------------------------------
## [1]
```

Pushing the boundary up for the number of days since last contact during previous campaign has made the profile only worse, so we will keep the widest that is $pdays > 0$.

```
print("Balance greater than 0")
```

```
## [1] "Balance greater than 0"
```

```
analyse_profile(data$balance > 0)
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        3798 samples = 84.0% of dataset
## [1] Positives:       461 samples = 12.1% of profile = 88.5% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.998
## [1] Exact Binomial test p-value:                              0.886
## [1] ----------------------------------------------------
## [1] Cohen's h value: 0.019
## [1] Risk difference: 0.006
## [1] Odds ratio:      1.046
## [1] ----------------------------------------------------
## [1]
```

```
print("Balance greater than 100")
```

```
## [1] "Balance greater than 100"
```

```
analyse_profile(data$balance > 100)
```

```
## [1] ----------------------------------------------------
## [1] Coverage:        3265 samples = 72.2% of dataset
## [1] Positives:       415 samples = 12.7% of profile = 79.7% of all positives
## [1] ----------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              0.983
```

```
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.036
## [1] Risk difference: 0.012
## [1] Odds ratio:      1.088
## [1] ------------------------------------------------
## [1]
```

```
print("Balance greater than 500")
```

```
## [1] "Balance greater than 500"
```

```
analyse_profile(data$balance > 500)
```

```
## [1] ------------------------------------------------
## [1] Coverage:        2134 samples = 47.2% of dataset
## [1] Positives:       302 samples = 14.2% of profile = 58.0% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.079
## [1] Risk difference: 0.026
## [1] Odds ratio:      1.192
## [1] ------------------------------------------------
## [1]
```

```
print("Balance greater than 1000")
```

```
## [1] "Balance greater than 1000"
```

```
analyse_profile(data$balance > 1000)
```

```
## [1] ------------------------------------------------
## [1] Coverage:        1481 samples = 32.8% of dataset
## [1] Positives:       221 samples = 14.9% of profile = 42.4% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ------------------------------------------------
## [1] Cohen's h value: 0.101
## [1] Risk difference: 0.034
## [1] Odds ratio:      1.245
## [1] ------------------------------------------------
## [1]
```

```
print("Balance greater than 5000")
```

```
## [1] "Balance greater than 5000"
```

```
analyse_profile(data$balance > 5000)
```

```
## [1] ------------------------------------------------
## [1] Coverage:        309 samples = 6.8% of dataset
## [1] Positives:       34 samples = 11.0% of profile = 6.5% of all positives
## [1] ------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.419
## [1] Exact Binomial test p-value:                              0.430
## [1] ------------------------------------------------
```

```
## [1] Cohen's h value: -0.016
## [1] Risk difference: -0.005
## [1] Odds ratio:      0.960
## [1] --------------------------------------------------
## [1]
```

Balance over one thousand has great coverage of 33% and a promising positive rate of 15%. We hope to improve with other features.

4.3.2 Bivariate

Here we will try to refine our profiles.

```
print("Previous outcome = success & housing = no")
```

```
## [1] "Previous outcome = success & housing = no"
```

```
analyse_profile((data$poutcome == "success") & (data$housing == "no"))
```

```
## [1] --------------------------------------------------
## [1] Coverage:        89 samples = 2.0% of dataset
## [1] Positives:       63 samples = 70.8% of profile = 12.1% of all positives
## [1] --------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] --------------------------------------------------
## [1] Cohen's h value: 1.307
## [1] Risk difference: 0.593
## [1] Odds ratio:      2.028
## [1] --------------------------------------------------
## [1]
```

```
print("Previous outcome = success & loan = no")
```

```
## [1] "Previous outcome = success & loan = no"
```

```
analyse_profile((data$poutcome == "success") & (data$loan == "no"))
```

```
## [1] --------------------------------------------------
## [1] Coverage:        123 samples = 2.7% of dataset
## [1] Positives:       78 samples = 63.4% of profile = 15.0% of all positives
## [1] --------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] --------------------------------------------------
## [1] Cohen's h value: 1.150
## [1] Risk difference: 0.519
## [1] Odds ratio:      2.275
## [1] --------------------------------------------------
## [1]
```

Intersection with **poutcome** covers a small patch of dataset, so we will not proceed with this.

```
print("Previous outcome = other & housing = no")
```

```
## [1] "Previous outcome = other & housing = no"
```

```
analyse_profile((data$poutcome == "other") & (data$housing == "no"))
```

```
## [1] --------------------------------------------------
```

```
## [1] Coverage:        64 samples = 1.4% of dataset
## [1] Positives:       17 samples = 26.6% of profile = 3.3% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.390
## [1] Risk difference: 0.150
## [1] Odds ratio:      1.913
## [1] -------------------------------------------------
## [1]
```

```
print("Previous outcome = other & loan = no")
```

```
## [1] "Previous outcome = other & loan = no"
```

```
analyse_profile((data$poutcome == "other") & (data$loan == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        173 samples = 3.8% of dataset
## [1] Positives:       34 samples = 19.7% of profile = 6.5% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              0.999
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.226
## [1] Risk difference: 0.081
## [1] Odds ratio:      1.549
## [1] -------------------------------------------------
## [1]
```

poutcome *other* profiles are also very small and additionally not as good.

```
print("Previous outcome = (other or success) & housing = no")
```

```
## [1] "Previous outcome = (other or success) & housing = no"
```

```
analyse_profile((data$poutcome %in% c("other", "success")) & (data$housing == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        153 samples = 3.4% of dataset
## [1] Positives:       80 samples = 52.3% of profile = 15.4% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.924
## [1] Risk difference: 0.408
## [1] Odds ratio:      2.447
## [1] -------------------------------------------------
## [1]
```

```
print("Previous outcome = (other or success) & loan = no")
```

```
## [1] "Previous outcome = (other or success) & loan = no"
```

```
analyse_profile((data$poutcome %in% c("other", "success")) & (data$loan == "no"))
```

```
## [1] -------------------------------------------------
```

```
## [1] Coverage:        296 samples = 6.5% of dataset
## [1] Positives:       112 samples = 37.8% of profile = 21.5% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.632
## [1] Risk difference: 0.263
## [1] Odds ratio:      2.307
## [1] -------------------------------------------------
## [1]
```

Union of previous profiles gives better results, but still covers little patch.

```
print("Job = retired & housing = no")
```

```
## [1] "Job = retired & housing = no"
```

```
analyse_profile((data$job == "retired") & (data$housing == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        180 samples = 4.0% of dataset
## [1] Positives:       47 samples = 26.1% of profile = 9.0% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.380
## [1] Risk difference: 0.146
## [1] Odds ratio:      1.892
## [1] -------------------------------------------------
## [1]
```

```
print("Job = retired & loan = no")
```

```
## [1] "Job = retired & loan = no"
```

```
analyse_profile((data$job == "retired") & (data$loan == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        198 samples = 4.4% of dataset
## [1] Positives:       52 samples = 26.3% of profile = 10.0% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.383
## [1] Risk difference: 0.147
## [1] Odds ratio:      1.899
## [1] -------------------------------------------------
## [1]
```

Those profiles are good, though coverage of 4% is not ideal, let us seek further.

```
print("Job = student & housing = no")
```

```
## [1] "Job = student & housing = no"
```

```r
analyse_profile((data$job == "student") & (data$housing == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        64 samples = 1.4% of dataset
## [1] Positives:       17 samples = 26.6% of profile = 3.3% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.390
## [1] Risk difference: 0.150
## [1] Odds ratio:      1.913
## [1] -------------------------------------------------
## [1]
```

```r
print("Job = student & loan = no")
```

```
## [1] "Job = student & loan = no"
```

```r
analyse_profile((data$job == "student") & (data$loan == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        83 samples = 1.8% of dataset
## [1] Positives:       19 samples = 22.9% of profile = 3.6% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     0.999
## [1] Exact Binomial test p-value:                              0.999
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.305
## [1] Risk difference: 0.114
## [1] Odds ratio:      1.731
## [1] -------------------------------------------------
## [1]
```

Student profiles was refined only into even smaller profile, so we will reject this profile.

```r
analyse_profile((data$pdays > 0) & ((data$poutcome == "other") | (data$poutcome == "success")))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        326 samples = 7.2% of dataset
## [1] Positives:       121 samples = 37.1% of profile = 23.2% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.617
## [1] Risk difference: 0.256
## [1] Odds ratio:      2.289
## [1] -------------------------------------------------
## [1]
```

Unsurprisingly this profile matches **poutcome** *other* or *success* completely, this was done more as an integrety check.

```r
analyse_profile((data$loan == "no") | (data$housing == "no"))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        4115 samples = 91.0% of dataset
```

```
## [1] Positives:       496 samples = 12.1% of profile = 95.2% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              0.861
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.016
## [1] Risk difference: 0.005
## [1] Odds ratio:      1.040
## [1] -------------------------------------------------
## [1]
```

Union of two of the widest profiles has resulted in the profile that spans more than 90% of data, but positive rate is close to the baseline, so we will not use this profile.

```
analyse_profile((data$job == "retired") & (data$balance > 0))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        200 samples = 4.4% of dataset
## [1] Positives:       47 samples = 23.5% of profile = 9.0% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.319
## [1] Risk difference: 0.120
## [1] Odds ratio:      1.763
## [1] -------------------------------------------------
## [1]
```

This is not the best profile, but it will be easy to search for such people so, we are going to use it later.

```
analyse_profile((data$housing == "no") & (data$pdays > 0))
```

```
## [1] -------------------------------------------------
## [1] Coverage:        300 samples = 6.6% of dataset
## [1] Positives:       107 samples = 35.7% of profile = 20.5% of all positives
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.587
## [1] Risk difference: 0.241
## [1] Odds ratio:      2.250
## [1] -------------------------------------------------
## [1]
```

This profile is extremely good as it shows coverage of 6.6% while more than its third had subscribed to the deposit.

4.4 Profiles

We have chosen three profiles:

- Previous Campaign Outcome = other or success (have good history with the bank)
- Housing = no and Previous days > 0 (have being previously contacted and have capacity to subscribe to a deposit)
- Job = retired and balance > 0 (retired people who have capacity and motivation to grow their fortune)

Let us take a look at individual profiles once more and than investigate how different they really are.

```
print("Previous Campaign Outcome = other or success")
```

```
## [1] "Previous Campaign Outcome = other or success"
```

```
analyse_profile(data$poutcome %in% c("other", "success"))
```

```
## [1] ---------------------------------------------------
## [1] Coverage:        326 samples = 7.2% of dataset
## [1] Positives:       121 samples = 37.1% of profile = 23.2% of all positives
## [1] ---------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ---------------------------------------------------
## [1] Cohen's h value: 0.617
## [1] Risk difference: 0.256
## [1] Odds ratio:      2.289
## [1] ---------------------------------------------------
## [1]
```

```
print("Housing = no and Previous days > 0")
```

```
## [1] "Housing = no and Previous days > 0"
```

```
analyse_profile((data$housing == "no") & (data$pdays > 0))
```

```
## [1] ---------------------------------------------------
## [1] Coverage:        300 samples = 6.6% of dataset
## [1] Positives:       107 samples = 35.7% of profile = 20.5% of all positives
## [1] ---------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ---------------------------------------------------
## [1] Cohen's h value: 0.587
## [1] Risk difference: 0.241
## [1] Odds ratio:      2.250
## [1] ---------------------------------------------------
## [1]
```

```
print("Job = retired and balance > 0")
```

```
## [1] "Job = retired and balance > 0"
```

```
analyse_profile((data$job == "retired") & (data$balance > 0))
```

```
## [1] ---------------------------------------------------
## [1] Coverage:        200 samples = 4.4% of dataset
## [1] Positives:       47 samples = 23.5% of profile = 9.0% of all positives
## [1] ---------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] ---------------------------------------------------
## [1] Cohen's h value: 0.319
## [1] Risk difference: 0.120
## [1] Odds ratio:      1.763
## [1] ---------------------------------------------------
## [1]
```

Now we will take a look at pairwise intersections of profiles

```
print("Previous Campaign Outcome = other or success and Housing = no and Previous days > 0")
```

```
## [1] "Previous Campaign Outcome = other or success and Housing = no and Previous days > 0"
```

```
prof <- data$poutcome %in% c("other", "success") & (data$housing == "no") & (data$pdays > 0)
print(sprintf("Number of samples: %d = %.1f%% of dataset", sum(prof), 100 * mean(prof)))
```

```
## [1] "Number of samples: 153 = 3.4% of dataset"
```

```
print("", quote=F)
```

```
## [1]
```

```
print("Housing = no and Previous days > 0 and Job = retired and balance > 0")
```

```
## [1] "Housing = no and Previous days > 0 and Job = retired and balance > 0"
```

```
prof <- (data$housing == "no") & (data$pdays > 0) & (data$job == "retired") & (data$balance > 0)
print(sprintf("Number of samples: %d = %.1f%% of dataset", sum(prof), 100 * mean(prof)))
```

```
## [1] "Number of samples: 40 = 0.9% of dataset"
```

```
print("", quote=F)
```

```
## [1]
```

```
print("Job = retired and balance > 0 and Previous Campaign Outcome = other or success")
```

```
## [1] "Job = retired and balance > 0 and Previous Campaign Outcome = other or success"
```

```
prof <- (data$job == "retired") & (data$balance > 0) & data$poutcome %in% c("other", "success")
print(sprintf("Number of samples: %d = %.1f%% of dataset", sum(prof), 100 * mean(prof)))
```

```
## [1] "Number of samples: 16 = 0.4% of dataset"
```

```
print("", quote=F)
```

```
## [1]
```

```
print("Intersection of all profiles")
```

```
## [1] "Intersection of all profiles"
```

```
prof <- (data$job == "retired") & (data$balance > 0) & data$poutcome %in% c("other", "success") & (data$
print(sprintf("Number of samples: %d = %.1f%% of dataset", sum(prof), 100 * mean(prof)))
```

```
## [1] "Number of samples: 16 = 0.4% of dataset"
```

Most of intersections have neglectable intersection, but **poutcome** = *other* or *success* with **housing** = *no* and **pdays** > 0 intersect roughly as a half of their profiles. They still represent different populations. Let us see union of all profiles.

```
analyse_profile(
    (data$poutcome %in% c("other", "success")) |
    (data$housing == "no") & (data$pdays > 0) |
    (data$job == "retired") & (data$balance > 0)
)
```

```
## [1] --------------------------------------------------
## [1] Coverage:        633 samples = 14.0% of dataset
## [1] Positives:       181 samples = 28.6% of profile = 34.7% of all positives
```

```
## [1] -------------------------------------------------
## [1] Proportion test p-value against profile's compliment:     1.000
## [1] Exact Binomial test p-value:                              1.000
## [1] -------------------------------------------------
## [1] Cohen's h value: 0.436
## [1] Risk difference: 0.171
## [1] Odds ratio:      2.003
## [1] -------------------------------------------------
## [1]
```

Together profiles cover more than a third of all who had subscribed to the deposit, while covering 14% of population. It also doubles the odds of subscription compared to raw dataset. We call it a success.

4.5 Conclusion

We have done thorough examination of profiles and arrived at three profiles, that cover total 35% of people who subscribe. The room for improvment is to check other multivariate profiles, those could be guided by a different model, perhaps more flexible like random forest, polynomial logistic regression or support vector machine.